AI Study Assistant — Complete Tech Stack, Architecture & Line-by-Line Code Guide

Comprehensive Technical Interview Preparation Manual | Java 17 • Spring Boot 3.4.5 • Spring Security 6 + JWT • Spring Data JPA + Hibernate • TiDB Cloud (MySQL 8) • OpenRouter AI • Apache PDFBox 3.0 & POI 5.2 • React 18.3 • Vite 5.3 • Tailwind CSS 3.4 • Chart.js 4.5

> **Purpose of This Guide:** This manual explains every technology stack, why it was chosen over alternatives, the complete system architecture, and **every single module and line of code** across all 61 source files in the cleaned project—written so clearly that even someone completely new to these stacks can understand it and explain it with authority to a technical interviewer.

1. Executive Summary & 30-Second Interview Elevator Pitch

What is the AI Study Assistant? It is a production-grade, full-stack AI-powered learning platform. A student can paste raw lecture notes or upload a PDF or PowerPoint (`.pptx`) presentation. The Spring Boot backend extracts the document text, constructs a strict JSON-schema prompt, and calls an LLM via the **OpenRouter AI API**. The AI returns a 3–5 paragraph summary, 10 flashcards, and 5 multiple-choice quiz questions, which are validated and saved atomically inside a database transaction. Students can then review flashcards using the **Leitner 5-Box Spaced Repetition System** (intervals of 1, 2, 4, 8, and 16 days), take **5-minute timed quizzes** with auto-submit and instant answer breakdowns, and track their mastery via **Chart.js learning analytics** and gamified achievements.

> **30-Second Interview Pitch:** "I built a full-stack AI Study Assistant using Java 17 and Spring Boot 3.4 on the backend and React 18 with Vite and Tailwind CSS on the frontend, backed by a serverless MySQL-compatible TiDB Cloud database. Users upload PDFs, PowerPoint decks, or text notes, which the backend parses using Apache PDFBox and Apache POI. It then orchestrates structured JSON generation through OpenRouter's LLM API inside a transactional boundary to create summaries, Leitner spaced-repetition flashcards, and timed quizzes. Every endpoint is protected by stateless Spring Security with HS256 JWTs, BCrypt cost-12 password hashing, and strict ownership checks to prevent IDOR vulnerabilities."

2. Dead Code & Unused Files Audit (What Was Removed & Why)

- **Deleted `PROJECT_EXPLANATION.md` & `PROJECT_EXPLANATION.pdf`:** Outdated files referencing a legacy "GroqService.java" prototype that no longer existed.
- **Deleted `backend/data/` (`studyassistant.mv.db`, `studyassistant.trace.db`):** Obsolete local H2 database binary files superseded by TiDB Cloud MySQL and in-memory H2 test profiles.
- **Deleted `backend/src/test/resources/application.properties` & `backend/src/main/resources/schema.sql`:** Removed the duplicate test config file and the outdated manual SQL DDL script (which lacked `summary`, `box_level`, and `next_review_date` columns and caused H2 test startup errors on `CREATE DATABASE`; Hibernate's `ddl-auto` manages schema generation automatically).
- **Removed Unused Dependencies:** Removed `lombok` from `backend/pom.xml` (all classes use explicit getters/setters or Java 17 `records`) and `react-markdown` from `frontend/package.json` and `StudyPage.jsx`.
- **Cleaned Dead Methods & Duplicate Checks:** Removed duplicate ownership check and dead methods in `QuizService.java` (`countStudentAttempts`, `getAverageScore`), unused methods in `NoteService.java` and `NoteRepository.java` (`getRecentNotes`, `countStudentNotes`, `findTop5ByStudentIdOrderByCreatedAtDesc`, `countByStudentId`), unused constructors in `FlashcardDTO.java`, unused `MAX_BOX` constant in `Flashcard.java`, dead `@JsonIgnore correctOption` field in `QuizQuestionDTO.java`, unused `relativeTime()` in `formatDate.js`, unused `stripQuestionFields` import in `HistoryPage.jsx`, redundant condition in `Timer.jsx`, unused CSS rules in `index.css`, and fixed mojibake emoji encoding in `NotesPage.jsx`.

3. Complete Tech Stack Deep-Dive (What Each Stack Is & Why It Is Used)

- **Java 17 (LTS):** Strongly-typed, object-oriented language running on the JVM. Chosen because it is the baseline for Spring Boot 3.x and gives us **Java Records** (immutable, zero-boilerplate DTOs like `AnalyticsResponse`, `DashboardResponse`, `PaginatedScoreHistory`, `ScorePoint`) and **Multi-line Text Blocks** (`"""..."""` in `OpenRouterService`).
- **Spring Boot 3.4.5:** Enterprise Java backend framework providing embedded Apache Tomcat, Inversion of Control (IoC) dependency injection, auto-configuration, and declarative `@Transactional` management. Chosen over Express/Node.js because multi-table AI persistence (updating Note + deleting old Flashcards/Quizzes + inserting 10 Flashcards + 1 Quiz + 5 QuizQuestions) requires strict ACID transaction rollback if any step fails.
- **Spring Security 6 + JJWT (0.11.5) + BCrypt (Cost 12):** Handles authentication and authorization. Because our React frontend (Vercel) and Spring Boot backend (Render) live on separate domains, stateful session cookies face cross-origin restrictions and don't scale horizontally. Stateless HMAC-SHA256 JWT tokens sent via `Authorization: Bearer <token>` require zero server session memory. Passwords are salted and hashed with BCrypt cost factor 12 (4,096 iterations).
- **Spring Data JPA + Hibernate 6.6:** Object-Relational Mapping (ORM) layer that maps Java `@Entity` classes to SQL tables, prevents SQL injection via prepared statements, and provides `@EntityGraph` to eliminate N+1 query bottlenecks.
- **TiDB Cloud (MySQL 8 Compatible) + H2 In-Memory Database:** TiDB Cloud provides a serverless, distributed MySQL 8 database with TLS encryption in production, while H2 runs in-memory (`jdbc:h2:mem:testdb;MODE=MySQL`) during `mvn test` so our 23 unit/integration tests run in 2 seconds with zero external dependencies.
- **Apache PDFBox 3.0.1 & Apache POI 5.2.5:** Java libraries that parse binary `.pdf` files (`Loader.loadPDF` + `PDFTextStripper`) and Microsoft PowerPoint `.pptx` slide decks (`XMLSlideShow` + `XSLFTextShape`) into plain text before calling the LLM.
- **OpenRouter AI API:** Unified OpenAI-compatible LLM API gateway. We use model `openrouter/free` with `response_format: {"type": "json_object"}` and custom retry backoff on HTTP 429 rate limits.
- **Jakarta Bean Validation & Springdoc OpenAPI 3 (Swagger UI):** Declarative request validation (`@NotBlank`, `@Email`, `@Size`, `@Pattern`) and auto-generated interactive API docs at `/swagger-ui.html`.
- **React 18.3 + Vite 5.3 + React Router DOM 6.23:** Component-based SPA frontend with instant HMR dev server, client-side routing, nested `<Outlet />` layouts, and route guards (`<ProtectedRoute>`).
- **Axios 1.7.2 + Tailwind CSS 3.4 + Chart.js 4.5 (react-chartjs-2):** Axios centralizes HTTP calls with JWT request/response interceptors; Tailwind CSS + custom CSS3 delivers the dark-mode aurora liquid-glass UI and 3D card-flip transforms; Chart.js renders interactive line and bar charts on HTML5 Canvas.

Part I: DevOps, Build & Config Files (Every Line Explained)

Module: GitHub Actions CI Workflow (`.github/workflows/ci.yml`)

What This File Is (Beginner-Friendly Analogy): An automated quality-control robot that runs on GitHub's cloud servers every time code is pushed or a Pull Request is opened against 'main'.

Why It Is Used & Architectural Role: Automates running all 23 backend JUnit tests, compiling the Spring Boot production JAR, and building the React Vite production bundle so broken code can never be merged unnoticed.

Complete Source Code (Every Line Numbered):

```
 1 | name: CI
 2 |
 3 | on:
 4 |   push:
 5 |     branches: [main]
 6 |   pull_request:
 7 |     branches: [main]
 8 |
 9 | jobs:
10 |   backend-test:
11 |     name: Backend Tests
12 |     runs-on: ubuntu-latest
13 |     defaults:
14 |       run:
15 |         working-directory: backend
16 |     steps:
17 |       - uses: actions/checkout@v4
18 |
19 |       - name: Set up JDK 17
20 |         uses: actions/setup-java@v4
21 |         with:
```

```
22 |           java-version: '17'
23 |           distribution: 'temurin'
24 |           cache: maven
25 |
26 |       - name: Run unit tests
27 |         run: mvn test -Dnet.bytebuddy.experimental=true
28 |
29 |       - name: Compile production build
30 |         run: mvn clean package -DskipTests
31 |
32 |   backend-compile:
33 |     name: Backend Build
34 |     runs-on: ubuntu-latest
35 |     defaults:
36 |       run:
37 |         working-directory: backend
38 |     steps:
39 |       - uses: actions/checkout@v4
40 |
41 |       - name: Set up JDK 17
42 |         uses: actions/setup-java@v4
43 |         with:
44 |           java-version: '17'
45 |           distribution: 'temurin'
46 |           cache: maven
47 |
48 |       - name: Compile
49 |         run: mvn clean compile -DskipTests
50 |
51 |   frontend-build:
52 |     name: Frontend Build
53 |     runs-on: ubuntu-latest
54 |     defaults:
55 |       run:
56 |         working-directory: frontend
57 |     steps:
58 |       - uses: actions/checkout@v4
59 |
60 |       - name: Set up Node.js
61 |         uses: actions/setup-node@v4
62 |         with:
63 |           node-version: 20
64 |           cache: npm
65 |           cache-dependency-path: frontend/package-lock.json
66 |
67 |       - name: Install dependencies
68 |         run: npm ci
69 |
70 |       - name: Lint (if configured)
71 |         run: npm run lint --if-present
72 |
73 |       - name: Build
74 |         run: npm run build
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–7:** Sets workflow name 'CI' and triggers on 'push' and 'pull_request' to branch 'main'.
- **Lines 9–30 (backend-test):** Runs on 'ubuntu-latest' in directory 'backend'. Checks out repo ('actions/checkout@v4'), sets up Eclipse Temurin JDK 17 with Maven cache ('actions/setup-java@v4'), runs 'mvn test -Dnet.bytebuddy.experimental=true', and packages the JAR ('mvn clean package -DskipTests').
- **Lines 32–49 (backend-compile):** Verifies clean compilation via 'mvn clean compile -DskipTests'.
- **Lines 51–73 (frontend-build):** Runs in 'frontend' with Node.js 20 ('actions/setup-node@v4'), installs exact lockfile dependencies via 'npm ci', runs 'npm run lint --if-present', and executes 'npm run build'.

> **How to Explain This Module in an Interview:** Explain that your CI pipeline runs 3 parallel jobs on Ubuntu runners with Maven and npm lockfile caching to validate both backend tests and frontend production bundling on every push.

Module: Backend Multi-Stage Dockerfile (`backend/Dockerfile`)

What This File Is (Beginner-Friendly Analogy): A two-stage container recipe: Stage 1 uses a full Maven + JDK 17 image to compile the Java code into a JAR; Stage 2 copies only that JAR into a slim JRE 17 runtime image.

Why It Is Used & Architectural Role: Containerizes the Spring Boot backend for Render deployment while keeping the final image small and secure.

Complete Source Code (Every Line Numbered):

```
# Use official Maven image for the build stage
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
# Download dependencies (cache layer)
RUN mvn dependency:go-offline -B
COPY src ./src
# Build the application
RUN mvn clean package -DskipTests

# Use lightweight JDK image for the runtime stage
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --from=build /app/target/study-assistant-1.0.0.jar app.jar

# Expose the default Spring Boot port
EXPOSE 8080

# Run the application using the prod profile
ENTRYPOINT ["java", "-jar", "app.jar", "--spring.profiles.active=prod"]
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–5:** Starts Stage 1 ('FROM maven:3.9.6-eclipse-temurin-17 AS build'), sets 'WORKDIR /app', copies 'pom.xml', and runs 'mvn dependency:go-offline -B' to cache Maven dependencies in a dedicated Docker layer.
- **Lines 6–8:** Copies 'src' into './src' and builds the executable JAR via 'mvn clean package -DskipTests'.
- **Lines 10–13:** Starts Stage 2 ('FROM eclipse-temurin:17-jre-jammy'), sets 'WORKDIR /app', and copies '/app/target/study-assistant-1.0.0.jar' from the 'build' stage as 'app.jar'.
- **Lines 15–19:** Exposes port 8080 and launches the server with 'ENTRYPOINT ["java", "-jar", "app.jar", "--spring.profiles.active=prod"]'.

How to Explain This Module in an Interview: Emphasize layer caching ('COPY pom.xml' before 'COPY src') and multi-stage image reduction (JDK build image -> JRE runtime image).

Module: Backend Maven POM (`backend/pom.xml`)

What This File Is (Beginner-Friendly Analogy): The master dependency and build configuration file for the Java Spring Boot backend.

Why It Is Used & Architectural Role: Declares Spring Boot 3.4.5 parent inheritance, Java 17 compiler level, and all third-party libraries (Spring Web, Security, Data JPA, MySQL, H2, JJWT, Validation, PDFBox, Apache POI, Springdoc OpenAPI).

Complete Source Code (Every Line Numbered):

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.4.5</version>
        <relativePath/>
    </parent>

    <groupId>com.studyassistant</groupId>
    <artifactId>study-assistant</artifactId>
    <version>1.0.0</version>
    <name>AI Study Assistant</name>
    <description>AI-powered study assistant with OpenRouter integration</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Spring Boot Core -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Spring Security -->
        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>

        <!-- Spring Data JPA -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- MySQL Driver -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- JWT -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.11.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.11.5</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.11.5</version>
            <scope>runtime</scope>
        </dependency>

        <!-- Validation -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- PDF Text Extraction -->
        <dependency>
            <groupId>org.apache.pdfbox</groupId>
            <artifactId>pdfbox</artifactId>
            <version>3.0.1</version>
        </dependency>

        <!-- PPTX Extraction (Apache POI) -->
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>5.2.5</version>
        </dependency>
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi</artifactId>
            <version>5.2.5</version>
        </dependency>

        <!-- Swagger / OpenAPI -->
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.3.0</version>
        </dependency>

        <!-- Testing -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- H2 Database (dev + test) -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
```

```
117 |             <scope>runtime</scope>
118 |         </dependency>
119 |     </dependencies>
120 |
121 |     <build>
122 |         <plugins>
123 |             <plugin>
124 |                 <groupId>org.springframework.boot</groupId>
125 |                 <artifactId>spring-boot-maven-plugin</artifactId>
126 |             </plugin>
127 |             <plugin>
128 |                 <groupId>org.apache.maven.plugins</groupId>
129 |                 <artifactId>maven-surefire-plugin</artifactId>
130 |                 <version>3.2.5</version>
131 |             </plugin>
132 |         </plugins>
133 |     </build>
134 | </project>
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–19:** Declares XML schema, inherits from 'spring-boot-starter-parent' 3.4.5, defines project coordinates ('com.studyassistant:study-assistant:1.0.0'), and sets '17'.
- **Lines 21–45:** Declares 'spring-boot-starter-web', 'spring-boot-starter-security', 'spring-boot-starter-data-jpa', and runtime 'mysql-connector-j'.
- **Lines 47–65:** Declares 'jjwt-api', 'jjwt-impl', and 'jjwt-jackson' (v0.11.5) for JWT token creation and parsing.
- **Lines 67–94:** Declares 'spring-boot-starter-validation', Apache 'pdfbox' 3.0.1, Apache 'poi' + 'poi-ooxml' 5.2.5, and 'springdoc-openapi-starter-webmvc-ui' 2.3.0.
- **Lines 96–124:** Declares test starters ('spring-boot-starter-test', 'spring-security-test'), 'h2' database, 'spring-boot-maven-plugin', and 'maven-surefire-plugin' 3.2.5.

How to Explain This Module in an Interview: Point out why 'jjwt-impl', 'jjwt-jackson', 'mysql-connector-j', and 'h2' use 'runtime': our code compiles against standard interfaces (JPA, JDBC, JJWT API), preventing tight coupling to implementation internals.

Module: Backend Default, Production & Test Properties (`backend/src/main/resources/application.properties`)

What This File Is (Beginner-Friendly Analogy): The main configuration file for the Spring Boot server, with environment variable overrides.

Why It Is Used & Architectural Role: Configures server port, TiDB Cloud MySQL connection with TLS, Hibernate DDL auto-update, JWT secret/expiration, OpenRouter API URL/key, 10MB file upload limits, and Swagger paths.

Complete Source Code (Every Line Numbered):

```
 1 | # Server
 2 | server.port=8080
 3 |
 4 | # ============================================================
 5 | # TiDB Cloud (MySQL-compatible, cloud DB)
 6 | # Free tier: gateway01.ap-southeast-1.prod.aws.tidbcloud.com:4000
 7 | # Secrets are injected via env vars (see application-local.properties for local dev):
 8 | #   SPRING_DATASOURCE_URL / SPRING_DATASOURCE_USERNAME / SPRING_DATASOURCE_PASSWORD
 9 | # ============================================================
10 | spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mysql://gateway01.ap-
southeast-1.prod.aws.tidbcloud.com:4000/study_assistant_db?
sslMode=VERIFY_IDENTITY&enabledTLSProtocols=TLSv1.2,TLSv1.3&createDatabaseIfNotExist=true}
11 | spring.datasource.username=${SPRING_DATASOURCE_USERNAME:4H1xmQtU8dEWWtm.root}
12 | spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:}
13 | spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
14 |
15 | # JPA / Hibernate
16 | spring.jpa.hibernate.ddl-auto=update
17 | spring.jpa.show-sql=true
18 | spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
19 |
20 | # JWT
21 | app.jwt.secret=${JWT_SECRET:YourSuperSecretKeyForJWTAuthenticationMustBeAtLeast32Characters}
22 | app.jwt.expiration=86400000
23 |
24 | # OpenRouter AI
25 | app.openrouter.api.key=${OPENROUTER_API_KEY:YOUR_API_KEY_HERE}
26 | app.openrouter.api.url=https://openrouter.ai/api/v1/chat/completions
```

```
 27 |
 28 | # File Upload
 29 | spring.servlet.multipart.max-file-size=10MB
 30 | spring.servlet.multipart.max-request-size=10MB
 31 |
 32 | # Swagger
 33 | springdoc.api-docs.path=/api-docs
 34 | springdoc.swagger-ui.path=/swagger-ui.html
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–12:** Sets 'server.port=8080' and configures 'spring.datasource.url', 'username', 'password', and 'com.mysql.cj.jdbc.Driver' using '${ENV_VAR:default}' syntax for TiDB Cloud.
- **Lines 14–17:** Sets 'spring.jpa.hibernate.ddl-auto=update', 'show-sql=true', and 'MySQL8Dialect'.
- **Lines 19–25:** Configures 'app.jwt.secret', 'app.jwt.expiration=86400000' (24 hours), 'app.openrouter.api.key', and 'app.openrouter.api.url'.
- **Lines 27–32:** Sets 'max-file-size=10MB', 'max-request-size=10MB', and Swagger paths ('/api-docs' and '/swagger-ui.html').

How to Explain This Module in an Interview: Also reference 'application-prod.yml' (which disables 'show-sql' and requires 'JWT_SECRET' and 'OPENROUTER_API_KEY' without defaults) and 'application-test.properties' (which configures in-memory H2 'jdbc:h2:mem:testdb;MODE=MySQL' with 'ddl-auto=create-drop').

Module: Production YAML Profile & Test Properties (`backend/src/main/resources/application-prod.yml`)

What This File Is (Beginner-Friendly Analogy): Environment-specific overrides for production ('application-prod.yml') and automated tests ('application-test.properties').

Why It Is Used & Architectural Role: Separates production security settings and in-memory test settings from local development defaults.

Complete Source Code (Every Line Numbered):

```
  1 | spring:
  2 |   datasource:
  3 |     # TiDB Cloud — connection details injected via Render environment variables
  4 |     url: ${SPRING_DATASOURCE_URL:jdbc:mysql://gateway01.ap-southeast-1.prod.aws.tidbcloud.com:4000/
study_assistant_db?sslMode=VERIFY_IDENTITY&enabledTLSProtocols=TLSv1.2,TLSv1.3&createDatabaseIfNotExist=true}
  5 |     username: ${SPRING_DATASOURCE_USERNAME:4H1xmQtU8dEWWtm.root}
  6 |     password: ${SPRING_DATASOURCE_PASSWORD}
  7 |     driver-class-name: com.mysql.cj.jdbc.Driver
  8 |   jpa:
  9 |     show-sql: false
 10 |     hibernate:
 11 |       ddl-auto: update
 12 |     properties:
 13 |       hibernate:
 14 |         dialect: org.hibernate.dialect.MySQL8Dialect
 15 |
 16 | app:
 17 |   jwt:
 18 |     # Required in production — the app refuses to start if JWT_SECRET is not set
 19 |     secret: ${JWT_SECRET}
 20 |   openrouter:
 21 |     api:
 22 |       key: ${OPENROUTER_API_KEY}
 23 |
 24 | cors:
 25 |   # Render environment variable to allow Vercel domain
 26 |   allowed-origins: ${ALLOWED_ORIGIN:http://localhost:5173}
```

Line-by-Line & Block-by-Block Explanation:

- **application-prod.yml Lines 1–25:** Injects TiDB Cloud credentials, sets 'show-sql: false', enforces mandatory '${JWT_SECRET}' and '${OPENROUTER_API_KEY}', and sets 'cors.allowed-origins' from '${ALLOWED_ORIGIN}'.
- **application-test.properties:** Configures 'jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE;MODE=MySQL', 'org.h2.Driver', 'ddl-auto=create-drop', and test JWT/OpenRouter keys so unit and MockMvc tests run isolated in RAM.

How to Explain This Module in an Interview: Explain how '@ActiveProfiles("test")' on controller tests automatically loads 'application-test.properties' to swap TiDB Cloud for an in-memory H2 instance.

Part II: Backend Entrypoint, Config & Security Modules

Module: StudyAssistantApplication (Main Class) (`backend/src/main/java/com/studyassistant/StudyAssistantApplication.java`)

What This File Is (Beginner-Friendly Analogy): The entry point of the Spring Boot application.

Why It Is Used & Architectural Role: Launches the Spring ApplicationContext, triggers component scanning across 'com.studyassistant.*', and boots embedded Tomcat.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant;
 2 |
 3 | import org.springframework.boot.SpringApplication;
 4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
 5 |
 6 | @SpringBootApplication
 7 | public class StudyAssistantApplication {
 8 |
 9 |     public static void main(String[] args) {
10 |         SpringApplication.run(StudyAssistantApplication.class, args);
11 |     }
12 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–4:** Package declaration and Spring Boot imports.
- **Lines 6–11:** '@SpringBootApplication' enables auto-configuration and component scanning; 'SpringApplication.run(StudyAssistantApplication.class, args)' boots the server.

How to Explain This Module in an Interview: Concise, standard Spring Boot entrypoint placed at the root package 'com.studyassistant'.

Module: CorsConfig (`backend/src/main/java/com/studyassistant/config/CorsConfig.java`)

What This File Is (Beginner-Friendly Analogy): Configures Cross-Origin Resource Sharing (CORS) so the React frontend can call the Spring Boot API from a different origin/port.

Why It Is Used & Architectural Role: Registers a 'CorsFilter' bean allowing configured origins, HTTP methods, and headers.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.config;
 2 |
 3 | import org.springframework.beans.factory.annotation.Value;
 4 | import org.springframework.context.annotation.Bean;
 5 | import org.springframework.context.annotation.Configuration;
 6 | import org.springframework.web.cors.CorsConfiguration;
 7 | import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
 8 | import org.springframework.web.filter.CorsFilter;
 9 |
10 | import java.util.Arrays;
11 | import java.util.List;
12 |
13 | @Configuration
14 | public class CorsConfig {
15 |
16 |     @Value("${cors.allowed-origins:http://localhost:5173,http://localhost:3000}")
17 |     private List<String> allowedOrigins;
18 |
19 |     @Bean
20 |     public CorsFilter corsFilter() {
21 |         CorsConfiguration config = new CorsConfiguration();
22 |         config.setAllowedOrigins(allowedOrigins);
23 |         config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS"));
24 |         config.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type", "Accept"));
25 |         config.setAllowCredentials(true);
26 |         config.setMaxAge(3600L);
27 |
28 |         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
29 |         source.registerCorsConfiguration("/**", config);
30 |         return new CorsFilter(source);
```

```
31 |     }
32 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–17:** Marks class with '@Configuration' and injects 'cors.allowed-origins' into a 'List'.
- **Lines 19–31:** Creates 'CorsConfiguration' allowing GET/POST/PUT/DELETE/OPTIONS, headers 'Authorization'/'Content-Type'/'Accept', credentials=true, maxAge=3600s, and registers it on '/**'.

> **How to Explain This Module in an Interview:** Explain how 'cors.allowed-origins' adapts seamlessly between localhost:5173 in development and your Vercel domain in production.

Module: SecurityConfig (`backend/src/main/java/com/studyassistant/config/SecurityConfig.java`)

What This File Is (Beginner-Friendly Analogy): The central security configuration defining stateless JWT authentication, public vs. protected endpoints, 401 entrypoint formatting, and BCrypt password encoding.

Why It Is Used & Architectural Role: Builds the 'SecurityFilterChain', 'PasswordEncoder', and 'AuthenticationManager' beans.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.config;
 2 |
 3 | import com.studyassistant.security.JwtAuthFilter;
 4 | import jakarta.servlet.http.HttpServletResponse;
 5 | import org.springframework.context.annotation.Bean;
 6 | import org.springframework.context.annotation.Configuration;
 7 | import org.springframework.security.authentication.AuthenticationManager;
 8 | import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
 9 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
10 | import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
11 | import org.springframework.security.config.http.SessionCreationPolicy;
12 | import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
13 | import org.springframework.security.crypto.password.PasswordEncoder;
14 | import org.springframework.security.web.SecurityFilterChain;
15 | import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
16 |
17 | @Configuration
18 | @EnableWebSecurity
19 | public class SecurityConfig {
20 |
21 |     private final JwtAuthFilter jwtAuthFilter;
22 |
23 |     public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
24 |         this.jwtAuthFilter = jwtAuthFilter;
25 |     }
26 |
27 |     @Bean
28 |     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
29 |         http
30 |             .csrf(csrf -> csrf.disable())
31 |             .cors(cors -> {})
32 |             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
33 |             .exceptionHandling(ex -> ex.authenticationEntryPoint((request, response, authException) -> {
34 |                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
35 |                 response.setContentType("application/json");
36 |                 response.getWriter().write("{\"status\":401,\"message\":\"Authentication required\"}");
37 |             }))
38 |             .authorizeHttpRequests(auth -> auth
39 |                 .requestMatchers(
40 |                     "/api/auth/**",
41 |                     "/swagger-ui/**",
42 |                     "/swagger-ui.html",
43 |                     "/v3/api-docs/**",
44 |                     "/api-docs/**"
45 |                 ).permitAll()
46 |                 .anyRequest().authenticated()
47 |             )
48 |             .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
49 |
50 |         return http.build();
51 |     }
52 |
53 |     @Bean
54 |     public PasswordEncoder passwordEncoder() {
55 |         return new BCryptPasswordEncoder(12);
```

```
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 17–25:** '@Configuration' and '@EnableWebSecurity', injecting 'JwtAuthFilter'.
- **Lines 27–36:** Disables CSRF (stateless Bearer API), enables CORS, sets 'SessionCreationPolicy.STATELESS', and configures a custom 'authenticationEntryPoint' that writes HTTP 401 JSON '{"status":401,"message":"Authentication required"}'.
- **Lines 37–49:** Permits '/api/auth/**' and Swagger paths ('/swagger-ui/**', '/v3/api-docs/**'), requires authentication for all other requests, and inserts 'jwtAuthFilter' before 'UsernamePasswordAuthenticationFilter'.
- **Lines 52–59:** Defines 'BCryptPasswordEncoder(12)' and 'AuthenticationManager' beans.

How to Explain This Module in an Interview: Highlight the custom 401 JSON AuthenticationEntryPoint, Stateless SessionCreationPolicy, and BCrypt cost factor 12.

Module: SwaggerConfig (`backend/src/main/java/com/studyassistant/config/SwaggerConfig.java`)

What This File Is (Beginner-Friendly Analogy): Configures OpenAPI 3.0 documentation and adds the JWT 'Authorize' button in Swagger UI.

Why It Is Used & Architectural Role: Provides a custom 'OpenAPI' bean with 'Bearer Authentication' security scheme.

Complete Source Code (Every Line Numbered):

```
package com.studyassistant.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SwaggerConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("AI Study Assistant API")
                        .version("1.0.0")
                        .description("API documentation for the AI Study Assistant platform. " +
                                "Upload notes, generate AI summaries & flashcards, take quizzes, and track progress.")
                        .contact(new Contact()
                                .name("Study Assistant Team")
                                .email("support@studyassistant.com")))
                .addSecurityItem(new SecurityRequirement().addList("Bearer Authentication"))
                .components(new Components()
                        .addSecuritySchemes("Bearer Authentication",
                                new SecurityScheme()
                                        .type(SecurityScheme.Type.HTTP)
                                        .bearerFormat("JWT")
                                        .scheme("bearer")));
    }
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–32:** Creates an 'OpenAPI' bean with API title, version, description, contact info, and a 'SecurityScheme' of type HTTP, scheme 'bearer', bearerFormat 'JWT'.

How to Explain This Module in an Interview: Enables interactive testing of protected endpoints directly inside '/swagger-ui.html'.

Module: CustomUserDetailsService (`backend/src/main/java/com/studyassistant/security/CustomUserDetailsService.java`)

What This File Is (Beginner-Friendly Analogy): Bridges our JPA 'StudentRepository' with Spring Security's 'UserDetailsService' contract.

Why It Is Used & Architectural Role: Loads a 'Student' by email during authentication and wraps it in a Spring Security 'User' principal.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.security;
 2 |
 3 | import com.studyassistant.model.Student;
 4 | import com.studyassistant.repository.StudentRepository;
 5 | import org.springframework.security.core.authority.SimpleGrantedAuthority;
 6 | import org.springframework.security.core.userdetails.User;
 7 | import org.springframework.security.core.userdetails.UserDetails;
 8 | import org.springframework.security.core.userdetails.UserDetailsService;
 9 | import org.springframework.security.core.userdetails.UsernameNotFoundException;
10 | import org.springframework.stereotype.Service;
11 |
12 | import java.util.Collections;
13 |
14 | @Service
15 | public class CustomUserDetailsService implements UserDetailsService {
16 |
17 |     private final StudentRepository studentRepository;
18 |
19 |     public CustomUserDetailsService(StudentRepository studentRepository) {
20 |         this.studentRepository = studentRepository;
21 |     }
22 |
23 |     @Override
24 |     public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
25 |         Student student = studentRepository.findByEmail(email)
26 |                 .orElseThrow(() -> new UsernameNotFoundException("Student not found with email: " + email));
27 |
28 |         return new User(
29 |                 student.getEmail(),
30 |                 student.getPassword(),
31 |                 Collections.singletonList(new SimpleGrantedAuthority("ROLE_" + student.getRole()))
32 |         );
33 |     }
34 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–21:** Annotated with '@Service' and injects 'StudentRepository'.
- **Lines 23–32:** Overrides 'loadUserByUsername(String email)', queries 'studentRepository.findByEmail(email)' (throwing 'UsernameNotFoundException' if absent), and returns a Spring 'User' with authority 'ROLE_STUDENT'.

> **How to Explain This Module in an Interview:** Shows clean integration between Spring Data JPA entities and Spring Security principals.

Module: JwtAuthFilter (`backend/src/main/java/com/studyassistant/security/JwtAuthFilter.java`)

What This File Is (Beginner-Friendly Analogy): Intercepts every incoming HTTP request once, extracts and validates the 'Bearer' JWT token, and marks the request as authenticated in Spring's SecurityContext.

Why It Is Used & Architectural Role: Extends 'OncePerRequestFilter' to enforce stateless JWT authentication on every request.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.security;
 2 |
 3 | import jakarta.servlet.FilterChain;
 4 | import jakarta.servlet.ServletException;
 5 | import jakarta.servlet.http.HttpServletRequest;
 6 | import jakarta.servlet.http.HttpServletResponse;
 7 | import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
 8 | import org.springframework.security.core.context.SecurityContextHolder;
 9 | import org.springframework.security.core.userdetails.UserDetails;
```

```
 10 | import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
 11 | import org.springframework.stereotype.Component;
 12 | import org.springframework.web.filter.OncePerRequestFilter;
 13 |
 14 | import java.io.IOException;
 15 |
 16 | @Component
 17 | public class JwtAuthFilter extends OncePerRequestFilter {
 18 |
 19 |     private final JwtUtil jwtUtil;
 20 |     private final CustomUserDetailsService userDetailsService;
 21 |
 22 |     public JwtAuthFilter(JwtUtil jwtUtil, CustomUserDetailsService userDetailsService) {
 23 |         this.jwtUtil = jwtUtil;
 24 |         this.userDetailsService = userDetailsService;
 25 |     }
 26 |
 27 |     @Override
 28 |     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
 29 |                                     FilterChain filterChain) throws ServletException, IOException {
 30 |         final String authHeader = request.getHeader("Authorization");
 31 |
 32 |         if (authHeader == null || !authHeader.startsWith("Bearer ")) {
 33 |             filterChain.doFilter(request, response);
 34 |             return;
 35 |         }
 36 |
 37 |         final String token = authHeader.substring(7);
 38 |
 39 |         try {
 40 |             final String email = jwtUtil.extractEmail(token);
 41 |
 42 |             if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
 43 |                 UserDetails userDetails = userDetailsService.loadUserByUsername(email);
 44 |
 45 |                 if (jwtUtil.isTokenValid(token, userDetails.getUsername())) {
 46 |                     UsernamePasswordAuthenticationToken authToken =
 47 |                             new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
 48 |                     authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
 49 |                     SecurityContextHolder.getContext().setAuthentication(authToken);
 50 |                 }
 51 |             }
 52 |         } catch (Exception e) {
 53 |             // Invalid token — continue without authentication
 54 |             logger.warn("JWT authentication failed: " + e.getMessage());
 55 |         }
 56 |
 57 |         filterChain.doFilter(request, response);
 58 |     }
 59 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 16–25:** '@Component' extending 'OncePerRequestFilter', injecting 'JwtUtil' and 'CustomUserDetailsService'.
- **Lines 27–35:** Checks 'request.getHeader("Authorization")'. If null or not starting with 'Bearer ', passes the request down the filter chain unauthenticated; otherwise extracts the token via 'substring(7)'.
- **Lines 37–57:** Extracts email via 'jwtUtil.extractEmail(token)', loads 'UserDetails', verifies 'jwtUtil.isTokenValid()', sets 'UsernamePasswordAuthenticationToken' in 'SecurityContextHolder', catches any token errors gracefully, and invokes 'filterChain.doFilter(request, response)'.

How to Explain This Module in an Interview: Walk through the exact steps inside 'doFilterInternal': Header check -> substring(7) -> extractEmail -> loadUserByUsername -> isTokenValid -> SecurityContextHolder.getContext().setAuthentication().

Module: JwtUtil (`backend/src/main/java/com/studyassistant/security/JwtUtil.java`)

What This File Is (Beginner-Friendly Analogy): Handles generating, signing, parsing, and validating HMAC-SHA256 JSON Web Tokens.

Why It Is Used & Architectural Role: Provides cryptographic token operations using 'io.jsonwebtoken' (JJWT).

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.security;
  2 |
  3 | import io.jsonwebtoken.*;
```

```
 4 | import io.jsonwebtoken.security.Keys;
 5 | import org.springframework.beans.factory.annotation.Value;
 6 | import org.springframework.stereotype.Component;
 7 |
 8 | import java.security.Key;
 9 | import java.util.Date;
10 | import java.util.function.Function;
11 |
12 | @Component
13 | public class JwtUtil {
14 |
15 |     @Value("${app.jwt.secret}")
16 |     private String secret;
17 |
18 |     @Value("${app.jwt.expiration}")
19 |     private long expiration;
20 |
21 |     private Key getSigningKey() {
22 |         byte[] keyBytes = secret.getBytes();
23 |         return Keys.hmacShaKeyFor(keyBytes);
24 |     }
25 |
26 |     public String generateToken(String email) {
27 |         return Jwts.builder()
28 |                 .setSubject(email)
29 |                 .setIssuedAt(new Date())
30 |                 .setExpiration(new Date(System.currentTimeMillis() + expiration))
31 |                 .signWith(getSigningKey(), SignatureAlgorithm.HS256)
32 |                 .compact();
33 |     }
34 |
35 |     public String extractEmail(String token) {
36 |         return extractClaim(token, Claims::getSubject);
37 |     }
38 |
39 |     public Date extractExpiration(String token) {
40 |         return extractClaim(token, Claims::getExpiration);
41 |     }
42 |
43 |     public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
44 |         final Claims claims = extractAllClaims(token);
45 |         return claimsResolver.apply(claims);
46 |     }
47 |
48 |     private Claims extractAllClaims(String token) {
49 |         return Jwts.parserBuilder()
50 |                 .setSigningKey(getSigningKey())
51 |                 .build()
52 |                 .parseClaimsJws(token)
53 |                 .getBody();
54 |     }
55 |
56 |     public boolean isTokenValid(String token, String email) {
57 |         final String extractedEmail = extractEmail(token);
58 |         return (extractedEmail.equals(email) && !isTokenExpired(token));
59 |     }
60 |
61 |     private boolean isTokenExpired(String token) {
62 |         return extractExpiration(token).before(new Date());
63 |     }
64 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–23:** Injects 'app.jwt.secret' and 'app.jwt.expiration' and derives the HMAC-SHA signing key via 'Keys.hmacShaKeyFor(secret.getBytes())'.
- **Lines 25–32:** 'generateToken(String email)' sets subject, issuedAt, expiration, and signs with 'SignatureAlgorithm.HS256'.
- **Lines 34–61:** Provides 'extractEmail', 'extractExpiration', generic 'extractClaim', 'extractAllClaims' (which verifies the signature), 'isTokenValid', and 'isTokenExpired'.

> **How to Explain This Module in an Interview:** Emphasize that 'parseClaimsJws(token)' both decodes the claims and verifies the HMAC-SHA256 signature against tampering.

Part III: JPA Entity Models (`model/` — Every Line Explained)

Module: Student Entity (`backend/src/main/java/com/studyassistant/model/Student.java`)

What This File Is (Beginner-Friendly Analogy): Maps to the 'students' table storing registered student accounts.

Why It Is Used & Architectural Role: Core user identity table referenced by notes and quiz attempts.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.model;
 2 |
 3 | import jakarta.persistence.*;
 4 | import java.time.LocalDateTime;
 5 |
 6 | @Entity
 7 | @Table(name = "students")
 8 | public class Student {
 9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     @Column(nullable = false, length = 100)
15 |     private String name;
16 |
17 |     @Column(nullable = false, unique = true, length = 150)
18 |     private String email;
19 |
20 |     @Column(nullable = false)
21 |     private String password;
22 |
23 |     @Column(length = 20)
24 |     private String role = "STUDENT";
25 |
26 |     @Column(name = "created_at")
27 |     private LocalDateTime createdAt;
28 |
29 |     @PrePersist
30 |     protected void onCreate() {
31 |         this.createdAt = LocalDateTime.now();
32 |     }
33 |
34 |     public Student() {}
35 |
36 |     public Student(String name, String email, String password) {
37 |         this.name = name;
38 |         this.email = email;
39 |         this.password = password;
40 |         this.role = "STUDENT";
41 |     }
42 |
43 |     public Long getId() { return id; }
44 |     public void setId(Long id) { this.id = id; }
45 |
46 |     public String getName() { return name; }
47 |     public void setName(String name) { this.name = name; }
48 |
49 |     public String getEmail() { return email; }
50 |     public void setEmail(String email) { this.email = email; }
51 |
52 |     public String getPassword() { return password; }
53 |     public void setPassword(String password) { this.password = password; }
54 |
55 |     public String getRole() { return role; }
56 |     public void setRole(String role) { this.role = role; }
57 |
58 |     public LocalDateTime getCreatedAt() { return createdAt; }
59 |     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
60 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 6–12:** '@Entity', '@Table(name = "students")', primary key 'id' with 'GenerationType.IDENTITY'.
- **Lines 14–27:** Columns 'name' (not null, 100 chars), 'email' (not null, unique, 150 chars), 'password' (not null, BCrypt hash), 'role' (defaults to 'STUDENT'), and 'created_at'.
- **Lines 29–60:** '@PrePersist onCreate()' sets 'createdAt = LocalDateTime.now()', constructors, and getters/setters.

> **How to Explain This Module in an Interview:** Mention unique constraint on email and automatic '@PrePersist' timestamping.

Module: Note Entity (`backend/src/main/java/com/studyassistant/model/Note.java`)

What This File Is (Beginner-Friendly Analogy): Maps to the 'notes' table storing uploaded text/PDF/PPTX content and the AI-generated summary.

Why It Is Used & Architectural Role: Core content entity owned by a Student.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.model;
 2 |
 3 | import jakarta.persistence.*;
 4 | import java.time.LocalDateTime;
 5 |
 6 | @Entity
 7 | @Table(name = "notes")
 8 | public class Note {
 9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     @ManyToOne(fetch = FetchType.LAZY)
15 |     @JoinColumn(name = "student_id", nullable = false)
16 |     private Student student;
17 |
18 |     @Column(nullable = false, length = 200)
19 |     private String title;
20 |
21 |     @Column(nullable = false, columnDefinition = "LONGTEXT")
22 |     private String content;
23 |
24 |     @Column(columnDefinition = "LONGTEXT")
25 |     private String summary;
26 |
27 |     @Column(name = "created_at")
28 |     private LocalDateTime createdAt;
29 |
30 |     @PrePersist
31 |     protected void onCreate() {
32 |         this.createdAt = LocalDateTime.now();
33 |     }
34 |
35 |     public Note() {}
36 |
37 |     public Note(Student student, String title, String content) {
38 |         this.student = student;
39 |         this.title = title;
40 |         this.content = content;
41 |     }
42 |
43 |     public Long getId() { return id; }
44 |     public void setId(Long id) { this.id = id; }
45 |
46 |     public Student getStudent() { return student; }
47 |     public void setStudent(Student student) { this.student = student; }
48 |
49 |     public String getTitle() { return title; }
50 |     public void setTitle(String title) { this.title = title; }
51 |
52 |     public String getContent() { return content; }
53 |     public void setContent(String content) { this.content = content; }
54 |
55 |     public String getSummary() { return summary; }
56 |     public void setSummary(String summary) { this.summary = summary; }
57 |
58 |     public LocalDateTime getCreatedAt() { return createdAt; }
59 |     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
60 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 6–15:** '@Entity', '@Table(name = "notes")', auto-increment 'id', and '@ManyToOne(fetch = FetchType.LAZY)' foreign key 'student_id'.
- **Lines 17–27:** 'title' (VARCHAR 200), 'content' (LONGTEXT), 'summary' (LONGTEXT), and 'created_at'.
- **Lines 29–60:** '@PrePersist onCreate()', constructors, and getters/setters.

How to Explain This Module in an Interview: Highlight 'FetchType.LAZY' on 'student' and 'LONGTEXT' column definitions for large extracted documents.

Module: Flashcard Entity (`backend/src/main/java/com/studyassistant/model/Flashcard.java`)

What This File Is (Beginner-Friendly Analogy): Maps to the 'flashcards' table storing Q&A pairs and Leitner spaced-repetition state.

Why It Is Used & Architectural Role: Stores flashcards linked to a Note along with 'boxLevel' (1–5) and 'nextReviewDate'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.model;
 2 |
 3 | import jakarta.persistence.*;
 4 | import java.time.LocalDateTime;
 5 |
 6 | @Entity
 7 | @Table(name = "flashcards")
 8 | public class Flashcard {
 9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     @ManyToOne(fetch = FetchType.LAZY)
15 |     @JoinColumn(name = "note_id", nullable = false)
16 |     private Note note;
17 |
18 |     @Column(nullable = false, columnDefinition = "TEXT")
19 |     private String question;
20 |
21 |     @Column(nullable = false, columnDefinition = "TEXT")
22 |     private String answer;
23 |
24 |     @Column(name = "box_level", nullable = false)
25 |     private Integer boxLevel = 1;
26 |
27 |     @Column(name = "next_review_date", nullable = false)
28 |     private LocalDateTime nextReviewDate = LocalDateTime.now();
29 |
30 |     public Flashcard() {}
31 |
32 |     public Flashcard(Note note, String question, String answer) {
33 |         this.note = note;
34 |         this.question = question;
35 |         this.answer = answer;
36 |     }
37 |
38 |     public Long getId() { return id; }
39 |     public void setId(Long id) { this.id = id; }
40 |
41 |     public Note getNote() { return note; }
42 |     public void setNote(Note note) { this.note = note; }
43 |
44 |     public String getQuestion() { return question; }
45 |     public void setQuestion(String question) { this.question = question; }
46 |
47 |     public String getAnswer() { return answer; }
48 |     public void setAnswer(String answer) { this.answer = answer; }
49 |
50 |     public Integer getBoxLevel() { return boxLevel; }
51 |     public void setBoxLevel(Integer boxLevel) { this.boxLevel = boxLevel; }
52 |
53 |     public LocalDateTime getNextReviewDate() { return nextReviewDate; }
54 |     public void setNextReviewDate(LocalDateTime nextReviewDate) { this.nextReviewDate = nextReviewDate; }
55 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 6–21:** '@Entity', '@Table(name = "flashcards")', 'id', lazy '@ManyToOne Note', and TEXT columns 'question' and 'answer'.
- **Lines 23–27:** 'boxLevel = 1' and 'nextReviewDate = LocalDateTime.now()' defaults.
- **Lines 29–55:** Constructors and getters/setters.

> **How to Explain This Module in an Interview:** Explain how 'boxLevel' and 'nextReviewDate' persist the Leitner spaced-repetition state directly on each flashcard row.

Module: Quiz Entity (`backend/src/main/java/com/studyassistant/model/Quiz.java`)

What This File Is (Beginner-Friendly Analogy): Maps to the 'quizzes' table representing a generated quiz for a Note.

Why It Is Used & Architectural Role: Parent entity for 'QuizQuestion' and 'QuizAttempt'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.model;
 2 |
 3 | import jakarta.persistence.*;
 4 | import java.time.LocalDateTime;
 5 |
 6 | @Entity
 7 | @Table(name = "quizzes")
 8 | public class Quiz {
 9 |
10 |     @Id
11 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
12 |     private Long id;
13 |
14 |     @ManyToOne(fetch = FetchType.LAZY)
15 |     @JoinColumn(name = "note_id", nullable = false)
16 |     private Note note;
17 |
18 |     @Column(length = 200)
19 |     private String title;
20 |
21 |     @Column(name = "created_at")
22 |     private LocalDateTime createdAt;
23 |
24 |     @PrePersist
25 |     protected void onCreate() {
26 |         this.createdAt = LocalDateTime.now();
27 |     }
28 |
29 |     public Quiz() {}
30 |
31 |     public Quiz(Note note, String title) {
32 |         this.note = note;
33 |         this.title = title;
34 |     }
35 |
36 |     public Long getId() { return id; }
37 |     public void setId(Long id) { this.id = id; }
38 |
39 |     public Note getNote() { return note; }
40 |     public void setNote(Note note) { this.note = note; }
41 |
42 |     public String getTitle() { return title; }
43 |     public void setTitle(String title) { this.title = title; }
44 |
45 |     public LocalDateTime getCreatedAt() { return createdAt; }
46 |     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
47 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 6–26:** '@Entity', '@Table(name = "quizzes")', 'id', lazy '@ManyToOne Note', 'title', 'createdAt', and '@PrePersist'.
- **Lines 28–48:** Constructors and getters/setters.

How to Explain This Module in an Interview: Normalizes quiz metadata separately from individual questions and student attempts.

Module: QuizQuestion Entity (`backend/src/main/java/com/studyassistant/model/QuizQuestion.java`)

What This File Is (Beginner-Friendly Analogy): Maps to the 'quiz_questions' table storing MCQ text, options A–D, and the correct option letter.

Why It Is Used & Architectural Role: Holds the 5 multiple-choice questions belonging to a Quiz.

Complete Source Code (Every Line Numbered):

```
1 | package com.studyassistant.model;
2 |
3 | import jakarta.persistence.*;
4 |
5 | @Entity
6 | @Table(name = "quiz_questions")
```

```
 7 | public class QuizQuestion {
 8 |
 9 |     @Id
10 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
11 |     private Long id;
12 |
13 |     @ManyToOne(fetch = FetchType.LAZY)
14 |     @JoinColumn(name = "quiz_id", nullable = false)
15 |     private Quiz quiz;
16 |
17 |     @Column(name = "question_text", nullable = false, columnDefinition = "TEXT")
18 |     private String questionText;
19 |
20 |     @Column(name = "option_a", nullable = false, length = 300)
21 |     private String optionA;
22 |
23 |     @Column(name = "option_b", nullable = false, length = 300)
24 |     private String optionB;
25 |
26 |     @Column(name = "option_c", nullable = false, length = 300)
27 |     private String optionC;
28 |
29 |     @Column(name = "option_d", nullable = false, length = 300)
30 |     private String optionD;
31 |
32 |     @Column(name = "correct_option", nullable = false, length = 1)
33 |     private String correctOption;
34 |
35 |     public QuizQuestion() {}
36 |
37 |     public QuizQuestion(Quiz quiz, String questionText, String optionA, String optionB,
38 |                         String optionC, String optionD, String correctOption) {
39 |         this.quiz = quiz;
40 |         this.questionText = questionText;
41 |         this.optionA = optionA;
42 |         this.optionB = optionB;
43 |         this.optionC = optionC;
44 |         this.optionD = optionD;
45 |         this.correctOption = correctOption;
46 |     }
47 |
48 |     public Long getId() { return id; }
49 |     public void setId(Long id) { this.id = id; }
50 |
51 |     public Quiz getQuiz() { return quiz; }
52 |     public void setQuiz(Quiz quiz) { this.quiz = quiz; }
53 |
54 |     public String getQuestionText() { return questionText; }
55 |     public void setQuestionText(String questionText) { this.questionText = questionText; }
56 |
57 |     public String getOptionA() { return optionA; }
58 |     public void setOptionA(String optionA) { this.optionA = optionA; }
59 |
60 |     public String getOptionB() { return optionB; }
61 |     public void setOptionB(String optionB) { this.optionB = optionB; }
62 |
63 |     public String getOptionC() { return optionC; }
64 |     public void setOptionC(String optionC) { this.optionC = optionC; }
65 |
66 |     public String getOptionD() { return optionD; }
67 |     public void setOptionD(String optionD) { this.optionD = optionD; }
68 |
69 |     public String getCorrectOption() { return correctOption; }
70 |     public void setCorrectOption(String correctOption) { this.correctOption = correctOption; }
71 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 5–32:** '@Entity', '@Table(name = "quiz_questions")', 'id', lazy '@ManyToOne Quiz', 'questionText' (TEXT), 'optionA'–'optionD' (VARCHAR 300), and 'correctOption' (length 1).
- **Lines 34–71:** Constructors and getters/setters.

> **How to Explain This Module in an Interview:** Contrast 'QuizQuestion' (stores 'correctOption' in DB) with 'QuizQuestionDTO' (omits 'correctOption' over the wire).

Module: QuizAttempt Entity (`backend/src/main/java/com/studyassistant/model/QuizAttempt.java`)

What This File Is (Beginner-Friendly Analogy): Maps to the 'quiz_attempts' table recording each completed quiz

submission.

Why It Is Used & Architectural Role: Tracks student scores over time for progress charts, analytics, and retake reminders.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.model;
  2 |
  3 | import jakarta.persistence.*;
  4 | import java.math.BigDecimal;
  5 | import java.time.LocalDateTime;
  6 |
  7 | @Entity
  8 | @Table(name = "quiz_attempts")
  9 | public class QuizAttempt {
 10 |
 11 |     @Id
 12 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
 13 |     private Long id;
 14 |
 15 |     @ManyToOne(fetch = FetchType.LAZY)
 16 |     @JoinColumn(name = "student_id", nullable = false)
 17 |     private Student student;
 18 |
 19 |     @ManyToOne(fetch = FetchType.LAZY)
 20 |     @JoinColumn(name = "quiz_id", nullable = false)
 21 |     private Quiz quiz;
 22 |
 23 |     @Column(nullable = false)
 24 |     private Integer score;
 25 |
 26 |     @Column(name = "total_questions", nullable = false)
 27 |     private Integer totalQuestions;
 28 |
 29 |     @Column(precision = 5, scale = 2)
 30 |     private BigDecimal percentage;
 31 |
 32 |     @Column(name = "attempted_at")
 33 |     private LocalDateTime attemptedAt;
 34 |
 35 |     @PrePersist
 36 |     protected void onCreate() {
 37 |         this.attemptedAt = LocalDateTime.now();
 38 |     }
 39 |
 40 |     public QuizAttempt() {}
 41 |
 42 |     public QuizAttempt(Student student, Quiz quiz, Integer score, Integer totalQuestions, BigDecimal
percentage) {
 43 |         this.student = student;
 44 |         this.quiz = quiz;
 45 |         this.score = score;
 46 |         this.totalQuestions = totalQuestions;
 47 |         this.percentage = percentage;
 48 |     }
 49 |
 50 |     public Long getId() { return id; }
 51 |     public void setId(Long id) { this.id = id; }
 52 |
 53 |     public Student getStudent() { return student; }
 54 |     public void setStudent(Student student) { this.student = student; }
 55 |
 56 |     public Quiz getQuiz() { return quiz; }
 57 |     public void setQuiz(Quiz quiz) { this.quiz = quiz; }
 58 |
 59 |     public Integer getScore() { return score; }
 60 |     public void setScore(Integer score) { this.score = score; }
 61 |
 62 |     public Integer getTotalQuestions() { return totalQuestions; }
 63 |     public void setTotalQuestions(Integer totalQuestions) { this.totalQuestions = totalQuestions; }
 64 |
 65 |     public BigDecimal getPercentage() { return percentage; }
 66 |     public void setPercentage(BigDecimal percentage) { this.percentage = percentage; }
 67 |
 68 |     public LocalDateTime getAttemptedAt() { return attemptedAt; }
 69 |     public void setAttemptedAt(LocalDateTime attemptedAt) { this.attemptedAt = attemptedAt; }
 70 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 7–29:** '@Entity', '@Table(name = "quiz_attempts")', 'id', lazy '@ManyToOne Student' and 'Quiz', 'score', 'totalQuestions', and 'BigDecimal percentage' (precision 5, scale 2).

- **Lines 31–69:** 'attemptedAt' with '@PrePersist', constructors, and getters/setters.

> **How to Explain This Module in an Interview:** Point out 'BigDecimal' with 'precision = 5, scale = 2' for exact score percentages.

Part IV: Spring Data JPA Repositories (`repository/` — Every Line Explained)

Module: StudentRepository (`backend/src/main/java/com/studyassistant/repository/StudentRepository.java`)

What This File Is (Beginner-Friendly Analogy): JPA repository for 'Student' entities.

Why It Is Used & Architectural Role: Provides email lookup and existence checks for authentication.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.repository;
 2 |
 3 | import com.studyassistant.model.Student;
 4 | import org.springframework.data.jpa.repository.JpaRepository;
 5 | import org.springframework.stereotype.Repository;
 6 |
 7 | import java.util.Optional;
 8 |
 9 | @Repository
10 | public interface StudentRepository extends JpaRepository<Student, Long> {
11 |     Optional<Student> findByEmail(String email);
12 |     boolean existsByEmail(String email);
13 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 9–13:** 'findByEmail(String email)' and 'existsByEmail(String email)'.

> **How to Explain This Module in an Interview:** Derived query methods automatically implemented by Spring Data JPA.

Module: NoteRepository (`backend/src/main/java/com/studyassistant/repository/NoteRepository.java`)

What This File Is (Beginner-Friendly Analogy): JPA repository for 'Note' entities.

Why It Is Used & Architectural Role: Provides chronological note listing and IDOR-safe ownership lookup.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.repository;
 2 |
 3 | import com.studyassistant.model.Note;
 4 | import org.springframework.data.jpa.repository.JpaRepository;
 5 | import org.springframework.stereotype.Repository;
 6 |
 7 | import java.util.List;
 8 | import java.util.Optional;
 9 |
10 | @Repository
11 | public interface NoteRepository extends JpaRepository<Note, Long> {
12 |     List<Note> findByStudentIdOrderByCreatedAtDesc(Long studentId);
13 |     Optional<Note> findByIdAndStudentId(Long id, Long studentId);
14 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 10–13:** 'findByStudentIdOrderByCreatedAtDesc(Long studentId)' and 'findByIdAndStudentId(Long id, Long studentId)'.

> **How to Explain This Module in an Interview:** 'findByIdAndStudentId' enforces ownership at the SQL query level.

Module: FlashcardRepository (`backend/src/main/java/com/studyassistant/repository/FlashcardRepository.java`)

What This File Is (Beginner-Friendly Analogy): JPA repository for 'Flashcard' entities and spaced-repetition queries.

Why It Is Used & Architectural Role: Fetches flashcards by note, counts total flashcards for a student, and queries due cards.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.repository;
 2 |
 3 | import com.studyassistant.model.Flashcard;
 4 | import org.springframework.data.jpa.repository.JpaRepository;
 5 | import org.springframework.stereotype.Repository;
 6 |
 7 | import java.util.List;
 8 | import org.springframework.data.jpa.repository.Query;
 9 | import org.springframework.data.repository.query.Param;
10 |
11 |
12 | @Repository
13 | public interface FlashcardRepository extends JpaRepository<Flashcard, Long> {
14 |     List<Flashcard> findByNoteId(Long noteId);
15 |
16 |     @Query("SELECT COUNT(f) FROM Flashcard f WHERE f.note.student.id = :studentId")
17 |     long countByStudentId(@Param("studentId") Long studentId);
18 |
19 |     @Query("SELECT f FROM Flashcard f WHERE f.note.student.id = :studentId AND f.nextReviewDate <= :now ORDER
BY f.nextReviewDate ASC")
20 |     List<Flashcard> findDueForStudent(@Param("studentId") Long studentId, @Param("now") java.time.LocalDateTime
now);
21 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 12–20:** 'findByNoteId', custom '@Query countByStudentId', and custom '@Query findDueForStudent' filtering by 'f.nextReviewDate <= :now ORDER BY f.nextReviewDate ASC'.

> **How to Explain This Module in an Interview:** Demonstrates custom JPQL queries across entity relationships ('f.note.student.id').

Module: QuizRepository (`backend/src/main/java/com/studyassistant/repository/QuizRepository.java`)

What This File Is (Beginner-Friendly Analogy): JPA repository for 'Quiz' entities.

Why It Is Used & Architectural Role: Finds the quiz associated with a note.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.repository;
 2 |
 3 | import com.studyassistant.model.Quiz;
 4 | import org.springframework.data.jpa.repository.JpaRepository;
 5 | import org.springframework.stereotype.Repository;
 6 |
 7 | import java.util.Optional;
 8 |
 9 | @Repository
10 | public interface QuizRepository extends JpaRepository<Quiz, Long> {
11 |     Optional<Quiz> findByNoteId(Long noteId);
12 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 9–12:** 'Optional findByNoteId(Long noteId)'.

> **How to Explain This Module in an Interview:** Returns an Optional to cleanly handle notes that haven't generated a quiz yet.

Module: QuizQuestionRepository (`backend/src/main/java/com/studyassistant/repository/QuizQuestionRepository.java`)

What This File Is (Beginner-Friendly Analogy): JPA repository for 'QuizQuestion' entities.

Why It Is Used & Architectural Role: Loads all questions for a given quiz ID.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.repository;
 2 |
 3 | import com.studyassistant.model.QuizQuestion;
 4 | import org.springframework.data.jpa.repository.JpaRepository;
 5 | import org.springframework.stereotype.Repository;
 6 |
 7 | import java.util.List;
 8 |
 9 | @Repository
10 | public interface QuizQuestionRepository extends JpaRepository<QuizQuestion, Long> {
11 |     List<QuizQuestion> findByQuizId(Long quizId);
12 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 9–12:** 'List findByQuizId(Long quizId)'.

How to Explain This Module in an Interview: Used by both QuizService (for grading) and NoteService (for history view).

Module: QuizAttemptRepository (`backend/src/main/java/com/studyassistant/repository/QuizAttemptRepository.java`)

What This File Is (Beginner-Friendly Analogy): JPA repository for 'QuizAttempt' entities with N+1 prevention and pagination.

Why It Is Used & Architectural Role: Powers Dashboard, Progress, Quizzes, Achievements, and Analytics.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.repository;
 2 |
 3 | import com.studyassistant.model.QuizAttempt;
 4 | import org.springframework.data.jpa.repository.JpaRepository;
 5 | import org.springframework.data.jpa.repository.Query;
 6 | import org.springframework.data.repository.query.Param;
 7 | import org.springframework.stereotype.Repository;
 8 | import org.springframework.data.domain.Page;
 9 | import org.springframework.data.domain.Pageable;
10 | import org.springframework.data.jpa.repository.EntityGraph;
11 |
12 | import java.util.List;
13 |
14 | @Repository
15 | public interface QuizAttemptRepository extends JpaRepository<QuizAttempt, Long> {
16 |     @EntityGraph(attributePaths = {"quiz", "quiz.note"})
17 |     List<QuizAttempt> findByStudentIdOrderByAttemptedAtDesc(Long studentId);
18 |
19 |     @EntityGraph(attributePaths = {"quiz", "quiz.note"})
20 |     List<QuizAttempt> findByStudentIdOrderByAttemptedAtAsc(Long studentId);
21 |
22 |     long countByStudentId(Long studentId);
23 |
24 |     @Query("SELECT AVG(qa.percentage) FROM QuizAttempt qa WHERE qa.student.id = :studentId")
25 |     Double findAveragePercentageByStudentId(@Param("studentId") Long studentId);
26 |
27 |     @EntityGraph(attributePaths = {"quiz", "quiz.note"})
28 |     @Query("SELECT q FROM QuizAttempt q WHERE q.student.id = :studentId ORDER BY q.attemptedAt DESC")
29 |     Page<QuizAttempt> findAttemptsByStudentIdPaged(@Param("studentId") Long studentId, Pageable pageable);
30 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–29:** Uses '@EntityGraph(attributePaths = {"quiz", "quiz.note"})' on 'findByStudentIdOrderByAttemptedAtDesc', 'findByStudentIdOrderByAttemptedAtAsc', and 'findAttemptsByStudentIdPaged', plus 'countByStudentId' and 'findAveragePercentageByStudentId'.

How to Explain This Module in an Interview: Highlight '@EntityGraph' solving the N+1 problem and 'Page' for server-side pagination.

Part V: Exception Handling & DTOs (`exception/` & `dto/` — Every Line Explained)

Module: ForbiddenException (`backend/src/main/java/com/studyassistant/exception/ForbiddenException.java`)

What This File Is (Beginner-Friendly Analogy): Custom runtime exception thrown when an authenticated student tries to access or modify another student's resource.

Why It Is Used & Architectural Role: Mapped to HTTP 403 Forbidden by GlobalExceptionHandler.

Complete Source Code (Every Line Numbered):

```
1 | package com.studyassistant.exception;
2 |
3 | public class ForbiddenException extends RuntimeException {
4 |     public ForbiddenException(String message) {
5 |         super(message);
6 |     }
7 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–7:** Extends 'RuntimeException' and passes 'message' to 'super(message)'.

How to Explain This Module in an Interview: Distinguishes 403 Forbidden (authenticated but not owner) from 401 Unauthorized (not logged in).

Module: ResourceNotFoundException (`backend/src/main/java/com/studyassistant/exception/ResourceNotFoundException.java`)

What This File Is (Beginner-Friendly Analogy): Custom runtime exception thrown when a requested Note, Quiz, Flashcard, or Student ID does not exist.

Why It Is Used & Architectural Role: Mapped to HTTP 404 Not Found by GlobalExceptionHandler.

Complete Source Code (Every Line Numbered):

```
1 | package com.studyassistant.exception;
2 |
3 | public class ResourceNotFoundException extends RuntimeException {
4 |     public ResourceNotFoundException(String message) {
5 |         super(message);
6 |     }
7 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–7:** Extends 'RuntimeException' and passes 'message' to 'super(message)'.

How to Explain This Module in an Interview: Clean domain exception for 404 responses.

Module: UnauthorizedException (`backend/src/main/java/com/studyassistant/exception/UnauthorizedException.java`)

What This File Is (Beginner-Friendly Analogy): Custom runtime exception thrown when login credentials (email or password) are invalid.

Why It Is Used & Architectural Role: Mapped to HTTP 401 Unauthorized by GlobalExceptionHandler.

Complete Source Code (Every Line Numbered):

```
1 | package com.studyassistant.exception;
2 |
3 | public class UnauthorizedException extends RuntimeException {
4 |     public UnauthorizedException(String message) {
5 |         super(message);
6 |     }
7 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–7:** Extends 'RuntimeException' and passes 'message' to 'super(message)'.

> **How to Explain This Module in an Interview:** Used by AuthService.login() on failed credential checks.

Module: GlobalExceptionHandler (`backend/src/main/java/com/studyassistant/exception/GlobalExceptionHandler.java`)

What This File Is (Beginner-Friendly Analogy): Centralized '@ControllerAdvice' interceptor that translates Java exceptions into structured JSON HTTP error responses.

Why It Is Used & Architectural Role: Guarantees consistent '{ status, message, timestamp }' error payloads across the entire API.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.exception;
 2 |
 3 | import org.slf4j.Logger;
 4 | import org.slf4j.LoggerFactory;
 5 | import org.springframework.dao.DataIntegrityViolationException;
 6 | import org.springframework.http.HttpStatus;
 7 | import org.springframework.http.ResponseEntity;
 8 | import org.springframework.http.converter.HttpMessageNotReadableException;
 9 | import org.springframework.validation.FieldError;
10 | import org.springframework.web.bind.MethodArgumentNotValidException;
11 | import org.springframework.web.bind.annotation.ControllerAdvice;
12 | import org.springframework.web.bind.annotation.ExceptionHandler;
13 | import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;
14 | import org.springframework.web.multipart.MaxUploadSizeExceededException;
15 |
16 | import java.time.LocalDateTime;
17 | import java.util.HashMap;
18 | import java.util.Map;
19 | import java.util.stream.Collectors;
20 |
21 | @ControllerAdvice
22 | public class GlobalExceptionHandler {
23 |
24 |     private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
25 |
26 |     @ExceptionHandler(ResourceNotFoundException.class)
27 |     public ResponseEntity<Map<String, Object>> handleResourceNotFound(ResourceNotFoundException ex) {
28 |         return buildErrorResponse(HttpStatus.NOT_FOUND, ex.getMessage());
29 |     }
30 |
31 |     @ExceptionHandler(UnauthorizedException.class)
32 |     public ResponseEntity<Map<String, Object>> handleUnauthorized(UnauthorizedException ex) {
33 |         return buildErrorResponse(HttpStatus.UNAUTHORIZED, ex.getMessage());
34 |     }
35 |
36 |     @ExceptionHandler(ForbiddenException.class)
37 |     public ResponseEntity<Map<String, Object>> handleForbidden(ForbiddenException ex) {
38 |         return buildErrorResponse(HttpStatus.FORBIDDEN, ex.getMessage());
39 |     }
40 |
41 |     @ExceptionHandler(IllegalArgumentException.class)
42 |     public ResponseEntity<Map<String, Object>> handleIllegalArgument(IllegalArgumentException ex) {
43 |         return buildErrorResponse(HttpStatus.BAD_REQUEST, ex.getMessage());
44 |     }
45 |
46 |     @ExceptionHandler(MethodArgumentNotValidException.class)
47 |     public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex) {
48 |         String message = ex.getBindingResult().getFieldErrors().stream()
49 |                 .map(FieldError::getDefaultMessage)
50 |                 .collect(Collectors.joining(", "));
51 |         return buildErrorResponse(HttpStatus.BAD_REQUEST, message);
52 |     }
53 |
54 |     @ExceptionHandler(HttpMessageNotReadableException.class)
55 |     public ResponseEntity<Map<String, Object>> handleUnreadable(HttpMessageNotReadableException ex) {
56 |         return buildErrorResponse(HttpStatus.BAD_REQUEST, "Malformed JSON request body");
57 |     }
58 |
59 |     @ExceptionHandler(MethodArgumentTypeMismatchException.class)
60 |     public ResponseEntity<Map<String, Object>> handleTypeMismatch(MethodArgumentTypeMismatchException ex) {
61 |         return buildErrorResponse(HttpStatus.BAD_REQUEST, "Invalid value for parameter: " + ex.getName());
62 |     }
63 |
```

```
64 |     @ExceptionHandler(DataIntegrityViolationException.class)
65 |     public ResponseEntity<Map<String, Object>> handleDataIntegrity(DataIntegrityViolationException ex) {
66 |         return buildErrorResponse(HttpStatus.CONFLICT, "Request conflicts with existing data");
67 |     }
68 |
69 |     @ExceptionHandler(MaxUploadSizeExceededException.class)
70 |     public ResponseEntity<Map<String, Object>> handleMaxUploadSize(MaxUploadSizeExceededException ex) {
71 |         return buildErrorResponse(HttpStatus.BAD_REQUEST, "File size exceeds maximum limit of 10MB");
72 |     }
73 |
74 |     @ExceptionHandler(Exception.class)
75 |     public ResponseEntity<Map<String, Object>> handleGeneral(Exception ex) {
76 |         log.error("Unhandled exception", ex);
77 |         return buildErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR, "An unexpected error occurred");
78 |     }
79 |
80 |     private ResponseEntity<Map<String, Object>> buildErrorResponse(HttpStatus status, String message) {
81 |         Map<String, Object> body = new HashMap<>();
82 |         body.put("status", status.value());
83 |         body.put("message", message);
84 |         body.put("timestamp", LocalDateTime.now().toString());
85 |         return new ResponseEntity<>(body, status);
86 |     }
87 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 20–84:** Handles ResourceNotFoundException (404), UnauthorizedException (401), ForbiddenException (403), IllegalArgumentException (400), MethodArgumentNotValidException (400), HttpMessageNotReadableException (400), MethodArgumentTypeMismatchException (400), DataIntegrityViolationException (409), MaxUploadSizeExceededException (400), and generic Exception (500).

> **How to Explain This Module in an Interview:** Centralizes error handling using Spring AOP '@ControllerAdvice'.

Module: AnalyticsResponse (`backend/src/main/java/com/studyassistant/dto/AnalyticsResponse.java`)

What This File Is (Beginner-Friendly Analogy): Java 17 record hierarchy representing the '/api/analytics' response.

Why It Is Used & Architectural Role: Packages OverallStats, TopicPerformance list, and ScoreTrendPoint list.

Complete Source Code (Every Line Numbered):

```
1  | package com.studyassistant.dto;
2  |
3  | import java.math.BigDecimal;
4  | import java.util.List;
5  |
6  | public record AnalyticsResponse(
7  |     OverallStats overall,
8  |     List<TopicPerformance> topics,
9  |     List<ScoreTrendPoint> trend
10 | ) {
11 |     public record OverallStats(
12 |         long totalAttempts,
13 |         BigDecimal averageScore,
14 |         double bestScore,
15 |         double worstScore,
16 |         long topicsAttempted,
17 |         long topicsPassed
18 |     ) {}
19 |
20 |     public record TopicPerformance(
21 |         Long noteId,
22 |         String noteTitle,
23 |         double averageScore,
24 |         double bestScore,
25 |         double worstScore,
26 |         long attempts,
27 |         long passed,
28 |         String recommendation
29 |     ) {}
30 |
31 |     public record ScoreTrendPoint(
32 |         String date,
33 |         double percentage
34 |     ) {}
35 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–36:** Defines 'AnalyticsResponse' record and nested records 'OverallStats', 'TopicPerformance', and 'ScoreTrendPoint'.

> **How to Explain This Module in an Interview:** Zero-boilerplate immutable Java 17 records.

Module: AuthResponse (`backend/src/main/java/com/studyassistant/dto/AuthResponse.java`)

What This File Is (Beginner-Friendly Analogy): Response DTO returned after successful login or registration.

Why It Is Used & Architectural Role: Delivers the signed JWT 'token', 'studentId', and 'name' to the React client.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | public class AuthResponse {
 4 |
 5 |     private String token;
 6 |     private Long studentId;
 7 |     private String name;
 8 |
 9 |     public AuthResponse() {}
10 |
11 |     public AuthResponse(String token, Long studentId, String name) {
12 |         this.token = token;
13 |         this.studentId = studentId;
14 |         this.name = name;
15 |     }
16 |
17 |     public String getToken() { return token; }
18 |     public void setToken(String token) { this.token = token; }
19 |
20 |     public Long getStudentId() { return studentId; }
21 |     public void setStudentId(Long studentId) { this.studentId = studentId; }
22 |
23 |     public String getName() { return name; }
24 |     public void setName(String name) { this.name = name; }
25 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–27:** Fields 'token', 'studentId', 'name', constructors, and getters/setters.

> **How to Explain This Module in an Interview:** Hydrates AuthContext on the frontend.

Module: DashboardResponse (`backend/src/main/java/com/studyassistant/dto/DashboardResponse.java`)

What This File Is (Beginner-Friendly Analogy): Java 17 record returned by 'GET /api/dashboard'.

Why It Is Used & Architectural Role: Bundles 'totalQuizzes', 'averageScore', 'PaginatedScoreHistory', and 'totalFlashcards'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | /**
 4 |  * DTO returned by the Dashboard endpoint. Contains aggregated statistics for a student.
 5 |  */
 6 | public record DashboardResponse(
 7 |         long totalQuizzes,
 8 |         java.math.BigDecimal averageScore,
 9 |         PaginatedScoreHistory scoreHistory,
10 |         long totalFlashcards) {}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–10:** Compact Java 17 record declaration.

> **How to Explain This Module in an Interview:** Single-request payload for the Dashboard KPIs and chart.

Module: FlashcardDTO (`backend/src/main/java/com/studyassistant/dto/FlashcardDTO.java`)

What This File Is (Beginner-Friendly Analogy): DTO representing a flashcard and its Leitner spaced-repetition metadata.

Why It Is Used & Architectural Role: Transfers 'id', 'question', 'answer', 'boxLevel', and 'nextReviewDate' to the frontend.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.dto;
  2 |
  3 | import java.time.LocalDateTime;
  4 |
  5 | public class FlashcardDTO {
  6 |
  7 |     private Long id;
  8 |     private String question;
  9 |     private String answer;
 10 |     private Integer boxLevel;
 11 |     private LocalDateTime nextReviewDate;
 12 |
 13 |     public FlashcardDTO() {}
 14 |
 15 |     public FlashcardDTO(Long id, String question, String answer, Integer boxLevel, LocalDateTime
nextReviewDate) {
 16 |         this.id = id;
 17 |         this.question = question;
 18 |         this.answer = answer;
 19 |         this.boxLevel = boxLevel;
 20 |         this.nextReviewDate = nextReviewDate;
 21 |     }
 22 |
 23 |     public Long getId() { return id; }
 24 |     public void setId(Long id) { this.id = id; }
 25 |
 26 |     public String getQuestion() { return question; }
 27 |     public void setQuestion(String question) { this.question = question; }
 28 |
 29 |     public String getAnswer() { return answer; }
 30 |     public void setAnswer(String answer) { this.answer = answer; }
 31 |
 32 |     public Integer getBoxLevel() { return boxLevel; }
 33 |     public void setBoxLevel(Integer boxLevel) { this.boxLevel = boxLevel; }
 34 |
 35 |     public LocalDateTime getNextReviewDate() { return nextReviewDate; }
 36 |     public void setNextReviewDate(LocalDateTime nextReviewDate) { this.nextReviewDate = nextReviewDate; }
 37 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–40:** Fields, 5-arg constructor, and getters/setters.

> **How to Explain This Module in an Interview:** Decouples the JPA Flashcard entity from API serialization.

Module: FlashcardReviewRequest (`backend/src/main/java/com/studyassistant/dto/FlashcardReviewRequest.java`)

What This File Is (Beginner-Friendly Analogy): Request DTO for 'POST /api/flashcards/review'.

Why It Is Used & Architectural Role: Validates that 'flashcardId' and 'correct' boolean are non-null.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.dto;
  2 |
  3 | import jakarta.validation.constraints.NotNull;
  4 |
  5 | public class FlashcardReviewRequest {
  6 |
  7 |     @NotNull(message = "flashcardId is required")
  8 |     private Long flashcardId;
  9 |
 10 |     @NotNull(message = "correct is required")
 11 |     private Boolean correct;
 12 |
 13 |     public FlashcardReviewRequest() {}
 14 |
 15 |     public FlashcardReviewRequest(Long flashcardId, Boolean correct) {
 16 |         this.flashcardId = flashcardId;
```

```
 17 |         this.correct = correct;
 18 |     }
 19 |
 20 |     public Long getFlashcardId() { return flashcardId; }
 21 |     public void setFlashcardId(Long flashcardId) { this.flashcardId = flashcardId; }
 22 |
 23 |     public Boolean getCorrect() { return correct; }
 24 |     public void setCorrect(Boolean correct) { this.correct = correct; }
 25 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–30:** '@NotNull' annotations on 'flashcardId' and 'correct', constructors, and getters/setters.

How to Explain This Module in an Interview: Uses boxed 'Boolean' with '@NotNull' so omitted fields fail validation instead of silently defaulting to 'false'.

Module: GenerateResponse (`backend/src/main/java/com/studyassistant/dto/GenerateResponse.java`)

What This File Is (Beginner-Friendly Analogy): Response DTO returned by 'POST /api/study/generate'.

Why It Is Used & Architectural Role: Contains the generated 'summary', 'List', 'quizId', and 'List'.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.dto;
  2 |
  3 | import java.util.List;
  4 |
  5 | public class GenerateResponse {
  6 |
  7 |     private String summary;
  8 |     private List<FlashcardDTO> flashcards;
  9 |     private Long quizId;
 10 |     private List<QuizQuestionDTO> questions;
 11 |
 12 |     public GenerateResponse() {}
 13 |
 14 |     public GenerateResponse(String summary, List<FlashcardDTO> flashcards, Long quizId, List<QuizQuestionDTO>
questions) {
 15 |         this.summary = summary;
 16 |         this.flashcards = flashcards;
 17 |         this.quizId = quizId;
 18 |         this.questions = questions;
 19 |     }
 20 |
 21 |     public String getSummary() { return summary; }
 22 |     public void setSummary(String summary) { this.summary = summary; }
 23 |
 24 |     public List<FlashcardDTO> getFlashcards() { return flashcards; }
 25 |     public void setFlashcards(List<FlashcardDTO> flashcards) { this.flashcards = flashcards; }
 26 |
 27 |     public Long getQuizId() { return quizId; }
 28 |     public void setQuizId(Long quizId) { this.quizId = quizId; }
 29 |
 30 |     public List<QuizQuestionDTO> getQuestions() { return questions; }
 31 |     public void setQuestions(List<QuizQuestionDTO> questions) { this.questions = questions; }
 32 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–38:** Fields, constructors, and getters/setters.

How to Explain This Module in an Interview: Allows StudyPage.jsx to render the summary, flashcards, and quiz start button immediately after generation.

Module: LoginRequest (`backend/src/main/java/com/studyassistant/dto/LoginRequest.java`)

What This File Is (Beginner-Friendly Analogy): Request DTO for 'POST /api/auth/login'.

Why It Is Used & Architectural Role: Validates non-blank email format and non-blank password.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | import jakarta.validation.constraints.Email;
 4 | import jakarta.validation.constraints.NotBlank;
 5 |
 6 | public class LoginRequest {
 7 |
 8 |     @NotBlank(message = "Email is required")
 9 |     @Email(message = "Invalid email format")
10 |     private String email;
11 |
12 |     @NotBlank(message = "Password is required")
13 |     private String password;
14 |
15 |     public LoginRequest() {}
16 |
17 |     public LoginRequest(String email, String password) {
18 |         this.email = email;
19 |         this.password = password;
20 |     }
21 |
22 |     public String getEmail() { return email; }
23 |     public void setEmail(String email) { this.email = email; }
24 |
25 |     public String getPassword() { return password; }
26 |     public void setPassword(String password) { this.password = password; }
27 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–31:** '@NotBlank' and '@Email' on 'email', '@NotBlank' on 'password'.

How to Explain This Module in an Interview: Rejects malformed login payloads at the controller boundary.

Module: NoteRequest (`backend/src/main/java/com/studyassistant/dto/NoteRequest.java`)

What This File Is (Beginner-Friendly Analogy): Request DTO for 'POST /api/notes/text'.

Why It Is Used & Architectural Role: Validates note title (max 200 chars) and content (max 100,000 chars).

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | import jakarta.validation.constraints.NotBlank;
 4 | import jakarta.validation.constraints.Size;
 5 |
 6 | public class NoteRequest {
 7 |
 8 |     @NotBlank(message = "Title is required")
 9 |     @Size(max = 200, message = "Title must be 200 characters or less")
10 |     private String title;
11 |
12 |     @NotBlank(message = "Content is required")
13 |     @Size(max = 100000, message = "Content must be 100,000 characters or less")
14 |     private String content;
15 |
16 |     public NoteRequest() {}
17 |
18 |     public NoteRequest(String title, String content) {
19 |         this.title = title;
20 |         this.content = content;
21 |     }
22 |
23 |     public String getTitle() { return title; }
24 |     public void setTitle(String title) { this.title = title; }
25 |
26 |     public String getContent() { return content; }
27 |     public void setContent(String content) { this.content = content; }
28 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–32:** '@NotBlank' and '@Size(max = 200)' on 'title'; '@NotBlank' and '@Size(max = 100000)' on 'content'.

How to Explain This Module in an Interview: Protects the database and AI service from empty or excessively huge text payloads.

Module: PaginatedScoreHistory (`backend/src/main/java/com/studyassistant/dto/PaginatedScoreHistory.java`)

What This File Is (Beginner-Friendly Analogy): Java 17 record wrapping a page of 'ScorePoint' entries.

Why It Is Used & Architectural Role: Mirrors Spring Data's 'Page' metadata ('content', 'pageNumber', 'pageSize', 'totalElements', 'totalPages', 'last').

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | import java.util.List;
 4 |
 5 | /**
 6 |  * Wrapper for a page of ScorePoint results. Mirrors Spring's Page metadata.
 7 |  */
 8 | public record PaginatedScoreHistory(
 9 |         List<ScorePoint> content,
10 |         int pageNumber,
11 |         int pageSize,
12 |         long totalElements,
13 |         int totalPages,
14 |         boolean last) {}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–15:** Record declaration with pagination metadata fields.

> **How to Explain This Module in an Interview:** Cleanly serializes pagination state without exposing internal Spring PageImpl classes.

Module: ProgressDTO (`backend/src/main/java/com/studyassistant/dto/ProgressDTO.java`)

What This File Is (Beginner-Friendly Analogy): DTO returned by 'GET /api/progress'.

Why It Is Used & Architectural Role: Carries 'noteTitle', 'score', 'totalQuestions', 'percentage', and 'attemptedAt' for each quiz attempt.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | import java.math.BigDecimal;
 4 | import java.time.LocalDateTime;
 5 |
 6 | public class ProgressDTO {
 7 |
 8 |     private String noteTitle;
 9 |     private int score;
10 |     private int totalQuestions;
11 |     private BigDecimal percentage;
12 |     private LocalDateTime attemptedAt;
13 |
14 |     public ProgressDTO() {}
15 |
16 |     public ProgressDTO(String noteTitle, int score, int totalQuestions, BigDecimal percentage, LocalDateTime
attemptedAt) {
17 |         this.noteTitle = noteTitle;
18 |         this.score = score;
19 |         this.totalQuestions = totalQuestions;
20 |         this.percentage = percentage;
21 |         this.attemptedAt = attemptedAt;
22 |     }
23 |
24 |     public String getNoteTitle() { return noteTitle; }
25 |     public void setNoteTitle(String noteTitle) { this.noteTitle = noteTitle; }
26 |
27 |     public int getScore() { return score; }
28 |     public void setScore(int score) { this.score = score; }
29 |
30 |     public int getTotalQuestions() { return totalQuestions; }
31 |     public void setTotalQuestions(int totalQuestions) { this.totalQuestions = totalQuestions; }
32 |
33 |     public BigDecimal getPercentage() { return percentage; }
34 |     public void setPercentage(BigDecimal percentage) { this.percentage = percentage; }
```

```
35 |
36 |     public LocalDateTime getAttemptedAt() { return attemptedAt; }
37 |     public void setAttemptedAt(LocalDateTime attemptedAt) { this.attemptedAt = attemptedAt; }
38 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–44:** Fields, constructor, and getters/setters.

> **How to Explain This Module in an Interview:** Powers the multi-topic line chart and summary table on ProgressPage.jsx.

Module: QuizAttemptRequest (`backend/src/main/java/com/studyassistant/dto/QuizAttemptRequest.java`)

What This File Is (Beginner-Friendly Analogy): Request DTO for 'POST /api/quiz/attempt'.

Why It Is Used & Architectural Role: Validates 'quizId' and each nested 'AnswerDTO' ('questionId' + 'selectedOption' matching A–D).

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | import jakarta.validation.Valid;
 4 | import jakarta.validation.constraints.NotBlank;
 5 | import jakarta.validation.constraints.NotEmpty;
 6 | import jakarta.validation.constraints.NotNull;
 7 | import jakarta.validation.constraints.Pattern;
 8 |
 9 | import java.util.List;
10 |
11 | public class QuizAttemptRequest {
12 |
13 |     @NotNull(message = "quizId is required")
14 |     private Long quizId;
15 |
16 |     @NotEmpty(message = "answers cannot be empty")
17 |     @Valid
18 |     private List<AnswerDTO> answers;
19 |
20 |     public QuizAttemptRequest() {}
21 |
22 |     public QuizAttemptRequest(Long quizId, List<AnswerDTO> answers) {
23 |         this.quizId = quizId;
24 |         this.answers = answers;
25 |     }
26 |
27 |     public Long getQuizId() { return quizId; }
28 |     public void setQuizId(Long quizId) { this.quizId = quizId; }
29 |
30 |     public List<AnswerDTO> getAnswers() { return answers; }
31 |     public void setAnswers(List<AnswerDTO> answers) { this.answers = answers; }
32 |
33 |     public static class AnswerDTO {
34 |         @NotNull(message = "questionId is required")
35 |         private Long questionId;
36 |
37 |         @NotBlank(message = "selectedOption is required")
38 |         @Pattern(regexp = "^[A-D]$", message = "selectedOption must be A, B, C, or D")
39 |         private String selectedOption;
40 |
41 |         public AnswerDTO() {}
42 |
43 |         public AnswerDTO(Long questionId, String selectedOption) {
44 |             this.questionId = questionId;
45 |             this.selectedOption = selectedOption;
46 |         }
47 |
48 |         public Long getQuestionId() { return questionId; }
49 |         public void setQuestionId(Long questionId) { this.questionId = questionId; }
50 |
51 |         public String getSelectedOption() { return selectedOption; }
52 |         public void setSelectedOption(String selectedOption) { this.selectedOption = selectedOption; }
53 |     }
54 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–55:** '@NotNull quizId', '@NotEmpty @Valid List answers', and inner class 'AnswerDTO' with '@Pattern(regexp =

"^[A-D]$")'.

How to Explain This Module in an Interview: Demonstrates nested bean validation with '@Valid' and regex '@Pattern'.

Module: QuizAttemptResponse (`backend/src/main/java/com/studyassistant/dto/QuizAttemptResponse.java`)

What This File Is (Beginner-Friendly Analogy): Response DTO returned after grading a quiz attempt.

Why It Is Used & Architectural Role: Contains 'score', 'totalQuestions', 'percentage', 'passed' boolean, and 'List'.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.dto;
  2 |
  3 | import java.util.List;
  4 |
  5 | public class QuizAttemptResponse {
  6 |
  7 |     private int score;
  8 |     private int totalQuestions;
  9 |     private double percentage;
 10 |     private boolean passed;
 11 |     private List<QuestionBreakdown> breakdown;
 12 |
 13 |     public QuizAttemptResponse() {}
 14 |
 15 |     public QuizAttemptResponse(int score, int totalQuestions, double percentage, boolean passed,
 16 |                                List<QuestionBreakdown> breakdown) {
 17 |         this.score = score;
 18 |         this.totalQuestions = totalQuestions;
 19 |         this.percentage = percentage;
 20 |         this.passed = passed;
 21 |         this.breakdown = breakdown;
 22 |     }
 23 |
 24 |     public int getScore() { return score; }
 25 |     public void setScore(int score) { this.score = score; }
 26 |
 27 |     public int getTotalQuestions() { return totalQuestions; }
 28 |     public void setTotalQuestions(int totalQuestions) { this.totalQuestions = totalQuestions; }
 29 |
 30 |     public double getPercentage() { return percentage; }
 31 |     public void setPercentage(double percentage) { this.percentage = percentage; }
 32 |
 33 |     public boolean isPassed() { return passed; }
 34 |     public void setPassed(boolean passed) { this.passed = passed; }
 35 |
 36 |     public List<QuestionBreakdown> getBreakdown() { return breakdown; }
 37 |     public void setBreakdown(List<QuestionBreakdown> breakdown) { this.breakdown = breakdown; }
 38 |
 39 |     public static class QuestionBreakdown {
 40 |         private Long questionId;
 41 |         private String selectedOption;
 42 |         private String correctOption;
 43 |         private boolean isCorrect;
 44 |
 45 |         public QuestionBreakdown() {}
 46 |
 47 |         public QuestionBreakdown(Long questionId, String selectedOption, String correctOption, boolean
isCorrect) {
 48 |             this.questionId = questionId;
 49 |             this.selectedOption = selectedOption;
 50 |             this.correctOption = correctOption;
 51 |             this.isCorrect = isCorrect;
 52 |         }
 53 |
 54 |         public Long getQuestionId() { return questionId; }
 55 |         public void setQuestionId(Long questionId) { this.questionId = questionId; }
 56 |
 57 |         public String getSelectedOption() { return selectedOption; }
 58 |         public void setSelectedOption(String selectedOption) { this.selectedOption = selectedOption; }
 59 |
 60 |         public String getCorrectOption() { return correctOption; }
 61 |         public void setCorrectOption(String correctOption) { this.correctOption = correctOption; }
 62 |
 63 |         public boolean isCorrect() { return isCorrect; }
 64 |         public void setCorrect(boolean correct) { isCorrect = correct; }
 65 |     }
 66 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–75:** Main response fields plus inner class 'QuestionBreakdown' ('questionId', 'selectedOption', 'correctOption', 'isCorrect').

> **How to Explain This Module in an Interview:** Provides the complete post-submission breakdown rendered on ResultPage.jsx.

Module: QuizQuestionDTO (`backend/src/main/java/com/studyassistant/dto/QuizQuestionDTO.java`)

What This File Is (Beginner-Friendly Analogy): Client-safe DTO for a multiple-choice question.

Why It Is Used & Architectural Role: Exposes 'id', 'questionText', and 'optionA'–'optionD' while omitting 'correctOption'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.dto;
 2 |
 3 | public class QuizQuestionDTO {
 4 |
 5 |     private Long id;
 6 |     private String questionText;
 7 |     private String optionA;
 8 |     private String optionB;
 9 |     private String optionC;
10 |     private String optionD;
11 |
12 |     public QuizQuestionDTO() {}
13 |
14 |     public QuizQuestionDTO(Long id, String questionText, String optionA, String optionB,
15 |                            String optionC, String optionD) {
16 |         this.id = id;
17 |         this.questionText = questionText;
18 |         this.optionA = optionA;
19 |         this.optionB = optionB;
20 |         this.optionC = optionC;
21 |         this.optionD = optionD;
22 |     }
23 |
24 |     public Long getId() { return id; }
25 |     public void setId(Long id) { this.id = id; }
26 |
27 |     public String getQuestionText() { return questionText; }
28 |     public void setQuestionText(String questionText) { this.questionText = questionText; }
29 |
30 |     public String getOptionA() { return optionA; }
31 |     public void setOptionA(String optionA) { this.optionA = optionA; }
32 |
33 |     public String getOptionB() { return optionB; }
34 |     public void setOptionB(String optionB) { this.optionB = optionB; }
35 |
36 |     public String getOptionC() { return optionC; }
37 |     public void setOptionC(String optionC) { this.optionC = optionC; }
38 |
39 |     public String getOptionD() { return optionD; }
40 |     public void setOptionD(String optionD) { this.optionD = optionD; }
41 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–42:** Fields, constructor, and getters/setters without 'correctOption'.

> **How to Explain This Module in an Interview:** Prevents users from inspecting network responses to find quiz answers.

Module: RegisterRequest (`backend/src/main/java/com/studyassistant/dto/RegisterRequest.java`)

What This File Is (Beginner-Friendly Analogy): Request DTO for 'POST /api/auth/register'.

Why It Is Used & Architectural Role: Validates 'name' ('@NotBlank'), 'email' ('@NotBlank @Email'), and 'password' ('@NotBlank @Size(min = 6)').

Complete Source Code (Every Line Numbered):

```
1  | package com.studyassistant.dto;
2  |
3  | import jakarta.validation.constraints.Email;
4  | import jakarta.validation.constraints.NotBlank;
5  | import jakarta.validation.constraints.Size;
6  |
7  | public class RegisterRequest {
8  |
9  |     @NotBlank(message = "Name is required")
10 |     private String name;
11 |
12 |     @NotBlank(message = "Email is required")
13 |     @Email(message = "Invalid email format")
14 |     private String email;
15 |
16 |     @NotBlank(message = "Password is required")
17 |     @Size(min = 6, message = "Password must be at least 6 characters")
18 |     private String password;
19 |
20 |     public RegisterRequest() {}
21 |
22 |     public RegisterRequest(String name, String email, String password) {
23 |         this.name = name;
24 |         this.email = email;
25 |         this.password = password;
26 |     }
27 |
28 |     public String getName() { return name; }
29 |     public void setName(String name) { this.name = name; }
30 |
31 |     public String getEmail() { return email; }
32 |     public void setEmail(String email) { this.email = email; }
33 |
34 |     public String getPassword() { return password; }
35 |     public void setPassword(String password) { this.password = password; }
36 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–39:** Validation annotations, constructors, and getters/setters.

> **How to Explain This Module in an Interview:** Enforces minimum 6-character passwords and valid email syntax on the server.

Module: ScorePoint (`backend/src/main/java/com/studyassistant/dto/ScorePoint.java`)

What This File Is (Beginner-Friendly Analogy): Java 17 record representing a single '(LocalDate date, BigDecimal percentage)' point on the Dashboard chart.

Why It Is Used & Architectural Role: Used inside 'PaginatedScoreHistory'.

Complete Source Code (Every Line Numbered):

```
1 | package com.studyassistant.dto;
2 |
3 | import java.math.BigDecimal;
4 | import java.time.LocalDate;
5 |
6 | /**
7 |  * Simple immutable record representing a single score data point.
8 |  */
9 | public record ScorePoint(LocalDate date, BigDecimal percentage) {}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–9:** Record declaration 'public record ScorePoint(LocalDate date, BigDecimal percentage) {}'.

> **How to Explain This Module in an Interview:** Concise immutable value object.

Part VI: Backend Services & Controllers (`service/` & `controller/` — Every Line Explained)

Module: AnalyticsService (`backend/src/main/java/com/studyassistant/service/AnalyticsService.java`)

What This File Is (Beginner-Friendly Analogy): Computes cumulative score trends, topic performance breakdowns, and study recommendations.

Why It Is Used & Architectural Role: Backs 'GET /api/analytics'.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.service;
  2 |
  3 | import com.studyassistant.dto.AnalyticsResponse;
  4 | import com.studyassistant.model.QuizAttempt;
  5 | import com.studyassistant.model.Student;
  6 | import com.studyassistant.repository.QuizAttemptRepository;
  7 | import com.studyassistant.repository.StudentRepository;
  8 | import com.studyassistant.exception.ResourceNotFoundException;
  9 | import org.springframework.stereotype.Service;
 10 |
 11 | import java.math.BigDecimal;
 12 | import java.math.RoundingMode;
 13 | import java.time.LocalDate;
 14 | import java.time.format.DateTimeFormatter;
 15 | import java.util.*;
 16 |
 17 | @Service
 18 | public class AnalyticsService {
 19 |
 20 |     private static final double PASS_THRESHOLD = 70.0;
 21 |     private static final DateTimeFormatter DATE_FMT = DateTimeFormatter.ofPattern("yyyy-MM-dd");
 22 |
 23 |     private final QuizAttemptRepository quizAttemptRepository;
 24 |     private final StudentRepository studentRepository;
 25 |
 26 |     public AnalyticsService(QuizAttemptRepository quizAttemptRepository, StudentRepository studentRepository) {
 27 |         this.quizAttemptRepository = quizAttemptRepository;
 28 |         this.studentRepository = studentRepository;
 29 |     }
 30 |
 31 |     public AnalyticsResponse getAnalytics(String email) {
 32 |         Student student = studentRepository.findByEmail(email)
 33 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
 34 |
 35 |         List<QuizAttempt> attempts =
quizAttemptRepository.findByStudentIdOrderByAttemptedAtAsc(student.getId());
 36 |
 37 |         Map<Long, List<QuizAttempt>> byTopic = new LinkedHashMap<>();
 38 |         List<AnalyticsResponse.ScoreTrendPoint> trend = new ArrayList<>();
 39 |         double runningSum = 0;
 40 |         int runningCount = 0;
 41 |
 42 |         for (QuizAttempt attempt : attempts) {
 43 |             byTopic.computeIfAbsent(attempt.getQuiz().getNote().getId(), k -> new ArrayList<>())
 44 |                     .add(attempt);
 45 |
 46 |             double pct = attempt.getPercentage() != null ? attempt.getPercentage().doubleValue() : 0;
 47 |             runningSum += pct;
 48 |             runningCount++;
 49 |             String date = attempt.getAttemptedAt() != null
 50 |                     ? attempt.getAttemptedAt().toLocalDate().format(DATE_FMT)
 51 |                     : LocalDate.now().format(DATE_FMT);
 52 |             trend.add(new AnalyticsResponse.ScoreTrendPoint(date, round(runningSum / runningCount)));
 53 |         }
 54 |
 55 |         List<AnalyticsResponse.TopicPerformance> topics = new ArrayList<>();
 56 |         long passedTopics = 0;
 57 |         for (Map.Entry<Long, List<QuizAttempt>> entry : byTopic.entrySet()) {
 58 |             List<QuizAttempt> topicAttempts = entry.getValue();
 59 |             double best = topicAttempts.stream().mapToDouble(a ->
a.getPercentage().doubleValue()).max().orElse(0);
 60 |             double worst = topicAttempts.stream().mapToDouble(a ->
a.getPercentage().doubleValue()).min().orElse(0);
 61 |             double avg = topicAttempts.stream().mapToDouble(a ->
a.getPercentage().doubleValue()).average().orElse(0);
 62 |             long passed = topicAttempts.stream().filter(a -> a.getPercentage().doubleValue() >=
PASS_THRESHOLD).count();
 63 |
 64 |             if (avg >= PASS_THRESHOLD) passedTopics++;
 65 |
 66 |             topics.add(new AnalyticsResponse.TopicPerformance(
 67 |                     entry.getKey(),
```

```
 68 |                     entry.getValue().get(0).getQuiz().getNote().getTitle(),
 69 |                     round(avg),
 70 |                     best,
 71 |                     worst,
 72 |                     topicAttempts.size(),
 73 |                     passed,
 74 |                     recommend(avg, worst)
 75 |             ));
 76 |         }
 77 |
 78 |         double overallAvg = attempts.isEmpty() ? 0 : runningSum / runningCount;
 79 |         double overallBest = attempts.stream().mapToDouble(a ->
a.getPercentage().doubleValue()).max().orElse(0);
 80 |         double overallWorst = attempts.isEmpty() ? 0 : attempts.stream().mapToDouble(a ->
a.getPercentage().doubleValue()).min().orElse(0);
 81 |
 82 |         AnalyticsResponse.OverallStats overall = new AnalyticsResponse.OverallStats(
 83 |                 attempts.size(),
 84 |                 BigDecimal.valueOf(overallAvg).setScale(1, RoundingMode.HALF_UP),
 85 |                 round(overallBest),
 86 |                 round(overallWorst),
 87 |                 topics.size(),
 88 |                 passedTopics
 89 |         );
 90 |
 91 |         return new AnalyticsResponse(overall, topics, trend);
 92 |     }
 93 |
 94 |     private String recommend(double avg, double worst) {
 95 |         if (avg >= 85) return "Mastered — great job, move on to a new topic";
 96 |         if (avg >= PASS_THRESHOLD) return "Solid — a quick review will keep it fresh";
 97 |         if (worst < 50) return "Needs work — re-study the weakest quiz and retake it";
 98 |         return "Keep practicing — retake the quiz to improve";
 99 |     }
100 |
101 |     private double round(double value) {
102 |         return Math.round(value * 10.0) / 10.0;
103 |     }
104 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 17–52:** Loads student attempts chronologically, groups by 'noteId' in a 'LinkedHashMap', and computes running cumulative average points.
- **Lines 54–104:** Computes per-topic best/worst/average/pass counts (70% threshold), overall stats, and rule-based recommendations ('recommend(avg, worst)').

> **How to Explain This Module in an Interview:** Single-pass cumulative trend calculation + Java Stream aggregations.

Module: AuthService (`backend/src/main/java/com/studyassistant/service/AuthService.java`)

What This File Is (Beginner-Friendly Analogy): Handles student registration and login verification.

Why It Is Used & Architectural Role: Backs '/api/auth/register' and '/api/auth/login'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.service;
 2 |
 3 | import com.studyassistant.dto.AuthResponse;
 4 | import com.studyassistant.dto.LoginRequest;
 5 | import com.studyassistant.dto.RegisterRequest;
 6 | import com.studyassistant.exception.UnauthorizedException;
 7 | import com.studyassistant.model.Student;
 8 | import com.studyassistant.repository.StudentRepository;
 9 | import com.studyassistant.security.JwtUtil;
10 | import org.springframework.security.crypto.password.PasswordEncoder;
11 | import org.springframework.stereotype.Service;
12 |
13 | @Service
14 | public class AuthService {
15 |
16 |     private final StudentRepository studentRepository;
17 |     private final PasswordEncoder passwordEncoder;
18 |     private final JwtUtil jwtUtil;
19 |
20 |     public AuthService(StudentRepository studentRepository, PasswordEncoder passwordEncoder, JwtUtil jwtUtil) {
21 |         this.studentRepository = studentRepository;
```

```
 22 |         this.passwordEncoder = passwordEncoder;
 23 |         this.jwtUtil = jwtUtil;
 24 |     }
 25 |
 26 |     public AuthResponse register(RegisterRequest request) {
 27 |         if (studentRepository.existsByEmail(request.getEmail())) {
 28 |             throw new IllegalArgumentException("Email already registered: " + request.getEmail());
 29 |         }
 30 |
 31 |         Student student = new Student();
 32 |         student.setName(request.getName());
 33 |         student.setEmail(request.getEmail());
 34 |         student.setPassword(passwordEncoder.encode(request.getPassword()));
 35 |         student.setRole("STUDENT");
 36 |
 37 |         student = studentRepository.save(student);
 38 |
 39 |         String token = jwtUtil.generateToken(student.getEmail());
 40 |         return new AuthResponse(token, student.getId(), student.getName());
 41 |     }
 42 |
 43 |     public AuthResponse login(LoginRequest request) {
 44 |         Student student = studentRepository.findByEmail(request.getEmail())
 45 |                 .orElseThrow(() -> new UnauthorizedException("Invalid email or password"));
 46 |
 47 |         if (!passwordEncoder.matches(request.getPassword(), student.getPassword())) {
 48 |             throw new UnauthorizedException("Invalid email or password");
 49 |         }
 50 |
 51 |         String token = jwtUtil.generateToken(student.getEmail());
 52 |         return new AuthResponse(token, student.getId(), student.getName());
 53 |     }
 54 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 15–54:** Checks email uniqueness on register, hashes password with 'passwordEncoder.encode()', verifies login with 'passwordEncoder.matches()', and mints JWTs via 'jwtUtil.generateToken()'.

> **How to Explain This Module in an Interview:** Prevents user enumeration by throwing identical 401 messages for unknown email and wrong password.

Module: DashboardService (`backend/src/main/java/com/studyassistant/service/DashboardService.java`)

What This File Is (Beginner-Friendly Analogy): Aggregates total quizzes, average score, total flashcards, and paginated score history.

Why It Is Used & Architectural Role: Backs 'GET /api/dashboard'.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.service;
  2 |
  3 | import com.studyassistant.dto.DashboardResponse;
  4 | import com.studyassistant.dto.ScorePoint;
  5 | import com.studyassistant.dto.PaginatedScoreHistory;
  6 | import com.studyassistant.model.QuizAttempt;
  7 | import com.studyassistant.repository.FlashcardRepository;
  8 | import com.studyassistant.repository.QuizAttemptRepository;
  9 | import org.springframework.data.domain.Page;
 10 | import org.springframework.data.domain.Pageable;
 11 | import org.springframework.stereotype.Service;
 12 |
 13 | import java.util.List;
 14 |
 15 | @Service
 16 | public class DashboardService {
 17 |
 18 |     private final QuizAttemptRepository quizAttemptRepository;
 19 |     private final FlashcardRepository flashcardRepository;
 20 |
 21 |     public DashboardService(QuizAttemptRepository quizAttemptRepository, FlashcardRepository
flashcardRepository) {
 22 |         this.quizAttemptRepository = quizAttemptRepository;
 23 |         this.flashcardRepository = flashcardRepository;
 24 |     }
 25 |
 26 |     /**
```

```
 27 |      * Retrieves aggregated dashboard data for a given student.
 28 |      *
 29 |      * @param studentId the ID of the authenticated student
 30 |      * @param pageable  pagination parameters for the score history (page, size)
 31 |      * @return DashboardResponse containing totals and paginated score history
 32 |      */
 33 |     public DashboardResponse getDashboard(Long studentId, Pageable pageable) {
 34 |         long totalQuizzes = quizAttemptRepository.countByStudentId(studentId);
 35 |         Double avg = quizAttemptRepository.findAveragePercentageByStudentId(studentId);
 36 |         java.math.BigDecimal averageScore = avg != null
 37 |                 ? java.math.BigDecimal.valueOf(avg).setScale(2, java.math.RoundingMode.HALF_UP)
 38 |                 : java.math.BigDecimal.ZERO;
 39 |
 40 |         Page<QuizAttempt> attemptPage = quizAttemptRepository.findAttemptsByStudentIdPaged(studentId,
pageable);
 41 |         List<ScorePoint> scorePoints = attemptPage.getContent().stream()
 42 |                 .map(a -> new ScorePoint(a.getAttemptedAt().toLocalDate(), a.getPercentage()))
 43 |                 .collect(java.util.stream.Collectors.toList());
 44 |
 45 |         PaginatedScoreHistory paginatedScoreHistory = new PaginatedScoreHistory(
 46 |                 scorePoints,
 47 |                 attemptPage.getNumber(),
 48 |                 attemptPage.getSize(),
 49 |                 attemptPage.getTotalElements(),
 50 |                 attemptPage.getTotalPages(),
 51 |                 attemptPage.isLast()
 52 |         );
 53 |         long totalFlashcards = flashcardRepository.countByStudentId(studentId);
 54 |         return new DashboardResponse(totalQuizzes, averageScore, paginatedScoreHistory, totalFlashcards);
 55 |     }
 56 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 16–56:** Queries 'countByStudentId', 'findAveragePercentageByStudentId', 'findAttemptsByStudentIdPaged', and 'flashcardRepository.countByStudentId'.

> **How to Explain This Module in an Interview:** Combines scalar aggregate queries with Spring Data Pageable.

Module: NoteService (`backend/src/main/java/com/studyassistant/service/NoteService.java`)

What This File Is (Beginner-Friendly Analogy): Saves text notes, extracts text from PDF/PPTX uploads, and retrieves stored study materials.

Why It Is Used & Architectural Role: Backs '/api/notes/*' endpoints.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.service;
  2 |
  3 | import com.studyassistant.exception.ResourceNotFoundException;
  4 | import com.studyassistant.model.Flashcard;
  5 | import com.studyassistant.model.Note;
  6 | import com.studyassistant.model.Quiz;
  7 | import com.studyassistant.model.QuizQuestion;
  8 | import com.studyassistant.model.Student;
  9 | import com.studyassistant.repository.FlashcardRepository;
 10 | import com.studyassistant.repository.NoteRepository;
 11 | import com.studyassistant.repository.QuizQuestionRepository;
 12 | import com.studyassistant.repository.QuizRepository;
 13 | import com.studyassistant.repository.StudentRepository;
 14 | import org.apache.pdfbox.Loader;
 15 | import org.apache.pdfbox.pdmodel.PDDocument;
 16 | import org.apache.pdfbox.text.PDFTextStripper;
 17 | import org.springframework.stereotype.Service;
 18 | import org.springframework.web.multipart.MultipartFile;
 19 |
 20 | import java.io.IOException;
 21 | import java.util.ArrayList;
 22 | import java.util.HashMap;
 23 | import java.util.List;
 24 | import java.util.Map;
 25 | import java.util.Optional;
 26 |
 27 | @Service
 28 | public class NoteService {
 29 |
 30 |     private final NoteRepository noteRepository;
```

```
    private final StudentRepository studentRepository;
    private final FlashcardRepository flashcardRepository;
    private final QuizRepository quizRepository;
    private final QuizQuestionRepository quizQuestionRepository;

    public NoteService(NoteRepository noteRepository, StudentRepository studentRepository,
                       FlashcardRepository flashcardRepository, QuizRepository quizRepository,
                       QuizQuestionRepository quizQuestionRepository) {
        this.noteRepository = noteRepository;
        this.studentRepository = studentRepository;
        this.flashcardRepository = flashcardRepository;
        this.quizRepository = quizRepository;
        this.quizQuestionRepository = quizQuestionRepository;
    }

    public Note saveTextNote(String email, String title, String content) {
        Student student = studentRepository.findByEmail(email)
                .orElseThrow(() -> new ResourceNotFoundException("Student not found"));

        Note note = new Note(student, title, content);
        return noteRepository.save(note);
    }

    public Note uploadFileNote(String email, String title, MultipartFile file) {
        Student student = studentRepository.findByEmail(email)
                .orElseThrow(() -> new ResourceNotFoundException("Student not found"));

        String extractedText = extractFileText(file);

        Note note = new Note(student, title, extractedText);
        return noteRepository.save(note);
    }

    public List<Note> getStudentNotes(String email) {
        Student student = studentRepository.findByEmail(email)
                .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
        return noteRepository.findByStudentIdOrderByCreatedAtDesc(student.getId());
    }

    public Note getNoteById(Long noteId, String email) {
        Student student = studentRepository.findByEmail(email)
                .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
        return noteRepository.findByIdAndStudentId(noteId, student.getId())
                .orElseThrow(() -> new ResourceNotFoundException("Note not found with id: " + noteId));
    }

    public Map<String, Object> getStoredStudyMaterials(Long noteId, String email) {
        Student student = studentRepository.findByEmail(email)
                .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
        Note note = noteRepository.findByIdAndStudentId(noteId, student.getId())
                .orElseThrow(() -> new ResourceNotFoundException("Note not found with id: " + noteId));

        Map<String, Object> result = new HashMap<>();
        result.put("summary", note.getSummary());
        result.put("content", note.getContent());
        result.put("hasSummary", note.getSummary() != null && !note.getSummary().trim().isEmpty());

        // Fetch flashcards
        List<Flashcard> flashcards = flashcardRepository.findByNoteId(noteId);
        List<Map<String, Object>> flashcardList = new ArrayList<>();
        for (Flashcard fc : flashcards) {
            Map<String, Object> fcMap = new HashMap<>();
            fcMap.put("id", fc.getId());
            fcMap.put("question", fc.getQuestion());
            fcMap.put("answer", fc.getAnswer());
            fcMap.put("boxLevel", fc.getBoxLevel());
            fcMap.put("nextReviewDate", fc.getNextReviewDate());
            flashcardList.add(fcMap);
        }
        result.put("flashcards", flashcardList);
        result.put("hasFlashcards", !flashcardList.isEmpty());

        // Fetch quiz questions (with correct answers — this is the history view)
        Optional<Quiz> quizOpt = quizRepository.findByNoteId(noteId);
        if (quizOpt.isPresent()) {
            Quiz quiz = quizOpt.get();
            List<QuizQuestion> questions = quizQuestionRepository.findByQuizId(quiz.getId());
            List<Map<String, Object>> questionList = new ArrayList<>();
            for (QuizQuestion q : questions) {
                Map<String, Object> qMap = new HashMap<>();
                qMap.put("id", q.getId());
                qMap.put("questionText", q.getQuestionText());
                qMap.put("optionA", q.getOptionA());
                qMap.put("optionB", q.getOptionB());
```

```
115 |                 qMap.put("optionC", q.getOptionC());
116 |                 qMap.put("optionD", q.getOptionD());
117 |                 qMap.put("correctOption", q.getCorrectOption());
118 |                 questionList.add(qMap);
119 |             }
120 |             result.put("questions", questionList);
121 |             result.put("quizId", quiz.getId());
122 |             result.put("hasQuiz", !questionList.isEmpty());
123 |         } else {
124 |             result.put("questions", new ArrayList<>());
125 |             result.put("hasQuiz", false);
126 |         }
127 |
128 |         return result;
129 |     }
130 |
131 |     private String extractFileText(MultipartFile file) {
132 |         String filename = file.getOriginalFilename();
133 |         if (filename != null && filename.toLowerCase().endsWith(".pdf")) {
134 |             return extractPdfText(file);
135 |         } else if (filename != null && (filename.toLowerCase().endsWith(".pptx") ||
filename.toLowerCase().endsWith(".ppt"))) {
136 |             return extractPptxText(file);
137 |         } else {
138 |             throw new RuntimeException("Unsupported file format. Please upload a PDF or PowerPoint (.pptx)
file.");
139 |         }
140 |     }
141 |
142 |     private String extractPdfText(MultipartFile file) {
143 |         try {
144 |             PDDocument document = Loader.loadPDF(file.getBytes());
145 |             PDFTextStripper stripper = new PDFTextStripper();
146 |             String text = stripper.getText(document);
147 |             document.close();
148 |             return text;
149 |         } catch (IOException e) {
150 |             throw new RuntimeException("Failed to extract text from PDF: " + e.getMessage());
151 |         }
152 |     }
153 |
154 |     private String extractPptxText(MultipartFile file) {
155 |         try {
156 |             org.apache.poi.xslf.usermodel.XMLSlideShow ppt = new
org.apache.poi.xslf.usermodel.XMLSlideShow(file.getInputStream());
157 |             StringBuilder text = new StringBuilder();
158 |             for (org.apache.poi.xslf.usermodel.XSLFSlide slide : ppt.getSlides()) {
159 |                 for (org.apache.poi.xslf.usermodel.XSLFShape shape : slide.getShapes()) {
160 |                     if (shape instanceof org.apache.poi.xslf.usermodel.XSLFTextShape) {
161 |                         org.apache.poi.xslf.usermodel.XSLFTextShape textShape =
(org.apache.poi.xslf.usermodel.XSLFTextShape) shape;
162 |                         text.append(textShape.getText()).append("\n");
163 |                     }
164 |                 }
165 |             }
166 |             ppt.close();
167 |             return text.toString();
168 |         } catch (Exception e) {
169 |             throw new RuntimeException("Failed to extract text from PowerPoint. Ensure it is a valid .pptx
file: " + e.getMessage());
170 |         }
171 |     }
172 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 26–121:** Handles 'saveTextNote', 'uploadFileNote', 'getStudentNotes', 'getNoteById', and 'getStoredStudyMaterials'.
- **Lines 123–162:** Extracts PDF text via Apache PDFBox ('Loader.loadPDF' + 'PDFTextStripper') and PPTX text via Apache POI ('XMLSlideShow' + 'XSLFTextShape').

> **How to Explain This Module in an Interview:** Demonstrates binary document parsing with PDFBox 3.0 and Apache POI 5.2.

Module: OpenRouterService (`backend/src/main/java/com/studyassistant/service/OpenRouterService.java`)

What This File Is (Beginner-Friendly Analogy): Orchestrates LLM prompt engineering, HTTP retry logic, JSON extraction, and transactional persistence of summaries, flashcards, and quizzes.

Why It Is Used & Architectural Role: Backs 'POST /api/study/generate'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.service;
 2 |
 3 | import com.fasterxml.jackson.databind.JsonNode;
 4 | import com.fasterxml.jackson.databind.ObjectMapper;
 5 | import com.studyassistant.dto.FlashcardDTO;
 6 | import com.studyassistant.dto.GenerateResponse;
 7 | import com.studyassistant.dto.QuizQuestionDTO;
 8 | import com.studyassistant.exception.ResourceNotFoundException;
 9 | import com.studyassistant.model.*;
10 | import com.studyassistant.repository.*;
11 | import org.slf4j.Logger;
12 | import org.slf4j.LoggerFactory;
13 | import org.springframework.beans.factory.annotation.Value;
14 | import org.springframework.http.*;
15 | import org.springframework.stereotype.Service;
16 | import org.springframework.transaction.annotation.Transactional;
17 | import org.springframework.web.client.ResourceAccessException;
18 | import org.springframework.web.client.RestTemplate;
19 |
20 | import java.time.Duration;
21 | import java.util.ArrayList;
22 | import java.util.List;
23 |
24 | @Service
25 | public class OpenRouterService {
26 |
27 |     private static final Logger logger = LoggerFactory.getLogger(OpenRouterService.class);
28 |
29 |     @Value("${app.openrouter.api.key}")
30 |     private String apiKey;
31 |
32 |     @Value("${app.openrouter.api.url:https://openrouter.ai/api/v1/chat/completions}")
33 |     private String apiUrl;
34 |
35 |     private final NoteRepository noteRepository;
36 |     private final FlashcardRepository flashcardRepository;
37 |     private final QuizRepository quizRepository;
38 |     private final QuizQuestionRepository quizQuestionRepository;
39 |     private final StudentRepository studentRepository;
40 |     private final ObjectMapper objectMapper;
41 |     private final RestTemplate restTemplate;
42 |
43 |     public OpenRouterService(NoteRepository noteRepository, FlashcardRepository flashcardRepository,
44 |                              QuizRepository quizRepository, QuizQuestionRepository quizQuestionRepository,
45 |                              StudentRepository studentRepository) {
46 |         this.noteRepository = noteRepository;
47 |         this.flashcardRepository = flashcardRepository;
48 |         this.quizRepository = quizRepository;
49 |         this.quizQuestionRepository = quizQuestionRepository;
50 |         this.studentRepository = studentRepository;
51 |         this.objectMapper = new ObjectMapper();
52 |
53 |         var factory = new org.springframework.http.client.SimpleClientHttpRequestFactory();
54 |         factory.setConnectTimeout(Duration.ofSeconds(5));
55 |         factory.setReadTimeout(Duration.ofSeconds(120));
56 |         this.restTemplate = new RestTemplate(factory);
57 |     }
58 |
59 |     @Transactional
60 |     public GenerateResponse generateStudyMaterials(Long noteId, String email) {
61 |         Student student = studentRepository.findByEmail(email)
62 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
63 |
64 |         Note note = noteRepository.findByIdAndStudentId(noteId, student.getId())
65 |                 .orElseThrow(() -> new ResourceNotFoundException("Note not found with id: " + noteId));
66 |
67 |         String prompt = buildPrompt(note.getContent());
68 |         String aiResponse = callOpenRouterApi(prompt);
69 |         String jsonContent = extractJsonFromResponse(aiResponse);
70 |
71 |         try {
72 |             JsonNode root = objectMapper.readTree(jsonContent);
73 |
74 |             // Parse and save summary directly onto the Note
75 |             String summary = root.get("summary").asText();
76 |             note.setSummary(summary);
77 |             noteRepository.save(note);
78 |
79 |             // Clean up any existing flashcards or quizzes for this note before saving new ones
```

```
            List<Flashcard> existingFc = flashcardRepository.findByNoteId(noteId);
            if (!existingFc.isEmpty()) {
                flashcardRepository.deleteAll(existingFc);
            }
            quizRepository.findByNoteId(noteId).ifPresent(existingQuiz -> {
                List<QuizQuestion> existingQq = quizQuestionRepository.findByQuizId(existingQuiz.getId());
                quizQuestionRepository.deleteAll(existingQq);
                quizRepository.delete(existingQuiz);
            });

            // Parse and save flashcards
            List<FlashcardDTO> flashcardDTOs = new ArrayList<>();
            JsonNode flashcardsNode = root.get("flashcards");
            if (flashcardsNode != null && flashcardsNode.isArray()) {
                for (JsonNode fc : flashcardsNode) {
                    Flashcard flashcard = new Flashcard(
                            note,
                            fc.get("question").asText(),
                            fc.get("answer").asText()
                    );
                    flashcard = flashcardRepository.save(flashcard);
                    flashcardDTOs.add(new FlashcardDTO(flashcard.getId(), flashcard.getQuestion(),
flashcard.getAnswer(), flashcard.getBoxLevel(), flashcard.getNextReviewDate()));
                }
            }

            // Create and save quiz
            Quiz quiz = new Quiz(note, "Quiz: " + note.getTitle());
            quiz = quizRepository.save(quiz);

            // Parse and save quiz questions
            List<QuizQuestionDTO> questionDTOs = new ArrayList<>();
            JsonNode questionsNode = root.get("questions");
            if (questionsNode != null && questionsNode.isArray()) {
                for (JsonNode q : questionsNode) {
                    QuizQuestion question = new QuizQuestion(
                            quiz,
                            q.get("questionText").asText(),
                            q.get("optionA").asText(),
                            q.get("optionB").asText(),
                            q.get("optionC").asText(),
                            q.get("optionD").asText(),
                            q.get("correctOption").asText()
                    );
                    question = quizQuestionRepository.save(question);

                    // NOTE: correctOption is NOT included in the DTO sent to frontend
                    questionDTOs.add(new QuizQuestionDTO(
                            question.getId(),
                            question.getQuestionText(),
                            question.getOptionA(),
                            question.getOptionB(),
                            question.getOptionC(),
                            question.getOptionD()
                    ));
                }
            }

            return new GenerateResponse(summary, flashcardDTOs, quiz.getId(), questionDTOs);

        } catch (Exception e) {
            logger.error("Failed to parse AI response: {}", e.getMessage());
            throw new RuntimeException("Failed to parse AI response. Please try again. Error: " +
e.getMessage());
        }
    }

    private String buildPrompt(String noteContent) {
        return """
                You are an expert study assistant.
                Given the following lecture notes or topic, do three things:

                1. Write a clear, concise summary of the topic in 3-5 paragraphs. The summary should cover the
main concepts, key points, and important takeaways in plain readable English. Do NOT use bullet trees or flowcharts —
write it as normal paragraphs.
                2. Generate exactly 10 flashcards as a JSON array:
                   [{"question": "...", "answer": "..."}]
                3. Generate exactly 5 multiple choice quiz questions as a JSON array:
                   [{"questionText": "...", "optionA": "...", "optionB": "...", "optionC": "...", "optionD":
"...", "correctOption": "A"}]

                Return ONLY valid JSON in this exact format, nothing else:
                {
                  "summary": "...",
```

```
                  "flashcards": [...],
                  "questions": [...]
                }
                CRITICAL: You must double-escape any backslashes in the text (e.g. write C:\\\\path instead of C:\\path) so the JSON is valid.

                Notes/Topic:
                """ + noteContent;
    }

    private String callOpenRouterApi(String prompt) {
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        headers.set("Authorization", "Bearer " + apiKey);
        headers.set("HTTP-Referer", "http://localhost:5173");
        headers.set("X-Title", "AI Study Assistant");

        String requestBody;
        try {
            java.util.Map<String, Object> requestMap = java.util.Map.of(
                "model", "openrouter/free",
                "messages", List.of(java.util.Map.of("role", "user", "content", prompt)),
                "response_format", java.util.Map.of("type", "json_object")
            );
            requestBody = objectMapper.writeValueAsString(requestMap);
        } catch (Exception e) {
            throw new RuntimeException("Failed to build OpenRouter request: " + e.getMessage());
        }

        HttpEntity<String> entity = new HttpEntity<>(requestBody, headers);

        int maxRetries = 2;
        for (int attempt = 0; attempt <= maxRetries; attempt++) {
            try {
                ResponseEntity<String> response = restTemplate.exchange(apiUrl, HttpMethod.POST, entity, String.class);

                if (response.getStatusCode() == HttpStatus.TOO_MANY_REQUESTS && attempt < maxRetries) {
                    long wait = 1000L * (attempt + 1);
                    logger.warn("Rate limited on attempt {}/{}, retrying after {}ms", attempt + 1, maxRetries + 1, wait);
                    try { Thread.sleep(wait); } catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
                    continue;
                }

                if (response.getStatusCode() != HttpStatus.OK) {
                    throw new RuntimeException("OpenRouter API returned status: " + response.getStatusCode());
                }

                return response.getBody();
            } catch (ResourceAccessException e) {
                if (attempt < maxRetries) {
                    logger.warn("Network error on attempt {}/{}, retrying: {}", attempt + 1, maxRetries + 1, e.getMessage());
                    try { Thread.sleep(1000L); } catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
                    continue;
                }
                logger.error("OpenRouter API call failed after {} attempts: {}", maxRetries + 1, e.getMessage());
                throw new RuntimeException("Failed to connect to AI service. Please try again later.");
            } catch (RuntimeException e) {
                throw e;
            } catch (Exception e) {
                logger.error("OpenRouter API call failed: {}", e.getMessage());
                throw new RuntimeException("Failed to call OpenRouter API: " + e.getMessage());
            }
        }
        throw new RuntimeException("Failed to call OpenRouter API after " + (maxRetries + 1) + " attempts");
    }

    private String extractJsonFromResponse(String responseStr) {
        try {
            JsonNode root = objectMapper.readTree(responseStr);
            String text = root.get("choices")
                    .get(0)
                    .get("message")
                    .get("content")
                    .asText();

            // Strip markdown code fences if present
            text = text.trim();
            if (text.startsWith("```json")) {
```

```
236 |                 text = text.substring(7);
237 |             } else if (text.startsWith("```")) {
238 |                 text = text.substring(3);
239 |             }
240 |             if (text.endsWith("```")) {
241 |                 text = text.substring(0, text.length() - 3);
242 |             }
243 |             return text.trim();
244 |
245 |         } catch (Exception e) {
246 |             throw new RuntimeException("Failed to extract content from OpenRouter response: " +
e.getMessage());
247 |         }
248 |     }
249 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 26–55:** Configures 'RestTemplate' with 5s connect / 120s read timeouts and Jackson 'ObjectMapper'.
- **Lines 57–142:** '@Transactional generateStudyMaterials' verifies note ownership, calls OpenRouter, parses JSON, updates 'note.summary', cleans up old flashcards/quizzes, and saves 10 flashcards + 1 quiz + 5 quiz questions.
- **Lines 144–244:** 'buildPrompt' (strict JSON schema), 'callOpenRouterApi' (3 attempts with backoff on 429/network error), and 'extractJsonFromResponse' (strips markdown fences).

How to Explain This Module in an Interview: Highlight '@Transactional' rollback safety, 120s read timeout, and 429 rate-limit retry backoff.

Module: ProgressService (`backend/src/main/java/com/studyassistant/service/ProgressService.java`)

What This File Is (Beginner-Friendly Analogy): Fetches chronological quiz attempts mapped to 'ProgressDTO'.

Why It Is Used & Architectural Role: Backs 'GET /api/progress'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.service;
 2 |
 3 | import com.studyassistant.dto.ProgressDTO;
 4 | import com.studyassistant.exception.ResourceNotFoundException;
 5 | import com.studyassistant.model.QuizAttempt;
 6 | import com.studyassistant.model.Student;
 7 | import com.studyassistant.repository.QuizAttemptRepository;
 8 | import com.studyassistant.repository.StudentRepository;
 9 | import org.springframework.stereotype.Service;
10 |
11 | import java.util.List;
12 | import java.util.stream.Collectors;
13 |
14 | @Service
15 | public class ProgressService {
16 |
17 |     private final QuizAttemptRepository quizAttemptRepository;
18 |     private final StudentRepository studentRepository;
19 |
20 |     public ProgressService(QuizAttemptRepository quizAttemptRepository, StudentRepository studentRepository) {
21 |         this.quizAttemptRepository = quizAttemptRepository;
22 |         this.studentRepository = studentRepository;
23 |     }
24 |
25 |     public List<ProgressDTO> getProgress(String email) {
26 |         Student student = studentRepository.findByEmail(email)
27 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
28 |
29 |         List<QuizAttempt> attempts =
quizAttemptRepository.findByStudentIdOrderByAttemptedAtAsc(student.getId());
30 |
31 |         return attempts.stream().map(attempt -> new ProgressDTO(
32 |                 attempt.getQuiz().getNote().getTitle(),
33 |                 attempt.getScore(),
34 |                 attempt.getTotalQuestions(),
35 |                 attempt.getPercentage(),
36 |                 attempt.getAttemptedAt()
37 |         )).collect(Collectors.toList());
38 |     }
39 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–39:** Fetches attempts via 'findByStudentIdOrderByAttemptedAtAsc' and maps to 'ProgressDTO'.

> **How to Explain This Module in an Interview:** Clean stream mapping over '@EntityGraph'-optimized attempts.

Module: QuizService (`backend/src/main/java/com/studyassistant/service/QuizService.java`)

What This File Is (Beginner-Friendly Analogy): Grades submitted quiz attempts and returns quiz attempt history.

Why It Is Used & Architectural Role: Backs '/api/quiz/attempt' and '/api/quiz/history'.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.service;
  2 |
  3 | import com.studyassistant.dto.QuizAttemptRequest;
  4 | import com.studyassistant.dto.QuizAttemptResponse;
  5 | import com.studyassistant.exception.ForbiddenException;
  6 | import com.studyassistant.exception.ResourceNotFoundException;
  7 | import com.studyassistant.model.*;
  8 | import com.studyassistant.repository.*;
  9 | import org.springframework.stereotype.Service;
 10 |
 11 | import java.math.BigDecimal;
 12 | import java.math.RoundingMode;
 13 | import java.util.ArrayList;
 14 | import java.util.List;
 15 | import java.util.Map;
 16 | import java.util.stream.Collectors;
 17 |
 18 | @Service
 19 | public class QuizService {
 20 |
 21 |     private final QuizRepository quizRepository;
 22 |     private final QuizQuestionRepository quizQuestionRepository;
 23 |     private final QuizAttemptRepository quizAttemptRepository;
 24 |     private final StudentRepository studentRepository;
 25 |
 26 |     public QuizService(QuizRepository quizRepository, QuizQuestionRepository quizQuestionRepository,
 27 |                        QuizAttemptRepository quizAttemptRepository, StudentRepository studentRepository) {
 28 |         this.quizRepository = quizRepository;
 29 |         this.quizQuestionRepository = quizQuestionRepository;
 30 |         this.quizAttemptRepository = quizAttemptRepository;
 31 |         this.studentRepository = studentRepository;
 32 |     }
 33 |
 34 |     public QuizAttemptResponse submitAttempt(String email, QuizAttemptRequest request) {
 35 |         Student student = studentRepository.findByEmail(email)
 36 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
 37 |
 38 |         Quiz quiz = quizRepository.findById(request.getQuizId())
 39 |                 .orElseThrow(() -> new ResourceNotFoundException("Quiz not found with id: " +
request.getQuizId()));
 40 |
 41 |         if (!quiz.getNote().getStudent().getId().equals(student.getId())) {
 42 |             throw new ForbiddenException("You do not have access to this quiz");
 43 |         }
 44 |
 45 |         List<QuizQuestion> questions = quizQuestionRepository.findByQuizId(quiz.getId());
 46 |
 47 |         // Build a map of questionId -> correctOption
 48 |         Map<Long, String> correctAnswers = questions.stream()
 49 |                 .collect(Collectors.toMap(QuizQuestion::getId, QuizQuestion::getCorrectOption));
 50 |
 51 |         int score = 0;
 52 |         List<QuizAttemptResponse.QuestionBreakdown> breakdown = new ArrayList<>();
 53 |
 54 |         for (QuizAttemptRequest.AnswerDTO answer : request.getAnswers()) {
 55 |             String correctOption = correctAnswers.get(answer.getQuestionId());
 56 |             boolean isCorrect = correctOption != null &&
 57 |                     correctOption.equalsIgnoreCase(answer.getSelectedOption());
 58 |
 59 |             if (isCorrect) {
 60 |                 score++;
 61 |             }
 62 |
 63 |             breakdown.add(new QuizAttemptResponse.QuestionBreakdown(
 64 |                     answer.getQuestionId(),
 65 |                     answer.getSelectedOption(),
```

```
 66 |                     correctOption != null ? correctOption : "N/A",
 67 |                     isCorrect
 68 |             ));
 69 |         }
 70 |
 71 |         int totalQuestions = questions.size();
 72 |         double percentage = totalQuestions > 0 ? (double) score / totalQuestions * 100 : 0;
 73 |         BigDecimal percentBD = BigDecimal.valueOf(percentage).setScale(2, RoundingMode.HALF_UP);
 74 |
 75 |         // Save attempt
 76 |         QuizAttempt attempt = new QuizAttempt(student, quiz, score, totalQuestions, percentBD);
 77 |         quizAttemptRepository.save(attempt);
 78 |
 79 |         boolean passed = percentage >= 60.0;
 80 |
 81 |         return new QuizAttemptResponse(score, totalQuestions, percentage, passed, breakdown);
 82 |     }
 83 |
 84 |     public List<Map<String, Object>> getQuizHistory(String email) {
 85 |         Student student = studentRepository.findByEmail(email)
 86 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
 87 |
 88 |         List<QuizAttempt> attempts =
quizAttemptRepository.findByStudentIdOrderByAttemptedAtDesc(student.getId());
 89 |
 90 |         return attempts.stream().map(attempt -> {
 91 |             Map<String, Object> map = new java.util.HashMap<>();
 92 |             map.put("quizId", attempt.getQuiz().getId());
 93 |             map.put("noteId", attempt.getQuiz().getNote().getId());
 94 |             map.put("noteTitle", attempt.getQuiz().getNote().getTitle());
 95 |             map.put("score", attempt.getScore());
 96 |             map.put("totalQuestions", attempt.getTotalQuestions());
 97 |             map.put("percentage", attempt.getPercentage());
 98 |             map.put("attemptedAt", attempt.getAttemptedAt());
 99 |             return map;
100 |         }).collect(Collectors.toList());
101 |     }
102 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 18–79:** 'submitAttempt' verifies ownership (throws 403 'ForbiddenException' if not owner), builds an O(1) 'correctAnswers' map, grades each answer, saves 'QuizAttempt', and returns 'QuizAttemptResponse'.
- **Lines 81–99:** 'getQuizHistory' returns attempts ordered newest-first.

> **How to Explain This Module in an Interview:** Server-side grading with O(1) Map lookup and 403 ownership enforcement.

Module: SpacedRepetitionService (`backend/src/main/java/com/studyassistant/service/SpacedRepetitionService.java`)

What This File Is (Beginner-Friendly Analogy): Implements the 5-box Leitner spaced-repetition algorithm for flashcards.

Why It Is Used & Architectural Role: Backs '/api/flashcards/review' and '/api/flashcards/due'.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.service;
  2 |
  3 | import com.studyassistant.dto.FlashcardDTO;
  4 | import com.studyassistant.exception.ForbiddenException;
  5 | import com.studyassistant.exception.ResourceNotFoundException;
  6 | import com.studyassistant.model.Flashcard;
  7 | import com.studyassistant.model.Student;
  8 | import com.studyassistant.repository.FlashcardRepository;
  9 | import com.studyassistant.repository.StudentRepository;
 10 | import org.springframework.stereotype.Service;
 11 | import org.springframework.transaction.annotation.Transactional;
 12 |
 13 | import java.time.LocalDateTime;
 14 | import java.util.List;
 15 |
 16 | @Service
 17 | public class SpacedRepetitionService {
 18 |
 19 |     private static final int MAX_BOX = 5;
 20 |
 21 |     // Each box N has a review interval of 2^(N-1) days: 1, 2, 4, 8, 16
 22 |     private static final int[] INTERVAL_DAYS = {1, 2, 4, 8, 16};
```

```
 23 |
 24 |     private final FlashcardRepository flashcardRepository;
 25 |     private final StudentRepository studentRepository;
 26 |
 27 |     public SpacedRepetitionService(FlashcardRepository flashcardRepository, StudentRepository
studentRepository) {
 28 |         this.flashcardRepository = flashcardRepository;
 29 |         this.studentRepository = studentRepository;
 30 |     }
 31 |
 32 |     /**
 33 |      * Leitner box algorithm:
 34 |      * - Correct answer  -> move card up one box (max 5)
 35 |      * - Wrong answer    -> move card back to box 1
 36 |      * - Next review date = now + 2^(box-1) days
 37 |      */
 38 |     @Transactional
 39 |     public FlashcardDTO reviewFlashcard(String email, Long flashcardId, boolean correct) {
 40 |         Student student = studentRepository.findByEmail(email)
 41 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
 42 |
 43 |         Flashcard flashcard = flashcardRepository.findById(flashcardId)
 44 |                 .orElseThrow(() -> new ResourceNotFoundException("Flashcard not found with id: " +
flashcardId));
 45 |
 46 |         if (!flashcard.getNote().getStudent().getId().equals(student.getId())) {
 47 |             throw new ForbiddenException("You do not have access to this flashcard");
 48 |         }
 49 |
 50 |         int currentBox = flashcard.getBoxLevel() != null ? flashcard.getBoxLevel() : 1;
 51 |         int newBox;
 52 |         if (correct) {
 53 |             newBox = Math.min(currentBox + 1, MAX_BOX);
 54 |         } else {
 55 |             newBox = 1;
 56 |         }
 57 |
 58 |         flashcard.setBoxLevel(newBox);
 59 |         flashcard.setNextReviewDate(LocalDateTime.now().plusDays(INTERVAL_DAYS[newBox - 1]));
 60 |         flashcardRepository.save(flashcard);
 61 |
 62 |         return toDTO(flashcard);
 63 |     }
 64 |
 65 |     public List<FlashcardDTO> getDueFlashcards(String email) {
 66 |         Student student = studentRepository.findByEmail(email)
 67 |                 .orElseThrow(() -> new ResourceNotFoundException("Student not found"));
 68 |
 69 |         return flashcardRepository.findDueForStudent(student.getId(), LocalDateTime.now())
 70 |                 .stream().map(this::toDTO).toList();
 71 |     }
 72 |
 73 |     private FlashcardDTO toDTO(Flashcard fc) {
 74 |         return new FlashcardDTO(fc.getId(), fc.getQuestion(), fc.getAnswer(), fc.getBoxLevel(),
fc.getNextReviewDate());
 75 |     }
 76 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 17–72:** Defines 'MAX_BOX = 5' and 'INTERVAL_DAYS = {1, 2, 4, 8, 16}'. 'reviewFlashcard' verifies ownership, promotes or resets 'boxLevel', and schedules 'nextReviewDate = LocalDateTime.now().plusDays(INTERVAL_DAYS[newBox - 1])'.

> **How to Explain This Module in an Interview:** Exponential review scheduling: 1, 2, 4, 8, and 16 days.

Module: AnalyticsController (`backend/src/main/java/com/studyassistant/controller/AnalyticsController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for 'GET /api/analytics'.

Why It Is Used & Architectural Role: Returns learning analytics for the authenticated student.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.controller;
  2 |
  3 | import com.studyassistant.dto.AnalyticsResponse;
  4 | import com.studyassistant.service.AnalyticsService;
```

```
 5 | import io.swagger.v3.oas.annotations.Operation;
 6 | import io.swagger.v3.oas.annotations.tags.Tag;
 7 | import org.springframework.http.ResponseEntity;
 8 | import org.springframework.security.core.Authentication;
 9 | import org.springframework.web.bind.annotation.GetMapping;
10 | import org.springframework.web.bind.annotation.RequestMapping;
11 | import org.springframework.web.bind.annotation.RestController;
12 |
13 | @RestController
14 | @RequestMapping("/api/analytics")
15 | @Tag(name = "Analytics", description = "Learning analytics and progress insights")
16 | public class AnalyticsController {
17 |
18 |     private final AnalyticsService analyticsService;
19 |
20 |     public AnalyticsController(AnalyticsService analyticsService) {
21 |         this.analyticsService = analyticsService;
22 |     }
23 |
24 |     @GetMapping
25 |     @Operation(summary = "Get topic-wise performance, trends, and study recommendations")
26 |     public ResponseEntity<AnalyticsResponse> getAnalytics(Authentication authentication) {
27 |         return ResponseEntity.ok(analyticsService.getAnalytics(authentication.getName()));
28 |     }
29 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–29:** Injects 'AnalyticsService' and calls 'getAnalytics(authentication.getName())'.

> **How to Explain This Module in an Interview:** Uses 'Authentication.getName()' for tenant isolation.

Module: AuthController (`backend/src/main/java/com/studyassistant/controller/AuthController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for '/api/auth/register' and '/api/auth/login'.

Why It Is Used & Architectural Role: Public endpoints for creating accounts and obtaining JWTs.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.controller;
 2 |
 3 | import com.studyassistant.dto.AuthResponse;
 4 | import com.studyassistant.dto.LoginRequest;
 5 | import com.studyassistant.dto.RegisterRequest;
 6 | import com.studyassistant.service.AuthService;
 7 | import io.swagger.v3.oas.annotations.Operation;
 8 | import io.swagger.v3.oas.annotations.tags.Tag;
 9 | import jakarta.validation.Valid;
10 | import org.springframework.http.ResponseEntity;
11 | import org.springframework.web.bind.annotation.*;
12 |
13 | @RestController
14 | @RequestMapping("/api/auth")
15 | @Tag(name = "Authentication", description = "Register and login endpoints")
16 | public class AuthController {
17 |
18 |     private final AuthService authService;
19 |
20 |     public AuthController(AuthService authService) {
21 |         this.authService = authService;
22 |     }
23 |
24 |     @PostMapping("/register")
25 |     @Operation(summary = "Register a new student")
26 |     public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest request) {
27 |         AuthResponse response = authService.register(request);
28 |         return ResponseEntity.ok(response);
29 |     }
30 |
31 |     @PostMapping("/login")
32 |     @Operation(summary = "Login with email and password")
33 |     public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
34 |         AuthResponse response = authService.login(request);
35 |         return ResponseEntity.ok(response);
36 |     }
37 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–37:** Validates '@Valid @RequestBody' and delegates to 'AuthService'.

> **How to Explain This Module in an Interview:** Permitted publicly in SecurityConfig.

Module: DashboardController (`backend/src/main/java/com/studyassistant/controller/DashboardController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for 'GET /api/dashboard'.

Why It Is Used & Architectural Role: Returns paginated dashboard stats.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.controller;
  2 |
  3 | import com.studyassistant.dto.DashboardResponse;
  4 | import com.studyassistant.service.DashboardService;
  5 | import io.swagger.v3.oas.annotations.Operation;
  6 | import io.swagger.v3.oas.annotations.tags.Tag;
  7 | import jakarta.validation.constraints.Min;
  8 | import org.springframework.data.domain.PageRequest;
  9 | import org.springframework.data.domain.Pageable;
 10 | import org.springframework.http.ResponseEntity;
 11 | import org.springframework.security.core.annotation.AuthenticationPrincipal;
 12 | import org.springframework.security.core.userdetails.UserDetails;
 13 | import org.springframework.web.bind.annotation.GetMapping;
 14 | import org.springframework.web.bind.annotation.RequestMapping;
 15 | import org.springframework.web.bind.annotation.RequestParam;
 16 | import org.springframework.web.bind.annotation.RestController;
 17 |
 18 | /**
 19 |  * Exposes study progress statistics for the authenticated student.
 20 |  * Returns a {@link DashboardResponse} containing totals and a paginated list of recent score points.
 21 |  */
 22 | @RestController
 23 | @RequestMapping("/api/dashboard")
 24 | @Tag(name = "Dashboard", description = "Study-progress overview")
 25 | public class DashboardController {
 26 |
 27 |     private final DashboardService dashboardService;
 28 |     private final com.studyassistant.repository.StudentRepository studentRepository;
 29 |
 30 |     public DashboardController(DashboardService dashboardService,
com.studyassistant.repository.StudentRepository studentRepository) {
 31 |         this.dashboardService = dashboardService;
 32 |         this.studentRepository = studentRepository;
 33 |     }
 34 |
 35 |     @GetMapping
 36 |     @Operation(summary = "Get paginated dashboard statistics for the current student")
 37 |     public ResponseEntity<DashboardResponse> getDashboard(
 38 |             @AuthenticationPrincipal UserDetails user,
 39 |             @RequestParam(name = "page", defaultValue = "0") @Min(0) int page,
 40 |             @RequestParam(name = "size", defaultValue = "7") @Min(1) int size) {
 41 |         // Resolve student ID from email
 42 |         Long studentId = studentRepository.findByEmail(user.getUsername())
 43 |                 .orElseThrow(() -> new RuntimeException("Student not found"))
 44 |                 .getId();
 45 |         Pageable pageable = PageRequest.of(page, size);
 46 |         DashboardResponse response = dashboardService.getDashboard(studentId, pageable);
 47 |         return ResponseEntity.ok(response);
 48 |     }
 49 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 21–45:** Resolves student ID from '@AuthenticationPrincipal UserDetails', builds 'PageRequest.of(page, size)', and calls 'dashboardService.getDashboard()'.

> **How to Explain This Module in an Interview:** Validates '@Min(0) page' and '@Min(1) size'.

Module: FlashcardController (`backend/src/main/java/com/studyassistant/controller/FlashcardController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for '/api/flashcards/review' and '/api/flashcards/due'.

Why It Is Used & Architectural Role: Exposes spaced-repetition endpoints.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.controller;
 2 |
 3 | import com.studyassistant.dto.FlashcardDTO;
 4 | import com.studyassistant.dto.FlashcardReviewRequest;
 5 | import com.studyassistant.service.SpacedRepetitionService;
 6 | import io.swagger.v3.oas.annotations.Operation;
 7 | import io.swagger.v3.oas.annotations.tags.Tag;
 8 | import jakarta.validation.Valid;
 9 | import org.springframework.http.ResponseEntity;
10 | import org.springframework.security.core.Authentication;
11 | import org.springframework.web.bind.annotation.*;
12 |
13 | import java.util.List;
14 |
15 | @RestController
16 | @RequestMapping("/api/flashcards")
17 | @Tag(name = "Flashcards", description = "Spaced-repetition flashcard review (Leitner system)")
18 | public class FlashcardController {
19 |
20 |     private final SpacedRepetitionService spacedRepetitionService;
21 |
22 |     public FlashcardController(SpacedRepetitionService spacedRepetitionService) {
23 |         this.spacedRepetitionService = spacedRepetitionService;
24 |     }
25 |
26 |     @PostMapping("/review")
27 |     @Operation(summary = "Record a flashcard review — moves card through Leitner boxes based on answer
correctness")
28 |     public ResponseEntity<FlashcardDTO> reviewFlashcard(
29 |             @Valid @RequestBody FlashcardReviewRequest request,
30 |             Authentication authentication) {
31 |         FlashcardDTO updated = spacedRepetitionService.reviewFlashcard(
32 |                 authentication.getName(), request.getFlashcardId(), request.getCorrect());
33 |         return ResponseEntity.ok(updated);
34 |     }
35 |
36 |     @GetMapping("/due")
37 |     @Operation(summary = "Get all flashcards due for review (nextReviewDate <= now)")
38 |     public ResponseEntity<List<FlashcardDTO>> getDueFlashcards(Authentication authentication) {
39 |         return ResponseEntity.ok(spacedRepetitionService.getDueFlashcards(authentication.getName()));
40 |     }
41 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 16–40:** Delegates POST '/review' and GET '/due' to 'SpacedRepetitionService'.

How to Explain This Module in an Interview: Clean REST interface for Leitner box updates.

Module: NoteController (`backend/src/main/java/com/studyassistant/controller/NoteController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for creating, uploading, listing, and fetching notes and study materials.

Why It Is Used & Architectural Role: Exposes '/api/notes/text', '/api/notes/upload', '/api/notes', '/api/notes/{noteId}', and '/api/notes/{noteId}/study-materials'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.controller;
 2 |
 3 | import com.studyassistant.dto.NoteRequest;
 4 | import com.studyassistant.model.Note;
 5 | import com.studyassistant.service.NoteService;
 6 | import io.swagger.v3.oas.annotations.Operation;
 7 | import io.swagger.v3.oas.annotations.tags.Tag;
 8 | import jakarta.validation.Valid;
 9 | import org.springframework.http.ResponseEntity;
10 | import org.springframework.security.core.Authentication;
11 | import org.springframework.web.bind.annotation.*;
12 | import org.springframework.web.multipart.MultipartFile;
13 |
```

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/notes")
@Tag(name = "Notes", description = "Note management endpoints")
public class NoteController {

    private final NoteService noteService;

    public NoteController(NoteService noteService) {
        this.noteService = noteService;
    }

    @PostMapping("/text")
    @Operation(summary = "Create a note from text input")
    public ResponseEntity<Map<String, Object>> createTextNote(
            @Valid @RequestBody NoteRequest request,
            Authentication authentication) {
        Note note = noteService.saveTextNote(authentication.getName(), request.getTitle(),
request.getContent());
        return ResponseEntity.ok(buildNoteResponse(note));
    }

    @PostMapping("/upload")
    @Operation(summary = "Create a note from PDF or PPTX upload")
    public ResponseEntity<Map<String, Object>> uploadFileNote(
            @RequestParam("file") MultipartFile file,
            @RequestParam("title") String title,
            Authentication authentication) {
        Note note = noteService.uploadFileNote(authentication.getName(), title, file);
        Map<String, Object> response = buildNoteResponse(note);
        response.put("extractedText", note.getContent());
        return ResponseEntity.ok(response);
    }

    @GetMapping
    @Operation(summary = "Get all notes for the authenticated student")
    public ResponseEntity<List<Map<String, Object>>> getAllNotes(Authentication authentication) {
        List<Note> notes = noteService.getStudentNotes(authentication.getName());
        List<Map<String, Object>> response = notes.stream()
                .map(this::buildNoteListItem)
                .collect(Collectors.toList());
        return ResponseEntity.ok(response);
    }

    @GetMapping("/{noteId}")
    @Operation(summary = "Get a specific note by ID (owner only)")
    public ResponseEntity<Map<String, Object>> getNoteById(@PathVariable Long noteId, Authentication
authentication) {
        Note note = noteService.getNoteById(noteId, authentication.getName());
        return ResponseEntity.ok(buildNoteResponse(note));
    }

    @GetMapping("/{noteId}/study-materials")
    @Operation(summary = "Get stored study materials (flashcards + quiz Q&A with answers) for a note (owner
only)")
    public ResponseEntity<Map<String, Object>> getStudyMaterials(@PathVariable Long noteId, Authentication
authentication) {
        Map<String, Object> materials = noteService.getStoredStudyMaterials(noteId, authentication.getName());
        return ResponseEntity.ok(materials);
    }

    private Map<String, Object> buildNoteResponse(Note note) {
        Map<String, Object> map = new HashMap<>();
        map.put("noteId", note.getId());
        map.put("title", note.getTitle());
        map.put("content", note.getContent());
        map.put("createdAt", note.getCreatedAt());
        return map;
    }

    private Map<String, Object> buildNoteListItem(Note note) {
        Map<String, Object> map = new HashMap<>();
        map.put("noteId", note.getId());
        map.put("title", note.getTitle());
        map.put("createdAt", note.getCreatedAt());
        return map;
    }
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 20–93:** Handles text creation, multipart file upload, lightweight note list serialization ('buildNoteListItem'), and full study material retrieval.

> **How to Explain This Module in an Interview:** Omits heavy note content in list endpoint to optimize bandwidth.

Module: ProgressController (`backend/src/main/java/com/studyassistant/controller/ProgressController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for 'GET /api/progress'.

Why It Is Used & Architectural Role: Returns chronological score history per topic.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.controller;
 2 |
 3 | import com.studyassistant.dto.ProgressDTO;
 4 | import com.studyassistant.service.ProgressService;
 5 | import io.swagger.v3.oas.annotations.Operation;
 6 | import io.swagger.v3.oas.annotations.tags.Tag;
 7 | import org.springframework.http.ResponseEntity;
 8 | import org.springframework.security.core.Authentication;
 9 | import org.springframework.web.bind.annotation.*;
10 |
11 | import java.util.List;
12 |
13 | @RestController
14 | @RequestMapping("/api/progress")
15 | @Tag(name = "Progress", description = "Student progress tracking endpoints")
16 | public class ProgressController {
17 |
18 |     private final ProgressService progressService;
19 |
20 |     public ProgressController(ProgressService progressService) {
21 |         this.progressService = progressService;
22 |     }
23 |
24 |     @GetMapping
25 |     @Operation(summary = "Get progress data for the authenticated student")
26 |     public ResponseEntity<List<ProgressDTO>> getProgress(Authentication authentication) {
27 |         List<ProgressDTO> progress = progressService.getProgress(authentication.getName());
28 |         return ResponseEntity.ok(progress);
29 |     }
30 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 14–30:** Delegates to 'progressService.getProgress(authentication.getName())'.

> **How to Explain This Module in an Interview:** Backs ProgressPage.jsx.

Module: QuizController (`backend/src/main/java/com/studyassistant/controller/QuizController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for 'POST /api/quiz/attempt' and 'GET /api/quiz/history'.

Why It Is Used & Architectural Role: Handles quiz submissions and attempt history.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.controller;
 2 |
 3 | import com.studyassistant.dto.QuizAttemptRequest;
 4 | import com.studyassistant.dto.QuizAttemptResponse;
 5 | import com.studyassistant.service.QuizService;
 6 | import io.swagger.v3.oas.annotations.Operation;
 7 | import io.swagger.v3.oas.annotations.tags.Tag;
 8 | import jakarta.validation.Valid;
 9 | import org.springframework.http.ResponseEntity;
10 | import org.springframework.security.core.Authentication;
11 | import org.springframework.web.bind.annotation.*;
12 |
13 | import java.util.List;
14 | import java.util.Map;
```

```
15 |
16 | @RestController
17 | @RequestMapping("/api/quiz")
18 | @Tag(name = "Quiz", description = "Quiz attempt and history endpoints")
19 | public class QuizController {
20 |
21 |     private final QuizService quizService;
22 |
23 |     public QuizController(QuizService quizService) {
24 |         this.quizService = quizService;
25 |     }
26 |
27 |     @PostMapping("/attempt")
28 |     @Operation(summary = "Submit a quiz attempt with answers")
29 |     public ResponseEntity<QuizAttemptResponse> submitAttempt(
30 |             @Valid @RequestBody QuizAttemptRequest request,
31 |             Authentication authentication) {
32 |         QuizAttemptResponse response = quizService.submitAttempt(authentication.getName(), request);
33 |         return ResponseEntity.ok(response);
34 |     }
35 |
36 |     @GetMapping("/history")
37 |     @Operation(summary = "Get quiz attempt history for the authenticated student")
38 |     public ResponseEntity<List<Map<String, Object>>> getHistory(Authentication authentication) {
39 |         List<Map<String, Object>> history = quizService.getQuizHistory(authentication.getName());
40 |         return ResponseEntity.ok(history);
41 |     }
42 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 16–39:** Validates '@Valid @RequestBody QuizAttemptRequest' and delegates to 'QuizService'.

> **How to Explain This Module in an Interview:** Backs QuizPage.jsx and QuizzesPage.jsx.

Module: StudyController (`backend/src/main/java/com/studyassistant/controller/StudyController.java`)

What This File Is (Beginner-Friendly Analogy): REST controller for 'POST /api/study/generate'.

Why It Is Used & Architectural Role: Triggers AI generation of summary, flashcards, and quiz for a note.

Complete Source Code (Every Line Numbered):

```
1  | package com.studyassistant.controller;
2  |
3  | import com.studyassistant.dto.GenerateResponse;
4  | import com.studyassistant.service.OpenRouterService;
5  | import io.swagger.v3.oas.annotations.Operation;
6  | import io.swagger.v3.oas.annotations.tags.Tag;
7  | import org.springframework.http.ResponseEntity;
8  | import org.springframework.security.core.Authentication;
9  | import org.springframework.web.bind.annotation.*;
10 |
11 | import java.util.Map;
12 |
13 | @RestController
14 | @RequestMapping("/api/study")
15 | @Tag(name = "Study", description = "AI-powered study material generation")
16 | public class StudyController {
17 |
18 |     private final OpenRouterService openRouterService;
19 |
20 |     public StudyController(OpenRouterService openRouterService) {
21 |         this.openRouterService = openRouterService;
22 |     }
23 |
24 |     @PostMapping("/generate")
25 |     @Operation(summary = "Generate summary, flashcards, and quiz from a note using AI (owner only)")
26 |     public ResponseEntity<GenerateResponse> generateStudyMaterials(
27 |             @RequestBody Map<String, Long> request, Authentication authentication) {
28 |         Long noteId = request.get("noteId");
29 |         if (noteId == null) {
30 |             throw new IllegalArgumentException("noteId is required");
31 |         }
32 |         GenerateResponse response = openRouterService.generateStudyMaterials(noteId, authentication.getName());
33 |         return ResponseEntity.ok(response);
34 |     }
35 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 13–34:** Extracts 'noteId' from request map, checks for null, and calls 'openRouterService.generateStudyMaterials(noteId, authentication.getName())'.

How to Explain This Module in an Interview: Backs StudyPage.jsx and HistoryPage.jsx.

Part VII: Backend Automated Test Suite (`src/test/` — Every Line Explained)

Module: AuthControllerTest (`backend/src/test/java/com/studyassistant/controller/AuthControllerTest.java`)

What This File Is (Beginner-Friendly Analogy): MockMvc integration tests for '/api/auth/register' and '/api/auth/login'.

Why It Is Used & Architectural Role: Verifies HTTP 200 JSON responses and 400 Bad Request on invalid email format.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.controller;
  2 |
  3 | import com.fasterxml.jackson.databind.ObjectMapper;
  4 | import com.studyassistant.dto.AuthResponse;
  5 | import com.studyassistant.dto.LoginRequest;
  6 | import com.studyassistant.dto.RegisterRequest;
  7 | import com.studyassistant.service.AuthService;
  8 | import org.junit.jupiter.api.DisplayName;
  9 | import org.junit.jupiter.api.Test;
 10 | import org.springframework.beans.factory.annotation.Autowired;
 11 | import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
 12 | import org.springframework.boot.test.context.SpringBootTest;
 13 | import org.springframework.test.context.ActiveProfiles;
 14 | import org.springframework.test.context.bean.override.mockito.MockitoBean;
 15 | import org.springframework.http.MediaType;
 16 | import org.springframework.test.web.servlet.MockMvc;
 17 |
 18 | import static org.mockito.ArgumentMatchers.any;
 19 | import static org.mockito.Mockito.when;
 20 | import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
 21 | import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
 22 |
 23 | @SpringBootTest
 24 | @AutoConfigureMockMvc
 25 | @ActiveProfiles("test")
 26 | class AuthControllerTest {
 27 |
 28 |     @Autowired private MockMvc mockMvc;
 29 |     @Autowired private ObjectMapper objectMapper;
 30 |     @MockitoBean private AuthService authService;
 31 |
 32 |     @Test
 33 |     @DisplayName("POST /api/auth/register - success")
 34 |     void registerSuccess() throws Exception {
 35 |         AuthResponse response = new AuthResponse("jwt-token", 1L, "John Doe");
 36 |         when(authService.register(any(RegisterRequest.class))).thenReturn(response);
 37 |
 38 |         mockMvc.perform(post("/api/auth/register")
 39 |                 .contentType(MediaType.APPLICATION_JSON)
 40 |                 .content(objectMapper.writeValueAsString(new RegisterRequest("John Doe", "john@example.com",
"password123"))))
 41 |                 .andExpect(status().isOk())
 42 |                 .andExpect(jsonPath("$.token").value("jwt-token"))
 43 |                 .andExpect(jsonPath("$.studentId").value(1))
 44 |                 .andExpect(jsonPath("$.name").value("John Doe"));
 45 |     }
 46 |
 47 |     @Test
 48 |     @DisplayName("POST /api/auth/login - success")
 49 |     void loginSuccess() throws Exception {
 50 |         AuthResponse response = new AuthResponse("jwt-token", 1L, "John Doe");
 51 |         when(authService.login(any(LoginRequest.class))).thenReturn(response);
 52 |
 53 |         mockMvc.perform(post("/api/auth/login")
 54 |                 .contentType(MediaType.APPLICATION_JSON)
 55 |                 .content(objectMapper.writeValueAsString(new LoginRequest("john@example.com", "password123"))))
```

```
56 |                 .andExpect(status().isOk())
57 |                 .andExpect(jsonPath("$.token").value("jwt-token"));
58 |     }
59 |
60 |     @Test
61 |     @DisplayName("POST /api/auth/register - invalid email returns 400")
62 |     void registerInvalidEmail() throws Exception {
63 |         mockMvc.perform(post("/api/auth/register")
64 |                 .contentType(MediaType.APPLICATION_JSON)
65 |                 .content("{\"name\":\"John\",\"email\":\"invalid\",\"password\":\"password123\"}"))
66 |                 .andExpect(status().isBadRequest());
67 |     }
68 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 23–66:** Uses '@SpringBootTest', '@AutoConfigureMockMvc', '@ActiveProfiles("test")', and '@MockitoBean AuthService' to test registerSuccess, loginSuccess, and registerInvalidEmail.

> **How to Explain This Module in an Interview:** Tests the full Spring MVC + Jakarta Validation pipeline.

Module: QuizControllerTest (`backend/src/test/java/com/studyassistant/controller/QuizControllerTest.java`)

What This File Is (Beginner-Friendly Analogy): MockMvc security integration tests for '/api/quiz/*'.

Why It Is Used & Architectural Role: Verifies that unauthenticated requests to '/api/quiz/attempt' and '/api/quiz/history' return 401 Unauthorized.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.controller;
 2 |
 3 | import com.studyassistant.security.JwtUtil;
 4 | import com.studyassistant.service.QuizService;
 5 | import org.junit.jupiter.api.DisplayName;
 6 | import org.junit.jupiter.api.Test;
 7 | import org.springframework.beans.factory.annotation.Autowired;
 8 | import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
 9 | import org.springframework.boot.test.context.SpringBootTest;
10 | import org.springframework.test.context.ActiveProfiles;
11 | import org.springframework.test.context.bean.override.mockito.MockitoBean;
12 | import org.springframework.http.MediaType;
13 | import org.springframework.test.web.servlet.MockMvc;
14 |
15 | import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
16 | import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
17 |
18 | @SpringBootTest
19 | @AutoConfigureMockMvc
20 | @ActiveProfiles("test")
21 | class QuizControllerTest {
22 |
23 |     @Autowired private MockMvc mockMvc;
24 |     @Autowired private JwtUtil jwtUtil;
25 |     @MockitoBean private QuizService quizService;
26 |
27 |     @Test
28 |     @DisplayName("POST /api/quiz/attempt - requires authentication")
29 |     void attemptRequiresAuth() throws Exception {
30 |         mockMvc.perform(post("/api/quiz/attempt")
31 |                 .contentType(MediaType.APPLICATION_JSON)
32 |                 .content("{}"))
33 |                 .andExpect(status().isUnauthorized());
34 |     }
35 |
36 |     @Test
37 |     @DisplayName("GET /api/quiz/history - requires authentication")
38 |     void historyRequiresAuth() throws Exception {
39 |         mockMvc.perform(get("/api/quiz/history"))
40 |                 .andExpect(status().isUnauthorized());
41 |     }
42 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 18–42:** Uses 'mockMvc.perform(post("/api/quiz/attempt"))' and 'get("/api/quiz/history")' without a JWT header and asserts 'status().isUnauthorized()'.

How to Explain This Module in an Interview: Proves SecurityConfig's 401 AuthenticationEntryPoint works end-to-end.

Module: AuthServiceTest (`backend/src/test/java/com/studyassistant/service/AuthServiceTest.java`)

What This File Is (Beginner-Friendly Analogy): Unit tests for 'AuthService'.

Why It Is Used & Architectural Role: Tests registration, duplicate email rejection, login success, wrong password (401), and unknown email (401).

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.service;
 2 |
 3 | import com.studyassistant.dto.AuthResponse;
 4 | import com.studyassistant.dto.LoginRequest;
 5 | import com.studyassistant.dto.RegisterRequest;
 6 | import com.studyassistant.exception.UnauthorizedException;
 7 | import com.studyassistant.model.Student;
 8 | import com.studyassistant.repository.StudentRepository;
 9 | import com.studyassistant.security.JwtUtil;
10 | import org.junit.jupiter.api.BeforeEach;
11 | import org.junit.jupiter.api.DisplayName;
12 | import org.junit.jupiter.api.Test;
13 | import org.junit.jupiter.api.extension.ExtendWith;
14 | import org.mockito.InjectMocks;
15 | import org.mockito.Mock;
16 | import org.mockito.junit.jupiter.MockitoExtension;
17 | import org.springframework.security.crypto.password.PasswordEncoder;
18 |
19 | import java.util.Optional;
20 |
21 | import static org.junit.jupiter.api.Assertions.*;
22 | import static org.mockito.ArgumentMatchers.any;
23 | import static org.mockito.Mockito.*;
24 |
25 | @ExtendWith(MockitoExtension.class)
26 | class AuthServiceTest {
27 |
28 |     @Mock private StudentRepository studentRepository;
29 |     @Mock private PasswordEncoder passwordEncoder;
30 |     @Mock private JwtUtil jwtUtil;
31 |     @InjectMocks private AuthService authService;
32 |
33 |     private Student testStudent;
34 |
35 |     @BeforeEach
36 |     void setUp() {
37 |         testStudent = new Student("John Doe", "john@example.com", "encoded_password");
38 |         testStudent.setId(1L);
39 |     }
40 |
41 |     @Test
42 |     @DisplayName("Register with valid data - student saved, token returned")
43 |     void registerWithValidData() {
44 |         RegisterRequest request = new RegisterRequest("John Doe", "john@example.com", "password123");
45 |         when(studentRepository.existsByEmail("john@example.com")).thenReturn(false);
46 |         when(passwordEncoder.encode("password123")).thenReturn("encoded_password");
47 |         when(studentRepository.save(any(Student.class))).thenReturn(testStudent);
48 |         when(jwtUtil.generateToken("john@example.com")).thenReturn("jwt-token");
49 |
50 |         AuthResponse response = authService.register(request);
51 |
52 |         assertNotNull(response);
53 |         assertEquals("jwt-token", response.getToken());
54 |         assertEquals(1L, response.getStudentId());
55 |         assertEquals("John Doe", response.getName());
56 |         verify(studentRepository).save(any(Student.class));
57 |     }
58 |
59 |     @Test
60 |     @DisplayName("Register with duplicate email - throws exception")
61 |     void registerDuplicateEmail() {
62 |         RegisterRequest request = new RegisterRequest("John", "john@example.com", "password");
63 |         when(studentRepository.existsByEmail("john@example.com")).thenReturn(true);
64 |
65 |         assertThrows(IllegalArgumentException.class, () -> authService.register(request));
66 |         verify(studentRepository, never()).save(any());
67 |     }
68 |
```

```
 69 |     @Test
 70 |     @DisplayName("Login with correct credentials - token returned")
 71 |     void loginSuccess() {
 72 |         LoginRequest request = new LoginRequest("john@example.com", "password123");
 73 |         when(studentRepository.findByEmail("john@example.com")).thenReturn(Optional.of(testStudent));
 74 |         when(passwordEncoder.matches("password123", "encoded_password")).thenReturn(true);
 75 |         when(jwtUtil.generateToken("john@example.com")).thenReturn("jwt-token");
 76 |
 77 |         AuthResponse response = authService.login(request);
 78 |
 79 |         assertNotNull(response);
 80 |         assertEquals("jwt-token", response.getToken());
 81 |     }
 82 |
 83 |     @Test
 84 |     @DisplayName("Login with wrong password - throws UnauthorizedException (401)")
 85 |     void loginWrongPassword() {
 86 |         LoginRequest request = new LoginRequest("john@example.com", "wrong");
 87 |         when(studentRepository.findByEmail("john@example.com")).thenReturn(Optional.of(testStudent));
 88 |         when(passwordEncoder.matches("wrong", "encoded_password")).thenReturn(false);
 89 |
 90 |         assertThrows(UnauthorizedException.class, () -> authService.login(request));
 91 |     }
 92 |
 93 |     @Test
 94 |     @DisplayName("Login with unknown email - throws UnauthorizedException (401)")
 95 |     void loginUnknownEmail() {
 96 |         LoginRequest request = new LoginRequest("ghost@example.com", "whatever");
 97 |         when(studentRepository.findByEmail("ghost@example.com")).thenReturn(Optional.empty());
 98 |
 99 |         assertThrows(UnauthorizedException.class, () -> authService.login(request));
100 |     }
101 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 24–99:** Uses '@ExtendWith(MockitoExtension.class)' to test all 5 authentication branches in isolation.

> **How to Explain This Module in an Interview:** Fast unit tests with mocked repository, encoder, and JwtUtil.

Module: NoteServiceTest (`backend/src/test/java/com/studyassistant/service/NoteServiceTest.java`)

What This File Is (Beginner-Friendly Analogy): Unit and IDOR security tests for 'NoteService'.

Why It Is Used & Architectural Role: Proves that an owner can read their note while another student is blocked with 'ResourceNotFoundException'.

Complete Source Code (Every Line Numbered):

```
 1 | package com.studyassistant.service;
 2 |
 3 | import com.studyassistant.exception.ResourceNotFoundException;
 4 | import com.studyassistant.model.Note;
 5 | import com.studyassistant.model.Student;
 6 | import com.studyassistant.repository.FlashcardRepository;
 7 | import com.studyassistant.repository.NoteRepository;
 8 | import com.studyassistant.repository.QuizQuestionRepository;
 9 | import com.studyassistant.repository.QuizRepository;
10 | import com.studyassistant.repository.StudentRepository;
11 | import org.junit.jupiter.api.BeforeEach;
12 | import org.junit.jupiter.api.DisplayName;
13 | import org.junit.jupiter.api.Test;
14 | import org.junit.jupiter.api.extension.ExtendWith;
15 | import org.mockito.InjectMocks;
16 | import org.mockito.Mock;
17 | import org.mockito.junit.jupiter.MockitoExtension;
18 |
19 | import java.util.Optional;
20 |
21 | import static org.junit.jupiter.api.Assertions.*;
22 | import static org.mockito.Mockito.when;
23 |
24 | @ExtendWith(MockitoExtension.class)
25 | class NoteServiceTest {
26 |
27 |     @Mock private NoteRepository noteRepository;
28 |     @Mock private StudentRepository studentRepository;
29 |     @Mock private FlashcardRepository flashcardRepository;
30 |     @Mock private QuizRepository quizRepository;
```

```
31 |     @Mock private QuizQuestionRepository quizQuestionRepository;
32 |     @InjectMocks private NoteService noteService;
33 |
34 |     private Student owner;
35 |     private Student other;
36 |     private Note note;
37 |
38 |     @BeforeEach
39 |     void setUp() {
40 |         owner = new Student("Owner", "owner@example.com", "pass");
41 |         owner.setId(1L);
42 |         other = new Student("Other", "other@example.com", "pass");
43 |         other.setId(2L);
44 |         note = new Note(owner, "My Note", "content");
45 |         note.setId(10L);
46 |     }
47 |
48 |     @Test
49 |     @DisplayName("Owner can fetch their own note")
50 |     void ownerCanFetchOwnNote() {
51 |         when(studentRepository.findByEmail("owner@example.com")).thenReturn(Optional.of(owner));
52 |         when(noteRepository.findByIdAndStudentId(10L, 1L)).thenReturn(Optional.of(note));
53 |
54 |         Note result = noteService.getNoteById(10L, "owner@example.com");
55 |
56 |         assertEquals(10L, result.getId());
57 |         assertEquals("My Note", result.getTitle());
58 |     }
59 |
60 |     @Test
61 |     @DisplayName("Another student cannot fetch someone else's note - ResourceNotFoundException")
62 |     void otherStudentCannotFetchNote() {
63 |         when(studentRepository.findByEmail("other@example.com")).thenReturn(Optional.of(other));
64 |         when(noteRepository.findByIdAndStudentId(10L, 2L)).thenReturn(Optional.empty());
65 |
66 |         assertThrows(ResourceNotFoundException.class,
67 |                 () -> noteService.getNoteById(10L, "other@example.com"));
68 |     }
69 |
70 |     @Test
71 |     @DisplayName("Another student cannot read someone else's study materials - ResourceNotFoundException")
72 |     void otherStudentCannotReadStudyMaterials() {
73 |         when(studentRepository.findByEmail("other@example.com")).thenReturn(Optional.of(other));
74 |         when(noteRepository.findByIdAndStudentId(10L, 2L)).thenReturn(Optional.empty());
75 |
76 |         assertThrows(ResourceNotFoundException.class,
77 |                 () -> noteService.getStoredStudyMaterials(10L, "other@example.com"));
78 |     }
79 |
80 |     @Test
81 |     @DisplayName("Unknown email throws ResourceNotFoundException")
82 |     void unknownEmailThrows() {
83 |         when(studentRepository.findByEmail("ghost@example.com")).thenReturn(Optional.empty());
84 |
85 |         assertThrows(ResourceNotFoundException.class,
86 |                 () -> noteService.getNoteById(10L, "ghost@example.com"));
87 |     }
88 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 22–82:** Tests 'ownerCanFetchOwnNote', 'otherStudentCannotFetchNote', 'otherStudentCannotReadStudyMaterials', and 'unknownEmailThrows'.

> **How to Explain This Module in an Interview:** Directly verifies IDOR protection on note and study material endpoints.

Module: OpenRouterServiceTest (`backend/src/test/java/com/studyassistant/service/OpenRouterServiceTest.java`)

What This File Is (Beginner-Friendly Analogy): Unit and IDOR security tests for 'OpenRouterService'.

Why It Is Used & Architectural Role: Verifies non-existent note handling, cross-student generation blocking, and repository invocation.

Complete Source Code (Every Line Numbered):

```
1 | package com.studyassistant.service;
2 |
3 | import com.studyassistant.exception.ResourceNotFoundException;
```

```
  4 | import com.studyassistant.model.Note;
  5 | import com.studyassistant.model.Student;
  6 | import com.studyassistant.repository.*;
  7 | import org.junit.jupiter.api.DisplayName;
  8 | import org.junit.jupiter.api.Test;
  9 | import org.junit.jupiter.api.extension.ExtendWith;
 10 | import org.mockito.InjectMocks;
 11 | import org.mockito.Mock;
 12 | import org.mockito.junit.jupiter.MockitoExtension;
 13 |
 14 | import java.util.Optional;
 15 |
 16 | import static org.junit.jupiter.api.Assertions.*;
 17 | import static org.mockito.Mockito.*;
 18 |
 19 | @ExtendWith(MockitoExtension.class)
 20 | class OpenRouterServiceTest {
 21 |
 22 |     @Mock private NoteRepository noteRepository;
 23 |     @Mock private StudentRepository studentRepository;
 24 |     @Mock private FlashcardRepository flashcardRepository;
 25 |     @Mock private QuizRepository quizRepository;
 26 |     @Mock private QuizQuestionRepository quizQuestionRepository;
 27 |     @InjectMocks private OpenRouterService openRouterService;
 28 |
 29 |     @Test
 30 |     @DisplayName("Non-existent note ID throws ResourceNotFoundException")
 31 |     void nonExistentNoteThrowsException() {
 32 |         Student student = new Student("Test", "test@test.com", "pass");
 33 |         student.setId(1L);
 34 |         when(studentRepository.findByEmail("test@test.com")).thenReturn(Optional.of(student));
 35 |         when(noteRepository.findByIdAndStudentId(999L, 1L)).thenReturn(Optional.empty());
 36 |
 37 |         assertThrows(ResourceNotFoundException.class,
 38 |                 () -> openRouterService.generateStudyMaterials(999L, "test@test.com"));
 39 |     }
 40 |
 41 |     @Test
 42 |     @DisplayName("Another student cannot generate materials for someone else's note -
ResourceNotFoundException")
 43 |     void otherStudentCannotGenerate() {
 44 |         Student owner = new Student("Owner", "owner@test.com", "pass");
 45 |         owner.setId(1L);
 46 |         Student other = new Student("Other", "other@test.com", "pass");
 47 |         other.setId(2L);
 48 |         Note note = new Note(owner, "Test", "Sample content for testing");
 49 |         note.setId(1L);
 50 |
 51 |         when(studentRepository.findByEmail("other@test.com")).thenReturn(Optional.of(other));
 52 |         when(noteRepository.findByIdAndStudentId(1L, 2L)).thenReturn(Optional.empty());
 53 |
 54 |         assertThrows(ResourceNotFoundException.class,
 55 |                 () -> openRouterService.generateStudyMaterials(1L, "other@test.com"));
 56 |         verify(noteRepository, never()).findById(any());
 57 |     }
 58 |
 59 |     @Test
 60 |     @DisplayName("Note content is used in AI generation call")
 61 |     void noteContentIsUsed() {
 62 |         Student student = new Student("Test", "test@test.com", "pass");
 63 |         student.setId(1L);
 64 |         Note note = new Note(student, "Test", "Sample content for testing");
 65 |         note.setId(1L);
 66 |         when(studentRepository.findByEmail("test@test.com")).thenReturn(Optional.of(student));
 67 |         when(noteRepository.findByIdAndStudentId(1L, 1L)).thenReturn(Optional.of(note));
 68 |
 69 |         // Will fail at API call since no real API key - this verifies the flow up to API call
 70 |         assertThrows(RuntimeException.class,
 71 |                 () -> openRouterService.generateStudyMaterials(1L, "test@test.com"));
 72 |         verify(noteRepository).findByIdAndStudentId(1L, 1L);
 73 |     }
 74 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 18–73:** Tests 'nonExistentNoteThrowsException', 'otherStudentCannotGenerate' (verifying 'findById' is never called without studentId), and 'noteContentIsUsed'.

> **How to Explain This Module in an Interview:** Confirms AI generation cannot be triggered on another student's note.

Module: QuizServiceTest (`backend/src/test/java/com/studyassistant/service/`

QuizServiceTest.java)

What This File Is (Beginner-Friendly Analogy): Unit and security tests for 'QuizService'.

Why It Is Used & Architectural Role: Tests 100%, 0%, and 60% quiz grading math, breakdown accuracy, DB persistence, and cross-student 403 Forbidden blocking.

Complete Source Code (Every Line Numbered):

```
  1 | package com.studyassistant.service;
  2 |
  3 | import com.studyassistant.dto.QuizAttemptRequest;
  4 | import com.studyassistant.dto.QuizAttemptResponse;
  5 | import com.studyassistant.exception.ForbiddenException;
  6 | import com.studyassistant.model.*;
  7 | import com.studyassistant.repository.*;
  8 | import org.junit.jupiter.api.BeforeEach;
  9 | import org.junit.jupiter.api.DisplayName;
 10 | import org.junit.jupiter.api.Test;
 11 | import org.junit.jupiter.api.extension.ExtendWith;
 12 | import org.mockito.InjectMocks;
 13 | import org.mockito.Mock;
 14 | import org.mockito.junit.jupiter.MockitoExtension;
 15 |
 16 | import java.util.Arrays;
 17 | import java.util.List;
 18 | import java.util.Optional;
 19 |
 20 | import static org.junit.jupiter.api.Assertions.*;
 21 | import static org.mockito.ArgumentMatchers.any;
 22 | import static org.mockito.Mockito.*;
 23 |
 24 | @ExtendWith(MockitoExtension.class)
 25 | class QuizServiceTest {
 26 |
 27 |     @Mock private QuizRepository quizRepository;
 28 |     @Mock private QuizQuestionRepository quizQuestionRepository;
 29 |     @Mock private QuizAttemptRepository quizAttemptRepository;
 30 |     @Mock private StudentRepository studentRepository;
 31 |     @InjectMocks private QuizService quizService;
 32 |
 33 |     private Student student;
 34 |     private Quiz quiz;
 35 |     private Note note;
 36 |     private List<QuizQuestion> questions;
 37 |
 38 |     @BeforeEach
 39 |     void setUp() {
 40 |         student = new Student("Test", "test@example.com", "pass");
 41 |         student.setId(1L);
 42 |         note = new Note(student, "Test Note", "content");
 43 |         note.setId(1L);
 44 |         quiz = new Quiz(note, "Test Quiz");
 45 |         quiz.setId(1L);
 46 |
 47 |         questions = Arrays.asList(
 48 |             createQuestion(1L, "Q1", "A", "B", "C", "D", "A"),
 49 |             createQuestion(2L, "Q2", "A", "B", "C", "D", "B"),
 50 |             createQuestion(3L, "Q3", "A", "B", "C", "D", "C"),
 51 |             createQuestion(4L, "Q4", "A", "B", "C", "D", "D"),
 52 |             createQuestion(5L, "Q5", "A", "B", "C", "D", "A")
 53 |         );
 54 |     }
 55 |
 56 |     private QuizQuestion createQuestion(Long id, String text, String a, String b, String c, String d, String
correct) {
 57 |         QuizQuestion q = new QuizQuestion(quiz, text, a, b, c, d, correct);
 58 |         q.setId(id);
 59 |         return q;
 60 |     }
 61 |
 62 |     private QuizAttemptRequest buildRequest(String... selectedOptions) {
 63 |         List<QuizAttemptRequest.AnswerDTO> answers = new java.util.ArrayList<>();
 64 |         for (int i = 0; i < selectedOptions.length; i++) {
 65 |             answers.add(new QuizAttemptRequest.AnswerDTO((long) (i + 1), selectedOptions[i]));
 66 |         }
 67 |         return new QuizAttemptRequest(1L, answers);
 68 |     }
 69 |
 70 |     @Test
 71 |     @DisplayName("All correct answers - score equals totalQuestions")
 72 |     void allCorrectAnswers() {
 73 |         when(studentRepository.findByEmail("test@example.com")).thenReturn(Optional.of(student));
```

```
 74 |          when(quizRepository.findById(1L)).thenReturn(Optional.of(quiz));
 75 |          when(quizQuestionRepository.findByQuizId(1L)).thenReturn(questions);
 76 |          when(quizAttemptRepository.save(any())).thenReturn(new QuizAttempt());
 77 |
 78 |          QuizAttemptResponse res = quizService.submitAttempt("test@example.com", buildRequest("A", "B", "C",
"D", "A"));
 79 |
 80 |          assertEquals(5, res.getScore());
 81 |          assertEquals(5, res.getTotalQuestions());
 82 |          assertEquals(100.0, res.getPercentage());
 83 |          assertTrue(res.isPassed());
 84 |      }
 85 |
 86 |      @Test
 87 |      @DisplayName("All wrong answers – score equals 0")
 88 |      void allWrongAnswers() {
 89 |          when(studentRepository.findByEmail("test@example.com")).thenReturn(Optional.of(student));
 90 |          when(quizRepository.findById(1L)).thenReturn(Optional.of(quiz));
 91 |          when(quizQuestionRepository.findByQuizId(1L)).thenReturn(questions);
 92 |          when(quizAttemptRepository.save(any())).thenReturn(new QuizAttempt());
 93 |
 94 |          QuizAttemptResponse res = quizService.submitAttempt("test@example.com", buildRequest("D", "D", "D",
"A", "D"));
 95 |
 96 |          assertEquals(0, res.getScore());
 97 |          assertFalse(res.isPassed());
 98 |      }
 99 |
100 |      @Test
101 |      @DisplayName("Partial correct answers – correct score calculated")
102 |      void partialCorrectAnswers() {
103 |          when(studentRepository.findByEmail("test@example.com")).thenReturn(Optional.of(student));
104 |          when(quizRepository.findById(1L)).thenReturn(Optional.of(quiz));
105 |          when(quizQuestionRepository.findByQuizId(1L)).thenReturn(questions);
106 |          when(quizAttemptRepository.save(any())).thenReturn(new QuizAttempt());
107 |
108 |          QuizAttemptResponse res = quizService.submitAttempt("test@example.com", buildRequest("A", "B", "A",
"A", "A"));
109 |
110 |          assertEquals(3, res.getScore());
111 |          assertEquals(60.0, res.getPercentage());
112 |          assertTrue(res.isPassed());
113 |      }
114 |
115 |      @Test
116 |      @DisplayName("Attempt saved to database with correct percentage")
117 |      void attemptSavedToDatabase() {
118 |          when(studentRepository.findByEmail("test@example.com")).thenReturn(Optional.of(student));
119 |          when(quizRepository.findById(1L)).thenReturn(Optional.of(quiz));
120 |          when(quizQuestionRepository.findByQuizId(1L)).thenReturn(questions);
121 |          when(quizAttemptRepository.save(any())).thenReturn(new QuizAttempt());
122 |
123 |          quizService.submitAttempt("test@example.com", buildRequest("A", "B", "C", "D", "A"));
124 |
125 |          verify(quizAttemptRepository).save(any(QuizAttempt.class));
126 |      }
127 |
128 |      @Test
129 |      @DisplayName("Breakdown contains correct and incorrect info")
130 |      void breakdownContainsCorrectInfo() {
131 |          when(studentRepository.findByEmail("test@example.com")).thenReturn(Optional.of(student));
132 |          when(quizRepository.findById(1L)).thenReturn(Optional.of(quiz));
133 |          when(quizQuestionRepository.findByQuizId(1L)).thenReturn(questions);
134 |          when(quizAttemptRepository.save(any())).thenReturn(new QuizAttempt());
135 |
136 |          QuizAttemptResponse res = quizService.submitAttempt("test@example.com", buildRequest("A", "A", "C",
"D", "A"));
137 |
138 |          assertEquals(4,
res.getBreakdown().stream().filter(QuizAttemptResponse.QuestionBreakdown::isCorrect).count());
139 |          assertEquals(1, res.getBreakdown().stream().filter(b -> !b.isCorrect()).count());
140 |      }
141 |
142 |      @Test
143 |      @DisplayName("Attempting a quiz owned by another student – throws ForbiddenException")
144 |      void attemptOtherStudentsQuizThrowsForbidden() {
145 |          Student other = new Student("Other", "other@example.com", "pass");
146 |          other.setId(2L);
147 |          note.setStudent(other);
148 |
149 |          when(studentRepository.findByEmail("test@example.com")).thenReturn(Optional.of(student));
150 |          when(quizRepository.findById(1L)).thenReturn(Optional.of(quiz));
151 |
152 |          assertThrows(ForbiddenException.class, () ->
```

```
153 |                 quizService.submitAttempt("test@example.com", buildRequest("A", "B", "C", "D", "A")));
154 |         verify(quizAttemptRepository, never()).save(any());
155 |     }
156 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 20–149:** Tests 'allCorrectAnswers', 'allWrongAnswers', 'partialCorrectAnswers', 'attemptSavedToDatabase', 'breakdownContainsCorrectInfo', and 'attemptOtherStudentsQuizThrowsForbidden'.

> **How to Explain This Module in an Interview:** Comprehensive test coverage of grading edge cases and 403 authorization.

Part VIII: Frontend Config, API, Context, Hooks, Utils & Styles (Every Line Explained)

Module: Frontend Package Manifest (`frontend/package.json`)

What This File Is (Beginner-Friendly Analogy): Declares React 18, Vite 5, Tailwind CSS 3, Chart.js 4, Axios, and React Router dependencies and build scripts.

Why It Is Used & Architectural Role: Manages frontend packages and npm scripts ('dev', 'build', 'preview').

Complete Source Code (Every Line Numbered):

```
 1 | {
 2 |   "name": "ai-study-assistant-frontend",
 3 |   "private": true,
 4 |   "version": "1.0.0",
 5 |   "type": "module",
 6 |   "scripts": {
 7 |     "dev": "vite",
 8 |     "build": "vite build",
 9 |     "preview": "vite preview"
10 |   },
11 |   "dependencies": {
12 |     "axios": "^1.7.2",
13 |     "chart.js": "^4.5.1",
14 |     "react": "^18.3.1",
15 |     "react-chartjs-2": "^5.3.1",
16 |     "react-dom": "^18.3.1",
17 |     "react-router-dom": "^6.23.1"
18 |   },
19 |   "devDependencies": {
20 |     "@types/react": "^18.3.3",
21 |     "@types/react-dom": "^18.3.0",
22 |     "@vitejs/plugin-react": "^4.3.1",
23 |     "autoprefixer": "^10.4.19",
24 |     "postcss": "^8.4.38",
25 |     "tailwindcss": "^3.4.4",
26 |     "vite": "^5.3.1"
27 |   }
28 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–27:** Defines 'dependencies' (axios, chart.js, react, react-chartjs-2, react-dom, react-router-dom) and 'devDependencies' (vite, tailwindcss, postcss, autoprefixer, @vitejs/plugin-react).

> **How to Explain This Module in an Interview:** Clean, minimal dependency tree with zero bloat.

Module: Vite Configuration (`frontend/vite.config.js`)

What This File Is (Beginner-Friendly Analogy): Configures the Vite dev server on port 5173 and proxies '/api' to localhost:8080.

Why It Is Used & Architectural Role: Connects the React dev server to the Spring Boot backend locally.

Complete Source Code (Every Line Numbered):

```
1 | import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
      }
    }
  }
})
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–15:** Configures '@vitejs/plugin-react', 'port: 5173', and '/api' proxy to 'http://localhost:8080'.

> **How to Explain This Module in an Interview:** Avoids local CORS issues when using relative '/api' paths.

Module: PostCSS Configuration (`frontend/postcss.config.js`)

What This File Is (Beginner-Friendly Analogy): Hooks Tailwind CSS and Autoprefixer into Vite's CSS build pipeline.

Why It Is Used & Architectural Role: Processes '@tailwind' directives and adds vendor prefixes.

Complete Source Code (Every Line Numbered):

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–6:** Enables 'tailwindcss' and 'autoprefixer' plugins.

> **How to Explain This Module in an Interview:** Standard PostCSS pipeline for Tailwind CSS 3.

Module: Tailwind CSS Config (`frontend/tailwind.config.js`)

What This File Is (Beginner-Friendly Analogy): Defines custom colors, fonts, animations, and keyframes.

Why It Is Used & Architectural Role: Provides design tokens for the dark-mode aurora theme.

Complete Source Code (Every Line Numbered):

```
export default {
  darkMode: 'class',
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}"
  ],
  theme: {
    extend: {
      colors: {
        // Primary: violet/purple
        primary: {
          50:  '#f5f3ff',
          100: '#ede9fe',
          200: '#ddd6fe',
          300: '#c4b5fd',
          400: '#a78bfa',
          500: '#8b5cf6',
          600: '#7c3aed',
          700: '#6d28d9',
          800: '#5b21b6',
          900: '#4c1d95',
          950: '#2e1065',
        },
        // Cyan accent
        cyan: {
          50:  '#ecfeff',
          100: '#cffafe',
          200: '#a5f3fc',
```

```
 29 |           300: '#67e8f9',
 30 |           400: '#22d3ee',
 31 |           500: '#06b6d4',
 32 |           600: '#0891b2',
 33 |           700: '#0e7490',
 34 |           800: '#155e75',
 35 |           900: '#164e63',
 36 |         },
 37 |         // Pink accent
 38 |         pink: {
 39 |           400: '#f472b6',
 40 |           500: '#ec4899',
 41 |           600: '#db2777',
 42 |         },
 43 |         // Surface (near-black for cards/glass)
 44 |         surface: {
 45 |           50:  '#f8fafc',
 46 |           100: '#f1f5f9',
 47 |           200: '#e2e8f0',
 48 |           300: '#cbd5e1',
 49 |           400: '#94a3b8',
 50 |           500: '#64748b',
 51 |           600: '#475569',
 52 |           700: '#334155',
 53 |           800: '#1e293b',
 54 |           900: '#0f172a',
 55 |           950: '#05060f',
 56 |         },
 57 |         // Accent shorthands
 58 |         violet: '#8b5cf6',
 59 |         aurora: {
 60 |           purple: '#7c3aed',
 61 |           cyan:   '#06b6d4',
 62 |           pink:   '#ec4899',
 63 |         }
 64 |       },
 65 |       fontFamily: {
 66 |         sans:    ['Inter', 'system-ui', '-apple-system', 'sans-serif'],
 67 |         display: ['Space Grotesk', 'Inter', 'system-ui', 'sans-serif'],
 68 |       },
 69 |       animation: {
 70 |         'fade-in':       'fadeIn 0.5s ease-out both',
 71 |         'slide-up':      'slideUp 0.55s ease-out both',
 72 |         'slide-in-right':'slideInRight 0.35s ease-out both',
 73 |         'glow-pulse':    'glowPulse 3s ease-in-out infinite',
 74 |         'float':         'float 6s ease-in-out infinite',
 75 |       },
 76 |       keyframes: {
 77 |         fadeIn:       { '0%': { opacity: '0' }, '100%': { opacity: '1' } },
 78 |         slideUp:      { '0%': { opacity: '0', transform: 'translateY(24px)' }, '100%': { opacity: '1',
transform: 'translateY(0)' } },
 79 |         slideInRight: { '0%': { opacity: '0', transform: 'translateX(20px)' }, '100%': { opacity: '1',
transform: 'translateX(0)' } },
 80 |         glowPulse:    { '0%,100%': { boxShadow: '0 0 20px rgba(139,92,246,0.2)' }, '50%': { boxShadow: '0 0
40px rgba(139,92,246,0.45)' } },
 81 |         float:        { '0%,100%': { transform: 'translateY(0)' }, '50%': { transform: 'translateY(-10px)' } },
 82 |       },
 83 |       backdropBlur: { xs: '2px', xl: '20px', '2xl': '40px' },
 84 |     }
 85 |   },
 86 |   plugins: []
 87 | };
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–82:** Configures 'darkMode: class', content globs, custom 'primary'/'cyan'/'pink'/'surface'/'aurora' colors, 'Inter'/'Space Grotesk' fonts, and keyframe animations.

> **How to Explain This Module in an Interview:** Custom design system built on Tailwind utility classes.

Module: Vercel SPA Config (`frontend/vercel.json`)

What This File Is (Beginner-Friendly Analogy): Rewrites all URLs to '/index.html' on Vercel.

Why It Is Used & Architectural Role: Prevents 404 errors when refreshing React Router client-side routes in production.

Complete Source Code (Every Line Numbered):

```
  1 | {
  2 |   "rewrites": [
  3 |     {
```

```
  4 |       "source": "/(.*)",
  5 |       "destination": "/index.html"
  6 |     }
  7 |   ]
  8 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–8:** Rewrites '/(.*)' to '/index.html'.

> **How to Explain This Module in an Interview:** Essential for deploying any Single-Page Application on static hosting.

Module: HTML Entrypoint (`frontend/index.html`)

What This File Is (Beginner-Friendly Analogy): The single HTML page served to the browser, loading Google Fonts ('Inter' & 'Space Grotesk') and '/src/main.jsx'.

Why It Is Used & Architectural Role: Provides the '

' mount point for React 18.

Complete Source Code (Every Line Numbered):

```
  1 | <!DOCTYPE html>
  2 | <html lang="en">
  3 |   <head>
  4 |     <meta charset="UTF-8" />
  5 |     <link rel="icon" type="image/svg+xml" href="/vite.svg" />
  6 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  7 |     <meta name="description" content="AI Study Assistant - Upload notes, generate AI summaries, flashcards &
quizzes, track your progress" />
  8 |     <link rel="preconnect" href="https://fonts.googleapis.com">
  9 |     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
 10 |     <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700;800&family=Space+Grotesk:wght@400;500;600;700&display=swap" rel="stylesheet">
 11 |     <title>AI Study Assistant</title>
 12 |   </head>
 13 |   <body>
 14 |     <div id="root"></div>
 15 |     <script type="module" src="/src/main.jsx"></script>
 16 |   </body>
 17 | </html>
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–18:** Loads Google Fonts, sets viewport and meta description, and mounts '/src/main.jsx' as an ES module.

> **How to Explain This Module in an Interview:** Single HTML shell of the SPA.

Module: React Entrypoint (main.jsx) (`frontend/src/main.jsx`)

What This File Is (Beginner-Friendly Analogy): Mounts '' wrapped in '' and '' into '#root'.

Why It Is Used & Architectural Role: Bootstraps the React 18 component tree and global CSS.

Complete Source Code (Every Line Numbered):

```
  1 | import React from 'react';
  2 | import ReactDOM from 'react-dom/client';
  3 | import App from './App';
  4 | import ErrorBoundary from './components/ErrorBoundary';
  5 | import './styles/index.css';
  6 |
  7 | ReactDOM.createRoot(document.getElementById('root')).render(
  8 |   <React.StrictMode>
  9 |     <ErrorBoundary>
 10 |       <App />
 11 |     </ErrorBoundary>
 12 |   </React.StrictMode>
 13 | );
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–14:** Imports React, ReactDOM, App, ErrorBoundary, and './styles/index.css', and calls 'ReactDOM.createRoot(...).render(...)'.

How to Explain This Module in an Interview: Uses React 18 concurrent root API.

Module: App Router (App.jsx) (`frontend/src/App.jsx`)

What This File Is (Beginner-Friendly Analogy): Defines all public and protected routes using React Router v6.

Why It Is Used & Architectural Role: Wraps the app in " and nests 12 private routes inside ".

Complete Source Code (Every Line Numbered):

```
 1 | import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
 2 | import { AuthProvider } from './context/AuthContext';
 3 | import Layout from './components/Layout';
 4 | import ProtectedRoute from './components/ProtectedRoute';
 5 | import LoginPage from './pages/LoginPage';
 6 | import RegisterPage from './pages/RegisterPage';
 7 | import DashboardPage from './pages/DashboardPage';
 8 | import NotesPage from './pages/NotesPage';
 9 | import StudyPage from './pages/StudyPage';
10 | import QuizPage from './pages/QuizPage';
11 | import ResultPage from './pages/ResultPage';
12 | import ProgressPage from './pages/ProgressPage';
13 | import AnalyticsPage from './pages/AnalyticsPage';
14 | import HistoryPage from './pages/HistoryPage';
15 | import QuizzesPage from './pages/QuizzesPage';
16 | import FlashcardsPage from './pages/FlashcardsPage';
17 | import AchievementsPage from './pages/AchievementsPage';
18 | import SettingsPage from './pages/SettingsPage';
19 |
20 | export default function App() {
21 |   return (
22 |     <AuthProvider>
23 |       <Router>
24 |         <Routes>
25 |           <Route path="/login" element={<LoginPage />} />
26 |           <Route path="/register" element={<RegisterPage />} />
27 |
28 |           <Route element={<ProtectedRoute><Layout /></ProtectedRoute>}>
29 |             <Route path="/dashboard" element={<DashboardPage />} />
30 |             <Route path="/notes" element={<NotesPage />} />
31 |             <Route path="/history" element={<HistoryPage />} />
32 |             <Route path="/study/:noteId" element={<StudyPage />} />
33 |             <Route path="/quiz/:quizId" element={<QuizPage />} />
34 |             <Route path="/result/:quizId" element={<ResultPage />} />
35 |             <Route path="/quizzes" element={<QuizzesPage />} />
36 |             <Route path="/flashcards" element={<FlashcardsPage />} />
37 |             <Route path="/progress" element={<ProgressPage />} />
38 |             <Route path="/analytics" element={<AnalyticsPage />} />
39 |             <Route path="/achievements" element={<AchievementsPage />} />
40 |             <Route path="/settings" element={<SettingsPage />} />
41 |           </Route>
42 |
43 |           <Route path="/" element={<Navigate to="/dashboard" replace />} />
44 |           <Route path="*" element={<Navigate to="/dashboard" replace />} />
45 |         </Routes>
46 |       </Router>
47 |     </AuthProvider>
48 |   );
49 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–48:** Declares '/login', '/register', protected layout routes for all study/quiz/analytics pages, and wildcard fallback redirects to '/dashboard'.

How to Explain This Module in an Interview: Uses React Router v6 layout routes with ".

Module: Axios Client & Interceptors (`frontend/src/api/axios.js`)

What This File Is (Beginner-Friendly Analogy): Creates the shared Axios instance and attaches JWT request/response interceptors.

Why It Is Used & Architectural Role: Automatically injects 'Authorization: Bearer ' and handles 401 token expiration.

Complete Source Code (Every Line Numbered):

```
1 | import axios from 'axios';
```

```
  2 |
  3 | const API = axios.create({
  4 |   baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8080',
  5 |   headers: {
  6 |     'Content-Type': 'application/json',
  7 |   },
  8 | });
  9 |
 10 | // Request interceptor — attach JWT token to every request
 11 | API.interceptors.request.use(
 12 |   (config) => {
 13 |     const token = localStorage.getItem('token');
 14 |     if (token) {
 15 |       config.headers.Authorization = `Bearer ${token}`;
 16 |     }
 17 |     return config;
 18 |   },
 19 |   (error) => Promise.reject(error)
 20 | );
 21 |
 22 | // Response interceptor — handle 401 (expired/invalid token)
 23 | API.interceptors.response.use(
 24 |   (response) => response,
 25 |   (error) => {
 26 |     if (error.response && error.response.status === 401) {
 27 |       const url = error.config?.url || '';
 28 |       if (!url.includes('/api/auth/')) {
 29 |         localStorage.removeItem('token');
 30 |         localStorage.removeItem('user');
 31 |         window.location.href = '/login';
 32 |       }
 33 |     }
 34 |     return Promise.reject(error);
 35 |   }
 36 | );
 37 |
 38 | export default API;
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–38:** Creates Axios instance with 'VITE_API_URL || http://localhost:8080', attaches Bearer token in request interceptor, and clears storage + redirects to '/login' on 401 responses (excluding '/api/auth/').

> **How to Explain This Module in an Interview:** Centralizes authentication headers and 401 session expiry handling.

Module: Dashboard API Helper (`frontend/src/api/dashboard.js`)

What This File Is (Beginner-Friendly Analogy): Helper function calling 'GET /api/dashboard?page=0&size=7'.

Why It Is Used & Architectural Role: Fetches paginated dashboard data for DashboardPage.jsx.

Complete Source Code (Every Line Numbered):

```
  1 | import API from './axios';
  2 |
  3 | /**
  4 |  * Fetch paginated dashboard data for the authenticated student.
  5 |  * @param {number} page - zero-based page index
  6 |  * @param {number} size - number of entries per page
  7 |  * @returns {Promise<Object>} DashboardResponse payload
  8 |  */
  9 | export const fetchDashboard = async (page = 0, size = 7) => {
 10 |   const response = await API.get(`/api/dashboard?page=${page}&size=${size}`);
 11 |   return response.data;
 12 | };
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–12:** Exports 'fetchDashboard(page = 0, size = 7)' returning 'response.data'.

> **How to Explain This Module in an Interview:** Clean API abstraction for paginated dashboard requests.

Module: AuthContext (`frontend/src/context/AuthContext.jsx`)

What This File Is (Beginner-Friendly Analogy): Provides global authentication state ('user', 'token', 'loading', 'isAuthenticated', 'login', 'logout').

Why It Is Used & Architectural Role: Synchronizes React auth state with 'localStorage'.

Complete Source Code (Every Line Numbered):

```
1  | import { createContext, useContext, useState, useEffect } from 'react';
2  |
3  | const AuthContext = createContext(null);
4  |
5  | export function AuthProvider({ children }) {
6  |   const [user, setUser] = useState(null);
7  |   const [token, setToken] = useState(null);
8  |   const [loading, setLoading] = useState(true);
9  |
10 |   useEffect(() => {
11 |     const savedToken = localStorage.getItem('token');
12 |     const savedUser = localStorage.getItem('user');
13 |     if (savedToken && savedUser) {
14 |       setToken(savedToken);
15 |       setUser(JSON.parse(savedUser));
16 |     }
17 |     setLoading(false);
18 |   }, []);
19 |
20 |   const login = (authResponse) => {
21 |     const { token: newToken, studentId, name } = authResponse;
22 |     const userData = { studentId, name };
23 |     localStorage.setItem('token', newToken);
24 |     localStorage.setItem('user', JSON.stringify(userData));
25 |     setToken(newToken);
26 |     setUser(userData);
27 |   };
28 |
29 |   const logout = () => {
30 |     localStorage.removeItem('token');
31 |     localStorage.removeItem('user');
32 |     setToken(null);
33 |     setUser(null);
34 |   };
35 |
36 |   const isAuthenticated = !!token;
37 |
38 |   return (
39 |     <AuthContext.Provider value={{ user, token, loading, isAuthenticated, login, logout }}>
40 |       {children}
41 |     </AuthContext.Provider>
42 |   );
43 | }
44 |
45 | export function useAuth() {
46 |   const context = useContext(AuthContext);
47 |   if (!context) {
48 |     throw new Error('useAuth must be used within an AuthProvider');
49 |   }
50 |   return context;
51 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–51:** Hydrates saved token/user on mount, defines 'login' and 'logout' helpers, and exports the 'useAuth()' hook.

How to Explain This Module in an Interview: Eliminates prop-drilling of user/token across the component tree.

Module: useFetch Hook (`frontend/src/hooks/useFetch.js`)

What This File Is (Beginner-Friendly Analogy): Custom React hook for declarative GET requests with 'data', 'error', 'loading', and 'refetch'.

Why It Is Used & Architectural Role: Used by AnalyticsPage.jsx to fetch '/api/analytics'.

Complete Source Code (Every Line Numbered):

```
1 | import { useState, useEffect, useCallback } from 'react';
2 | import API from '../api/axios';
3 |
4 | export default function useFetch(url, options = {}) {
5 |   const [data, setData] = useState(null);
6 |   const [error, setError] = useState(null);
7 |   const [loading, setLoading] = useState(true);
8 |
9 |   const fetchData = useCallback(async () => {
```

```
    setLoading(true);
    setError(null);
    try {
      const res = await API.get(url, options);
      setData(res.data);
    } catch (err) {
      const message = err.response?.data?.message || err.message || 'Request failed';
      setError(message);
    } finally {
      setLoading(false);
    }
  }, [url]);

  useEffect(() => {
    fetchData();
  }, [fetchData]);

  return { data, error, loading, refetch: fetchData };
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–29:** Uses 'useState', 'useCallback', and 'useEffect' to manage async request lifecycle.

> **How to Explain This Module in an Interview:** Reusable custom hook pattern.

Module: Date Formatting Utility (`frontend/src/utils/formatDate.js`)

What This File Is (Beginner-Friendly Analogy): Formats ISO timestamp strings into readable dates ('Aug 9, 2026') and date-times.

Why It Is Used & Architectural Role: Used across HistoryPage, QuizzesPage, and FlashcardsPage.

Complete Source Code (Every Line Numbered):

```
export function formatDate(dateStr, opts = {}) {
  if (!dateStr) return '';
  const d = new Date(dateStr);
  const defaults = { day: 'numeric', month: 'short', year: 'numeric' };
  return d.toLocaleDateString('en-US', { ...defaults, ...opts });
}

export function formatDateTime(dateStr) {
  if (!dateStr) return '';
  return new Date(dateStr).toLocaleString('en-US', {
    day: 'numeric', month: 'short', year: 'numeric',
    hour: 'numeric', minute: '2-digit',
  });
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–14:** Exports 'formatDate(dateStr, opts)' and 'formatDateTime(dateStr)'.

> **How to Explain This Module in an Interview:** Standardizes date formatting across the UI.

Module: Quiz Utility (`frontend/src/utils/quizUtils.js`)

What This File Is (Beginner-Friendly Analogy): Strips non-question fields (like 'correctOption') when preparing stored questions for a quiz retake.

Why It Is Used & Architectural Role: Used by QuizzesPage.jsx before navigating to '/quiz/:quizId'.

Complete Source Code (Every Line Numbered):

```
export function stripQuestionFields(questions) {
  if (!questions?.length) return [];
  return questions.map(({ id, questionText, optionA, optionB, optionC, optionD }) => ({
    id, questionText, optionA, optionB, optionC, optionD,
  }));
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–6:** Maps questions to '{ id, questionText, optionA, optionB, optionC, optionD }'.

How to Explain This Module in an Interview: Ensures clean state payloads when retaking quizzes.

Module: Global Stylesheet & Liquid Glass Design (`frontend/src/styles/index.css`)

What This File Is (Beginner-Friendly Analogy): Defines the aurora background, liquid glass cards, buttons, inputs, 3D flashcard flip classes, quiz option states, and responsive media queries.

Why It Is Used & Architectural Role: Implements the visual design system and mobile responsiveness.

Complete Source Code (Every Line Numbered):

```
  1 | @tailwind base;
  2 | @tailwind components;
  3 | @tailwind utilities;
  4 |
  5 | @layer base {
  6 |   * {
  7 |     margin: 0;
  8 |     padding: 0;
  9 |     box-sizing: border-box;
 10 |   }
 11 |
 12 |   body {
 13 |     font-family: 'Inter', system-ui, -apple-system, sans-serif;
 14 |     background: #05060f;
 15 |     color: #e2e8f0;
 16 |     min-height: 100vh;
 17 |     -webkit-font-smoothing: antialiased;
 18 |     -moz-osx-font-smoothing: grayscale;
 19 |     overflow-x: hidden;
 20 |   }
 21 |
 22 |   /* Aurora background */
 23 |   body::before {
 24 |     content: '';
 25 |     position: fixed;
 26 |     inset: 0;
 27 |     background:
 28 |       radial-gradient(ellipse 80% 60% at 20% -10%, rgba(124, 58, 237, 0.35) 0%, transparent 60%),
 29 |       radial-gradient(ellipse 60% 50% at 80% 10%, rgba(6, 182, 212, 0.25) 0%, transparent 55%),
 30 |       radial-gradient(ellipse 70% 70% at 50% 100%, rgba(99, 38, 196, 0.3) 0%, transparent 65%),
 31 |       radial-gradient(ellipse 50% 40% at 10% 70%, rgba(236, 72, 153, 0.15) 0%, transparent 50%),
 32 |       linear-gradient(135deg, #05060f 0%, #0d0a1e 40%, #050f1e 100%);
 33 |     z-index: -1;
 34 |     pointer-events: none;
 35 |   }
 36 |
 37 |   ::selection {
 38 |     background-color: rgba(139, 92, 246, 0.4);
 39 |     color: #ede9fe;
 40 |   }
 41 |
 42 |   ::-webkit-scrollbar { width: 5px; }
 43 |   ::-webkit-scrollbar-track { background: rgba(255,255,255,0.03); }
 44 |   ::-webkit-scrollbar-thumb { background: rgba(139, 92, 246, 0.4); border-radius: 10px; }
 45 |   ::-webkit-scrollbar-thumb:hover { background: rgba(139, 92, 246, 0.6); }
 46 | }
 47 |
 48 | @layer components {
 49 |
 50 |   /* ===== LIQUID GLASS CARD ===== */
 51 |   .glass-card {
 52 |     position: relative;
 53 |     background: rgba(255, 255, 255, 0.04);
 54 |     backdrop-filter: blur(24px) saturate(180%);
 55 |     -webkit-backdrop-filter: blur(24px) saturate(180%);
 56 |     border: 1px solid rgba(255, 255, 255, 0.10);
 57 |     border-radius: 20px;
 58 |     box-shadow:
 59 |       0 4px 24px rgba(0, 0, 0, 0.35),
 60 |       0 1px 0 rgba(255, 255, 255, 0.08) inset,
 61 |       0 -1px 0 rgba(0, 0, 0, 0.2) inset;
 62 |     overflow: hidden;
 63 |     -webkit-mask-image: -webkit-radial-gradient(white, black);
 64 |     transform: translateZ(0);
 65 |     transition: all 0.3s ease;
 66 |   }
 67 |
 68 |   .glass-card::before {
 69 |     content: '';
 70 |     position: absolute;
```

```
    inset: 0;
    background: linear-gradient(
      135deg,
      rgba(255, 255, 255, 0.07) 0%,
      transparent 50%,
      rgba(255, 255, 255, 0.02) 100%
    );
    border-radius: inherit;
    pointer-events: none;
  }

  .glass-card:hover {
    border-color: rgba(139, 92, 246, 0.30);
    box-shadow:
      0 8px 40px rgba(0, 0, 0, 0.4),
      0 0 0 1px rgba(139, 92, 246, 0.15) inset,
      0 1px 0 rgba(255, 255, 255, 0.10) inset;
    transform: translateY(-2px) translateZ(0);
  }

  /* ===== BUTTONS ===== */
  .btn-primary {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 0.5rem;
    padding: 0.65rem 1.5rem;
    background: linear-gradient(135deg, #7c3aed 0%, #6d28d9 50%, #5b21b6 100%);
    color: #fff;
    font-weight: 600;
    font-size: 0.9rem;
    border-radius: 12px;
    border: 1px solid rgba(139, 92, 246, 0.4);
    box-shadow: 0 4px 16px rgba(124, 58, 237, 0.35), 0 1px 0 rgba(255,255,255,0.1) inset;
    cursor: pointer;
    transition: all 0.25s ease;
    text-decoration: none;
    white-space: nowrap;
  }
  .btn-primary:hover {
    background: linear-gradient(135deg, #8b5cf6 0%, #7c3aed 50%, #6d28d9 100%);
    box-shadow: 0 6px 24px rgba(124, 58, 237, 0.5), 0 1px 0 rgba(255,255,255,0.15) inset;
    transform: translateY(-1px);
  }
  .btn-primary:active { transform: translateY(0); }
  .btn-primary:disabled {
    opacity: 0.45;
    cursor: not-allowed;
    transform: none;
  }

  .btn-secondary {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 0.5rem;
    padding: 0.65rem 1.5rem;
    background: rgba(255, 255, 255, 0.06);
    color: #c4b5fd;
    font-weight: 600;
    font-size: 0.9rem;
    border-radius: 12px;
    border: 1px solid rgba(139, 92, 246, 0.25);
    cursor: pointer;
    transition: all 0.25s ease;
    text-decoration: none;
    backdrop-filter: blur(12px);
  }
  .btn-secondary:hover {
    background: rgba(139, 92, 246, 0.12);
    border-color: rgba(139, 92, 246, 0.45);
    color: #ede9fe;
    transform: translateY(-1px);
  }

  .btn-danger {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    padding: 0.65rem 1.5rem;
    background: linear-gradient(135deg, #be123c, #e11d48);
    color: #fff;
    font-weight: 600;
    font-size: 0.9rem;
```

```
    border-radius: 12px;
    border: 1px solid rgba(244, 63, 94, 0.3);
    box-shadow: 0 4px 16px rgba(225, 29, 72, 0.3);
    cursor: pointer;
    transition: all 0.25s ease;
  }
  .btn-danger:hover {
    background: linear-gradient(135deg, #e11d48, #f43f5e);
    box-shadow: 0 6px 24px rgba(225, 29, 72, 0.45);
    transform: translateY(-1px);
  }

  /* ===== INPUT ===== */
  .input-field {
    width: 100%;
    padding: 0.75rem 1rem;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 12px;
    color: #e2e8f0;
    font-size: 0.95rem;
    backdrop-filter: blur(12px);
    transition: all 0.25s ease;
    outline: none;
  }
  .input-field::placeholder { color: rgba(148, 163, 184, 0.6); }
  .input-field:focus {
    border-color: rgba(139, 92, 246, 0.5);
    background: rgba(139, 92, 246, 0.06);
    box-shadow: 0 0 0 3px rgba(139, 92, 246, 0.15);
  }

  /* ===== SPINNER ===== */
  .spinner {
    width: 2.5rem;
    height: 2.5rem;
    border: 3px solid rgba(139, 92, 246, 0.15);
    border-top-color: #8b5cf6;
    border-radius: 50%;
    animation: spin 0.8s linear infinite;
  }
  @keyframes spin { to { transform: rotate(360deg); } }

  /* ===== FLASHCARD ===== */
  .flashcard-container {
    perspective: 1200px;
    width: 100%;
    max-width: 520px;
    height: 340px;
  }
  .flashcard-inner {
    position: relative;
    width: 100%;
    height: 100%;
    transition: transform 0.65s cubic-bezier(0.4, 0, 0.2, 1);
    transform-style: preserve-3d;
  }
  .flashcard-inner.flipped { transform: rotateY(180deg); }
  .flashcard-front,
  .flashcard-back {
    position: absolute;
    width: 100%;
    height: 100%;
    backface-visibility: hidden;
    -webkit-backface-visibility: hidden;
    border-radius: 20px;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 2rem;
    text-align: center;
    font-size: 1.1rem;
    line-height: 1.6;
  }
  .flashcard-front {
    background: linear-gradient(135deg, rgba(91, 33, 182, 0.6), rgba(109, 40, 217, 0.5));
    border: 1px solid rgba(139, 92, 246, 0.35);
    color: #ede9fe;
    box-shadow: 0 20px 60px rgba(109, 40, 217, 0.2);
    backdrop-filter: blur(20px);
  }
  .flashcard-back {
    background: linear-gradient(135deg, rgba(6, 78, 59, 0.6), rgba(4, 120, 87, 0.5));
    border: 1px solid rgba(16, 185, 129, 0.35);
```

```
    color: #d1fae5;
    transform: rotateY(180deg);
    box-shadow: 0 20px 60px rgba(16, 185, 129, 0.15);
    backdrop-filter: blur(20px);
  }

  /* ===== QUIZ OPTIONS ===== */
  .quiz-option {
    width: 100%;
    padding: 1rem;
    text-align: left;
    border-radius: 14px;
    border: 1px solid rgba(255, 255, 255, 0.08);
    background: rgba(255, 255, 255, 0.03);
    color: #cbd5e1;
    cursor: pointer;
    transition: all 0.2s ease;
    backdrop-filter: blur(12px);
  }
  .quiz-option:hover {
    border-color: rgba(139, 92, 246, 0.4);
    background: rgba(139, 92, 246, 0.08);
    color: #e2e8f0;
  }
  .quiz-option.selected {
    border-color: #8b5cf6;
    background: rgba(139, 92, 246, 0.15);
    color: #ede9fe;
  }
  .quiz-option.correct {
    border-color: #10b981;
    background: rgba(16, 185, 129, 0.12);
    color: #d1fae5;
  }
  .quiz-option.incorrect {
    border-color: #f43f5e;
    background: rgba(244, 63, 94, 0.10);
    color: #fecdd3;
  }

  /* ===== PROGRESS BAR ===== */
  .progress-bar {
    height: 6px;
    background: rgba(255, 255, 255, 0.06);
    border-radius: 99px;
    overflow: hidden;
  }
  .progress-bar-fill {
    height: 100%;
    background: linear-gradient(90deg, #7c3aed, #06b6d4);
    border-radius: 99px;
    transition: width 0.5s ease-out;
  }

  /* ===== TAB BUTTONS ===== */
  .tab-btn { padding: 0.55rem 1.25rem; font-weight: 500; border-radius: 10px; transition: all 0.25s; }
  .tab-btn.active {
    background: rgba(139, 92, 246, 0.18);
    color: #c4b5fd;
    border: 1px solid rgba(139, 92, 246, 0.35);
  }
  .tab-btn.inactive { color: #64748b; }
  .tab-btn.inactive:hover { color: #94a3b8; background: rgba(255,255,255,0.04); }
}

@layer utilities {
  .animate-fade-in { animation: fadeIn 0.5s ease-out both; }
  .animate-slide-up { animation: slideUp 0.55s ease-out both; }

  @keyframes fadeIn {
    from { opacity: 0; }
    to { opacity: 1; }
  }
  @keyframes slideUp {
    from { opacity: 0; transform: translateY(24px); }
    to { opacity: 1; transform: translateY(0); }
  }
}

/* Mobile responsive */
@media (max-width: 768px) {
  .app-sidebar {
    position: fixed !important;
    left: 0; top: 0;
```

```
323 |     z-index: 50;
324 |     transform: translateX(-100%);
325 |     transition: transform 0.25s ease;
326 |     width: 280px !important;
327 |   }
328 |   .app-sidebar[data-open] { transform: translateX(0); }
329 |   .sidebar-close-btn { display: block !important; }
330 |   .sidebar-backdrop { display: block !important; }
331 |   .mobile-menu-btn { display: flex !important; }
332 |   .header-search { display: none !important; }
333 |   .logo-mobile { display: block !important; }
334 | }
335 |
336 | @media (min-width: 769px) {
337 |   .sidebar-close-btn { display: none !important; }
338 |   .logo-mobile { display: none !important; }
339 | }
340 |
341 | /* Responsive grids */
342 | @media (max-width: 1024px) {
343 |   .dashboard-grid-2col { grid-template-columns: 1fr !important; }
344 | }
345 | @media (max-width: 900px) {
346 |   .analytics-2col { grid-template-columns: 1fr !important; }
347 |   .two-panel-grid { grid-template-columns: 1fr !important; }
348 |   .sidebar-note-list { grid-template-columns: 1fr !important; }
349 |   .sidebar-note-list > div:first-child { position: static !important; width: 100% !important; max-height: none
!important; }
350 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–46:** '@layer base': resets box-sizing, sets dark '#05060f' background, renders the fixed multi-radial aurora gradient in 'body::before', and styles custom scrollbars.
- **Lines 48–280:** '@layer components': defines '.glass-card', '.btn-primary', '.btn-secondary', '.btn-danger', '.input-field', '.spinner', '.flashcard-container' / '.flashcard-inner.flipped' (with 'perspective: 1200px', 'transform-style: preserve-3d', 'backface-visibility: hidden', 'rotateY(180deg)'), '.quiz-option', '.progress-bar', and '.tab-btn'.
- **Lines 282–330:** '@layer utilities' and '@media' queries for mobile drawer navigation ('max-width: 768px') and responsive 1-column grid collapse ('max-width: 1024px' / '900px').

How to Explain This Module in an Interview: Explain how 3D flashcards work in CSS: '.flashcard-container' sets 'perspective: 1200px', '.flashcard-inner' sets 'transform-style: preserve-3d' and rotates '180deg' on '.flipped', while '.flashcard-front' and '.flashcard-back' set 'backface-visibility: hidden' with the back face pre-rotated '180deg'.

Part IX: Frontend Components & Pages (`components/` & `pages/` — Every Line Explained)

Module: ErrorBoundary Component (`frontend/src/components/ErrorBoundary.jsx`)

What This File Is (Beginner-Friendly Analogy): React class component that catches unhandled JS errors in child components and renders a recovery screen.

Why It Is Used & Architectural Role: Prevents a runtime error in one chart or page from crashing the whole browser tab into a blank screen.

Complete Source Code (Every Line Numbered):

```
 1 | import { Component } from 'react';
 2 |
 3 | export default class ErrorBoundary extends Component {
 4 |   constructor(props) {
 5 |     super(props);
 6 |     this.state = { hasError: false, error: null };
 7 |   }
 8 |
 9 |   static getDerivedStateFromError(error) {
10 |     return { hasError: true, error };
11 |   }
12 |
13 |   componentDidCatch(error, errorInfo) {
14 |     console.error('ErrorBoundary caught:', error, errorInfo);
```

```
  }

  render() {
    if (this.state.hasError) {
      return (
        <div style={{
          display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: 'center',
          minHeight: '60vh', padding: '2rem', textAlign: 'center',
        }}>
          <div style={{ fontSize: '3rem', marginBottom: '1rem' }}>⚠ </div>
          <h2 style={{ fontSize: '1.3rem', fontWeight: 700, color: '#e2e8f0', margin: '0 0 0.5rem' }}>
            Something went wrong
          </h2>
          <p style={{ color: '#64748b', fontSize: '0.9rem', marginBottom: '1.5rem', maxWidth: '400px' }}>
            An unexpected error occurred. Please try refreshing the page.
          </p>
          <button
            onClick={() => {
              this.setState({ hasError: false, error: null });
              window.location.reload();
            }}
            style={{
              padding: '0.6rem 1.5rem', borderRadius: '10px', border: 'none',
              background: 'linear-gradient(135deg, #7c3aed, #06b6d4)',
              color: 'white', fontWeight: 600, cursor: 'pointer', fontSize: '0.9rem',
            }}
          >
            Reload page
          </button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–49:** Implements 'getDerivedStateFromError' and 'componentDidCatch', rendering a fallback UI with a 'Reload page' button when 'this.state.hasError' is true.

How to Explain This Module in an Interview: Production resilience pattern in React.

Module: FlashcardFlip Component (`frontend/src/components/FlashcardFlip.jsx`)

What This File Is (Beginner-Friendly Analogy): Interactive 3D flashcard carousel used on StudyPage.jsx.

Why It Is Used & Architectural Role: Allows flipping cards on click and stepping forward/backward with dot indicators.

Complete Source Code (Every Line Numbered):

```
import { useState } from 'react';

export default function FlashcardFlip({ flashcards }) {
  const [currentIndex, setCurrentIndex] = useState(0);
  const [isFlipped, setIsFlipped] = useState(false);

  if (!flashcards || flashcards.length === 0) {
    return <p className="text-surface-400 text-center">No flashcards available.</p>;
  }

  const card = flashcards[currentIndex];

  const handleNext = () => {
    setIsFlipped(false);
    setTimeout(() => {
      setCurrentIndex((prev) => (prev + 1) % flashcards.length);
    }, 150);
  };

  const handlePrev = () => {
    setIsFlipped(false);
    setTimeout(() => {
      setCurrentIndex((prev) => (prev - 1 + flashcards.length) % flashcards.length);
    }, 150);
  };

  return (
    <div className="flex flex-col items-center gap-6">
      {/* Card counter */}
      <div className="flex items-center gap-2 text-surface-400 text-sm">
```

```
 31 |          <span className="px-3 py-1 bg-surface-700/50 rounded-full">
 32 |            Card {currentIndex + 1} of {flashcards.length}
 33 |          </span>
 34 |        </div>
 35 |
 36 |        {/* Flashcard */}
 37 |        <div
 38 |          className="flashcard-container cursor-pointer"
 39 |          onClick={() => setIsFlipped(!isFlipped)}
 40 |        >
 41 |          <div className={`flashcard-inner ${isFlipped ? 'flipped' : ''}`}>
 42 |            <div className="flashcard-front">
 43 |              <div>
 44 |                <div className="text-xs uppercase tracking-wider text-primary-300/60 mb-4 font-
semibold">Question</div>
 45 |                <p className="text-lg">{card.question}</p>
 46 |                <div className="mt-4 text-xs text-primary-300/40">Click to reveal answer</div>
 47 |              </div>
 48 |            </div>
 49 |            <div className="flashcard-back">
 50 |              <div>
 51 |                <div className="text-xs uppercase tracking-wider text-emerald-300/60 mb-4 font-semibold">Answer</
div>
 52 |                <p className="text-lg">{card.answer}</p>
 53 |              </div>
 54 |            </div>
 55 |          </div>
 56 |        </div>
 57 |
 58 |        {/* Navigation arrows */}
 59 |        <div className="flex items-center gap-4">
 60 |          <button
 61 |            onClick={handlePrev}
 62 |            className="w-12 h-12 flex items-center justify-center rounded-xl bg-surface-700/50 border border-
surface-600/50 text-surface-300 hover:text-white hover:bg-surface-600/50 transition-all"
 63 |          >
 64 |            <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
 65 |              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M15 19l-7-7 7-7" />
 66 |            </svg>
 67 |          </button>
 68 |
 69 |          {/* Dot indicators */}
 70 |          <div className="flex gap-1.5">
 71 |            {flashcards.map((_, i) => (
 72 |              <button
 73 |                key={i}
 74 |                onClick={() => { setIsFlipped(false); setCurrentIndex(i); }}
 75 |                className={`w-2 h-2 rounded-full transition-all duration-300 ${
 76 |                  i === currentIndex ? 'bg-primary-500 w-6' : 'bg-surface-600 hover:bg-surface-500'
 77 |                }`}
 78 |              />
 79 |            ))}
 80 |          </div>
 81 |
 82 |          <button
 83 |            onClick={handleNext}
 84 |            className="w-12 h-12 flex items-center justify-center rounded-xl bg-surface-700/50 border border-
surface-600/50 text-surface-300 hover:text-white hover:bg-surface-600/50 transition-all"
 85 |          >
 86 |            <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
 87 |              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9 5l7 7-7 7" />
 88 |            </svg>
 89 |          </button>
 90 |        </div>
 91 |      </div>
 92 |    );
 93 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–92:** Manages 'currentIndex' and 'isFlipped', resets flip state with a 150ms timeout before changing index on Next/Prev, and renders front/back faces plus dot navigation.

> **How to Explain This Module in an Interview:** 150ms transition delay prevents exposing the next card's answer while unflipping.

Module: Layout Component (`frontend/src/components/Layout.jsx`)

What This File Is (Beginner-Friendly Analogy): Persistent app frame containing the Sidebar, mobile backdrop, top Header bar (search, logout, user initials avatar), and ".

Why It Is Used & Architectural Role: Wraps all authenticated pages.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState } from 'react';
  2 | import { Outlet, useNavigate } from 'react-router-dom';
  3 | import Sidebar from './Sidebar';
  4 | import { useAuth } from '../context/AuthContext';
  5 |
  6 | export default function Layout() {
  7 |   const { user, logout } = useAuth();
  8 |   const navigate = useNavigate();
  9 |   const [sidebarOpen, setSidebarOpen] = useState(false);
 10 |
 11 |   const handleLogout = () => {
 12 |     logout();
 13 |     navigate('/login');
 14 |   };
 15 |
 16 |   return (
 17 |     <div style={{ display: 'flex', minHeight: '100vh', background: '#05060f' }}>
 18 |       <Sidebar open={sidebarOpen} onClose={() => setSidebarOpen(false)} />
 19 |
 20 |       {sidebarOpen && (
 21 |         <div
 22 |           onClick={() => setSidebarOpen(false)}
 23 |           style={{
 24 |             position: 'fixed', inset: 0, zIndex: 40,
 25 |             background: 'rgba(0,0,0,0.5)', display: 'none',
 26 |           }}
 27 |           className="sidebar-backdrop"
 28 |         />
 29 |       )}
 30 |
 31 |       <div style={{ flex: 1, display: 'flex', flexDirection: 'column', height: '100vh', overflow: 'hidden',
minWidth: 0 }}>
 32 |         <header style={{
 33 |           height: '64px', padding: '0 1.5rem',
 34 |           display: 'flex', alignItems: 'center', justifyContent: 'space-between', gap: '1rem',
 35 |           borderBottom: '1px solid rgba(255,255,255,0.05)',
 36 |           background: 'rgba(5,6,15,0.5)', backdropFilter: 'blur(12px)', zIndex: 10, flexShrink: 0,
 37 |         }}>
 38 |           <div style={{ display: 'flex', alignItems: 'center', gap: '0.75rem' }}>
 39 |             <button
 40 |               onClick={() => setSidebarOpen(!sidebarOpen)}
 41 |               className="mobile-menu-btn"
 42 |               style={{
 43 |                 display: 'none', background: 'none', border: 'none',
 44 |                 color: '#94a3b8', fontSize: '1.25rem', cursor: 'pointer', padding: '0.25rem',
 45 |               }}
 46 |               aria-label="Toggle menu"
 47 |             >
 48 |               ≡
 49 |             </button>
 50 |             <span style={{ fontSize: '1.1rem', fontWeight: 700, color: '#e2e8f0' }} className="logo-
mobile">StudyAI</span>
 51 |           </div>
 52 |
 53 |           <div style={{ flex: 1, maxWidth: '400px' }} className="header-search">
 54 |             <div style={{
 55 |               display: 'flex', alignItems: 'center', gap: '0.5rem',
 56 |               background: 'rgba(255,255,255,0.03)',
 57 |               border: '1px solid rgba(255,255,255,0.08)',
 58 |               borderRadius: '12px', padding: '0.5rem 1rem',
 59 |             }}>
 60 |               <span style={{ color: '#64748b' }}>🔍</span>
 61 |               <input
 62 |                 type="text" placeholder="Search..."
 63 |                 style={{ background: 'transparent', border: 'none', outline: 'none', color: '#e2e8f0', width:
'100%', fontSize: '0.85rem' }}
 64 |               />
 65 |             </div>
 66 |           </div>
 67 |
 68 |           <div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>
 69 |             <button
 70 |               onClick={handleLogout}
 71 |               title="Logout"
 72 |               style={{
 73 |                 background: 'none', border: '1px solid rgba(255,255,255,0.1)',
 74 |                 borderRadius: '50%', padding: '0.35rem',
 75 |                 color: '#94a3b8', cursor: 'pointer', display: 'flex', alignItems: 'center', justifyContent:
'center',
```

```
 76 |               }}
 77 |             >
 78 |               <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" strokeLinejoin="round">
 79 |                 <path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4" />
 80 |                 <polyline points="16 17 21 12 16 7" />
 81 |                 <line x1="21" y1="12" x2="9" y2="12" />
 82 |               </svg>
 83 |             </button>
 84 |
 85 |             <div style={{
 86 |               width: '34px', height: '34px', borderRadius: '50%',
 87 |               background: 'linear-gradient(135deg, #7c3aed, #06b6d4)',
 88 |               display: 'flex', alignItems: 'center', justifyContent: 'center',
 89 |               color: 'white', fontWeight: 600, fontSize: '0.8rem',
 90 |             }}>
 91 |               {user?.name?.substring(0, 2).toUpperCase() || 'SA'}
 92 |             </div>
 93 |           </div>
 94 |         </header>
 95 |
 96 |         <main style={{ flex: 1, overflowY: 'auto', position: 'relative' }}>
 97 |           <Outlet />
 98 |         </main>
 99 |       </div>
100 |     </div>
101 |   );
102 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–94:** Manages 'sidebarOpen' state, 'handleLogout', header bar, and scrollable ' '.

> **How to Explain This Module in an Interview:** Uses React Router " so the sidebar and header never remount on navigation.

Module: ProgressChart Component (`frontend/src/components/ProgressChart.jsx`)

What This File Is (Beginner-Friendly Analogy): Multi-line Chart.js component plotting quiz score percentages over time grouped by topic.

Why It Is Used & Architectural Role: Used on ProgressPage.jsx.

Complete Source Code (Every Line Numbered):

```
  1 | import { Line } from 'react-chartjs-2';
  2 | import {
  3 |   Chart as ChartJS,
  4 |   CategoryScale,
  5 |   LinearScale,
  6 |   PointElement,
  7 |   LineElement,
  8 |   Title,
  9 |   Tooltip,
 10 |   Legend,
 11 |   Filler,
 12 | } from 'chart.js';
 13 |
 14 | ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip, Legend, Filler);
 15 |
 16 | export default function ProgressChart({ progressData }) {
 17 |   if (!progressData || progressData.length < 2) {
 18 |     return (
 19 |       <div className="glass-card p-8 text-center">
 20 |         <div className="text-surface-400 mb-2">📊</div>
 21 |         <p className="text-surface-400">Take at least 2 quizzes to see your progress graph.</p>
 22 |       </div>
 23 |     );
 24 |   }
 25 |
 26 |   // Group data by noteTitle for multiple lines
 27 |   const groupedData = {};
 28 |   progressData.forEach((item) => {
 29 |     if (!groupedData[item.noteTitle]) {
 30 |       groupedData[item.noteTitle] = [];
 31 |     }
 32 |     groupedData[item.noteTitle].push(item);
 33 |   });
 34 |
```

```
  const colors = [
    { border: '#6366f1', bg: 'rgba(99, 102, 241, 0.1)' },
    { border: '#10b981', bg: 'rgba(16, 185, 129, 0.1)' },
    { border: '#f59e0b', bg: 'rgba(245, 158, 11, 0.1)' },
    { border: '#06b6d4', bg: 'rgba(6, 182, 212, 0.1)' },
    { border: '#f43f5e', bg: 'rgba(244, 63, 94, 0.1)' },
    { border: '#8b5cf6', bg: 'rgba(139, 92, 246, 0.1)' },
  ];

  // All unique dates
  const allDates = [...new Set(progressData.map((d) =>
    new Date(d.attemptedAt).toLocaleDateString('en-US', { month: 'short', day: 'numeric' })
  ))];

  const datasets = Object.keys(groupedData).map((title, index) => {
    const color = colors[index % colors.length];
    return {
      label: title.length > 25 ? title.substring(0, 25) + '...' : title,
      data: groupedData[title].map((d) => d.percentage),
      borderColor: color.border,
      backgroundColor: color.bg,
      fill: true,
      tension: 0.4,
      pointRadius: 6,
      pointHoverRadius: 8,
      pointBackgroundColor: color.border,
      pointBorderColor: '#0f172a',
      pointBorderWidth: 2,
      borderWidth: 2.5,
    };
  });

  const data = { labels: allDates, datasets };

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: {
        display: datasets.length > 1,
        position: 'bottom',
        labels: {
          color: '#94a3b8',
          font: { family: 'Inter', size: 12 },
          padding: 15,
          usePointStyle: true,
          pointStyle: 'circle',
          boxWidth: 8,
          boxHeight: 8,
        },
      },
      tooltip: {
        backgroundColor: 'rgba(15, 23, 42, 0.9)',
        titleColor: '#e2e8f0',
        bodyColor: '#94a3b8',
        borderColor: 'rgba(99, 102, 241, 0.3)',
        borderWidth: 1,
        padding: 12,
        cornerRadius: 8,
        titleFont: { family: 'Inter', weight: '600' },
        bodyFont: { family: 'Inter' },
        callbacks: {
          label: (context) => ` ${context.dataset.label}: ${context.parsed.y}%`,
        },
      },
    },
    scales: {
      x: {
        grid: { color: 'rgba(51, 65, 85, 0.3)', drawBorder: false },
        ticks: { color: '#64748b', font: { family: 'Inter', size: 11 } },
      },
      y: {
        min: 0,
        max: 100,
        grid: { color: 'rgba(51, 65, 85, 0.3)', drawBorder: false },
        ticks: {
          color: '#64748b',
          font: { family: 'Inter', size: 11 },
          callback: (value) => value + '%',
          stepSize: 20,
        },
      },
    },
    interaction: {
```

```
119 |       intersect: false,
120 |       mode: 'index',
121 |     },
122 |   };
123 |
124 |   return (
125 |     <div className="glass-card p-6">
126 |       <h3 className="text-lg font-semibold text-surface-100 mb-6">Score Progress Over Time</h3>
127 |       <div style={{ height: '350px' }}>
128 |         <Line data={data} options={options} />
129 |       </div>
130 |     </div>
131 |   );
132 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–132:** Registers Chart.js modules, shows an empty state if '< 2' attempts exist, groups data by 'noteTitle' into color-coded datasets, and renders ''.

> **How to Explain This Module in an Interview:** Dynamically assigns distinct border/fill colors per topic.

Module: ProtectedRoute Component (`frontend/src/components/ProtectedRoute.jsx`)

What This File Is (Beginner-Friendly Analogy): Route guard component that blocks unauthenticated access to private routes.

Why It Is Used & Architectural Role: Redirects to '/login' if not authenticated after 'AuthContext' finishes loading.

Complete Source Code (Every Line Numbered):

```
 1 | import { Navigate } from 'react-router-dom';
 2 | import { useAuth } from '../context/AuthContext';
 3 |
 4 | export default function ProtectedRoute({ children }) {
 5 |   const { isAuthenticated, loading } = useAuth();
 6 |
 7 |   if (loading) {
 8 |     return (
 9 |       <div className="min-h-screen flex items-center justify-center">
10 |         <div className="spinner"></div>
11 |       </div>
12 |     );
13 |   }
14 |
15 |   if (!isAuthenticated) {
16 |     return <Navigate to="/login" replace />;
17 |   }
18 |
19 |   return children;
20 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–20:** Checks 'loading' (renders spinner) and '!isAuthenticated' (renders ''), otherwise returns 'children'.

> **How to Explain This Module in an Interview:** Prevents unauthorized access and flash-of-unauthenticated-content.

Module: QuizQuestion Component (`frontend/src/components/QuizQuestion.jsx`)

What This File Is (Beginner-Friendly Analogy): Renders a single multiple-choice question and its 4 option buttons (A–D).

Why It Is Used & Architectural Role: Used by QuizPage.jsx.

Complete Source Code (Every Line Numbered):

```
 1 | export default function QuizQuestion({ question, selectedOption, onSelect, showResult, correctOption }) {
 2 |   const options = [
 3 |     { key: 'A', text: question.optionA },
 4 |     { key: 'B', text: question.optionB },
 5 |     { key: 'C', text: question.optionC },
 6 |     { key: 'D', text: question.optionD },
 7 |   ];
 8 |
 9 |   const getOptionClass = (optionKey) => {
10 |     if (!showResult) {
11 |       return selectedOption === optionKey ? 'quiz-option selected' : 'quiz-option';
```

```
  12 |     }
  13 |     if (optionKey === correctOption) return 'quiz-option correct';
  14 |     if (optionKey === selectedOption && selectedOption !== correctOption) return 'quiz-option incorrect';
  15 |     return 'quiz-option opacity-50';
  16 |   };
  17 |
  18 |   return (
  19 |     <div className="animate-fade-in">
  20 |       <h3 className="text-xl font-semibold text-surface-100 mb-6 leading-relaxed">
  21 |         {question.questionText}
  22 |       </h3>
  23 |
  24 |       <div className="grid gap-3">
  25 |         {options.map(({ key, text }) => (
  26 |           <button
  27 |             key={key}
  28 |             onClick={() => !showResult && onSelect(key)}
  29 |             disabled={showResult}
  30 |             className={getOptionClass(key)}
  31 |           >
  32 |             <div className="flex items-start gap-3">
  33 |               <span className={`flex-shrink-0 w-8 h-8 rounded-lg flex items-center justify-center text-sm font-
bold transition-all ${
  34 |                 selectedOption === key && !showResult
  35 |                   ? 'bg-primary-500 text-white'
  36 |                   : showResult && key === correctOption
  37 |                   ? 'bg-emerald-500 text-white'
  38 |                   : showResult && key === selectedOption
  39 |                   ? 'bg-rose-500 text-white'
  40 |                   : 'bg-surface-700/50 text-surface-300'
  41 |               }`}>
  42 |                 {key}
  43 |               </span>
  44 |               <span className="pt-1">{text}</span>
  45 |             </div>
  46 |           </button>
  47 |         ))}
  48 |       </div>
  49 |     </div>
  50 |   );
  51 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–50:** Maps options A–D into buttons, applying '.selected', '.correct', or '.incorrect' classes and badge colors.

> **How to Explain This Module in an Interview:** Clean presentational component controlled by QuizPage state.

Module: RemindersPanel Component (`frontend/src/components/RemindersPanel.jsx`)

What This File Is (Beginner-Friendly Analogy): Floating bottom-right widget alerting the student to retake quizzes where they scored below 60%.

Why It Is Used & Architectural Role: Drives active mastery by surfacing failing quizzes on the Dashboard.

Complete Source Code (Every Line Numbered):

```
   1 | import { useState, useEffect, useCallback, useRef } from 'react';
   2 |
   3 | const STAGGER_MS = 400;
   4 | const AUTO_COLLAPSE_MS = 800;
   5 |
   6 | const panelStyle = {
   7 |   position: 'fixed',
   8 |   bottom: '1.5rem',
   9 |   right: '1.5rem',
  10 |   zIndex: 50,
  11 |   background: 'rgba(10, 10, 25, 0.7)',
  12 |   backdropFilter: 'blur(24px) saturate(180%)',
  13 |   WebkitBackdropFilter: 'blur(24px) saturate(180%)',
  14 |   border: '1px solid rgba(245, 158, 11, 0.2)',
  15 |   borderRadius: '16px',
  16 |   boxShadow: '0 8px 32px rgba(0,0,0,0.5), 0 0 40px rgba(245, 158, 11, 0.08)',
  17 |   overflow: 'hidden',
  18 |   transition: 'all 0.35s cubic-bezier(0.4, 0, 0.2, 1)',
  19 | };
  20 |
  21 | const cardStyle = {
  22 |   display: 'flex',
  23 |   alignItems: 'center',
  24 |   gap: '0.85rem',
```

```
  padding: '0.85rem 1.1rem',
  background: 'rgba(245, 158, 11, 0.06)',
  border: '1px solid rgba(245, 158, 11, 0.18)',
  borderRadius: '12px',
  cursor: 'pointer',
  transition: 'all 0.2s ease',
  textAlign: 'left',
  width: '100%',
  color: 'inherit',
  fontSize: 'inherit',
  fontFamily: 'inherit',
};

export default function RemindersPanel({ failingQuizzes, onRetake, quizRetakeMap }) {
  const [phase, setPhase] = useState('hidden');
  const [expanded, setExpanded] = useState(false);
  const collapseTimer = useRef(null);

  const startAnimation = useCallback(() => {
    if (failingQuizzes.length === 0) return;
    setPhase('animating');
    setExpanded(false);

    if (collapseTimer.current) clearTimeout(collapseTimer.current);
    collapseTimer.current = setTimeout(() => {
      setPhase('stacked');
    }, failingQuizzes.length * STAGGER_MS + AUTO_COLLAPSE_MS);
  }, [failingQuizzes.length]);

  useEffect(() => {
    startAnimation();
    return () => { if (collapseTimer.current) clearTimeout(collapseTimer.current); };
  }, [startAnimation]);

  useEffect(() => {
    if (failingQuizzes.length === 0) setPhase('hidden');
  }, [failingQuizzes.length]);

  if (phase === 'hidden' || failingQuizzes.length === 0) return null;

  const toggleExpand = () => {
    if (collapseTimer.current) clearTimeout(collapseTimer.current);
    setExpanded(!expanded);
  };

  const handleClickCard = (quiz) => {
    if (!quizRetakeMap[quiz.quizId]) return;
    onRetake(quiz);
  };

  if (!expanded && phase === 'stacked') {
    return (
      <div style={{ ...panelStyle, padding: '0.8rem 1.15rem' }}>
        <button
          onClick={toggleExpand}
          style={{
            display: 'flex', alignItems: 'center', gap: '0.7rem',
            background: 'none', border: 'none', cursor: 'pointer', color: '#fbbf24',
            padding: 0, fontFamily: 'inherit',
          }}
        >
          <span style={{
            width: '32px', height: '32px', borderRadius: '10px', flexShrink: 0,
            background: 'rgba(245,158,11,0.15)',
            display: 'flex', alignItems: 'center', justifyContent: 'center', fontSize: '1rem',
          }}>⚠ </span>
          <span style={{ fontSize: '0.85rem', fontWeight: 600, whiteSpace: 'nowrap' }}>
            {failingQuizzes.length} quiz{failingQuizzes.length > 1 ? 'zes' : ''} to retake
          </span>
          <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5"
strokeLinecap="round" strokeLinejoin="round" style={{ flexShrink: 0 }}>
            <path d="M18 15l-6-6-6 6" />
          </svg>
        </button>
      </div>
    );
  }

  return (
    <div style={{ ...panelStyle, padding: '1rem', width: '340px' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'0.65rem' }}>
        <p style={{ fontSize: '0.8rem', fontWeight: 600, color: '#fbbf24', margin: 0 }}>⚠  Quizzes to Retake</
p>
```

```
106 |         {phase === 'animating' && (
107 |           <button onClick={toggleExpand} style={{
108 |             background: 'none', border: 'none', cursor: 'pointer', color: '#94a3b8',
109 |             padding: '0.2rem', display: 'flex', alignItems: 'center',
110 |           }}>
111 |             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5"
strokeLinecap="round" strokeLinejoin="round">
112 |               <path d="M6 9l6 6 6-6" />
113 |             </svg>
114 |           </button>
115 |         )}
116 |       </div>
117 |
118 |       <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem' }}>
119 |         {failingQuizzes.map((quiz, i) => {
120 |           const hasQuestions = !!quizRetakeMap[quiz.quizId];
121 |           return (
122 |             <button
123 |               key={`${quiz.quizId}-${quiz.attemptedAt}`}
124 |               onClick={() => handleClickCard(quiz)}
125 |               disabled={!hasQuestions}
126 |               className="reminder-anim"
127 |               style={{
128 |                 ...cardStyle,
129 |                 animationDelay: `${i * STAGGER_MS}ms`,
130 |                 opacity: hasQuestions ? 1 : 0.45,
131 |                 cursor: hasQuestions ? 'pointer' : 'not-allowed',
132 |               }}
133 |               onMouseEnter={e => { if (hasQuestions) { e.currentTarget.style.borderColor =
'rgba(245,158,11,0.45)'; e.currentTarget.style.background = 'rgba(245,158,11,0.12)'; } }}
134 |               onMouseLeave={e => { e.currentTarget.style.borderColor = 'rgba(245,158,11,0.18)';
e.currentTarget.style.background = 'rgba(245,158,11,0.06)'; }}
135 |             >
136 |               <div style={{
137 |                 width: '36px', height: '36px', borderRadius: '10px', flexShrink: 0,
138 |                 background: 'rgba(244,63,94,0.12)',
139 |                 display: 'flex', alignItems: 'center', justifyContent: 'center', fontSize: '0.9rem',
140 |               }}>↳</div>
141 |               <div style={{ flex: 1, minWidth: 0 }}>
142 |                 <p style={{ fontSize: '0.85rem', fontWeight: 600, color: '#e2e8f0', margin: 0, whiteSpace:
'nowrap', overflow: 'hidden', textOverflow: 'ellipsis' }}>
143 |                   {quiz.noteTitle}
144 |                 </p>
145 |                 <p style={{ fontSize: '0.72rem', color: '#94a3b8', margin: '0.1rem 0 0' }}>
146 |                   Previous: <span style={{ color: '#fb7185', fontWeight: 600 }}
>{Number(quiz.percentage).toFixed(0)}%</span>
147 |                 </p>
148 |               </div>
149 |               <span style={{ fontSize: '0.75rem', fontWeight: 600, color: hasQuestions ? '#fbbf24' : '#64748b',
flexShrink: 0, whiteSpace: 'nowrap' }}>
150 |                 {hasQuestions ? 'Retake →' : 'Generate first'}
151 |               </span>
152 |             </button>
153 |           );
154 |         })}
155 |       </div>
156 |
157 |       <style>{`
158 |         @keyframes spin { to { transform: rotate(360deg); } }
159 |         @keyframes reminderSlideUp {
160 |           from { opacity: 0; transform: translateY(16px); }
161 |           to { opacity: 1; transform: translateY(0); }
162 |         }
163 |         .reminder-anim {
164 |           animation: reminderSlideUp 0.4s ease-out both;
165 |         }
166 |       `}</style>
167 |     </div>
168 |   );
169 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–160:** Animates failing quiz cards in with a 400ms stagger, auto-collapses into a compact pill after 800ms, expands on click, and launches a retake when clicked.

> **How to Explain This Module in an Interview:** Polished UX micro-interaction using staged animation phases ('hidden' -> 'animating' -> 'stacked').

Module: Sidebar Component (`frontend/src/components/Sidebar.jsx`)

What This File Is (Beginner-Friendly Analogy): Left navigation sidebar with 9 app links and active-route highlighting.

Why It Is Used & Architectural Role: Provides primary navigation across desktop and mobile viewports.

Complete Source Code (Every Line Numbered):

```
  1 | import { Link, useLocation } from 'react-router-dom';
  2 |
  3 | export default function Sidebar({ open, onClose }) {
  4 |   const location = useLocation();
  5 |
  6 |   const navLinks = [
  7 |     { to: '/dashboard', label: 'Dashboard', icon: '🏠' },
  8 |     { to: '/notes', label: 'Upload Notes', icon: '📤' },
  9 |     { to: '/history', label: 'Notes History', icon: '🕒' },
 10 |     { to: '/quizzes', label: 'Quizzes', icon: '📝' },
 11 |     { to: '/flashcards', label: 'Flashcards', icon: '🃏' },
 12 |     { to: '/progress', label: 'Progress', icon: '📈' },
 13 |     { to: '/analytics', label: 'Analytics', icon: '📊' },
 14 |     { to: '/achievements', label: 'Achievements', icon: '🏆' },
 15 |     { to: '/settings', label: 'Settings', icon: '⚙ ' },
 16 |   ];
 17 |
 18 |   return (
 19 |     <>
 20 |       <aside className="app-sidebar" data-open={open || undefined} style={{
 21 |         width: '260px',
 22 |         background: 'rgba(10, 10, 20, 0.95)',
 23 |         backdropFilter: 'blur(20px)',
 24 |         borderRight: '1px solid rgba(255,255,255,0.05)',
 25 |         display: 'flex',
 26 |         flexDirection: 'column',
 27 |         height: '100vh',
 28 |         flexShrink: 0,
 29 |       }}>
 30 |         <div style={{ padding: '1.5rem 1.5rem 1rem', display: 'flex', alignItems: 'center', justifyContent:
'space-between' }}>
 31 |           <div style={{ display: 'flex', alignItems: 'center', gap: '0.75rem' }}>
 32 |             <div style={{
 33 |               width: '32px', height: '32px',
 34 |               background: 'linear-gradient(135deg, #7c3aed, #06b6d4)',
 35 |               borderRadius: '8px', display: 'flex', alignItems: 'center', justifyContent: 'center',
 36 |               boxShadow: '0 4px 12px rgba(124,58,237,0.3)',
 37 |             }}>
 38 |               <span style={{ color: 'white', fontWeight: 'bold', fontSize: '1rem' }}>S</span>
 39 |             </div>
 40 |             <span style={{ fontSize: '1.1rem', fontWeight: 700, color: '#e2e8f0' }}>StudyAI</span>
 41 |           </div>
 42 |           <button
 43 |             onClick={onClose}
 44 |             className="sidebar-close-btn"
 45 |             style={{
 46 |               display: 'none', background: 'none', border: 'none',
 47 |               color: '#64748b', fontSize: '1.2rem', cursor: 'pointer',
 48 |             }}
 49 |             aria-label="Close menu"
 50 |           >
 51 |             ✕
 52 |           </button>
 53 |         </div>
 54 |
 55 |         <nav style={{ flex: 1, padding: '0 0.75rem', display: 'flex', flexDirection: 'column', gap: '0.15rem',
overflowY: 'auto' }}>
 56 |           {navLinks.map((link) => {
 57 |             const active = location.pathname === link.to || (link.to !== '/dashboard' &&
location.pathname.startsWith(link.to));
 58 |             return (
 59 |               <Link
 60 |                 key={link.to}
 61 |                 to={link.to}
 62 |                 onClick={onClose}
 63 |                 style={{
 64 |                   display: 'flex', alignItems: 'center', gap: '0.75rem',
 65 |                   padding: '0.7rem 1rem', borderRadius: '10px',
 66 |                   color: active ? '#fff' : '#64748b',
 67 |                   background: active ? 'linear-gradient(90deg, rgba(124,58,237,0.2) 0%, transparent 100%)' :
'transparent',
 68 |                   borderLeft: active ? '3px solid #7c3aed' : '3px solid transparent',
 69 |                   textDecoration: 'none', fontWeight: active ? 600 : 500,
 70 |                   transition: 'all 0.15s ease',
 71 |                 }}
 72 |               >
 73 |                 <span style={{ fontSize: '1rem', opacity: active ? 1 : 0.7 }}>{link.icon}</span>
 74 |                 <span style={{ fontSize: '0.875rem' }}>{link.label}</span>
```

```
              </Link>
            );
          })}
        </nav>
      </aside>
    </>
  );
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–80:** Maps 'navLinks' ('/dashboard', '/notes', '/history', '/quizzes', '/flashcards', '/progress', '/analytics', '/achievements', '/settings') to React Router '' elements with active state styling.

How to Explain This Module in an Interview: Responsive drawer on mobile via 'data-open' attribute.

Module: Timer Component (`frontend/src/components/Timer.jsx`)

What This File Is (Beginner-Friendly Analogy): 5-minute countdown timer with visual progress bar and urgency color shifts.

Why It Is Used & Architectural Role: Auto-submits the quiz on QuizPage.jsx when time reaches 00:00.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useEffect, useRef } from 'react';
  2 |
  3 | export default function Timer({ totalSeconds, onTimeUp }) {
  4 |   const [secondsLeft, setSecondsLeft] = useState(totalSeconds);
  5 |   const intervalRef = useRef(null);
  6 |   const onTimeUpRef = useRef(onTimeUp);
  7 |   onTimeUpRef.current = onTimeUp;
  8 |
  9 |   useEffect(() => {
 10 |     setSecondsLeft(totalSeconds);
 11 |     intervalRef.current = setInterval(() => {
 12 |       setSecondsLeft((prev) => {
 13 |         const next = prev - 1;
 14 |         return next > 0 ? next : 0;
 15 |       });
 16 |     }, 1000);
 17 |
 18 |     return () => clearInterval(intervalRef.current);
 19 |   }, [totalSeconds]);
 20 |
 21 |   useEffect(() => {
 22 |     if (secondsLeft === 0) {
 23 |       clearInterval(intervalRef.current);
 24 |       onTimeUpRef.current?.();
 25 |     }
 26 |   }, [secondsLeft]);
 27 |
 28 |   const minutes = Math.floor(secondsLeft / 60);
 29 |   const seconds = secondsLeft % 60;
 30 |   const progress = (secondsLeft / totalSeconds) * 100;
 31 |
 32 |   const isLow = secondsLeft <= 30;
 33 |   const isCritical = secondsLeft <= 10;
 34 |
 35 |   return (
 36 |     <div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>
 37 |       <div style={{
 38 |         display: 'flex', alignItems: 'center', gap: '0.5rem',
 39 |         padding: '0.5rem 1rem', borderRadius: '12px', border: '1px solid',
 40 |         borderColor: isCritical ? 'rgba(244,63,94,0.3)' : isLow ? 'rgba(245,158,11,0.3)' :
'rgba(255,255,255,0.1)',
 41 |         background: isCritical ? 'rgba(244,63,94,0.1)' : isLow ? 'rgba(245,158,11,0.1)' :
'rgba(255,255,255,0.05)',
 42 |         color: isCritical ? '#fb7185' : isLow ? '#fbbf24' : '#e2e8f0',
 43 |         transition: 'all 0.3s ease',
 44 |       }}>
 45 |         <span style={{ fontSize: '1.1rem' }}>{isCritical ? '🔴' : isLow ? '🟡' : '⏱'}</span>
 46 |         <span style={{ fontFamily: 'monospace', fontWeight: 700, fontSize: '1.1rem', letterSpacing: '0.05em' }}
>
 47 |           {String(minutes).padStart(2, '0')}:{String(seconds).padStart(2, '0')}
 48 |         </span>
 49 |       </div>
 50 |
 51 |       <div style={{ flex: 1, height: '6px', background: 'rgba(255,255,255,0.08)', borderRadius: '99px',
overflow: 'hidden', maxWidth: '180px' }}>
 52 |         <div style={{
 53 |           width: `${progress}%`, height: '100%', borderRadius: '99px',
```

```
 54 |           background: isCritical ? 'linear-gradient(90deg, #f43f5e, #fb7185)' : isLow ? 'linear-gradient(90deg,
#f59e0b, #fbbf24)' : 'linear-gradient(90deg, #7c3aed, #38bdf8)',
 55 |           transition: 'width 1s linear',
 56 |         }} />
 57 |       </div>
 58 |     </div>
 59 |   );
 60 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–59:** Uses 'useRef' for 'intervalRef' and 'onTimeUpRef', decrements 'secondsLeft' every second, triggers 'onTimeUpRef.current()' at 0, and styles low (<=30s) and critical (<=10s) states.

How to Explain This Module in an Interview: Uses 'onTimeUpRef' to avoid stale closures without restarting the interval.

Module: AchievementsPage (`frontend/src/pages/AchievementsPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Gamified badges page evaluating 8 study milestones against the student's quiz and note history.

Why It Is Used & Architectural Role: Boosts student engagement with unlockable badges and progress bars.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useEffect } from 'react';
  2 | import API from '../api/axios';
  3 |
  4 | const ACHIEVEMENTS = [
  5 |   {
  6 |     id: 'first-quiz',
  7 |     icon: '🎯',
  8 |     title: 'First Steps',
  9 |     desc: 'Take your first quiz',
 10 |     test: (s) => s.attempts >= 1,
 11 |     progress: (s) => Math.min(s.attempts, 1),
 12 |     max: 1,
 13 |     color: '#38bdf8',
 14 |   },
 15 |   {
 16 |     id: 'quiz-whiz',
 17 |     icon: '⚡',
 18 |     title: 'Quiz Whiz',
 19 |     desc: 'Score 90% or higher on any quiz',
 20 |     test: (s) => s.best >= 90,
 21 |     progress: (s) => (s.best >= 90 ? 1 : 0),
 22 |     max: 1,
 23 |     color: '#a78bfa',
 24 |   },
 25 |   {
 26 |     id: 'perfect-score',
 27 |     icon: '🏆',
 28 |     title: 'Perfect Score',
 29 |     desc: 'Score 100% on a quiz',
 30 |     test: (s) => s.hasPerfect,
 31 |     progress: (s) => (s.hasPerfect ? 1 : 0),
 32 |     max: 1,
 33 |     color: '#fbbf24',
 34 |   },
 35 |   {
 36 |     id: 'marathon',
 37 |     icon: '🏃',
 38 |     title: 'Marathon Runner',
 39 |     desc: 'Take 5 quizzes',
 40 |     test: (s) => s.attempts >= 5,
 41 |     progress: (s) => Math.min(s.attempts, 5),
 42 |     max: 5,
 43 |     color: '#34d399',
 44 |   },
 45 |   {
 46 |     id: 'topic-explorer',
 47 |     icon: '🗺️',
 48 |     title: 'Topic Explorer',
 49 |     desc: 'Take quizzes on 3 different topics',
 50 |     test: (s) => s.topics >= 3,
 51 |     progress: (s) => Math.min(s.topics, 3),
 52 |     max: 3,
 53 |     color: '#f472b6',
 54 |   },
 55 |   {
```

```
    id: 'collector',
    icon: '📚',
    title: 'Knowledge Collector',
    desc: 'Create 3 notes',
    test: (s) => s.notes >= 3,
    progress: (s) => Math.min(s.notes, 3),
    max: 3,
    color: '#fb923c',
  },
  {
    id: 'consistent',
    icon: '📈',
    title: 'Consistent Learner',
    desc: 'Keep an average score of 70% or higher',
    test: (s) => s.attempts > 0 && s.average >= 70,
    progress: (s) => (s.attempts > 0 && s.average >= 70 ? 1 : 0),
    max: 1,
    color: '#06b6d4',
  },
  {
    id: 'double-digit',
    icon: '🔥',
    title: 'Study Streak',
    desc: 'Take 10 quizzes',
    test: (s) => s.attempts >= 10,
    progress: (s) => Math.min(s.attempts, 10),
    max: 10,
    color: '#f43f5e',
  },
];

export default function AchievementsPage() {
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const [historyRes, notesRes] = await Promise.all([
          API.get('/api/quiz/history'),
          API.get('/api/notes'),
        ]);
        const attempts = historyRes.data;
        const percentages = attempts.map(a => Number(a.percentage));
        const uniqueTopics = new Set(attempts.map(a => a.noteTitle)).size;
        setStats({
          attempts: attempts.length,
          best: percentages.length ? Math.max(...percentages) : 0,
          average: percentages.length ? (percentages.reduce((a, b) => a + b, 0) / percentages.length) : 0,
          hasPerfect: percentages.some(p => p === 100),
          topics: uniqueTopics,
          notes: notesRes.data.length,
        });
      } catch (err) {
        console.error('Failed to fetch achievements data', err);
      } finally {
        setLoading(false);
      }
    };
    fetchData();
  }, []);

  if (loading) {
    return (
      <div style={{ display: 'flex', justifyContent: 'center', padding: '4rem' }}>
        <div className="spinner" />
      </div>
    );
  }

  const unlockedCount = ACHIEVEMENTS.filter(a => stats && a.test(stats)).length;

  return (
    <div style={{ padding: '2rem 1rem', maxWidth: '1100px', margin: '0 auto' }}>
      <div style={{ marginBottom: '2rem' }}>
        <h1 style={{ fontSize: '2rem', fontWeight: 800, background: 'linear-gradient(135deg, #a78bfa,
#38bdf8)', WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent' }}>
          🏆 Achievements
        </h1>
        <p style={{ color: '#64748b', marginTop: '0.4rem' }}>
          {stats && stats.attempts > 0
            ? `You've unlocked ${unlockedCount} of ${ACHIEVEMENTS.length} achievements. Keep studying!`
            : 'Take quizzes and create notes to unlock achievements.'}
        </p>
```

```
139 |       </div>
140 |
141 |       {/* Overall progress bar */}
142 |       <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.07)',
borderRadius: '1.25rem', padding: '1.5rem', marginBottom: '1.5rem' }}>
143 |         <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'0.75rem' }}>
144 |           <p style={{ fontSize: '0.9rem', fontWeight: 600, color: '#e2e8f0', margin: 0 }}>Overall Progress</p>
145 |           <p style={{ fontSize: '0.85rem', color: '#a78bfa', fontWeight: 700, margin: 0 }}>{unlockedCount}/
{ACHIEVEMENTS.length}</p>
146 |         </div>
147 |         <div style={{ height: '8px', background: 'rgba(255,255,255,0.07)', borderRadius: '99px', overflow:
'hidden' }}>
148 |           <div style={{
149 |             width: `${(unlockedCount / ACHIEVEMENTS.length) * 100}%`, height: '100%',
150 |             background: 'linear-gradient(90deg, #7c3aed, #06b6d4)', borderRadius: '99px',
151 |             transition: 'width 0.6s ease',
152 |           }} />
153 |         </div>
154 |       </div>
155 |
156 |       {/* Achievement cards */}
157 |       <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(240px, 1fr))', gap:
'1.25rem' }}>
158 |         {ACHIEVEMENTS.map((a) => {
159 |           const unlocked = stats && a.test(stats);
160 |           const prog = stats ? a.progress(stats) : 0;
161 |           return (
162 |             <div
163 |               key={a.id}
164 |               style={{
165 |                 padding: '1.5rem',
166 |                 borderRadius: '1.25rem',
167 |                 background: unlocked ? `${a.color}0F` : 'rgba(255,255,255,0.02)',
168 |                 border: `1px solid ${unlocked ? `${a.color}50` : 'rgba(255,255,255,0.07)'}`,
169 |                 opacity: unlocked ? 1 : 0.6,
170 |                 transition: 'all 0.25s ease',
171 |               }}
172 |               onMouseEnter={e => { e.currentTarget.style.transform = 'translateY(-3px)'; }}
173 |               onMouseLeave={e => { e.currentTarget.style.transform = 'translateY(0)'; }}
174 |             >
175 |               <div style={{ display: 'flex', alignItems: 'center', gap: '0.85rem', marginBottom: '0.75rem' }}>
176 |                 <div style={{
177 |                   width: '44px', height: '44px', borderRadius: '12px', flexShrink: 0,
178 |                   background: unlocked ? `${a.color}22` : 'rgba(255,255,255,0.05)',
179 |                   display: 'flex', alignItems: 'center', justifyContent: 'center', fontSize: '1.4rem',
180 |                   filter: unlocked ? 'none' : 'grayscale(1)',
181 |                 }}>
182 |                   {a.icon}
183 |                 </div>
184 |                 <div>
185 |                   <p style={{ fontSize: '0.95rem', fontWeight: 700, color: unlocked ? '#f8fafc' : '#94a3b8',
margin: 0 }}>{a.title}</p>
186 |                   <p style={{ fontSize: '0.72rem', color: unlocked ? a.color : '#64748b', fontWeight: 600,
margin: '0.1rem 0 0' }}>
187 |                     {unlocked ? 'UNLOCKED' : 'LOCKED'}
188 |                   </p>
189 |                 </div>
190 |               </div>
191 |               <p style={{ fontSize: '0.82rem', color: '#64748b', lineHeight: 1.5, margin: '0 0 0.9rem' }}
>{a.desc}</p>
192 |               {!unlocked && a.max > 1 && (
193 |                 <div style={{ display: 'flex', alignItems: 'center', gap: '0.6rem' }}>
194 |                   <div style={{ flex: 1, height: '5px', background: 'rgba(255,255,255,0.07)', borderRadius:
'99px', overflow: 'hidden' }}>
195 |                     <div style={{ width: `${(prog / a.max) * 100}%`, height: '100%', background: a.color,
borderRadius: '99px' }} />
196 |                   </div>
197 |                   <span style={{ fontSize: '0.72rem', color: '#64748b', fontWeight: 600 }}>{prog}/{a.max}</
span>
198 |                 </div>
199 |               )}
200 |             </div>
201 |           );
202 |         })}
203 |       </div>
204 |     </div>
205 |   );
206 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–203:** Defines 8 'ACHIEVEMENTS' with 'test(stats)' and 'progress(stats)' predicates, fetches '/api/quiz/history' and '/

api/notes' in parallel via 'Promise.all', and renders overall progress + badge cards.

How to Explain This Module in an Interview: Declarative predicate-driven achievement engine.

Module: AnalyticsPage (`frontend/src/pages/AnalyticsPage.jsx`)

What This File Is (Beginner-Friendly Analogy): AI-powered learning analytics dashboard showing cumulative score trends, topic performance bar chart, weakest/strongest topics, and recommendations.

Why It Is Used & Architectural Role: Visualizes '/api/analytics' data using Chart.js " and " charts.

Complete Source Code (Every Line Numbered):

```
  1 | import { Line, Bar } from 'react-chartjs-2';
  2 | import {
  3 |   Chart as ChartJS,
  4 |   CategoryScale,
  5 |   LinearScale,
  6 |   PointElement,
  7 |   LineElement,
  8 |   BarElement,
  9 |   Title,
 10 |   Tooltip,
 11 |   Legend,
 12 |   Filler,
 13 | } from 'chart.js';
 14 | import { useNavigate } from 'react-router-dom';
 15 | import useFetch from '../hooks/useFetch';
 16 |
 17 | ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, BarElement, Title, Tooltip, Legend,
Filler);
 18 |
 19 | const PASS_THRESHOLD = 70;
 20 |
 21 | export default function AnalyticsPage() {
 22 |   const navigate = useNavigate();
 23 |   const { data, error, loading } = useFetch('/api/analytics');
 24 |
 25 |   if (loading) {
 26 |     return <div style={{ display: 'flex', justifyContent: 'center', padding: '4rem' }}><div className="spinner"
/></div>;
 27 |   }
 28 |
 29 |   if (error) {
 30 |     return (
 31 |       <div style={{ textAlign: 'center', padding: '4rem' }}>
 32 |         <div style={{ fontSize: '2rem', marginBottom: '0.5rem' }}>🤖</div>
 33 |         <p style={{ color: '#f87171' }}>{error}</p>
 34 |       </div>
 35 |     );
 36 |   }
 37 |
 38 |   if (!data || data.overall.totalAttempts === 0) {
 39 |     return (
 40 |       <div style={{ padding: '2rem 1rem', maxWidth: '1200px', margin: '0 auto' }}>
 41 |         <h1 style={{ fontSize: '1.75rem', fontWeight: 700, color: '#f8fafc', marginBottom: '0.25rem' }}
>Learning Analytics</h1>
 42 |         <p style={{ color: '#64748b', marginBottom: '2rem' }}>AI-powered insights into how you learn</p>
 43 |         <div style={{ textAlign: 'center', padding: '4rem 1rem', background: 'rgba(255,255,255,0.02)', border:
'1px solid rgba(255,255,255,0.06)', borderRadius: '20px' }}>
 44 |           <div style={{ fontSize: '3rem', marginBottom: '1rem' }}>📊</div>
 45 |           <p style={{ color: '#e2e8f0', fontWeight: 600, marginBottom: '0.25rem' }}>No quiz attempts yet</p>
 46 |           <p style={{ color: '#64748b', fontSize: '0.9rem', marginBottom: '1.25rem' }}>Take a quiz to unlock
personalized learning analytics</p>
 47 |           <button onClick={() => navigate('/notes')} className="btn-primary" style={{ padding: '0.5rem
1.25rem', fontSize: '0.85rem' }}>
 48 |             Upload a note to get started →
 49 |           </button>
 50 |         </div>
 51 |       </div>
 52 |     );
 53 |   }
 54 |
 55 |   const { overall, topics, trend } = data;
 56 |
 57 |   const sortedTopics = [...topics].sort((a, b) => a.averageScore - b.averageScore);
 58 |   const weakest = sortedTopics[0];
 59 |   const strongest = sortedTopics[sortedTopics.length - 1];
 60 |
 61 |   const trendChart = {
 62 |     labels: trend.map((p) => p.date),
```

```
 63 |     datasets: [{
 64 |       label: 'Cumulative average score',
 65 |       data: trend.map((p) => p.percentage),
 66 |       borderColor: '#8b5cf6',
 67 |       backgroundColor: 'rgba(139, 92, 246, 0.1)',
 68 |       fill: true,
 69 |       tension: 0.35,
 70 |       pointRadius: 3,
 71 |       pointBackgroundColor: '#a78bfa',
 72 |     }],
 73 |   };
 74 |
 75 |   const topicChart = {
 76 |     labels: topics.map((t) => t.noteTitle.length > 20 ? t.noteTitle.substring(0, 20) + '…' : t.noteTitle),
 77 |     datasets: [{
 78 |       label: 'Average score (%)',
 79 |       data: topics.map((t) => t.averageScore),
 80 |       backgroundColor: topics.map((t) => t.averageScore >= PASS_THRESHOLD ? 'rgba(16, 185, 129, 0.7)' :
'rgba(244, 63, 94, 0.7)'),
 81 |       borderRadius: 8,
 82 |     }],
 83 |   };
 84 |
 85 |   const chartOptions = {
 86 |     responsive: true,
 87 |     plugins: {
 88 |       legend: { labels: { color: '#94a3b8', font: { size: 12 } } },
 89 |       tooltip: { backgroundColor: '#1e293b', titleColor: '#e2e8f0', bodyColor: '#94a3b8' },
 90 |     },
 91 |     scales: {
 92 |       x: { ticks: { color: '#64748b', maxRotation: 45 }, grid: { color: 'rgba(255,255,255,0.05)' } },
 93 |       y: { min: 0, max: 100, ticks: { color: '#64748b', callback: (v) => v + '%' }, grid: { color:
'rgba(255,255,255,0.05)' } },
 94 |     },
 95 |   };
 96 |
 97 |   const statCards = [
 98 |     { label: 'Average Score', value: `${overall.averageScore}%`, icon: '🎯', color: '#8b5cf6' },
 99 |     { label: 'Best Score', value: `${overall.bestScore}%`, icon: '🏆', color: '#38bdf8' },
100 |     { label: 'Quizzes Taken', value: overall.totalAttempts, icon: '📝', color: '#10b981' },
101 |     { label: 'Topics Passed', value: `${overall.topicsPassed}/${overall.topicsAttempted}`, icon: '✓', color:
'#f59e0b' },
102 |   ];
103 |
104 |   return (
105 |     <div style={{ padding: '2rem 1rem', maxWidth: '1200px', margin: '0 auto' }}>
106 |       <div style={{ marginBottom: '2rem' }}>
107 |         <h1 style={{ fontSize: '1.75rem', fontWeight: 700, color: '#f8fafc', marginBottom: '0.25rem' }}
>Learning Analytics</h1>
108 |         <p style={{ color: '#64748b' }}>AI-powered insights into how you learn — weakest topics, score trends,
and what to study next.</p>
109 |       </div>
110 |
111 |       {/* Stat cards */}
112 |       <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(200px, 1fr))', gap: '1rem',
marginBottom: '1.5rem' }}>
113 |         {statCards.map((s) => (
114 |           <div key={s.label} style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid
rgba(255,255,255,0.06)', borderRadius: '16px', padding: '1.25rem' }}>
115 |             <div style={{ fontSize: '1.25rem', marginBottom: '0.5rem' }}>{s.icon}</div>
116 |             <div style={{ fontSize: '1.5rem', fontWeight: 700, color: '#f8fafc' }}>{s.value}</div>
117 |             <div style={{ fontSize: '0.8rem', color: '#64748b', marginTop: '0.25rem' }}>{s.label}</div>
118 |           </div>
119 |         ))}
120 |       </div>
121 |
122 |       {/* Charts */}
123 |       <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '1.5rem', marginBottom: '1.5rem' }}
className="analytics-2col">
124 |         <div style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid rgba(255,255,255,0.06)',
borderRadius: '20px', padding: '1.5rem' }}>
125 |           <h3 style={{ color: '#f8fafc', fontWeight: 600, marginBottom: '1rem' }}>Score Trend</h3>
126 |           <Line data={trendChart} options={chartOptions} />
127 |         </div>
128 |         <div style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid rgba(255,255,255,0.06)',
borderRadius: '20px', padding: '1.5rem' }}>
129 |           <h3 style={{ color: '#f8fafc', fontWeight: 600, marginBottom: '1rem' }}>Topic Performance</h3>
130 |           <Bar data={topicChart} options={chartOptions} />
131 |         </div>
132 |       </div>
133 |
134 |       {/* Weakest / strongest */}
135 |       {weakest && (
136 |         <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '1.5rem', marginBottom: '1.5rem' }}
```

```
className="analytics-2col">
137 |           <div style={{ background: 'rgba(244,63,94,0.05)', border: '1px solid rgba(244,63,94,0.15)',
borderRadius: '16px', padding: '1.25rem' }}>
138 |             <div style={{ fontSize: '0.75rem', fontWeight: 700, color: '#fb7185', textTransform: 'uppercase',
letterSpacing: '0.05em', marginBottom: '0.5rem' }}>Weakest topic</div>
139 |             <div style={{ color: '#f8fafc', fontWeight: 600, marginBottom: '0.25rem' }}>{weakest.noteTitle}</
div>
140 |             <div style={{ color: '#64748b', fontSize: '0.85rem', marginBottom: '0.75rem' }}>Average
{weakest.averageScore}% across {weakest.attempts} attempt{weakest.attempts !== 1 ? 's' : ''}</div>
141 |             <div style={{ color: '#94a3b8', fontSize: '0.85rem' }}>{weakest.recommendation}</div>
142 |           </div>
143 |           {strongest && (
144 |             <div style={{ background: 'rgba(16,185,129,0.05)', border: '1px solid rgba(16,185,129,0.15)',
borderRadius: '16px', padding: '1.25rem' }}>
145 |               <div style={{ fontSize: '0.75rem', fontWeight: 700, color: '#34d399', textTransform: 'uppercase',
letterSpacing: '0.05em', marginBottom: '0.5rem' }}>Strongest topic</div>
146 |               <div style={{ color: '#f8fafc', fontWeight: 600, marginBottom: '0.25rem' }}>{strongest.noteTitle}
</div>
147 |               <div style={{ color: '#64748b', fontSize: '0.85rem', marginBottom: '0.75rem' }}>Average
{strongest.averageScore}% across {strongest.attempts} attempt{strongest.attempts !== 1 ? 's' : ''}</div>
148 |               <div style={{ color: '#94a3b8', fontSize: '0.85rem' }}>{strongest.recommendation}</div>
149 |             </div>
150 |           )}
151 |         </div>
152 |       )}
153 |
154 |       {/* Topic table */}
155 |       <div style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid rgba(255,255,255,0.06)',
borderRadius: '20px', overflow: 'hidden' }}>
156 |         <div style={{ padding: '1.5rem 1.5rem 1rem' }}>
157 |           <h3 style={{ color: '#f8fafc', fontWeight: 600 }}>Topic Breakdown</h3>
158 |           <p style={{ color: '#64748b', fontSize: '0.8rem', marginTop: '0.25rem' }}>Performance per uploaded
note</p>
159 |         </div>
160 |         <div style={{ overflowX: 'auto' }}>
161 |           <table style={{ width: '100%', borderCollapse: 'collapse' }}>
162 |             <thead>
163 |               <tr style={{ background: 'rgba(255,255,255,0.03)' }}>
164 |                 {['Topic', 'Attempts', 'Avg', 'Best', 'Worst', 'Pass Rate', 'Recommendation'].map((h) => (
165 |                   <th key={h} style={{ textAlign: 'left', padding: '0.75rem 1.25rem', fontSize: '0.75rem',
fontWeight: 600, color: '#64748b', textTransform: 'uppercase', letterSpacing: '0.05em' }}>
166 |                     {h}
167 |                   </th>
168 |                 ))}
169 |               </tr>
170 |             </thead>
171 |             <tbody>
172 |               {topics.map((t) => {
173 |                 const passRate = Math.round((t.passed / t.attempts) * 100);
174 |                 const isWeak = t.averageScore < PASS_THRESHOLD;
175 |                 return (
176 |                   <tr key={t.noteId} style={{ borderTop: '1px solid rgba(255,255,255,0.04)' }}>
177 |                     <td style={{ padding: '0.9rem 1.25rem', color: '#e2e8f0', fontWeight: 500 }}>{t.noteTitle}
</td>
178 |                     <td style={{ padding: '0.9rem 1.25rem', color: '#94a3b8' }}>{t.attempts}</td>
179 |                     <td style={{ padding: '0.9rem 1.25rem', color: isWeak ? '#fb7185' : '#34d399', fontWeight:
600 }}>{t.averageScore}%</td>
180 |                     <td style={{ padding: '0.9rem 1.25rem', color: '#94a3b8' }}>{t.bestScore}%</td>
181 |                     <td style={{ padding: '0.9rem 1.25rem', color: '#94a3b8' }}>{t.worstScore}%</td>
182 |                     <td style={{ padding: '0.9rem 1.25rem', color: '#94a3b8' }}>{passRate}%</td>
183 |                     <td style={{ padding: '0.9rem 1.25rem', color: '#94a3b8', fontSize: '0.85rem' }}
>{t.recommendation}</td>
184 |                   </tr>
185 |                 );
186 |               })}
187 |             </tbody>
188 |           </table>
189 |         </div>
190 |       </div>
191 |     </div>
192 |   );
193 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–194:** Fetches '/api/analytics' via 'useFetch', sorts topics to highlight Weakest and Strongest topics, renders cumulative trend Line chart and pass/fail color-coded Bar chart, and builds the Topic Breakdown table.

> **How to Explain This Module in an Interview:** Combines Chart.js Line and Bar charts with rule-based recommendations.

Module: DashboardPage (`frontend/src/pages/DashboardPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Main student dashboard displaying 4 KPI cards with week-over-week trends, 7-day score chart, recent notes, 'Continue Learning' progress bar, and RemindersPanel.

Why It Is Used & Architectural Role: Landing page for authenticated students.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useEffect } from 'react';
  2 | import { Link, useNavigate } from 'react-router-dom';
  3 | import { useAuth } from '../context/AuthContext';
  4 | import API from '../api/axios';
  5 | import { fetchDashboard } from '../api/dashboard';
  6 | import { Line } from 'react-chartjs-2';
  7 | import RemindersPanel from '../components/RemindersPanel';
  8 | import {
  9 |   Chart as ChartJS, CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip, Legend, Filler
 10 | } from 'chart.js';
 11 |
 12 | ChartJS.register(CategoryScale, LinearScale, PointElement, LineElement, Title, Tooltip, Legend, Filler);
 13 |
 14 | function StatCard({ label, value, icon, subtext, gradientFrom, gradientTo, glowColor }) {
 15 |   return (
 16 |     <div style={{
 17 |       position: 'relative',
 18 |       background: 'rgba(255,255,255,0.03)',
 19 |       backdropFilter: 'blur(20px)',
 20 |       WebkitBackdropFilter: 'blur(20px)',
 21 |       border: '1px solid rgba(255,255,255,0.08)',
 22 |       borderRadius: '20px',
 23 |       padding: '1.5rem',
 24 |       boxShadow: '0 4px 24px rgba(0,0,0,0.2)',
 25 |       overflow: 'hidden',
 26 |       WebkitMaskImage: '-webkit-radial-gradient(white, black)',
 27 |       transform: 'translateZ(0)',
 28 |       transition: 'all 0.3s ease',
 29 |       display: 'flex',
 30 |       flexDirection: 'column',
 31 |     }}
 32 |       onMouseEnter={e => { e.currentTarget.style.transform='translateY(-4px) translateZ(0)';
e.currentTarget.style.boxShadow=`0 12px 40px rgba(0,0,0,0.4), 0 0 40px ${glowColor}1A`;
e.currentTarget.style.borderColor=`${glowColor}40`; }}
 33 |       onMouseLeave={e => { e.currentTarget.style.transform='translateY(0) translateZ(0)';
e.currentTarget.style.boxShadow=`0 4px 24px rgba(0,0,0,0.2)`;
e.currentTarget.style.borderColor='rgba(255,255,255,0.08)'; }}
 34 |     >
 35 |       <div style={{
 36 |         position: 'absolute', top: 0, left: 0, right: 0, height: '1px',
 37 |         background: `linear-gradient(90deg, transparent, ${glowColor}60, transparent)`,
 38 |       }} />
 39 |       <div style={{ display: 'flex', gap: '1rem', alignItems: 'flex-start' }}>
 40 |         <div style={{
 41 |           width: '46px', height: '46px',
 42 |           background: `linear-gradient(135deg, ${gradientFrom}30, ${gradientTo}20)`,
 43 |           border: `1px solid ${gradientFrom}40`,
 44 |           borderRadius: '14px',
 45 |           display: 'flex', alignItems: 'center', justifyContent: 'center',
 46 |           fontSize: '1.4rem', flexShrink: 0,
 47 |         }}>
 48 |           {icon}
 49 |         </div>
 50 |         <div>
 51 |           <p style={{ fontSize: '0.85rem', color: '#cbd5e1', fontWeight: 500, marginBottom: '0.2rem' }}>{label}
</p>
 52 |           <p style={{ fontSize: '1.8rem', fontWeight: 700, color: '#f8fafc', lineHeight: 1, marginBottom:
'0.4rem' }}>{value}</p>
 53 |           <p style={{ fontSize: '0.75rem', color: gradientFrom, display: 'flex', alignItems: 'center', gap:
'0.25rem' }}>
 54 |             {subtext}
 55 |           </p>
 56 |         </div>
 57 |       </div>
 58 |     </div>
 59 |   );
 60 | }
 61 |
 62 | const WEEK_MS = 7 * 24 * 60 * 60 * 1000;
 63 |
 64 | function avg(arr) {
 65 |   return arr.length ? arr.reduce((s, v) => s + v, 0) / arr.length : null;
 66 | }
 67 |
 68 | function countTrend(items, getDate) {
 69 |   const now = Date.now();
```

```
  let recent = 0;
  let previous = 0;
  items.forEach((item) => {
    const t = new Date(getDate(item)).getTime();
    if (Number.isNaN(t)) return;
    if (t >= now - WEEK_MS) recent += 1;
    else if (t >= now - 2 * WEEK_MS) previous += 1;
  });
  const delta = recent - previous;
  if (delta > 0) return `↑ ${delta} from last week`;
  if (delta < 0) return `↓ ${Math.abs(delta)} vs last week`;
  if (recent > 0) return '→ No change this week';
  return '— No activity yet';
}

function weightedTrend(items, getDate, getWeight) {
  const now = Date.now();
  let recent = 0;
  let previous = 0;
  items.forEach((item) => {
    const t = new Date(getDate(item)).getTime();
    if (Number.isNaN(t)) return;
    const w = getWeight(item) || 0;
    if (t >= now - WEEK_MS) recent += w;
    else if (t >= now - 2 * WEEK_MS) previous += w;
  });
  const delta = recent - previous;
  if (delta > 0) return `↑ ${delta} new cards this week`;
  if (delta < 0) return `↓ ${Math.abs(delta)} cards vs last week`;
  if (recent > 0) return '→ No new cards this week';
  return '— No cards yet';
}

function scoreTrend(attempts) {
  const now = Date.now();
  const recent = [];
  const previous = [];
  attempts.forEach((a) => {
    const t = new Date(a.attemptedAt).getTime();
    if (Number.isNaN(t)) return;
    const p = Number(a.percentage);
    if (t >= now - WEEK_MS) recent.push(p);
    else if (t >= now - 2 * WEEK_MS) previous.push(p);
  });
  const r = avg(recent);
  const p = avg(previous);
  if (r === null && p === null) return { sub: 'No quizzes yet', pill: '🎯 Take your first quiz' };
  if (r === null) return { sub: 'No quizzes this week', pill: '📉 No quizzes this week' };
  if (p === null) return { sub: `↑ ${r.toFixed(1)}% avg this week`, pill: `📈 ${r.toFixed(0)}% avg this week`
};
  const delta = r - p;
  if (delta > 0.5) return { sub: `↑ +${delta.toFixed(1)}% improvement`, pill: `📈 +${delta.toFixed(0)}%
improvement` };
  if (delta < -0.5) return { sub: `↓ ${Math.abs(delta).toFixed(1)}% from last week`, pill: `📉 -
${Math.abs(delta).toFixed(0)}% this week` };
  return { sub: '→ Steady performance', pill: '📊 Scores steady' };
}

export default function DashboardPage() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const [stats, setStats] = useState({ totalNotes: 0, totalQuizzes: 0, averageScore: 0, totalFlashcards: 0 });
  const [scoreHistory, setScoreHistory] = useState({ content: [], pageNumber: 0, pageSize: 7, totalElements: 0,
totalPages: 0, last: true });
  const [recentNotes, setRecentNotes] = useState([]);
  const [trends, setTrends] = useState({ notes: '', quizzes: '', score: '', scorePill: '', cards: '',
pendingQuizzes: 0, best: null, lowest: null, passRate: null });
  const [failingQuizzes, setFailingQuizzes] = useState([]);
  const [quizRetakeMap, setQuizRetakeMap] = useState({});
  const [materialsStatus, setMaterialsStatus] = useState({});
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const dashRes = await fetchDashboard(0, 7);
        const { totalQuizzes, averageScore, totalFlashcards, scoreHistory: sh } = dashRes;
        setStats(prev => ({ ...prev, totalQuizzes, averageScore, totalFlashcards }));
        setScoreHistory(sh);
        const notesRes = await API.get('/api/notes');
        const notes = notesRes.data;
        setStats(prev => ({ ...prev, totalNotes: notes.length }));
        setRecentNotes(notes.slice(0, 4));

```

```
149 |         const historyRes = await API.get('/api/quiz/history');
150 |         const attempts = historyRes.data;
151 |         setFailingQuizzes(attempts.filter((a) => Number(a.percentage) < 60));
152 |
153 |         const materialsRes = await Promise.allSettled(
154 |           notes.map((n) => API.get(`/api/notes/${n.noteId}/study-materials`))
155 |         );
156 |         const materialsByNote = {};
157 |         notes.forEach((n, i) => {
158 |           if (materialsRes[i].status === 'fulfilled') materialsByNote[n.noteId] = materialsRes[i].value.data;
159 |         });
160 |
161 |         const cardItems = notes.map((n) => ({
162 |           date: n.createdAt,
163 |           count: (materialsByNote[n.noteId]?.flashcards || []).length,
164 |         }));
165 |         const pendingQuizzes = notes.filter((n) => !materialsByNote[n.noteId]?.hasQuiz).length;
166 |
167 |         const retakeMap = {};
168 |         Object.values(materialsByNote).forEach((mat) => {
169 |           if (mat.quizId && mat.questions?.length) {
170 |             retakeMap[mat.quizId] = mat.questions.map(({ id, questionText, optionA, optionB, optionC, optionD
}) => ({ id, questionText, optionA, optionB, optionC, optionD }));
171 |           }
172 |         });
173 |         setQuizRetakeMap(retakeMap);
174 |
175 |         const statusMap = {};
176 |         Object.entries(materialsByNote).forEach(([noteId, mat]) => {
177 |           statusMap[noteId] = {
178 |             hasSummary: !!(mat.summary && String(mat.summary).trim().length > 0),
179 |             hasFlashcards: (mat.flashcards || []).length > 0,
180 |             hasQuiz: !!mat.hasQuiz,
181 |           };
182 |         });
183 |         setMaterialsStatus(statusMap);
184 |         const st = scoreTrend(attempts);
185 |         const percentages = attempts.map((a) => Number(a.percentage));
186 |
187 |         setTrends({
188 |           notes: countTrend(notes, (n) => n.createdAt),
189 |           quizzes: countTrend(attempts, (a) => a.attemptedAt),
190 |           score: st.sub,
191 |           scorePill: st.pill,
192 |           cards: weightedTrend(cardItems, (c) => c.date, (c) => c.count),
193 |           pendingQuizzes,
194 |           best: percentages.length ? Math.max(...percentages) : null,
195 |           lowest: percentages.length ? Math.min(...percentages) : null,
196 |           passRate: attempts.length ? Math.round((attempts.filter((a) => Number(a.percentage) >= 60).length /
attempts.length) * 100) : null,
197 |         });
198 |       } catch (e) {
199 |         console.error(e);
200 |       } finally {
201 |         setLoading(false);
202 |       }
203 |     };
204 |     fetchData();
205 |   }, []);
206 |
207 |   if (loading) return (
208 |     <div style={{ minHeight: '80vh', display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
209 |       <div className="spinner" />
210 |     </div>
211 |   );
212 |
213 |   const handleRetake = (quiz) => {
214 |     const questions = quizRetakeMap[quiz.quizId];
215 |     if (!questions) return;
216 |     navigate(`/quiz/${quiz.quizId}`, { state: { questions } });
217 |   };
218 |
219 |   const hasHistory = scoreHistory.content.length > 0;
220 |
221 |   const chartData = {
222 |     labels: hasHistory ? scoreHistory.content.map(p => new Date(p.date).toLocaleDateString('en-US', { month:
'short', day: 'numeric' })) : [],
223 |     datasets: [{
224 |       label: 'Score %',
225 |       data: hasHistory ? scoreHistory.content.map(p => Number(p.percentage)) : [],
226 |       fill: true,
227 |       borderColor: '#38bdf8',
228 |       backgroundColor: (ctx) => {
229 |         const gradient = ctx.chart.ctx.createLinearGradient(0, 0, 0, ctx.chart.height);
```

```
230 |         gradient.addColorStop(0, 'rgba(56,189,248,0.4)');
231 |         gradient.addColorStop(1, 'rgba(124,58,237,0.05)');
232 |         return gradient;
233 |       },
234 |       tension: 0.4,
235 |       pointBackgroundColor: '#fff',
236 |       pointBorderColor: '#38bdf8',
237 |       pointBorderWidth: 2,
238 |       pointRadius: 4,
239 |     }],
240 |   };
241 |
242 |   const chartOptions = {
243 |     responsive: true,
244 |     maintainAspectRatio: false,
245 |     plugins: { legend: { display: false } },
246 |     scales: {
247 |       x: { grid: { color: 'rgba(255,255,255,0.05)', drawBorder: false }, ticks: { color: '#64748b' } },
248 |       y: { beginAtZero: true, max: 100, grid: { color: 'rgba(255,255,255,0.05)', drawBorder: false, borderDash:
[5, 5] }, ticks: { color: '#64748b', callback: v => `${v}%` } },
249 |     },
250 |   };
251 |
252 |   return (
253 |     <div style={{ padding: '2rem 1rem', maxWidth: '1400px', margin: '0 auto' }}>
254 |       {/* Header row */}
255 |       <div style={{ marginBottom: '2rem' }}>
256 |         <h1 style={{ fontSize: '2rem', fontWeight: 600, color: '#f8fafc', marginBottom: '0.75rem' }}>
257 |           Good <span style={{ color: '#c084fc', fontWeight: 700 }}>evening</span>, {user?.name}! ✊
258 |         </h1>
259 |         <p style={{ color: '#94a3b8', fontSize: '0.95rem', marginBottom: '1.25rem' }}>
260 |           Let's continue your AI learning journey and achieve your goals.
261 |         </p>
262 |         <div style={{ display: 'flex', gap: '1rem', flexWrap: 'wrap' }}>
263 |           <span style={{ background: 'rgba(245,158,11,0.1)', color: '#fcd34d', padding: '0.3rem 0.8rem',
borderRadius: '99px', fontSize: '0.8rem', border: '1px solid rgba(245,158,11,0.2)' }}>
264 |             {recentNotes.length === 0
265 |               ? '🔥 Upload notes to start'
266 |               : trends.pendingQuizzes > 0
267 |                 ? `🔥 ${trends.pendingQuizzes} quizzes pending`
268 |                 : '✓ All quizzes completed'}
269 |           </span>
270 |           <span style={{ background: 'rgba(16,185,129,0.1)', color: '#6ee7b7', padding: '0.3rem 0.8rem',
borderRadius: '99px', fontSize: '0.8rem', border: '1px solid rgba(16,185,129,0.2)' }}>
271 |             {trends.scorePill}
272 |           </span>
273 |           <span style={{ background: 'rgba(56,189,248,0.1)', color: '#7dd3fc', padding: '0.3rem 0.8rem',
borderRadius: '99px', fontSize: '0.8rem', border: '1px solid rgba(56,189,248,0.2)' }}>🎯 Keep it up!</span>
274 |         </div>
275 |       </div>
276 |
277 |       {/* Stat Cards */}
278 |       <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(240px, 1fr))', gap:
'1.25rem', marginBottom: '1.5rem' }}>
279 |         <StatCard label="Total Notes" value={stats.totalNotes} icon="🗐" subtext={trends.notes}
gradientFrom="#38bdf8" gradientTo="#0284c7" glowColor="#38bdf8" />
280 |         <StatCard label="Quizzes Taken" value={stats.totalQuizzes} icon="✓" subtext={trends.quizzes}
gradientFrom="#10b981" gradientTo="#047857" glowColor="#10b981" />
281 |         <StatCard label="Average Score" value={stats.averageScore ? `${Number(stats.averageScore).toFixed(1)}%`
: '—'} icon="📊" subtext={trends.score} gradientFrom="#a855f7" gradientTo="#7e22ce" glowColor="#a855f7" />
282 |         <StatCard label="Flashcards" value={stats.totalFlashcards} icon="📚" subtext={trends.cards}
gradientFrom="#f59e0b" gradientTo="#b45309" glowColor="#f59e0b" />
283 |       </div>
284 |
285 |       {/* Middle Grid */}
286 |       <div className="two-panel-grid" style={{ display: 'grid', gridTemplateColumns: '1fr 350px', gap:
'1.5rem', marginBottom: '1.5rem' }}>
287 |         {/* Chart */}
288 |         <div style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid rgba(255,255,255,0.06)',
borderRadius: '20px', padding: '1.5rem' }}>
289 |           <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '1rem' }}>
290 |             <div>
291 |               <h3 style={{ fontSize: '1.1rem', color: '#f8fafc', fontWeight: 600 }}>Score Progress</h3>
292 |               <p style={{ fontSize: '0.8rem', color: '#64748b' }}>Your quiz performance over time</p>
293 |             </div>
294 |             <select style={{ background: 'rgba(255,255,255,0.05)', color: '#cbd5e1', border: '1px solid
rgba(255,255,255,0.1)', borderRadius: '8px', padding: '0.3rem 0.5rem', fontSize: '0.8rem' }}>
295 |               <option>Last 7 Days</option>
296 |             </select>
297 |           </div>
298 |           <div style={{ height: '240px', width: '100%' }}>
299 |             {hasHistory ? (
300 |               <Line data={chartData} options={chartOptions} />
301 |             ) : (
```

```
302 |               <div style={{ height: '100%', display: 'flex', flexDirection: 'column', alignItems: 'center',
justifyContent: 'center', gap: '0.5rem', textAlign: 'center' }}>
303 |                 <div style={{ fontSize: '2.5rem' }}>📊</div>
304 |                 <p style={{ color: '#64748b', fontSize: '0.9rem', margin: 0 }}>No quiz data yet</p>
305 |                 <Link to="/notes" style={{ fontSize: '0.85rem', color: '#8b5cf6', textDecoration: 'none' }}
>Take your first quiz</Link>
306 |               </div>
307 |             )}
308 |           </div>
309 |
310 |           <div style={{ display: 'grid', gridTemplateColumns: 'repeat(4, 1fr)', gap: '1rem', marginTop:
'1.5rem', paddingTop: '1.5rem', borderTop: '1px dashed rgba(255,255,255,0.1)' }}>
311 |             <div><p style={{ fontSize: '0.75rem', color: '#64748b' }}>Best Score</p><p style={{ color:
'#10b981', fontWeight: 600 }}>{trends.best !== null ? `${trends.best.toFixed(0)}%` : '—'}</p></div>
312 |             <div><p style={{ fontSize: '0.75rem', color: '#64748b' }}>Lowest Score</p><p style={{ color:
'#f43f5e', fontWeight: 600 }}>{trends.lowest !== null ? `${trends.lowest.toFixed(0)}%` : '—'}</p></div>
313 |             <div><p style={{ fontSize: '0.75rem', color: '#64748b' }}>Total Quizzes</p><p style={{ color:
'#f8fafc', fontWeight: 600 }}>{stats.totalQuizzes}</p></div>
314 |             <div><p style={{ fontSize: '0.75rem', color: '#64748b' }}>Pass Rate</p><p style={{ color:
'#f59e0b', fontWeight: 600 }}>{trends.passRate !== null ? `${trends.passRate}%` : '—'}</p></div>
315 |           </div>
316 |         </div>
317 |
318 |         {/* Recent Notes */}
319 |         <div style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid rgba(255,255,255,0.06)',
borderRadius: '20px', padding: '1.5rem' }}>
320 |           <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'1.25rem' }}>
321 |             <h3 style={{ fontSize: '1.1rem', color: '#f8fafc', fontWeight: 600 }}>Recent Notes</h3>
322 |             <Link to="/notes" style={{ fontSize: '0.8rem', color: '#8b5cf6', textDecoration: 'none' }}>View
all</Link>
323 |           </div>
324 |           <div style={{ display: 'flex', flexDirection: 'column', gap: '0.75rem' }}>
325 |             {recentNotes.length === 0 ? (
326 |               <div style={{ textAlign: 'center', padding: '2rem 1rem' }}>
327 |                 <div style={{ fontSize: '2.5rem', marginBottom: '0.75rem', opacity: 0.5 }}>📝</div>
328 |                 <p style={{ color: '#64748b', fontSize: '0.9rem', marginBottom: '1rem' }}>No notes yet. Add
your first note to start studying.</p>
329 |                 <Link to="/notes" className="btn-primary" style={{ fontSize: '0.85rem' }}>Add Note</Link>
330 |               </div>
331 |             ) : recentNotes.map((n, i) => (
332 |               <div key={i} style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between',
padding: '0.75rem', background: 'rgba(255,255,255,0.03)', borderRadius: '12px', border: '1px solid
rgba(255,255,255,0.05)' }}>
333 |                 <div style={{ display: 'flex', gap: '0.75rem', alignItems: 'center' }}>
334 |                   <div style={{ width: '32px', height: '32px', borderRadius: '8px', background:
['rgba(59,130,246,0.2)', 'rgba(168,85,247,0.2)', 'rgba(16,185,129,0.2)', 'rgba(245,158,11,0.2)'][i%4], color:
['#3b82f6', '#a855f7', '#10b981', '#f59e0b'][i%4], display: 'flex', alignItems: 'center', justifyContent: 'center' }}
>📄</div>
335 |                   <div>
336 |                     <p style={{ fontSize: '0.9rem', color: '#e2e8f0', fontWeight: 500 }}>{n.title}</p>
337 |                     <p style={{ fontSize: '0.75rem', color: '#64748b' }}>{new
Date(n.createdAt).toLocaleDateString('en-US', { day: 'numeric', month: 'short', year: 'numeric' })}</p>
338 |                   </div>
339 |                 </div>
340 |                 <button onClick={() => navigate('/history', { state: { selectedNoteId: n.noteId } })} style={{
background: 'rgba(255,255,255,0.05)', border: '1px solid rgba(255,255,255,0.1)', color: '#c4b5fd', padding: '0.3rem
0.6rem', borderRadius: '6px', fontSize: '0.75rem', cursor: 'pointer' }}>Study →</button>
341 |               </div>
342 |             ))}
343 |           </div>
344 |         </div>
345 |       </div>
346 |
347 |       {/* Bottom Grid */}
348 |       <div className="dashboard-grid-2col" style={{ display: 'grid', gridTemplateColumns: '1fr 350px', gap:
'1.5rem' }}>
349 |         {/* Continue Learning */}
350 |         <div style={{ background: 'rgba(255,255,255,0.02)', border: '1px solid rgba(255,255,255,0.06)',
borderRadius: '20px', padding: '1.5rem' }}>
351 |           <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom:
'1rem' }}>
352 |             <div>
353 |               <h3 style={{ fontSize: '1.1rem', color: '#f8fafc', fontWeight: 600 }}>Continue Learning</h3>
354 |               <p style={{ fontSize: '0.8rem', color: '#64748b' }}>{recentNotes.length > 0 ? 'Pick up where you
left off' : 'Start your learning journey'}</p>
355 |             </div>
356 |             <Link to="/history" style={{ fontSize: '0.8rem', color: '#8b5cf6', textDecoration: 'none' }}>View
all</Link>
357 |           </div>
358 |           {recentNotes.length === 0 ? (
359 |             <div style={{ textAlign: 'center', padding: '1.75rem 1rem' }}>
360 |               <div style={{ fontSize: '2.25rem', marginBottom: '0.75rem' }}>🚀</div>
361 |               <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: '0 0 0.25rem' }}>No notes yet</p>
```

```
362 |               <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0 0 1rem' }}>Upload your first note
and let AI generate study materials</p>
363 |               <button onClick={() => navigate('/notes')} className="btn-primary" style={{ padding: '0.5rem
1rem', fontSize: '0.85rem' }}>
364 |                 Upload your first note →
365 |               </button>
366 |             </div>
367 |           ) : (() => {
368 |             const cont = recentNotes[0];
369 |             const status = materialsStatus[cont.noteId];
370 |             const done = status ? [status.hasSummary, status.hasFlashcards,
status.hasQuiz].filter(Boolean).length : 0;
371 |             const pct = status ? Math.round((done / 3) * 100) : 0;
372 |             return (
373 |               <div style={{ display: 'flex', alignItems: 'center', gap: '1rem', background:
'rgba(255,255,255,0.03)', padding: '1.25rem', borderRadius: '16px', border: '1px solid rgba(255,255,255,0.05)' }}>
374 |                 <div style={{ width: '40px', height: '40px', borderRadius: '10px', background:
'rgba(168,85,247,0.2)', color: '#a855f7', display: 'flex', alignItems: 'center', justifyContent: 'center', fontSize:
'1.2rem' }}>📄</div>
375 |                 <div style={{ flex: 1, minWidth: 0 }}>
376 |                   <p style={{ fontSize: '0.95rem', color: '#f8fafc', fontWeight: 500, marginBottom: '0.4rem',
whiteSpace: 'nowrap', overflow: 'hidden', textOverflow: 'ellipsis' }}>
377 |                     {cont.title}
378 |                   </p>
379 |                   <div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>
380 |                     <div style={{ flex: 1, height: '6px', background: 'rgba(255,255,255,0.1)', borderRadius:
'99px' }}>
381 |                       <div style={{ width: `${pct}%`, height: '100%', background: 'linear-gradient(90deg,
#a855f7, #38bdf8)', borderRadius: '99px' }} />
382 |                     </div>
383 |                     <span style={{ fontSize: '0.75rem', color: '#64748b', whiteSpace: 'nowrap' }}>{pct}%
Complete</span>
384 |                   </div>
385 |                 </div>
386 |                 <button
387 |                   onClick={() => navigate('/history', { state: { selectedNoteId: cont.noteId } })}
388 |                   className="btn-primary"
389 |                   style={{ padding: '0.5rem 1rem', fontSize: '0.85rem' }}
390 |                 >
391 |                   Continue Study →
392 |                 </button>
393 |               </div>
394 |             );
395 |           })()}
396 |         </div>
397 |       </div>
398 |
399 |       <RemindersPanel failingQuizzes={failingQuizzes} onRetake={handleRetake} quizRetakeMap={quizRetakeMap} />
400 |
401 |     </div>
402 |   );
403 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–331:** Defines 'StatCard' and trend helpers ('countTrend', 'weightedTrend', 'scoreTrend'), loads dashboard/notes/history/materials via 'Promise.allSettled', and renders the responsive grid.

> **How to Explain This Module in an Interview:** Fault-tolerant parallel loading with 'Promise.allSettled'.

Module: FlashcardsPage (`frontend/src/pages/FlashcardsPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Spaced-repetition flashcard review hub and per-note flashcard browser.

Why It Is Used & Architectural Role: Integrates with '/api/flashcards/due' and '/api/flashcards/review' to move cards through Leitner boxes 1–5.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useEffect } from 'react';
  2 | import { useNavigate } from 'react-router-dom';
  3 | import API from '../api/axios';
  4 | import { formatDate } from '../utils/formatDate';
  5 |
  6 | export default function FlashcardsPage() {
  7 |   const navigate = useNavigate();
  8 |   const [notes, setNotes] = useState([]);
  9 |   const [selectedNote, setSelectedNote] = useState(null);
 10 |   const [flashcards, setFlashcards] = useState([]);
 11 |   const [hasFlashcards, setHasFlashcards] = useState(false);
```

```
  const [flipped, setFlipped] = useState({});
  const [loadingNotes, setLoadingNotes] = useState(true);
  const [loadingCards, setLoadingCards] = useState(false);
  const [dueCards, setDueCards] = useState([]);
  const [reviewMode, setReviewMode] = useState(false);
  const [reviewFlipped, setReviewFlipped] = useState(false);
  const [reviewError, setReviewError] = useState('');

  const fetchDueCards = async () => {
    try {
      const res = await API.get('/api/flashcards/due');
      setDueCards(res.data || []);
    } catch { setDueCards([]); }
  };

  useEffect(() => {
    const fetchNotes = async () => {
      try {
        const res = await API.get('/api/notes');
        setNotes(res.data);
      } catch (err) {
        console.error('Failed to fetch notes', err);
      } finally {
        setLoadingNotes(false);
      }
    };
    fetchNotes();
    fetchDueCards();
  }, []);

  const handleSelectNote = async (note) => {
    setSelectedNote(note);
    setFlashcards([]);
    setHasFlashcards(false);
    setFlipped({});
    setReviewMode(false);
    setLoadingCards(true);
    try {
      const res = await API.get(`/api/notes/${note.noteId}/study-materials`);
      setFlashcards(res.data.flashcards || []);
      setHasFlashcards(!!res.data.hasFlashcards);
    } catch (err) {
      console.error('Failed to fetch flashcards', err);
    } finally {
      setLoadingCards(false);
    }
  };

  const handleReview = async (correct) => {
    const card = dueCards[0];
    if (!card) { setReviewMode(false); return; }
    setReviewError('');
    try {
      const res = await API.post('/api/flashcards/review', { flashcardId: card.id, correct });
      const updated = res.data;
      setDueCards(prev => prev.filter(c => c.id !== updated.id));
      setFlashcards(prev => prev.map(f => f.id === updated.id ? { ...f, boxLevel: updated.boxLevel,
nextReviewDate: updated.nextReviewDate } : f));
      setReviewFlipped(false);
    } catch (err) {
      setReviewError(err.response?.data?.message || 'Failed to save review');
    }
  };

  return (
    <div style={{ padding: '2rem 1rem', maxWidth: '1200px', margin: '0 auto' }}>
      <div style={{ marginBottom: '2rem' }}>
        <h1 style={{ fontSize: '2rem', fontWeight: 800, background: 'linear-gradient(135deg, #a78bfa,
#38bdf8)', WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent' }}>
          🃏 Flashcards
        </h1>
        <p style={{ color: '#64748b', marginTop: '0.4rem' }}>Spaced repetition (Leitner boxes) — review cards
daily to move them to higher boxes and longer intervals.</p>
      </div>

      {/* Spaced repetition review panel */}
      <div style={{ marginBottom: '1.5rem', background: 'rgba(255,255,255,0.02)', border: '1px solid
rgba(255,255,255,0.06)', borderRadius: '16px', padding: '1.25rem' }}>
        {!reviewMode && dueCards.length === 0 && (
          <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', flexWrap:
'wrap', gap: '1rem' }}>
            <div style={{ display: 'flex', alignItems: 'center', gap: '0.75rem' }}>
              <span style={{ fontSize: '1.5rem' }}>✓</span>
              <div>
```

```
 91 |                 <p style={{ fontSize: '0.95rem', fontWeight: 600, color: '#e2e8f0', margin: 0 }}>No cards due
for review</p>
 92 |                 <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0.15rem 0 0' }}>All caught up! New
cards are due as their review interval expires.</p>
 93 |               </div>
 94 |             </div>
 95 |             {flashcards.length > 0 && (
 96 |               <button
 97 |                 onClick={() => setReviewMode(true)}
 98 |                 className="btn-primary"
 99 |                 style={{ padding: '0.5rem 1rem', fontSize: '0.85rem' }}
100 |               >
101 |                 Browse all cards →
102 |               </button>
103 |             )}
104 |           </div>
105 |         )}
106 |
107 |         {!reviewMode && dueCards.length > 0 && (
108 |           <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', flexWrap:
'wrap', gap: '1rem' }}>
109 |             <div style={{ display: 'flex', alignItems: 'center', gap: '0.75rem' }}>
110 |               <span style={{ fontSize: '1.5rem' }}>⏰</span>
111 |               <div>
112 |                 <p style={{ fontSize: '0.95rem', fontWeight: 600, color: '#e2e8f0', margin: 0 }}
>{dueCards.length} card{dueCards.length !== 1 ? 's' : ''} due for review</p>
113 |                 <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0.15rem 0 0' }}>Answer correctly to
move cards up the Leitner boxes</p>
114 |               </div>
115 |             </div>
116 |             <button
117 |               onClick={() => { setReviewMode(true); setReviewFlipped(false); }}
118 |               className="btn-primary"
119 |               style={{ padding: '0.6rem 1.25rem', fontSize: '0.9rem' }}
120 |             >
121 |               Start review →
122 |             </button>
123 |           </div>
124 |         )}
125 |
126 |         {reviewMode && dueCards.length > 0 && (
127 |           <div>
128 |             <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', marginBottom:
'1rem' }}>
129 |               <p style={{ fontSize: '0.9rem', fontWeight: 600, color: '#e2e8f0', margin: 0 }}>
130 |                 Reviewing card 1 of {dueCards.length}
131 |               </p>
132 |               <button onClick={() => setReviewMode(false)} style={{ background: 'none', border: 'none', color:
'#64748b', cursor: 'pointer', fontSize: '0.85rem' }}>
133 |                 Exit review ✕
134 |               </button>
135 |             </div>
136 |             <div
137 |               onClick={() => setReviewFlipped(!reviewFlipped)}
138 |               title="Click to flip"
139 |               style={{ perspective: '1000px', cursor: 'pointer', height: '260px', maxWidth: '560px', margin: '0
auto' }}
140 |             >
141 |               <div style={{
142 |                 position: 'relative', width: '100%', height: '100%',
143 |                 transformStyle: 'preserve-3d',
144 |                 transform: reviewFlipped ? 'rotateY(180deg)' : 'rotateY(0deg)',
145 |                 transition: 'transform 0.5s cubic-bezier(0.4,0,0.2,1)',
146 |               }}>
147 |                 <div style={{
148 |                   position: 'absolute', inset: 0, backfaceVisibility: 'hidden',
149 |                   background: 'linear-gradient(135deg, rgba(91,33,182,0.4), rgba(6,182,212,0.2))',
150 |                   border: '1px solid rgba(139,92,246,0.4)',
151 |                   borderRadius: '1rem', padding: '1.5rem',
152 |                   display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: 'center',
textAlign: 'center',
153 |                 }}>
154 |                   <p style={{ fontSize: '0.75rem', color: '#a78bfa', fontWeight: 600, textTransform:
'uppercase', letterSpacing: '0.06em', margin: '0 0 0.75rem' }}>Question</p>
155 |                   <p style={{ fontSize: '1.05rem', color: '#ede9fe', margin: 0, lineHeight: 1.6 }}
>{dueCards[0].question}</p>
156 |                   <p style={{ fontSize: '0.7rem', color: '#475569', marginTop: '1rem' }}>Click to flip</p>
157 |                 </div>
158 |                 <div style={{
159 |                   position: 'absolute', inset: 0, backfaceVisibility: 'hidden',
160 |                   transform: 'rotateY(180deg)',
161 |                   background: 'linear-gradient(135deg, rgba(6,182,212,0.3), rgba(16,185,129,0.2))',
162 |                   border: '1px solid rgba(56,189,248,0.35)',
163 |                   borderRadius: '1rem', padding: '1.5rem',
```

```
                  display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: 'center',
textAlign: 'center',
                }}>
                  <p style={{ fontSize: '0.75rem', color: '#38bdf8', fontWeight: 600, textTransform:
'uppercase', letterSpacing: '0.06em', margin: '0 0 0.75rem' }}>Answer</p>
                  <p style={{ fontSize: '1rem', color: '#f0f9ff', margin: 0, lineHeight: 1.6 }}
>{dueCards[0].answer}</p>
                </div>
              </div>
            </div>
            {reviewError && <p style={{ color: '#f87171', fontSize: '0.85rem', textAlign: 'center', marginTop:
'0.75rem' }}>{reviewError}</p>}
            <div style={{ display: 'flex', justifyContent: 'center', gap: '1rem', marginTop: '1.25rem' }}>
              <button
                onClick={() => handleReview(false)}
                style={{
                  padding: '0.6rem 1.75rem', borderRadius: '12px', border: '1px solid rgba(244,63,94,0.4)',
                  background: 'rgba(244,63,94,0.1)', color: '#fb7185', fontWeight: 600, cursor: 'pointer',
fontSize: '0.9rem',
                }}
              >
                🔄 Still learning
              </button>
              <button
                onClick={() => handleReview(true)}
                style={{
                  padding: '0.6rem 1.75rem', borderRadius: '12px', border: '1px solid rgba(16,185,129,0.4)',
                  background: 'rgba(16,185,129,0.1)', color: '#34d399', fontWeight: 600, cursor: 'pointer',
fontSize: '0.9rem',
                }}
              >
                ✓ Got it
              </button>
            </div>
          </div>
        )}

        {reviewMode && dueCards.length === 0 && (
          <div style={{ textAlign: 'center', padding: '1.5rem' }}>
            <div style={{ fontSize: '2.5rem', marginBottom: '0.5rem' }}>🎉</div>
            <p style={{ color: '#e2e8f0', fontWeight: 600, margin: '0 0 0.25rem' }}>Review complete!</p>
            <p style={{ color: '#64748b', fontSize: '0.85rem', margin: '0 0 1rem' }}>All due cards reviewed.
Check back tomorrow for the next round.</p>
            <button onClick={() => setReviewMode(false)} className="btn-primary" style={{ padding: '0.5rem
1.25rem', fontSize: '0.85rem' }}>
              Done
            </button>
          </div>
        )}
      </div>

      <div className="sidebar-note-list" style={{ display: 'grid', gridTemplateColumns: '280px 1fr', gap:
'1.5rem', alignItems: 'start' }}>

        {/* Notes list */}
        <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.07)',
borderRadius: '1.25rem', overflow: 'hidden' }}>
          <div style={{ padding: '1rem 1.25rem', borderBottom: '1px solid rgba(255,255,255,0.06)', background:
'rgba(124,58,237,0.08)' }}>
            <p style={{ fontSize: '0.8rem', fontWeight: 600, color: '#a78bfa', textTransform: 'uppercase',
letterSpacing: '0.08em', margin: 0 }}>Notes</p>
          </div>

          {loadingNotes ? (
            <div style={{ padding: '2rem', textAlign: 'center' }}>
              <div className="spinner" style={{ margin: '0 auto', width: '2rem', height: '2rem' }} />
            </div>
          ) : notes.length === 0 ? (
            <div style={{ padding: '2rem', textAlign: 'center', color: '#64748b' }}>
              <p style={{ fontSize: '0.875rem' }}>No notes yet.</p>
              <button onClick={() => navigate('/notes')} className="btn-primary" style={{ marginTop: '1rem',
padding: '0.5rem 1.2rem', fontSize: '0.8rem' }}>
                Upload Notes
              </button>
            </div>
          ) : (
            <div style={{ maxHeight: 'calc(100vh - 260px)', overflowY: 'auto' }}>
              {notes.map((note) => {
                const isActive = selectedNote?.noteId === note.noteId;
                return (
                  <button
                    key={note.noteId}
                    onClick={() => handleSelectNote(note)}
                    style={{
```

```
                        display: 'block', width: '100%', textAlign: 'left',
                        padding: '0.9rem 1.25rem', border: 'none', cursor: 'pointer',
                        background: isActive ? 'rgba(124,58,237,0.15)' : 'transparent',
                        borderLeft: isActive ? '3px solid #7c3aed' : '3px solid transparent',
                        borderBottom: '1px solid rgba(255,255,255,0.04)',
                        transition: 'all 0.15s ease',
                      }}
                      onMouseEnter={e => { if (!isActive) e.currentTarget.style.background =
'rgba(255,255,255,0.04)'; }}
                      onMouseLeave={e => { if (!isActive) e.currentTarget.style.background = 'transparent'; }}
                    >
                      <p style={{ fontSize: '0.875rem', fontWeight: isActive ? 600 : 400, color: isActive ?
'#c4b5fd' : '#cbd5e1', margin: 0, lineHeight: 1.4 }}>
                        {note.title}
                      </p>
                      <p style={{ fontSize: '0.72rem', color: '#475569', margin: '0.15rem 0 0' }}
>{formatDate(note.createdAt)}</p>
                    </button>
                  );
                })}
              </div>
            )}
          </div>

          {/* Flashcards panel */}
          <div>
            {!selectedNote ? (
              <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent:
'center', minHeight: '380px', background: 'rgba(255,255,255,0.02)', border: '2px dashed rgba(255,255,255,0.08)',
borderRadius: '1.25rem', color: '#475569', textAlign: 'center', padding: '2rem' }}>
                <div style={{ fontSize: '4rem', marginBottom: '1rem' }}>☜</div>
                <p style={{ fontSize: '1.1rem', fontWeight: 600, color: '#64748b' }}>Select a note from the
left</p>
                <p style={{ fontSize: '0.875rem', marginTop: '0.4rem' }}>to view its flashcards</p>
              </div>
            ) : loadingCards ? (
              <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent:
'center', minHeight: '380px', gap: '1rem' }}>
                <div className="spinner" />
                <p style={{ color: '#64748b' }}>Loading flashcards...</p>
              </div>
            ) : !hasFlashcards ? (
              <div style={{ padding: '3.5rem 2rem', textAlign: 'center', background: 'rgba(255,255,255,0.02)',
borderRadius: '1.25rem', border: '1px dashed rgba(255,255,255,0.08)' }}>
                <div style={{ fontSize: '3.5rem', marginBottom: '1rem' }}> </div>
                <h3 style={{ fontSize: '1.2rem', fontWeight: 700, color: '#e2e8f0', margin: '0 0 0.5rem' }}>No
Flashcards Generated</h3>
                <p style={{ color: '#64748b', fontSize: '0.9rem', maxWidth: '420px', margin: '0 auto 1.5rem',
lineHeight: 1.6 }}>
                  Generate AI flashcards for "{selectedNote.title}" with a single click.
                </p>
                <button onClick={() => navigate(`/study/${selectedNote.noteId}`)} className="btn-primary">
                  ✨ Generate Flashcards
                </button>
              </div>
            ) : (
              <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(280px, 1fr))', gap:
'1rem' }}>
                {flashcards.map((fc, idx) => (
                  <div
                    key={fc.id}
                    onClick={() => setFlipped(prev => ({ ...prev, [idx]: !prev[idx] }))}
                    title="Click to flip"
                    style={{ perspective: '1000px', cursor: 'pointer', height: '230px' }}
                  >
                    <div style={{
                      position: 'relative', width: '100%', height: '100%',
                      transformStyle: 'preserve-3d',
                      transform: flipped[idx] ? 'rotateY(180deg)' : 'rotateY(0deg)',
                      transition: 'transform 0.5s cubic-bezier(0.4,0,0.2,1)',
                    }}>
                      <div style={{
                        position: 'absolute', inset: 0, backfaceVisibility: 'hidden',
                        background: 'linear-gradient(135deg, rgba(91,33,182,0.35), rgba(109,40,217,0.25))',
                        border: '1px solid rgba(139,92,246,0.35)',
                        borderRadius: '1rem', padding: '1.25rem', display: 'flex', flexDirection: 'column',
justifyContent: 'space-between',
                      }}>
                        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
flexShrink: 0 }}>
                          <p style={{ fontSize: '0.75rem', color: '#a78bfa', fontWeight: 600, textTransform:
'uppercase', letterSpacing: '0.06em', margin: 0 }}>Question {idx + 1}</p>
                          <span
                            title="Leitner box level — higher = longer review interval"
```

```
                            style={{
                              fontSize: '0.7rem', fontWeight: 700, color: '#c4b5fd',
                              background: 'rgba(124,58,237,0.2)', border: '1px solid rgba(139,92,246,0.3)',
                              borderRadius: '99px', padding: '0.15rem 0.6rem',
                            }}
                          >
                            Box {fc.boxLevel || 1}/5
                          </span>
                        </div>
                        <div style={{ flexGrow: 1, overflowY: 'auto', padding: '0.5rem 0', margin: '0.2rem 0' }}>
                          <p style={{ fontSize: '0.9rem', color: '#ede9fe', margin: 0, lineHeight: 1.5 }}
>{fc.question}</p>
                        </div>
                        <p style={{ fontSize: '0.7rem', color: '#7c6fb0', margin: 0, textAlign: 'right' }}>Click
to reveal answer →</p>
                      </div>
                      <div style={{
                        position: 'absolute', inset: 0, backfaceVisibility: 'hidden',
                        transform: 'rotateY(180deg)',
                        background: 'linear-gradient(135deg, rgba(4,78,59,0.4), rgba(4,120,87,0.3))',
                        border: '1px solid rgba(16,185,129,0.35)',
                        borderRadius: '1rem', padding: '1.25rem', display: 'flex', flexDirection: 'column',
justifyContent: 'space-between',
                      }}>
                        <p style={{ fontSize: '0.75rem', color: '#34d399', fontWeight: 600, textTransform:
'uppercase', letterSpacing: '0.06em', margin: 0, flexShrink: 0 }}>Answer</p>
                        <div style={{ flexGrow: 1, overflowY: 'auto', padding: '0.5rem 0', margin: '0.2rem 0' }}>
                          <p style={{ fontSize: '0.9rem', color: '#d1fae5', margin: 0, lineHeight: 1.5 }}
>{fc.answer}</p>
                        </div>
                        <p style={{ fontSize: '0.7rem', color: '#6ab79b', margin: 0, textAlign: 'right' }}>←
Click to flip back</p>
                      </div>
                    </div>
                  </div>
                ))}
              </div>
            )}
          </div>
        </div>
      </div>
    );
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–295:** Loads due cards and notes on mount, provides interactive Review Mode ('🔁 Still learning' vs '✓ Got it'), and displays per-note 3D flashcards with 'Box X/5' badges.

How to Explain This Module in an Interview: Full frontend implementation of the Leitner spaced-repetition workflow.

Module: HistoryPage (`frontend/src/pages/HistoryPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Two-panel master-detail archive of all uploaded notes and their generated summaries, flashcards, quiz Q&A keys, and raw text.

Why It Is Used & Architectural Role: Allows students to revisit any note's materials or trigger generation if not yet generated.

Complete Source Code (Every Line Numbered):

```
 1 | import { useState, useEffect, useRef } from 'react';
 2 | import { useNavigate, useLocation } from 'react-router-dom';
 3 | import API from '../api/axios';
 4 | import { formatDate } from '../utils/formatDate';
 5 |
 6 | const optionBg = {
 7 |   A: 'rgba(124,58,237,0.12)',
 8 |   B: 'rgba(6,182,212,0.12)',
 9 |   C: 'rgba(236,72,153,0.12)',
10 |   D: 'rgba(245,158,11,0.12)',
11 | };
12 | const optionBorder = {
13 |   A: 'rgba(124,58,237,0.35)',
14 |   B: 'rgba(6,182,212,0.35)',
15 |   C: 'rgba(236,72,153,0.35)',
16 |   D: 'rgba(245,158,11,0.35)',
17 | };
18 | const optionColor = {
19 |   A: '#a78bfa',
20 |   B: '#38bdf8',
```

```
  C: '#f472b6',
  D: '#fbbf24',
};

export default function HistoryPage() {
  const navigate = useNavigate();
  const location = useLocation();
  const autoSelectedRef = useRef(false);
  const [notes, setNotes] = useState([]);
  const [selectedNote, setSelectedNote] = useState(null);
  const [materials, setMaterials] = useState(null);
  const [loadingNotes, setLoadingNotes] = useState(true);
  const [loadingMaterials, setLoadingMaterials] = useState(false);
  const [generating, setGenerating] = useState(false);
  const [genError, setGenError] = useState('');
  const [activeSection, setActiveSection] = useState('summary');
  const [flippedCards, setFlippedCards] = useState({});

  useEffect(() => {
    fetchNotes();
  }, []);

  const fetchNotes = async () => {
    try {
      const res = await API.get('/api/notes');
      setNotes(res.data);
      const targetId = location.state?.selectedNoteId;
      if (targetId && !autoSelectedRef.current) {
        autoSelectedRef.current = true;
        const target = res.data.find((n) => n.noteId === targetId);
        if (target) {
          setLoadingNotes(false);
          setSelectedNote(target);
          setMaterials(null);
          setActiveSection('summary');
          setFlippedCards({});
          setLoadingMaterials(true);
          try {
            const matRes = await API.get(`/api/notes/${targetId}/study-materials`);
            setMaterials(matRes.data);
          } catch (err) {
            console.error('Failed to load materials', err);
            setMaterials({ error: 'Failed to load materials.' });
          } finally {
            setLoadingMaterials(false);
          }
          navigate(location.pathname, { replace: true, state: null });
          return;
        }
      }
    } catch (err) {
      console.error('Failed to fetch notes', err);
    } finally {
      setLoadingNotes(false);
    }
  };

  const handleSelectNote = async (note) => {
    setSelectedNote(note);
    setMaterials(null);
    setFlippedCards({});
    setActiveSection('summary');
    setGenError('');
    setLoadingMaterials(true);
    try {
      const res = await API.get(`/api/notes/${note.noteId}/study-materials`);
      setMaterials(res.data);
    } catch (err) {
      console.error('Failed to fetch study materials', err);
      setMaterials({ error: 'Failed to load materials for this note.' });
    } finally {
      setLoadingMaterials(false);
    }
  };

  const handleGenerateNow = async () => {
    if (!selectedNote) return;
    setGenerating(true);
    setGenError('');
    try {
      await API.post('/api/study/generate', { noteId: selectedNote.noteId });
      // Re-fetch materials after generation
      await handleSelectNote(selectedNote);
    } catch (err) {
```

```
105 |       console.error('Generation failed', err);
106 |       setGenError(err.response?.data?.message || 'Failed to generate study materials. Please try again.');
107 |     } finally {
108 |       setGenerating(false);
109 |     }
110 |   };
111 |
112 |   const toggleFlip = (idx) => {
113 |     setFlippedCards(prev => ({ ...prev, [idx]: !prev[idx] }));
114 |   };
115 |
116 |   const hasAnyMaterials = materials && (materials.hasSummary || materials.hasFlashcards || materials.hasQuiz);
117 |
118 |   return (
119 |     <div style={{ padding: '2rem 1rem', maxWidth: '1400px', margin: '0 auto' }}>
120 |       <div style={{ margin: '0 auto' }}>
121 |
122 |         {/* Header */}
123 |         <div style={{ marginBottom: '2rem' }}>
124 |           <h1 style={{ fontSize: '2rem', fontWeight: 800, background: 'linear-gradient(135deg, #a78bfa,
#38bdf8)', WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent' }}>
125 |             📚 Notes History
126 |           </h1>
127 |           <p style={{ color: '#64748b', marginTop: '0.4rem' }}>Select a note to revisit your generated
summaries, flashcards, and quizzes.</p>
128 |         </div>
129 |
130 |         <div className="sidebar-note-list" style={{ display: 'grid', gridTemplateColumns: '300px 1fr', gap:
'1.5rem', alignItems: 'start' }}>
131 |
132 |           {/* Left Panel: Notes List */}
133 |           <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.07)',
borderRadius: '1.25rem', overflow: 'hidden' }}>
134 |             <div style={{ padding: '1rem 1.25rem', borderBottom: '1px solid rgba(255,255,255,0.06)',
background: 'rgba(124,58,237,0.08)', display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
135 |               <p style={{ fontSize: '0.8rem', fontWeight: 600, color: '#a78bfa', textTransform: 'uppercase',
letterSpacing: '0.08em', margin: 0 }}>Your Notes</p>
136 |               <span style={{ fontSize: '0.75rem', color: '#64748b', background: 'rgba(255,255,255,0.05)',
padding: '0.15rem 0.5rem', borderRadius: '6px' }}>{notes.length}</span>
137 |             </div>
138 |
139 |             {loadingNotes ? (
140 |               <div style={{ padding: '2rem', textAlign: 'center' }}>
141 |                 <div style={{ width: '2rem', height: '2rem', border: '2px solid rgba(124,58,237,0.2)',
borderTopColor: '#7c3aed', borderRadius: '50%', animation: 'spin 0.8s linear infinite', margin: '0 auto' }} />
142 |               </div>
143 |             ) : notes.length === 0 ? (
144 |               <div style={{ padding: '2rem', textAlign: 'center', color: '#475569' }}>
145 |                 <div style={{ fontSize: '2.5rem', marginBottom: '0.5rem' }}>📝</div>
146 |                 <p style={{ fontSize: '0.875rem' }}>No notes uploaded yet.</p>
147 |                 <button onClick={() => navigate('/notes')} style={{ marginTop: '1rem', padding: '0.5rem
1.2rem', background: 'linear-gradient(135deg, #7c3aed, #06b6d4)', border: 'none', borderRadius: '8px', color:
'white', cursor: 'pointer', fontSize: '0.8rem', fontWeight: 600 }}>
148 |                   Upload Notes
149 |                 </button>
150 |               </div>
151 |             ) : (
152 |               <div style={{ maxHeight: 'calc(100vh - 220px)', overflowY: 'auto' }}>
153 |                 {notes.map((note) => {
154 |                   const isActive = selectedNote?.noteId === note.noteId;
155 |                   return (
156 |                     <button
157 |                       key={note.noteId}
158 |                       onClick={() => handleSelectNote(note)}
159 |                       style={{
160 |                         display: 'block', width: '100%', textAlign: 'left',
161 |                         padding: '0.9rem 1.25rem', border: 'none', cursor: 'pointer',
162 |                         background: isActive ? 'rgba(124,58,237,0.15)' : 'transparent',
163 |                         borderLeft: isActive ? '3px solid #7c3aed' : '3px solid transparent',
164 |                         borderBottom: '1px solid rgba(255,255,255,0.04)',
165 |                         transition: 'all 0.15s ease',
166 |                       }}
167 |                       onMouseEnter={e => { if (!isActive) e.currentTarget.style.background =
'rgba(255,255,255,0.04)'; }}
168 |                       onMouseLeave={e => { if (!isActive) e.currentTarget.style.background = 'transparent'; }}
169 |                     >
170 |                       <p style={{ fontSize: '0.875rem', fontWeight: isActive ? 600 : 400, color: isActive ?
'#c4b5fd' : '#cbd5e1', margin: 0, marginBottom: '0.2rem', lineHeight: 1.4 }}>
171 |                         {note.title}
172 |                       </p>
173 |                       <p style={{ fontSize: '0.72rem', color: '#475569', margin: 0 }}>
174 |                         {formatDate(note.createdAt)}
175 |                       </p>
176 |                     </button>
```

```
177 |                   );
178 |                 })}
179 |               </div>
180 |             )}
181 |           </div>
182 |
183 |           {/* Right Panel: Study Materials */}
184 |           <div>
185 |             {!selectedNote && (
186 |               <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent:
'center', minHeight: '400px', background: 'rgba(255,255,255,0.02)', border: '2px dashed rgba(255,255,255,0.08)',
borderRadius: '1.25rem', color: '#475569', textAlign: 'center', padding: '2rem' }}>
187 |                 <div style={{ fontSize: '4rem', marginBottom: '1rem' }}>☜</div>
188 |                 <p style={{ fontSize: '1.1rem', fontWeight: 600, color: '#64748b' }}>Select a note from the
left</p>
189 |                 <p style={{ fontSize: '0.875rem', marginTop: '0.4rem' }}>to view its summary, flashcards, and
quiz Q&A</p>
190 |               </div>
191 |             )}
192 |
193 |             {loadingMaterials && (
194 |               <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent:
'center', minHeight: '400px', gap: '1rem' }}>
195 |                 <div style={{ width: '3rem', height: '3rem', border: '3px solid rgba(124,58,237,0.2)',
borderTopColor: '#7c3aed', borderRadius: '50%', animation: 'spin 0.8s linear infinite' }} />
196 |                 <p style={{ color: '#64748b' }}>Loading study materials...</p>
197 |               </div>
198 |             )}
199 |
200 |             {materials && !loadingMaterials && (
201 |               <>
202 |                 {/* Selected note title & retake button */}
203 |                 <div style={{ marginBottom: '1.25rem', display: 'flex', alignItems: 'center', justifyContent:
'space-between', flexWrap: 'wrap', gap: '0.75rem' }}>
204 |                   <div>
205 |                     <h2 style={{ fontSize: '1.3rem', fontWeight: 700, color: '#e2e8f0', margin: 0 }}
>{selectedNote.title}</h2>
206 |                     <p style={{ fontSize: '0.8rem', color: '#475569', margin: 0, marginTop: '0.2rem' }}
>Uploaded on {formatDate(selectedNote.createdAt)}</p>
207 |                   </div>
208 |                   {materials.hasQuiz && (
209 |                     <button
210 |                       onClick={() => navigate(`/quiz/${materials.quizId}`, { state: { questions:
materials.questions?.map(q => ({ id: q.id, questionText: q.questionText, optionA: q.optionA, optionB: q.optionB,
optionC: q.optionC, optionD: q.optionD })) } })}
211 |                       style={{ padding: '0.6rem 1.4rem', background: 'linear-gradient(135deg, #7c3aed,
#06b6d4)', border: 'none', borderRadius: '10px', color: 'white', cursor: 'pointer', fontWeight: 600, fontSize:
'0.875rem' }}
212 |                     >
213 |                         Retake Quiz
214 |                     </button>
215 |                   )}
216 |                 </div>
217 |
218 |                 {genError && (
219 |                   <div style={{ marginBottom: '1.25rem', padding: '0.9rem 1.2rem', background:
'rgba(244,63,94,0.1)', border: '1px solid rgba(244,63,94,0.3)', borderRadius: '10px', color: '#f43f5e', fontSize:
'0.875rem' }}>
220 |                     ⚠  {genError}
221 |                   </div>
222 |                 )}
223 |
224 |                 {/* If no materials generated yet */}
225 |                 {!hasAnyMaterials ? (
226 |                   <div style={{ padding: '3.5rem 2rem', textAlign: 'center', background:
'rgba(255,255,255,0.02)', borderRadius: '1.25rem', border: '1px dashed rgba(255,255,255,0.08)' }}>
227 |                     <div style={{ fontSize: '3.5rem', marginBottom: '1rem' }}> </div>
228 |                     <h3 style={{ fontSize: '1.2rem', fontWeight: 700, color: '#e2e8f0', margin: '0 0 0.5rem' }}
>No Study Materials Generated Yet</h3>
229 |                     <p style={{ color: '#64748b', fontSize: '0.9rem', maxWidth: '420px', margin: '0 auto
1.5rem', lineHeight: 1.6 }}>
230 |                       Generate AI summaries, flashcards, and quizzes for this note with a single click.
231 |                     </p>
232 |                     <button
233 |                       onClick={handleGenerateNow}
234 |                       disabled={generating}
235 |                       style={{
236 |                         padding: '0.8rem 1.8rem', background: 'linear-gradient(135deg, #7c3aed 0%, #06b6d4
100%)',
237 |                         border: 'none', borderRadius: '12px', color: 'white', fontWeight: 700, fontSize:
'0.95rem',
238 |                         cursor: generating ? 'not-allowed' : 'pointer', opacity: generating ? 0.7 : 1,
239 |                         boxShadow: '0 4px 20px rgba(124,58,237,0.4)', display: 'inline-flex', alignItems:
'center', gap: '0.6rem'
```

```
240 |                       }}
241 |                     >
242 |                       {generating ? (
243 |                         <>
244 |                           <div style={{ width: '1.2rem', height: '1.2rem', border: '2px solid
rgba(255,255,255,0.3)', borderTopColor: 'white', borderRadius: '50%', animation: 'spin 0.8s linear infinite' }} />
245 |                           Generating Study Materials...
246 |                         </>
247 |                       ) : (
248 |                         '🪁 Generate Study Materials'
249 |                       )}
250 |                     </button>
251 |                   </div>
252 |                 ) : (
253 |                   <>
254 |                     {/* Section Tabs */}
255 |                     <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1.25rem', flexWrap: 'wrap' }}>
256 |                       {[
257 |                         { key: 'summary', label: '📖 Summary' },
258 |                         { key: 'flashcards', label: `🃏 Flashcards (${materials.flashcards?.length || 0})` },
259 |                         { key: 'quiz', label: `🏆 Quiz Q&A (${materials.questions?.length || 0})` },
260 |                         { key: 'content', label: '📝 Original Note' },
261 |                       ].map(tab => (
262 |                         <button
263 |                           key={tab.key}
264 |                           onClick={() => setActiveSection(tab.key)}
265 |                           style={{
266 |                             padding: '0.55rem 1.2rem', border: '1px solid',
267 |                             borderColor: activeSection === tab.key ? 'rgba(124,58,237,0.5)' :
'rgba(255,255,255,0.08)',
268 |                             borderRadius: '10px', cursor: 'pointer', fontSize: '0.875rem', fontWeight:
activeSection === tab.key ? 600 : 400,
269 |                             color: activeSection === tab.key ? '#c4b5fd' : '#64748b',
270 |                             background: activeSection === tab.key ? 'rgba(124,58,237,0.12)' : 'transparent',
271 |                             transition: 'all 0.2s',
272 |                           }}
273 |                         >
274 |                           {tab.label}
275 |                         </button>
276 |                       ))}
277 |                     </div>
278 |
279 |                     {/* Summary Tab */}
280 |                     {activeSection === 'summary' && (
281 |                       <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid
rgba(255,255,255,0.07)', borderRadius: '1.25rem', padding: '1.5rem 1.75rem' }}>
282 |                         <h3 style={{ fontSize: '1.1rem', fontWeight: 700, color: '#e2e8f0', marginBottom:
'1rem', display: 'flex', alignItems: 'center', gap: '0.5rem' }}>
283 |                           <span style={{ padding: '0.2rem 0.5rem', background: 'rgba(124,58,237,0.2)',
borderRadius: '6px', fontSize: '0.85rem' }}>📖</span>
284 |                           Topic Summary
285 |                         </h3>
286 |                         {materials.hasSummary ? (
287 |                           <div style={{ color: '#cbd5e1', lineHeight: '1.85', fontSize: '0.95rem' }}>
288 |                             {materials.summary.split('\n').filter(p => p.trim()).map((para, i) => (
289 |                               <p key={i} style={{ marginBottom: '1rem' }}>{para}</p>
290 |                             ))}
291 |                           </div>
292 |                         ) : (
293 |                           <p style={{ color: '#64748b', fontSize: '0.9rem' }}>No summary stored for this
note.</p>
294 |                         )}
295 |                       </div>
296 |                     )}
297 |
298 |                     {/* Flashcards Tab */}
299 |                     {activeSection === 'flashcards' && (
300 |                       <div>
301 |                         {!materials.hasFlashcards ? (
302 |                           <div style={{ padding: '3rem', textAlign: 'center', color: '#475569', background:
'rgba(255,255,255,0.02)', borderRadius: '1rem', border: '1px dashed rgba(255,255,255,0.07)' }}>
303 |                             No flashcards generated for this note yet.
304 |                           </div>
305 |                         ) : (
306 |                           <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(280px,
1fr))', gap: '1rem' }}>
307 |                             {materials.flashcards.map((fc, idx) => (
308 |                               <div
309 |                                 key={fc.id}
310 |                                 onClick={() => toggleFlip(idx)}
311 |                                 title="Click to flip"
312 |                                 style={{ perspective: '1000px', cursor: 'pointer', height: '240px' }}
313 |                               >
314 |                                 <div style={{
```

```
315 |                                   position: 'relative', width: '100%', height: '100%',
316 |                                   transformStyle: 'preserve-3d',
317 |                                   transform: flippedCards[idx] ? 'rotateY(180deg)' : 'rotateY(0deg)',
318 |                                   transition: 'transform 0.5s cubic-bezier(0.4,0,0.2,1)',
319 |                                 }}>
320 |                                   {/* Front */}
321 |                                   <div style={{
322 |                                     position: 'absolute', inset: 0, backfaceVisibility: 'hidden',
323 |                                     background: 'rgba(124,58,237,0.1)', border: '1px solid
rgba(124,58,237,0.25)',
324 |                                     borderRadius: '1rem', padding: '1.25rem', display: 'flex', flexDirection:
'column', justifyContent: 'space-between',
325 |                                   }}>
326 |                                     <p style={{ fontSize: '0.75rem', color: '#7c3aed', fontWeight: 600,
textTransform: 'uppercase', letterSpacing: '0.06em', margin: 0, flexShrink: 0 }}>Question {idx + 1}</p>
327 |                                     <div style={{ flexGrow: 1, overflowY: 'auto', padding: '0.5rem 0', margin:
'0.2rem 0' }}>
328 |                                       <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0, lineHeight:
1.5 }}>{fc.question}</p>
329 |                                     </div>
330 |                                     <p style={{ fontSize: '0.7rem', color: '#475569', margin: 0, textAlign:
'right' }}>Click to reveal answer →</p>
331 |                                   </div>
332 |                                   {/* Back */}
333 |                                   <div style={{
334 |                                     position: 'absolute', inset: 0, backfaceVisibility: 'hidden',
335 |                                     transform: 'rotateY(180deg)',
336 |                                     background: 'rgba(6,182,212,0.1)', border: '1px solid
rgba(6,182,212,0.25)',
337 |                                     borderRadius: '1rem', padding: '1.25rem', display: 'flex', flexDirection:
'column', justifyContent: 'space-between',
338 |                                   }}>
339 |                                     <p style={{ fontSize: '0.75rem', color: '#06b6d4', fontWeight: 600,
textTransform: 'uppercase', letterSpacing: '0.06em', margin: 0, flexShrink: 0 }}>Answer</p>
340 |                                     <div style={{ flexGrow: 1, overflowY: 'auto', padding: '0.5rem 0', margin:
'0.2rem 0' }}>
341 |                                       <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0, lineHeight:
1.5 }}>{fc.answer}</p>
342 |                                     </div>
343 |                                     <p style={{ fontSize: '0.7rem', color: '#475569', margin: 0, textAlign:
'right' }}>← Click to flip back</p>
344 |                                   </div>
345 |                                 </div>
346 |                               </div>
347 |                             ))}
348 |                           </div>
349 |                         )}
350 |                       </div>
351 |                     )}
352 |
353 |                     {/* Quiz Q&A Tab */}
354 |                     {activeSection === 'quiz' && (
355 |                       <div>
356 |                         {!materials.hasQuiz ? (
357 |                           <div style={{ padding: '3rem', textAlign: 'center', color: '#475569', background:
'rgba(255,255,255,0.02)', borderRadius: '1rem', border: '1px dashed rgba(255,255,255,0.07)' }}>
358 |                             No quiz generated for this note yet.
359 |                           </div>
360 |                         ) : (
361 |                           <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>
362 |                             {materials.questions.map((q, idx) => (
363 |                               <div key={q.id} style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid
rgba(255,255,255,0.07)', borderRadius: '1rem', padding: '1.25rem' }}>
364 |                                 <p style={{ fontSize: '0.75rem', color: '#64748b', fontWeight: 600,
textTransform: 'uppercase', letterSpacing: '0.06em', margin: '0 0 0.5rem' }}>Q{idx + 1}</p>
365 |                                 <p style={{ fontSize: '1rem', color: '#e2e8f0', fontWeight: 500, margin: '0 0
1rem', lineHeight: 1.5 }}>{q.questionText}</p>
366 |                                 <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.5rem' }}
>
367 |                                   {['A', 'B', 'C', 'D'].map(opt => {
368 |                                     const isCorrect = q.correctOption === opt;
369 |                                     return (
370 |                                       <div
371 |                                         key={opt}
372 |                                         style={{
373 |                                           padding: '0.6rem 0.9rem', borderRadius: '8px',
374 |                                           background: isCorrect ? 'rgba(34,197,94,0.15)' : optionBg[opt],
375 |                                           border: `1px solid ${isCorrect ? 'rgba(34,197,94,0.5)' :
optionBorder[opt]}`,
376 |                                           display: 'flex', alignItems: 'center', gap: '0.6rem',
377 |                                         }}
378 |                                       >
379 |                                         <span style={{
380 |                                           width: '22px', height: '22px', borderRadius: '50%', flexShrink: 0,
```

```
381 |                                             background: isCorrect ? 'rgba(34,197,94,0.2)' :
'rgba(255,255,255,0.05)',
382 |                                             border: `1px solid ${isCorrect ? 'rgba(34,197,94,0.6)' :
optionBorder[opt]}`,
383 |                                             display: 'flex', alignItems: 'center', justifyContent: 'center',
384 |                                             fontSize: '0.7rem', fontWeight: 700,
385 |                                             color: isCorrect ? '#4ade80' : optionColor[opt],
386 |                                           }}>
387 |                                             {isCorrect ? '✓' : opt}
388 |                                           </span>
389 |                                           <span style={{ fontSize: '0.82rem', color: isCorrect ? '#86efac' :
'#cbd5e1', lineHeight: 1.3 }}>
390 |                                             {q[`option${opt}`]}
391 |                                           </span>
392 |                                         </div>
393 |                                       );
394 |                                     })}
395 |                                   </div>
396 |                                 </div>
397 |                               ))}
398 |                             </div>
399 |                           )}
400 |                         </div>
401 |                       )}
402 |
403 |                       {/* Original Note Tab */}
404 |                       {activeSection === 'content' && (
405 |                         <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid
rgba(255,255,255,0.07)', borderRadius: '1.25rem', padding: '1.5rem 1.75rem' }}>
406 |                           <h3 style={{ fontSize: '1.1rem', fontWeight: 700, color: '#e2e8f0', marginBottom:
'1rem', display: 'flex', alignItems: 'center', gap: '0.5rem' }}>
407 |                             <span style={{ padding: '0.2rem 0.5rem', background: 'rgba(6,182,212,0.2)',
borderRadius: '6px', fontSize: '0.85rem' }}>📄</span>
408 |                             Original Uploaded Text
409 |                           </h3>
410 |                           <div style={{ color: '#94a3b8', fontFamily: 'monospace', fontSize: '0.85rem',
whiteSpace: 'pre-wrap', background: 'rgba(0,0,0,0.3)', padding: '1.25rem', borderRadius: '10px', border: '1px solid
rgba(255,255,255,0.05)', maxHeight: '450px', overflowY: 'auto' }}>
411 |                             {materials.content || 'No text available.'}
412 |                           </div>
413 |                         </div>
414 |                       )}
415 |                     </>
416 |                   )}
417 |                 </>
418 |               )}
419 |             </div>
420 |           </div>
421 |         </div>
422 |
423 |         <style>{`
424 |           @keyframes spin { from { transform: rotate(0deg); } to { transform: rotate(360deg); } }
425 |         `}</style>
426 |       </div>
427 |     );
428 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–328:** Supports deep-linking via 'location.state.selectedNoteId', fetches '/api/notes/{id}/study-materials', triggers '/api/study/generate' on demand, and renders 4 tabbed views.

> **How to Explain This Module in an Interview:** Master-detail architecture with deep-link state support from the Dashboard.

Module: LoginPage (`frontend/src/pages/LoginPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Student sign-in page.

Why It Is Used & Architectural Role: Authenticates via 'POST /api/auth/login', stores JWT in AuthContext, and navigates to '/dashboard'.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState } from 'react';
  2 | import { Link, useNavigate } from 'react-router-dom';
  3 | import { useAuth } from '../context/AuthContext';
  4 | import API from '../api/axios';
  5 |
  6 | export default function LoginPage() {
```

```
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);
    try {
      const response = await API.post('/api/auth/login', { email, password });
      login(response.data);
      navigate('/dashboard');
    } catch (err) {
      setError(err.response?.data?.message || 'Login failed. Please check your credentials.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div style={{ minHeight: '100vh', display: 'flex', alignItems: 'center', justifyContent: 'center', padding:
'5rem 1.5rem 2rem' }}>
      <div style={{ width: '100%', maxWidth: '420px', animation: 'slideUp 0.5s ease-out both' }}>

        {/* Logo + Header */}
        <div style={{ textAlign: 'center', marginBottom: '2rem' }}>
          <div style={{
            width: '64px', height: '64px', margin: '0 auto 1.25rem',
            background: 'linear-gradient(135deg, #7c3aed, #06b6d4)',
            borderRadius: '20px',
            display: 'flex', alignItems: 'center', justifyContent: 'center',
            boxShadow: '0 8px 32px rgba(124,58,237,0.4)',
          }}>
            <svg width="32" height="32" fill="none" viewBox="0 0 24 24" stroke="white" strokeWidth={2}>
              <path strokeLinecap="round" strokeLinejoin="round" d="M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5
5S4.168 5.477 3 6.253v13C4.168 18.477 5.754 18 7.5 18s3.332.477 4.5 1.253m0-13C13.168 5.477 14.754 5 16.5 5c1.747 0
3.332.477 4.5 1.253v13C19.832 18.477 18.247 18 16.5 18c-1.746 0-3.332.477-4.5 1.253" />
            </svg>
          </div>
          <h1 style={{
            fontSize: '1.8rem', fontWeight: 800, fontFamily: "'Space Grotesk', sans-serif",
            background: 'linear-gradient(135deg, #f1f5f9 0%, #c4b5fd 60%, #38bdf8 100%)',
            WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent', backgroundClip: 'text',
            marginBottom: '0.5rem',
          }}>Welcome back</h1>
          <p style={{ color: '#64748b', fontSize: '0.95rem' }}>Sign in to continue your studies</p>
        </div>

        {/* Card */}
        <div style={{
          background: 'rgba(255,255,255,0.04)',
          backdropFilter: 'blur(24px) saturate(180%)',
          WebkitBackdropFilter: 'blur(24px) saturate(180%)',
          border: '1px solid rgba(255,255,255,0.09)',
          borderRadius: '24px',
          padding: '2rem',
          boxShadow: '0 8px 40px rgba(0,0,0,0.35)',
          position: 'relative', overflow: 'hidden',
        }}>
          {/* Top glow */}
          <div style={{ position: 'absolute', top: 0, left: 0, right: 0, height: '1px', background: 'linear-
gradient(90deg, transparent, rgba(139,92,246,0.7), rgba(6,182,212,0.5), transparent)' }} />

          {error && (
            <div style={{ marginBottom: '1.25rem', padding: '0.85rem 1rem', background: 'rgba(244,63,94,0.1)',
border: '1px solid rgba(244,63,94,0.25)', borderRadius: '12px', color: '#fda4af', fontSize: '0.875rem' }}>
              {error}
            </div>
          )}

          <form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1.25rem' }}>
            <div>
              <label htmlFor="login-email" style={{ display: 'block', fontSize: '0.85rem', fontWeight: 500,
color: '#94a3b8', marginBottom: '0.5rem' }}>Email</label>
              <input id="login-email" type="email" value={email} onChange={e => setEmail(e.target.value)}
                className="input-field" placeholder="you@example.com" required />
            </div>
            <div>
              <label htmlFor="login-password" style={{ display: 'block', fontSize: '0.85rem', fontWeight: 500,
color: '#94a3b8', marginBottom: '0.5rem' }}>Password</label>
              <input id="login-password" type="password" value={password} onChange={e =>
```

```
setPassword(e.target.value)}
 84 |                 className="input-field" placeholder="••••••••" required />
 85 |             </div>
 86 |
 87 |             <button type="submit" disabled={loading} className="btn-primary" style={{ width: '100%', marginTop:
'0.25rem', padding: '0.8rem' }}>
 88 |               {loading ? (
 89 |                 <><div style={{ width: '18px', height: '18px', border: '2px solid rgba(255,255,255,0.3)',
borderTopColor: '#fff', borderRadius: '50%', animation: 'spin 0.8s linear infinite' }} />Signing in...</>
 90 |               ) : 'Sign In →'}
 91 |             </button>
 92 |           </form>
 93 |         </div>
 94 |
 95 |         <p style={{ textAlign: 'center', color: '#475569', fontSize: '0.875rem', marginTop: '1.5rem' }}>
 96 |           Don't have an account?{' '}
 97 |           <Link to="/register" style={{ color: '#a78bfa', fontWeight: 600, textDecoration: 'none' }}
 98 |             onMouseEnter={e => e.target.style.color='#c4b5fd'}
 99 |             onMouseLeave={e => e.target.style.color='#a78bfa'}
100 |           >Create one →</Link>
101 |         </p>
102 |       </div>
103 |     </div>
104 |   );
105 | }
106 |
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–102:** Controlled form for 'email' and 'password' with loading spinner and error banner.

> **How to Explain This Module in an Interview:** Accessible form inputs with 'htmlFor'/'id' bindings.

Module: NotesPage (`frontend/src/pages/NotesPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Note creation page supporting both pasted text and drag-and-drop PDF/PPTX file uploads.

Why It Is Used & Architectural Role: Posts to '/api/notes/text' or '/api/notes/upload' and redirects to '/study/:noteId'.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useRef } from 'react';
  2 | import { useNavigate } from 'react-router-dom';
  3 | import API from '../api/axios';
  4 |
  5 | export default function NotesPage() {
  6 |   const [activeTab, setActiveTab] = useState('text');
  7 |   const [title, setTitle] = useState('');
  8 |   const [content, setContent] = useState('');
  9 |   const [file, setFile] = useState(null);
 10 |   const [dragOver, setDragOver] = useState(false);
 11 |   const [loading, setLoading] = useState(false);
 12 |   const [error, setError] = useState('');
 13 |   const fileRef = useRef(null);
 14 |   const navigate = useNavigate();
 15 |
 16 |   const handleTextSubmit = async (e) => {
 17 |     e.preventDefault();
 18 |     setError('');
 19 |     setLoading(true);
 20 |     try {
 21 |       const res = await API.post('/api/notes/text', { title, content });
 22 |       navigate(`/study/${res.data.noteId}`);
 23 |     } catch (err) {
 24 |       setError(err.response?.data?.message || 'Failed to save note');
 25 |     } finally { setLoading(false); }
 26 |   };
 27 |
 28 |   const handleFileSubmit = async (e) => {
 29 |     e.preventDefault();
 30 |     if (!file) { setError('Please select a PDF or PowerPoint file'); return; }
 31 |     setError('');
 32 |     setLoading(true);
 33 |     try {
 34 |       const formData = new FormData();
 35 |       formData.append('file', file);
 36 |       formData.append('title', title);
 37 |       const res = await API.post('/api/notes/upload', formData, {
 38 |         headers: { 'Content-Type': undefined }
 39 |       });
```

```
      navigate(`/study/${res.data.noteId}`);
    } catch (err) {
      setError(err.response?.data?.message || 'Failed to upload document');
    } finally { setLoading(false); }
  };

  const handleDrop = (e) => {
    e.preventDefault();
    setDragOver(false);
    const droppedFile = e.dataTransfer.files[0];
    if (droppedFile?.name.match(/\.(pdf|pptx?)$/i)) setFile(droppedFile);
    else setError('Only PDF and PowerPoint (.pptx) files are accepted');
  };

  return (
    <div className="min-h-screen p-8 max-w-3xl mx-auto">
      <div className="mb-8 animate-fade-in">
        <h1 className="text-3xl font-bold text-surface-100">Add New Note</h1>
        <p className="text-surface-400 mt-2">Paste your notes or upload a PDF to get started.</p>
      </div>

      {/* Tabs */}
      <div className="flex gap-2 mb-6">
        <button onClick={() => setActiveTab('text')} className={`tab-btn ${activeTab === 'text' ? 'active' :
'inactive'}`}>
          📝 Paste Text
        </button>
        <button onClick={() => setActiveTab('upload')} className={`tab-btn ${activeTab === 'upload' ? 'active'
: 'inactive'}`}>
          📄 Upload Document
        </button>
      </div>

      {error && (
        <div className="mb-6 p-4 bg-rose-500/10 border border-rose-500/30 rounded-xl text-rose-300 text-
sm">{error}</div>
      )}

      <div className="glass-card p-6 sm:p-8 animate-slide-up">
        {/* Title field (shared) */}
        <div className="mb-5">
          <label className="block text-sm font-medium text-surface-300 mb-2">Title</label>
          <input type="text" value={title} onChange={(e) => setTitle(e.target.value)} className="input-field"
placeholder="e.g. Chapter 5: Data Structures" required />
        </div>

        {activeTab === 'text' ? (
          <form onSubmit={handleTextSubmit}>
            <div className="mb-5">
              <label className="block text-sm font-medium text-surface-300 mb-2">Notes Content</label>
              <textarea value={content} onChange={(e) => setContent(e.target.value)} className="input-field
min-h-[250px] resize-y" placeholder="Paste your lecture notes, topic content, or any text here..." required />
            </div>
            <button type="submit" disabled={loading || !title || !content} className="btn-primary w-full flex
items-center justify-center gap-2">
              {loading ? <><div className="w-5 h-5 border-2 border-white/30 border-t-white rounded-full
animate-spin" /> Saving...</> : '✨ Save & Generate Study Materials'}
            </button>
          </form>
        ) : (
          <form onSubmit={handleFileSubmit}>
            <div className="mb-5">
              <label className="block text-sm font-medium text-surface-300 mb-2">Document File (PDF, PPTX)</
label>
              <div
                className={`border-2 border-dashed rounded-xl p-8 text-center cursor-pointer transition-all ${
                  dragOver ? 'border-primary-500 bg-primary-500/5' : 'border-surface-600/50 hover:border-
surface-500/50'
                }`}
                onDragOver={(e) => { e.preventDefault(); setDragOver(true); }}
                onDragLeave={() => setDragOver(false)}
                onDrop={handleDrop}
                onClick={() => fileRef.current?.click()}
              >
                <input ref={fileRef} type="file" accept=".pdf,.pptx,.ppt" className="hidden" onChange={(e) =>
setFile(e.target.files[0])} />
                {file ? (
                  <div>
                    <div className="text-4xl mb-2">📄</div>
                    <p className="text-surface-200 font-medium">{file.name}</p>
                    <p className="text-surface-400 text-sm mt-1">{(file.size / 1024 / 1024).toFixed(2)} MB</p>
                  </div>
                ) : (
                  <div>
```

```
114 |                     <div className="text-4xl mb-2">📤</div>
115 |                     <p className="text-surface-300">Drop your file here or click to browse</p>
116 |                     <p className="text-surface-500 text-sm mt-1">Max file size: 10MB</p>
117 |                   </div>
118 |                 )}
119 |               </div>
120 |             </div>
121 |             <button type="submit" disabled={loading || !title || !file} className="btn-primary w-full flex
items-center justify-center gap-2">
122 |               {loading ? <><div className="w-5 h-5 border-2 border-white/30 border-t-white rounded-full
animate-spin" /> Uploading...</> : '🚀 Upload & Generate Study Materials'}
123 |             </button>
124 |           </form>
125 |         )}
126 |       </div>
127 |     </div>
128 |   );
129 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–133:** Manages 'text' vs 'upload' tabs, drag-and-drop file validation ('.pdf', '.pptx', '.ppt'), 'FormData' upload, and navigation to StudyPage.

> **How to Explain This Module in an Interview:** Handles both JSON text payloads and multipart binary uploads.

Module: ProgressPage (`frontend/src/pages/ProgressPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Displays the multi-topic 'ProgressChart' and a Topic Summary table with best score, last score, and trend arrows ('↑', '↓', '→').

Why It Is Used & Architectural Role: Visualizes '/api/progress' data.

Complete Source Code (Every Line Numbered):

```
 1 | import { useState, useEffect } from 'react';
 2 | import API from '../api/axios';
 3 | import ProgressChart from '../components/ProgressChart';
 4 |
 5 | export default function ProgressPage() {
 6 |   const [progressData, setProgressData] = useState([]);
 7 |   const [loading, setLoading] = useState(true);
 8 |
 9 |   useEffect(() => {
10 |     const fetchProgress = async () => {
11 |       try {
12 |         const res = await API.get('/api/progress');
13 |         setProgressData(res.data);
14 |       } catch (err) { console.error(err); }
15 |       finally { setLoading(false); }
16 |     };
17 |     fetchProgress();
18 |   }, []);
19 |
20 |   if (loading) return <div className="flex items-center justify-center p-16"><div className="spinner" /></div>;
21 |
22 |   // Build summary table data: group by noteTitle
23 |   const topicMap = {};
24 |   progressData.forEach((item) => {
25 |     if (!topicMap[item.noteTitle]) topicMap[item.noteTitle] = [];
26 |     topicMap[item.noteTitle].push(item);
27 |   });
28 |
29 |   const summaryRows = Object.entries(topicMap).map(([topic, attempts]) => {
30 |     const scores = attempts.map((a) => parseFloat(a.percentage));
31 |     const best = Math.max(...scores);
32 |     const last = scores[scores.length - 1];
33 |     const prev = scores.length >= 2 ? scores[scores.length - 2] : null;
34 |     const trend = prev !== null ? (last > prev ? '↑' : last < prev ? '↓' : '→') : '—';
35 |     return { topic, attempts: attempts.length, best, last, trend };
36 |   });
37 |
38 |   return (
39 |     <div className="min-h-screen p-8 max-w-5xl mx-auto">
40 |       <div className="mb-8 animate-fade-in">
41 |         <h1 className="text-3xl font-bold text-surface-100">Your Progress</h1>
42 |         <p className="text-surface-400 mt-2">Track your performance across all topics.</p>
43 |       </div>
44 |
45 |       {/* Chart */}
```

```
 46 |       <div className="mb-8 animate-slide-up">
 47 |         <ProgressChart progressData={progressData} />
 48 |       </div>
 49 |
 50 |       {/* Summary Table */}
 51 |       {summaryRows.length > 0 && (
 52 |         <div className="glass-card overflow-hidden animate-slide-up">
 53 |           <div className="p-6 border-b border-surface-700/50">
 54 |             <h3 className="text-lg font-semibold text-surface-100">Topic Summary</h3>
 55 |           </div>
 56 |           <div className="overflow-x-auto">
 57 |             <table className="w-full">
 58 |               <thead>
 59 |                 <tr className="border-b border-surface-700/50">
 60 |                   <th className="px-6 py-4 text-left text-xs font-semibold text-surface-400 uppercase tracking-
wider">Topic</th>
 61 |                   <th className="px-6 py-4 text-center text-xs font-semibold text-surface-400 uppercase
tracking-wider">Attempts</th>
 62 |                   <th className="px-6 py-4 text-center text-xs font-semibold text-surface-400 uppercase
tracking-wider">Best Score</th>
 63 |                   <th className="px-6 py-4 text-center text-xs font-semibold text-surface-400 uppercase
tracking-wider">Last Score</th>
 64 |                   <th className="px-6 py-4 text-center text-xs font-semibold text-surface-400 uppercase
tracking-wider">Trend</th>
 65 |                 </tr>
 66 |               </thead>
 67 |               <tbody>
 68 |                 {summaryRows.map((row, i) => (
 69 |                   <tr key={i} className="border-b border-surface-700/30 hover:bg-surface-700/20 transition-
colors">
 70 |                     <td className="px-6 py-4 text-surface-200 font-medium">{row.topic}</td>
 71 |                     <td className="px-6 py-4 text-center text-surface-300">{row.attempts}</td>
 72 |                     <td className="px-6 py-4 text-center">
 73 |                       <span className="text-emerald-400 font-semibold">{row.best.toFixed(0)}%</span>
 74 |                     </td>
 75 |                     <td className="px-6 py-4 text-center text-surface-300">{row.last.toFixed(0)}%</td>
 76 |                     <td className="px-6 py-4 text-center text-xl">
 77 |                       <span className={row.trend === '↑' ? 'text-emerald-400' : row.trend === '↓' ? 'text-
rose-400' : 'text-surface-400'}>
 78 |                         {row.trend}
 79 |                       </span>
 80 |                     </td>
 81 |                   </tr>
 82 |                 ))}
 83 |               </tbody>
 84 |             </table>
 85 |           </div>
 86 |         </div>
 87 |       )}
 88 |
 89 |       {progressData.length === 0 && (
 90 |         <div className="glass-card p-12 text-center">
 91 |           <div className="text-5xl mb-4">📊</div>
 92 |           <p className="text-surface-400 text-lg">No progress data yet. Take some quizzes to see your stats!</
p>
 93 |         </div>
 94 |       )}
 95 |     </div>
 96 |   );
 97 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–98:** Groups attempts by topic, computes attempt count, best score, last score, and direction arrow compared to the previous attempt.

> **How to Explain This Module in an Interview:** Complements AnalyticsPage with per-topic chronological trajectory.

Module: QuizPage (`frontend/src/pages/QuizPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Interactive timed quiz assessment interface.

Why It Is Used & Architectural Role: Renders 'Timer' (300s), progress bar, 'QuizQuestion', Previous/Next/Submit buttons, and a numbered question dot navigator.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useCallback } from 'react';
  2 | import { useParams, useLocation, useNavigate } from 'react-router-dom';
  3 | import API from '../api/axios';
```

```
import QuizQuestionComp from '../components/QuizQuestion';
import Timer from '../components/Timer';

export default function QuizPage() {
  const { quizId } = useParams();
  const location = useLocation();
  const navigate = useNavigate();
  const questions = location.state?.questions || [];

  const [currentIndex, setCurrentIndex] = useState(0);
  const [answers, setAnswers] = useState({});
  const [submitting, setSubmitting] = useState(false);

  const handleSelect = (optionKey) => {
    setAnswers((prev) => ({ ...prev, [questions[currentIndex].id]: optionKey }));
  };

  const handleSubmit = useCallback(async () => {
    if (submitting) return;
    setSubmitting(true);
    const answerList = questions.map((q) => ({
      questionId: q.id,
      selectedOption: answers[q.id] || '',
    }));
    try {
      const res = await API.post('/api/quiz/attempt', { quizId: parseInt(quizId), answers: answerList });
      navigate(`/result/${quizId}`, { state: { result: res.data, questions } });
    } catch (err) {
      console.error('Submit failed:', err);
      setSubmitting(false);
    }
  }, [submitting, questions, answers, quizId, navigate]);

  if (questions.length === 0) {
    return (
      <div className="flex items-center justify-center p-8">
        <div className="glass-card p-8 text-center">
          <p className="text-surface-400 mb-4">No quiz data found. Please generate study materials first.</p>
          <button onClick={() => navigate('/dashboard')} className="btn-primary">Go to Dashboard</button>
        </div>
      </div>
    );
  }

  const current = questions[currentIndex];
  const isLast = currentIndex === questions.length - 1;
  const progress = ((currentIndex + 1) / questions.length) * 100;
  const answeredCount = Object.keys(answers).length;
  const totalQuestions = questions.length;

  return (
    <div className="min-h-screen p-8 max-w-3xl mx-auto">
      {/* Header */}
      <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 mb-6">
        <div>
          <h1 className="text-2xl font-bold text-surface-100">Quiz Time</h1>
          <p className="text-surface-400 text-sm">
            Question {currentIndex + 1} of {totalQuestions}
             ·
            <span style={{ color: answeredCount === totalQuestions ? '#4ade80' : '#a78bfa' }}>
              {answeredCount}/{totalQuestions} answered
            </span>
          </p>
        </div>
        <Timer totalSeconds={300} onTimeUp={handleSubmit} />
      </div>

      {/* Progress bar */}
      <div className="progress-bar mb-6">
        <div className="progress-bar-fill" style={{ width: `${progress}%` }} />
      </div>

      {/* Question */}
      <div className="glass-card p-6 sm:p-8 mb-6">
        <QuizQuestionComp
          question={current}
          selectedOption={answers[current.id]}
          onSelect={handleSelect}
          showResult={false}
        />
      </div>

      {/* Navigation */}
      <div className="flex justify-between items-center gap-3">
```

```
 88 |          <button
 89 |            onClick={() => setCurrentIndex((p) => Math.max(0, p - 1))}
 90 |            disabled={currentIndex === 0}
 91 |            className="btn-secondary disabled:opacity-30"
 92 |          >
 93 |            ← Previous
 94 |          </button>
 95 |
 96 |          <div className="flex gap-3">
 97 |            {/* Submit always visible */}
 98 |            <button
 99 |              onClick={handleSubmit}
100 |              disabled={submitting}
101 |              className="btn-primary flex items-center gap-2"
102 |              style={{ background: 'linear-gradient(135deg, #16a34a, #15803d)', boxShadow: '0 4px 14px
rgba(22,163,74,0.35)' }}
103 |            >
104 |              {submitting ? <div className="w-5 h-5 border-2 border-white/30 border-t-white rounded-full animate-
spin" /> : '✓'}
105 |              Submit Quiz
106 |            </button>
107 |
108 |            {/* Next only if not on last */}
109 |            {!isLast && (
110 |              <button onClick={() => setCurrentIndex((p) => p + 1)} className="btn-primary">
111 |                Next →
112 |              </button>
113 |            )}
114 |          </div>
115 |        </div>
116 |
117 |        {/* Question dots navigator */}
118 |        <div style={{ marginTop: '1.5rem', display: 'flex', flexWrap: 'wrap', gap: '0.4rem', justifyContent:
'center' }}>
119 |          {questions.map((q, idx) => (
120 |            <button
121 |              key={idx}
122 |              onClick={() => setCurrentIndex(idx)}
123 |              title={`Question ${idx + 1}`}
124 |              style={{
125 |                width: '32px', height: '32px', borderRadius: '50%', border: 'none', cursor: 'pointer',
126 |                fontSize: '0.7rem', fontWeight: 600, transition: 'all 0.15s',
127 |                background: idx === currentIndex
128 |                  ? 'linear-gradient(135deg, #7c3aed, #06b6d4)'
129 |                  : answers[q.id]
130 |                  ? 'rgba(124,58,237,0.35)'
131 |                  : 'rgba(255,255,255,0.07)',
132 |                color: idx === currentIndex ? 'white' : answers[q.id] ? '#c4b5fd' : '#64748b',
133 |                boxShadow: idx === currentIndex ? '0 2px 10px rgba(124,58,237,0.5)' : 'none',
134 |              }}
135 |            >
136 |              {idx + 1}
137 |            </button>
138 |          ))}
139 |        </div>
140 |      </div>
141 |    );
142 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–132:** Tracks 'currentIndex' and 'answers' map, memoizes 'handleSubmit' with 'useCallback' for Timer auto-submit, and posts to '/api/quiz/attempt'.

How to Explain This Module in an Interview: Memoized 'handleSubmit' integrates cleanly with Timer's auto-submit callback.

Module: QuizzesPage (`frontend/src/pages/QuizzesPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Historical log of all quiz attempts with summary KPIs and 1-click retake buttons.

Why It Is Used & Architectural Role: Fetches '/api/quiz/history' and loads stripped questions for retakes.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useEffect } from 'react';
  2 | import { useNavigate } from 'react-router-dom';
  3 | import API from '../api/axios';
  4 | import { formatDateTime } from '../utils/formatDate';
```

```
  5 | import { stripQuestionFields } from '../utils/quizUtils';
  6 |
  7 | export default function QuizzesPage() {
  8 |   const navigate = useNavigate();
  9 |   const [attempts, setAttempts] = useState([]);
 10 |   const [loading, setLoading] = useState(true);
 11 |   const [retakingId, setRetakingId] = useState(null);
 12 |
 13 |   const handleRetake = async (quiz) => {
 14 |     if (!quiz.noteId || retakingId) return;
 15 |     setRetakingId(quiz.quizId);
 16 |     try {
 17 |       const res = await API.get(`/api/notes/${quiz.noteId}/study-materials`);
 18 |       const { questions, quizId } = res.data;
 19 |       if (questions?.length) {
 20 |         const stripped = stripQuestionFields(questions);
 21 |         navigate(`/quiz/${quizId || quiz.quizId}`, { state: { questions: stripped } });
 22 |       }
 23 |     } catch (err) {
 24 |       console.error('Failed to load quiz:', err);
 25 |       setRetakingId(null);
 26 |     }
 27 |   };
 28 |
 29 |   useEffect(() => {
 30 |     const fetchHistory = async () => {
 31 |       try {
 32 |         const res = await API.get('/api/quiz/history');
 33 |         setAttempts(res.data);
 34 |       } catch (err) {
 35 |         console.error('Failed to fetch quiz history', err);
 36 |       } finally {
 37 |         setLoading(false);
 38 |       }
 39 |     };
 40 |     fetchHistory();
 41 |   }, []);
 42 |
 43 |   const average = attempts.length > 0
 44 |     ? (attempts.reduce((sum, a) => sum + Number(a.percentage), 0) / attempts.length).toFixed(1)
 45 |     : 0;
 46 |   const best = attempts.length > 0 ? Math.max(...attempts.map(a => Number(a.percentage))) : 0;
 47 |   const passed = attempts.filter(a => Number(a.percentage) >= 60).length;
 48 |
 49 |   return (
 50 |     <div style={{ padding: '2rem 1rem', maxWidth: '1100px', margin: '0 auto' }}>
 51 |       <div style={{ marginBottom: '2rem' }}>
 52 |         <h1 style={{ fontSize: '2rem', fontWeight: 800, background: 'linear-gradient(135deg, #a78bfa,
#38bdf8)', WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent' }}>
 53 |           📝 Quizzes
 54 |         </h1>
 55 |         <p style={{ color: '#64748b', marginTop: '0.4rem' }}>Review all your quiz attempts and scores.</p>
 56 |       </div>
 57 |
 58 |       {loading ? (
 59 |         <div style={{ display: 'flex', justifyContent: 'center', padding: '4rem' }}>
 60 |           <div className="spinner" />
 61 |         </div>
 62 |       ) : attempts.length === 0 ? (
 63 |         <div style={{ padding: '4rem 2rem', textAlign: 'center', background: 'rgba(255,255,255,0.02)',
borderRadius: '1.25rem', border: '2px dashed rgba(255,255,255,0.08)' }}>
 64 |           <div style={{ fontSize: '3.5rem', marginBottom: '1rem' }}>📝</div>
 65 |           <h3 style={{ fontSize: '1.2rem', fontWeight: 700, color: '#e2e8f0', margin: '0 0 0.5rem' }}>No
Quizzes Taken Yet</h3>
 66 |           <p style={{ color: '#64748b', fontSize: '0.9rem', marginBottom: '1.5rem' }}>
 67 |             Generate study materials from a note, then take the quiz to see your scores here.
 68 |           </p>
 69 |           <button onClick={() => navigate('/notes')} className="btn-primary">Upload Notes</button>
 70 |         </div>
 71 |       ) : (
 72 |         <>
 73 |           {/* Summary cards */}
 74 |           <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(200px, 1fr))', gap:
'1.25rem', marginBottom: '1.5rem' }}>
 75 |             <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.08)',
borderRadius: '1.25rem', padding: '1.25rem' }}>
 76 |               <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0 0 0.25rem' }}>Total Attempts</p>
 77 |               <p style={{ fontSize: '1.8rem', fontWeight: 700, color: '#f8fafc', margin: 0 }}>{attempts.length}
</p>
 78 |             </div>
 79 |             <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.08)',
borderRadius: '1.25rem', padding: '1.25rem' }}>
 80 |               <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0 0 0.25rem' }}>Average Score</p>
 81 |               <p style={{ fontSize: '1.8rem', fontWeight: 700, color: '#a78bfa', margin: 0 }}>{average}%</p>
```

```
 82 |             </div>
 83 |             <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.08)',
borderRadius: '1.25rem', padding: '1.25rem' }}>
 84 |               <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0 0 0.25rem' }}>Best Score</p>
 85 |               <p style={{ fontSize: '1.8rem', fontWeight: 700, color: '#34d399', margin: 0 }}>{best}%</p>
 86 |             </div>
 87 |             <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.08)',
borderRadius: '1.25rem', padding: '1.25rem' }}>
 88 |               <p style={{ fontSize: '0.8rem', color: '#64748b', margin: '0 0 0.25rem' }}>Passed</p>
 89 |               <p style={{ fontSize: '1.8rem', fontWeight: 700, color: '#38bdf8', margin: 0 }}>{passed}/
{attempts.length}</p>
 90 |             </div>
 91 |           </div>
 92 |
 93 |           {/* Attempt list */}
 94 |           <div style={{ display: 'flex', flexDirection: 'column', gap: '0.85rem' }}>
 95 |             {attempts.map((a) => {
 96 |               const pct = Number(a.percentage);
 97 |               const ok = pct >= 60;
 98 |               return (
 99 |                 <div
100 |                   key={`${a.quizId}-${a.attemptedAt}`}
101 |                   style={{
102 |                     display: 'flex', alignItems: 'center', justifyContent: 'space-between', gap: '1rem',
flexWrap: 'wrap',
103 |                     padding: '1.1rem 1.4rem',
104 |                     background: 'rgba(255,255,255,0.03)',
105 |                     border: `1px solid ${ok ? 'rgba(16,185,129,0.2)' : 'rgba(244,63,94,0.2)'}`,
106 |                     borderRadius: '1rem',
107 |                   }}
108 |                 >
109 |                   <div style={{ display: 'flex', alignItems: 'center', gap: '1rem', minWidth: 0 }}>
110 |                     <div style={{
111 |                       width: '44px', height: '44px', borderRadius: '12px', flexShrink: 0,
112 |                       background: ok ? 'rgba(16,185,129,0.15)' : 'rgba(244,63,94,0.15)',
113 |                       display: 'flex', alignItems: 'center', justifyContent: 'center', fontSize: '1.3rem',
114 |                     }}>
115 |                       {ok ? '✓' : '✗'}
116 |                     </div>
117 |                     <div style={{ minWidth: 0 }}>
118 |                       <p style={{ fontSize: '0.95rem', fontWeight: 600, color: '#e2e8f0', margin: 0,
whiteSpace: 'nowrap', overflow: 'hidden', textOverflow: 'ellipsis' }}>
119 |                         {a.noteTitle}
120 |                       </p>
121 |                       <p style={{ fontSize: '0.78rem', color: '#64748b', margin: '0.15rem 0 0' }}
>{formatDateTime(a.attemptedAt)}</p>
122 |                     </div>
123 |                   </div>
124 |                   <div style={{ display: 'flex', alignItems: 'center', gap: '1.25rem' }}>
125 |                     <div style={{ textAlign: 'right' }}>
126 |                       <p style={{ fontSize: '0.75rem', color: '#64748b', margin: 0 }}>Score</p>
127 |                       <p style={{ fontSize: '1rem', fontWeight: 700, color: '#f8fafc', margin: 0 }}>{a.score}/
{a.totalQuestions}</p>
128 |                     </div>
129 |                     <div style={{ width: '110px', height: '6px', background: 'rgba(255,255,255,0.08)',
borderRadius: '99px', overflow: 'hidden' }}>
130 |                       <div style={{
131 |                         width: `${pct}%`, height: '100%', borderRadius: '99px',
132 |                         background: ok ? 'linear-gradient(90deg, #10b981, #34d399)' : 'linear-gradient(90deg,
#f43f5e, #fb7185)',
133 |                       }} />
134 |                     </div>
135 |                     <span style={{
136 |                       fontSize: '0.85rem', fontWeight: 700, width: '52px', textAlign: 'right',
137 |                       color: ok ? '#34d399' : '#fb7185',
138 |                     }}>
139 |                       {pct.toFixed(0)}%
140 |                     </span>
141 |                     {a.noteId && (
142 |                       <button
143 |                         onClick={() => handleRetake(a)}
144 |                         disabled={retakingId === a.quizId}
145 |                         style={{
146 |                           padding: '0.4rem 1rem', borderRadius: '8px', border: 'none',
147 |                           background: retakingId === a.quizId ? 'rgba(124,58,237,0.2)' : 'linear-
gradient(135deg, #7c3aed, #06b6d4)',
148 |                           color: 'white', cursor: retakingId === a.quizId ? 'wait' : 'pointer',
149 |                           fontSize: '0.78rem', fontWeight: 600, whiteSpace: 'nowrap', flexShrink: 0,
150 |                           opacity: retakingId === a.quizId ? 0.7 : 1,
151 |                         }}
152 |                       >
153 |                         {retakingId === a.quizId ? '...' : 'Retake →'}
154 |                       </button>
155 |                     )}
```

```
                  </div>
                </div>
              );
            })}
          </div>
        </>
      )}
    </div>
  );
}
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–145:** Calculates total attempts, average score, best score, and passed count, and renders each attempt card with a progress bar and 'Retake →' button.

How to Explain This Module in an Interview: Uses 'stripQuestionFields' before launching a retake.

Module: RegisterPage (`frontend/src/pages/RegisterPage.jsx`)

What This File Is (Beginner-Friendly Analogy): New student account registration page.

Why It Is Used & Architectural Role: Validates password confirmation and min length (6 chars), calls 'POST /api/auth/register', logs the user in, and redirects to '/dashboard'.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState } from 'react';
  2 | import { Link, useNavigate } from 'react-router-dom';
  3 | import { useAuth } from '../context/AuthContext';
  4 | import API from '../api/axios';
  5 |
  6 | export default function RegisterPage() {
  7 |   const [name, setName] = useState('');
  8 |   const [email, setEmail] = useState('');
  9 |   const [password, setPassword] = useState('');
 10 |   const [confirmPassword, setConfirmPassword] = useState('');
 11 |   const [error, setError] = useState('');
 12 |   const [loading, setLoading] = useState(false);
 13 |   const { login } = useAuth();
 14 |   const navigate = useNavigate();
 15 |
 16 |   const handleSubmit = async (e) => {
 17 |     e.preventDefault();
 18 |     setError('');
 19 |
 20 |     if (password !== confirmPassword) {
 21 |       setError('Passwords do not match');
 22 |       return;
 23 |     }
 24 |
 25 |     if (password.length < 6) {
 26 |       setError('Password must be at least 6 characters');
 27 |       return;
 28 |     }
 29 |
 30 |     setLoading(true);
 31 |
 32 |     try {
 33 |       const response = await API.post('/api/auth/register', { name, email, password });
 34 |       login(response.data);
 35 |       navigate('/dashboard');
 36 |     } catch (err) {
 37 |       setError(err.response?.data?.message || 'Registration failed. Please try again.');
 38 |     } finally {
 39 |       setLoading(false);
 40 |     }
 41 |   };
 42 |
 43 |   return (
 44 |     <div style={{ minHeight: '100vh', display: 'flex', alignItems: 'center', justifyContent: 'center', padding:
'5rem 1.5rem 2rem' }}>
 45 |       <div style={{ width: '100%', maxWidth: '420px', animation: 'slideUp 0.5s ease-out both' }}>
 46 |
 47 |         {/* Logo + Header */}
 48 |         <div style={{ textAlign: 'center', marginBottom: '2rem' }}>
 49 |           <div style={{
 50 |             width: '64px', height: '64px', margin: '0 auto 1.25rem',
 51 |             background: 'linear-gradient(135deg, #10b981, #06b6d4)',
 52 |             borderRadius: '20px',
```

```
 53 |             display: 'flex', alignItems: 'center', justifyContent: 'center',
 54 |             boxShadow: '0 8px 32px rgba(16,185,129,0.3)',
 55 |           }}>
 56 |             <svg width="32" height="32" fill="none" viewBox="0 0 24 24" stroke="white" strokeWidth={2}>
 57 |               <path strokeLinecap="round" strokeLinejoin="round" d="M18 9v3m0 0v3m0-3h3m-3 0h-3m-2-5a4 4 0 11-8
0 4 4 0 018 0zM3 20a6 6 0 0112 0v1H3v-1z" />
 58 |             </svg>
 59 |           </div>
 60 |           <h1 style={{
 61 |             fontSize: '1.8rem', fontWeight: 800, fontFamily: "'Space Grotesk', sans-serif",
 62 |             background: 'linear-gradient(135deg, #f1f5f9 0%, #34d399 60%, #38bdf8 100%)',
 63 |             WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent', backgroundClip: 'text',
 64 |             marginBottom: '0.5rem',
 65 |           }}>Create Account</h1>
 66 |           <p style={{ color: '#64748b', fontSize: '0.95rem' }}>Start your learning journey</p>
 67 |         </div>
 68 |
 69 |         {/* Card */}
 70 |         <div style={{
 71 |           background: 'rgba(255,255,255,0.04)',
 72 |           backdropFilter: 'blur(24px) saturate(180%)',
 73 |           WebkitBackdropFilter: 'blur(24px) saturate(180%)',
 74 |           border: '1px solid rgba(255,255,255,0.09)',
 75 |           borderRadius: '24px',
 76 |           padding: '2rem',
 77 |           boxShadow: '0 8px 40px rgba(0,0,0,0.35)',
 78 |           position: 'relative', overflow: 'hidden',
 79 |         }}>
 80 |           {/* Top glow */}
 81 |           <div style={{ position: 'absolute', top: 0, left: 0, right: 0, height: '1px', background: 'linear-
gradient(90deg, transparent, rgba(16,185,129,0.7), rgba(6,182,212,0.5), transparent)' }} />
 82 |
 83 |           {error && (
 84 |             <div style={{ marginBottom: '1.25rem', padding: '0.85rem 1rem', background: 'rgba(244,63,94,0.1)',
border: '1px solid rgba(244,63,94,0.25)', borderRadius: '12px', color: '#fda4af', fontSize: '0.875rem' }}>
 85 |               {error}
 86 |             </div>
 87 |           )}
 88 |
 89 |           <form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1.25rem' }}>
 90 |             <div>
 91 |               <label htmlFor="name" style={{ display: 'block', fontSize: '0.85rem', fontWeight: 500, color:
'#94a3b8', marginBottom: '0.5rem' }}>Full Name</label>
 92 |               <input id="name" type="text" value={name} onChange={e => setName(e.target.value)}
 93 |                 className="input-field" placeholder="John Doe" required />
 94 |             </div>
 95 |             <div>
 96 |               <label htmlFor="reg-email" style={{ display: 'block', fontSize: '0.85rem', fontWeight: 500,
color: '#94a3b8', marginBottom: '0.5rem' }}>Email</label>
 97 |               <input id="reg-email" type="email" value={email} onChange={e => setEmail(e.target.value)}
 98 |                 className="input-field" placeholder="you@example.com" required />
 99 |             </div>
100 |             <div>
101 |               <label htmlFor="reg-password" style={{ display: 'block', fontSize: '0.85rem', fontWeight: 500,
color: '#94a3b8', marginBottom: '0.5rem' }}>Password</label>
102 |               <input id="reg-password" type="password" value={password} onChange={e =>
setPassword(e.target.value)}
103 |                 className="input-field" placeholder="••••••••" required minLength={6} />
104 |             </div>
105 |             <div>
106 |               <label htmlFor="confirm-password" style={{ display: 'block', fontSize: '0.85rem', fontWeight:
500, color: '#94a3b8', marginBottom: '0.5rem' }}>Confirm Password</label>
107 |               <input id="confirm-password" type="password" value={confirmPassword} onChange={e =>
setConfirmPassword(e.target.value)}
108 |                 className="input-field" placeholder="••••••••" required />
109 |             </div>
110 |
111 |             <button type="submit" disabled={loading} className="btn-primary" style={{ width: '100%', marginTop:
'0.25rem', padding: '0.8rem', background: 'linear-gradient(135deg, #10b981, #059669)', border: '1px solid
rgba(16,185,129,0.4)', boxShadow: '0 4px 16px rgba(16,185,129,0.3)' }}>
112 |               {loading ? (
113 |                 <><div style={{ width: '18px', height: '18px', border: '2px solid rgba(255,255,255,0.3)',
borderTopColor: '#fff', borderRadius: '50%', animation: 'spin 0.8s linear infinite' }} />Creating account...</>
114 |               ) : 'Create Account →'}
115 |             </button>
116 |           </form>
117 |         </div>
118 |
119 |         <p style={{ textAlign: 'center', color: '#475569', fontSize: '0.875rem', marginTop: '1.5rem' }}>
120 |           Already have an account?{' '}
121 |           <Link to="/login" style={{ color: '#10b981', fontWeight: 600, textDecoration: 'none' }}
122 |             onMouseEnter={e => e.target.style.color='#34d399'}
123 |             onMouseLeave={e => e.target.style.color='#10b981'}
124 |           >Sign in →</Link>
```

```
125 |         </p>
126 |       </div>
127 |     </div>
128 |   );
129 | }
130 |
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–122:** Controlled form for 'name', 'email', 'password', and 'confirmPassword' with client-side validation and error display.

> **How to Explain This Module in an Interview:** Auto-logs the student in immediately upon registration.

Module: ResultPage (`frontend/src/pages/ResultPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Post-quiz scorecard and question-by-question answer breakdown.

Why It Is Used & Architectural Role: Displays score, percentage, PASSED/FAILED badge, and each question's selected vs. correct answer.

Complete Source Code (Every Line Numbered):

```
  1 | import { useLocation, useNavigate, useParams } from 'react-router-dom';
  2 |
  3 | export default function ResultPage() {
  4 |   const { quizId } = useParams();
  5 |   const location = useLocation();
  6 |   const navigate = useNavigate();
  7 |   const { result, questions } = location.state || {};
  8 |
  9 |   if (!result) {
 10 |     return (
 11 |       <div className="flex items-center justify-center p-8">
 12 |         <div className="glass-card p-8 text-center">
 13 |           <p className="text-surface-400 mb-4">No result data found.</p>
 14 |           <button onClick={() => navigate('/dashboard')} className="btn-primary">Go to Dashboard</button>
 15 |         </div>
 16 |       </div>
 17 |     );
 18 |   }
 19 |
 20 |   const { score, totalQuestions, percentage, passed, breakdown } = result;
 21 |
 22 |   return (
 23 |     <div className="min-h-screen p-8 max-w-3xl mx-auto">
 24 |       {/* Score Card */}
 25 |       <div className="glass-card p-8 text-center mb-8 animate-fade-in">
 26 |         <div className={`inline-flex items-center justify-center w-32 h-32 rounded-full mb-6 ${
 27 |           passed
 28 |             ? 'bg-gradient-to-br from-emerald-500/20 to-emerald-600/10 border-2 border-emerald-500/30'
 29 |             : 'bg-gradient-to-br from-rose-500/20 to-rose-600/10 border-2 border-rose-500/30'
 30 |         }`}>
 31 |           <div>
 32 |             <p className={`text-4xl font-bold ${passed ? 'text-emerald-400' : 'text-rose-400'}`}>
 33 |               {score}/{totalQuestions}
 34 |             </p>
 35 |             <p className={`text-lg font-semibold ${passed ? 'text-emerald-300' : 'text-rose-300'}`}>
 36 |               {percentage.toFixed(0)}%
 37 |             </p>
 38 |           </div>
 39 |         </div>
 40 |
 41 |         <div className={`inline-block px-6 py-2 rounded-full text-sm font-bold mb-4 ${
 42 |           passed
 43 |             ? 'bg-emerald-500/10 text-emerald-400 border border-emerald-500/30'
 44 |             : 'bg-rose-500/10 text-rose-400 border border-rose-500/30'
 45 |         }`}>
 46 |           {passed ? '🎉 PASSED' : '❌ FAILED'}
 47 |         </div>
 48 |
 49 |         <p className="text-surface-400">
 50 |           {passed ? 'Great job! Keep up the good work.' : 'Don\'t worry, review the material and try again!'}
 51 |         </p>
 52 |       </div>
 53 |
 54 |       {/* Breakdown Table */}
 55 |       <div className="glass-card p-6 mb-8 animate-slide-up">
 56 |         <h3 className="text-lg font-semibold text-surface-100 mb-4">Question Breakdown</h3>
 57 |         <div className="space-y-3">
```

```
 58 |           {breakdown.map((item, i) => {
 59 |             const q = questions?.find((q) => q.id === item.questionId);
 60 |             return (
 61 |               <div key={i} className={`p-4 rounded-xl border ${
 62 |                 item.correct
 63 |                   ? 'bg-emerald-500/5 border-emerald-500/20'
 64 |                   : 'bg-rose-500/5 border-rose-500/20'
 65 |               }`}>
 66 |                 <div className="flex items-start justify-between gap-3">
 67 |                   <div className="min-w-0 flex-1">
 68 |                     <p className="text-sm font-medium text-surface-200 mb-2">
 69 |                       Q{i + 1}: {q?.questionText || `Question ${item.questionId}`}
 70 |                     </p>
 71 |                     <div className="flex gap-4 text-xs">
 72 |                       <span className="text-surface-400">
 73 |                         Your answer: <span className={item.correct ? 'text-emerald-400' : 'text-rose-400'}
>{item.selectedOption || '—'}</span>
 74 |                       </span>
 75 |                       {!item.correct && (
 76 |                         <span className="text-surface-400">
 77 |                           Correct: <span className="text-emerald-400">{item.correctOption}</span>
 78 |                         </span>
 79 |                       )}
 80 |                     </div>
 81 |                   </div>
 82 |                   <span className="text-xl flex-shrink-0">{item.correct ? '✓' : '✗'}</span>
 83 |                 </div>
 84 |               </div>
 85 |             );
 86 |           })}
 87 |         </div>
 88 |       </div>
 89 |
 90 |       {/* Action buttons */}
 91 |       <div className="flex gap-4 justify-center">
 92 |         <button onClick={() => questions?.length && navigate(`/quiz/${quizId}`, { state: { questions } })}
className="btn-secondary">Retake Quiz</button>
 93 |         <button onClick={() => navigate('/progress')} className="btn-primary">View Progress</button>
 94 |       </div>
 95 |     </div>
 96 |   );
 97 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–94:** Reads '{ result, questions }' from React Router 'location.state', renders circular pass/fail badge, maps over 'breakdown', and provides 'Retake Quiz' and 'View Progress' buttons.

How to Explain This Module in an Interview: Immediate visual feedback loop reinforcing correct answers.

Module: SettingsPage (`frontend/src/pages/SettingsPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Account profile overview, app preferences display, and session logout page.

Why It Is Used & Architectural Role: Displays logged-in student's name, ID, avatar initials, and logout action.

Complete Source Code (Every Line Numbered):

```
  1 | import { useNavigate } from 'react-router-dom';
  2 | import { useAuth } from '../context/AuthContext';
  3 |
  4 | export default function SettingsPage() {
  5 |   const { user, logout } = useAuth();
  6 |   const navigate = useNavigate();
  7 |
  8 |   const handleLogout = () => {
  9 |     logout();
 10 |     navigate('/login');
 11 |   };
 12 |
 13 |   return (
 14 |     <div style={{ padding: '2rem 1rem', maxWidth: '900px', margin: '0 auto' }}>
 15 |       <div style={{ marginBottom: '2rem' }}>
 16 |         <h1 style={{ fontSize: '2rem', fontWeight: 800, background: 'linear-gradient(135deg, #a78bfa,
#38bdf8)', WebkitBackgroundClip: 'text', WebkitTextFillColor: 'transparent' }}>
 17 |           ⚙  Settings
 18 |         </h1>
 19 |         <p style={{ color: '#64748b', marginTop: '0.4rem' }}>Manage your account and preferences.</p>
 20 |       </div>
 21 |
 22 |       {/* Profile */}
```

```
 23 |       <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.07)',
borderRadius: '1.25rem', padding: '1.75rem', marginBottom: '1.5rem' }}>
 24 |         <h3 style={{ fontSize: '1.05rem', fontWeight: 700, color: '#e2e8f0', margin: '0 0 1.25rem' }}>Profile</
h3>
 25 |         <div style={{ display: 'flex', alignItems: 'center', gap: '1.25rem', marginBottom: '1.5rem' }}>
 26 |           <div style={{
 27 |             width: '64px', height: '64px', borderRadius: '50%', flexShrink: 0,
 28 |             background: 'linear-gradient(135deg, #7c3aed, #06b6d4)',
 29 |             display: 'flex', alignItems: 'center', justifyContent: 'center',
 30 |             color: 'white', fontWeight: 700, fontSize: '1.5rem',
 31 |             boxShadow: '0 8px 24px rgba(124,58,237,0.35)',
 32 |           }}>
 33 |             {user?.name?.substring(0, 2).toUpperCase() || 'SA'}
 34 |           </div>
 35 |           <div>
 36 |             <p style={{ fontSize: '1.1rem', fontWeight: 600, color: '#f8fafc', margin: 0 }}>{user?.name}</p>
 37 |             <p style={{ fontSize: '0.82rem', color: '#64748b', margin: '0.25rem 0 0' }}>Student ID:
{user?.studentId}</p>
 38 |           </div>
 39 |         </div>
 40 |
 41 |         <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(220px, 1fr))', gap: '1rem'
}}>
 42 |           <div style={{ padding: '1rem 1.25rem', background: 'rgba(255,255,255,0.02)', border: '1px solid
rgba(255,255,255,0.06)', borderRadius: '0.85rem' }}>
 43 |             <p style={{ fontSize: '0.75rem', color: '#64748b', margin: '0 0 0.25rem' }}>Account</p>
 44 |             <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0 }}>Student</p>
 45 |           </div>
 46 |           <div style={{ padding: '1rem 1.25rem', background: 'rgba(255,255,255,0.02)', border: '1px solid
rgba(255,255,255,0.06)', borderRadius: '0.85rem' }}>
 47 |             <p style={{ fontSize: '0.75rem', color: '#64748b', margin: '0 0 0.25rem' }}>Member Since</p>
 48 |             <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0 }}>Account active</p>
 49 |           </div>
 50 |         </div>
 51 |       </div>
 52 |
 53 |       {/* Preferences */}
 54 |       <div style={{ background: 'rgba(255,255,255,0.03)', border: '1px solid rgba(255,255,255,0.07)',
borderRadius: '1.25rem', padding: '1.75rem', marginBottom: '1.5rem' }}>
 55 |         <h3 style={{ fontSize: '1.05rem', fontWeight: 700, color: '#e2e8f0', margin: '0 0 1.25rem' }}
>Preferences</h3>
 56 |         <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', padding: '1rem
0', borderBottom: '1px solid rgba(255,255,255,0.05)' }}>
 57 |           <div>
 58 |             <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0 }}>Appearance</p>
 59 |             <p style={{ fontSize: '0.78rem', color: '#64748b', margin: '0.15rem 0 0' }}>Dark theme is always
on</p>
 60 |           </div>
 61 |           <span style={{ fontSize: '0.8rem', color: '#a78bfa', fontWeight: 600 }}>🌙 Dark</span>
 62 |         </div>
 63 |         <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', padding: '1rem
0', borderBottom: '1px solid rgba(255,255,255,0.05)' }}>
 64 |           <div>
 65 |             <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0 }}>Quiz Timer</p>
 66 |             <p style={{ fontSize: '0.78rem', color: '#64748b', margin: '0.15rem 0 0' }}>5 minutes per quiz</p>
 67 |           </div>
 68 |           <span style={{ fontSize: '0.8rem', color: '#a78bfa', fontWeight: 600 }}>05:00</span>
 69 |         </div>
 70 |         <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', padding: '1rem 0'
}}>
 71 |           <div>
 72 |             <p style={{ fontSize: '0.9rem', color: '#e2e8f0', margin: 0 }}>Notifications</p>
 73 |             <p style={{ fontSize: '0.78rem', color: '#64748b', margin: '0.15rem 0 0' }}>Study reminders</p>
 74 |           </div>
 75 |           <span style={{ fontSize: '0.8rem', color: '#a78bfa', fontWeight: 600 }}>On</span>
 76 |         </div>
 77 |       </div>
 78 |
 79 |       {/* Danger zone */}
 80 |       <div style={{ background: 'rgba(244,63,94,0.04)', border: '1px solid rgba(244,63,94,0.2)', borderRadius:
'1.25rem', padding: '1.75rem' }}>
 81 |         <h3 style={{ fontSize: '1.05rem', fontWeight: 700, color: '#fda4af', margin: '0 0 0.75rem' }}>Session</
h3>
 82 |         <p style={{ fontSize: '0.85rem', color: '#64748b', margin: '0 0 1.25rem' }}>Log out of your account on
this device.</p>
 83 |         <button onClick={handleLogout} className="btn-danger">Logout</button>
 84 |       </div>
 85 |     </div>
 86 |   );
 87 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–90:** Reads 'user' and 'logout' from 'useAuth()' and renders Profile, Preferences, and Session cards.

> **How to Explain This Module in an Interview:** Clean account management view.

Module: StudyPage (`frontend/src/pages/StudyPage.jsx`)

What This File Is (Beginner-Friendly Analogy): Triggers AI generation for a newly created note and presents the generated Summary, 3D Flashcards carousel, and 'Start Quiz' CTA.

Why It Is Used & Architectural Role: Calls 'POST /api/study/generate' with '{ noteId }' on mount.

Complete Source Code (Every Line Numbered):

```
  1 | import { useState, useEffect } from 'react';
  2 | import { useParams, useNavigate } from 'react-router-dom';
  3 | import API from '../api/axios';
  4 | import FlashcardFlip from '../components/FlashcardFlip';
  5 |
  6 | export default function StudyPage() {
  7 |   const { noteId } = useParams();
  8 |   const navigate = useNavigate();
  9 |   const [data, setData] = useState(null);
 10 |   const [loading, setLoading] = useState(true);
 11 |   const [error, setError] = useState('');
 12 |
 13 |   useEffect(() => {
 14 |     const generate = async () => {
 15 |       try {
 16 |         const res = await API.post('/api/study/generate', { noteId: parseInt(noteId) });
 17 |         setData(res.data);
 18 |       } catch (err) {
 19 |         setError(err.response?.data?.message || 'Failed to generate study materials. Please try again.');
 20 |       } finally { setLoading(false); }
 21 |     };
 22 |     generate();
 23 |   }, [noteId]);
 24 |
 25 |   if (loading) {
 26 |     return (
 27 |       <div className="flex flex-col items-center justify-center p-16 gap-4">
 28 |         <div className="w-16 h-16 border-4 border-primary-500/20 border-t-primary-500 rounded-full animate-
spin" />
 29 |         <div className="text-center">
 30 |           <p className="text-xl font-semibold text-surface-100">Generating Study Materials</p>
 31 |           <p className="text-surface-400 mt-2">Our AI is analyzing your notes...</p>
 32 |         </div>
 33 |       </div>
 34 |     );
 35 |   }
 36 |
 37 |   if (error) {
 38 |     return (
 39 |       <div className="flex items-center justify-center p-16 px-4">
 40 |         <div className="glass-card p-8 max-w-md text-center">
 41 |           <div className="text-4xl mb-4">⚠ </div>
 42 |           <p className="text-rose-300 mb-4">{error}</p>
 43 |           <button onClick={() => window.location.reload()} className="btn-primary">Try Again</button>
 44 |         </div>
 45 |       </div>
 46 |     );
 47 |   }
 48 |
 49 |   return (
 50 |     <div className="min-h-screen p-8 max-w-4xl mx-auto">
 51 |       {/* Section A: Summary */}
 52 |       <section className="mb-12 animate-fade-in">
 53 |         <h2 className="text-2xl font-bold text-surface-100 mb-4 flex items-center gap-2">
 54 |           <span className="w-8 h-8 rounded-lg bg-primary-500/20 flex items-center justify-center text-sm">📖</
span>
 55 |           Summary
 56 |         </h2>
 57 |         <div className="glass-card p-6 sm:p-8">
 58 |           <div className="text-surface-300 leading-relaxed" style={{ fontSize: '0.95rem', lineHeight: '1.85' }}
>
 59 |             {data.summary.split('\n').filter(p => p.trim()).map((para, i) => (
 60 |               <p key={i} style={{ marginBottom: '1rem' }}>{para}</p>
 61 |             ))}
 62 |           </div>
 63 |         </div>
 64 |       </section>
 65 |
```

```
 66 |       {/* Section B: Flashcards */}
 67 |       <section className="mb-12 animate-slide-up">
 68 |         <h2 className="text-2xl font-bold text-surface-100 mb-4 flex items-center gap-2">
 69 |           <span className="w-8 h-8 rounded-lg bg-amber-500/20 flex items-center justify-center text-sm">🃏</
span>
 70 |           Flashcards
 71 |         </h2>
 72 |         <FlashcardFlip flashcards={data.flashcards} />
 73 |       </section>
 74 |
 75 |       {/* Section C: Start Quiz */}
 76 |       <section className="animate-slide-up">
 77 |         <div className="glass-card p-8 text-center">
 78 |           <div className="text-5xl mb-4">🎯</div>
 79 |           <h2 className="text-2xl font-bold text-surface-100 mb-2">Ready for the Quiz?</h2>
 80 |           <p className="text-surface-400 mb-6">Test your knowledge with {data.questions.length} multiple choice
questions.</p>
 81 |           <button
 82 |             onClick={() => navigate(`/quiz/${data.quizId}`, { state: { questions: data.questions } })}
 83 |             className="btn-primary text-lg px-8 py-4"
 84 |           >
 85 |             🚀 Start Quiz
 86 |           </button>
 87 |         </div>
 88 |       </section>
 89 |     </div>
 90 |   );
 91 | }
```

Line-by-Line & Block-by-Block Explanation:

- **Lines 1–88:** Manages loading spinner while AI generates materials, error retry state, Section A (Summary paragraphs), Section B (''), and Section C ('Start Quiz' button).

How to Explain This Module in an Interview: Seamless single-page workflow from note creation to studying and quizzing.

Part X: Top 10 Interview Questions & Winning Answers Cheat Sheet

1. **Why did you choose Spring Boot 3 + React 18 over a MERN stack?**
   Spring Boot 3 + Java 17 gives us compile-time type safety, native multi-threading, mature PDF/PPTX binary parsing (Apache PDFBox & Apache POI), and declarative ACID transactions (`@Transactional`) when persisting multi-entity AI outputs across 6 relational tables. React 18 + Vite gives us a fast, reactive SPA for 3D flashcard flips, 1-second timer ticks, and Chart.js canvas visualizations.
2. **How do you prevent users from cheating by inspecting network responses during a quiz?**
   When generating or fetching quiz questions for an active quiz, the backend maps `QuizQuestion` entities to `QuizQuestionDTO`, which completely omits `correctOption`. Grading happens strictly on the server in `QuizService.submitAttempt()`, and correct answers are only revealed in `QuizAttemptResponse` after the attempt is saved.
3. **How did you prevent IDOR (Insecure Direct Object Reference) attacks?**
   Every endpoint extracts the authenticated student's email from the verified JWT principal (`Authentication.getName()`)—never from request parameters. All database lookups enforce ownership via `noteRepository.findByIdAndStudentId(noteId, student.getId())` or explicit ownership checks in `QuizService` and `SpacedRepetitionService` that throw a 403 `ForbiddenException`, verified by dedicated JUnit security tests.
4. **How did you solve the Hibernate N+1 query problem?**
   In `QuizAttemptRepository`, `QuizAttempt` has a lazy `@ManyToOne` to `Quiz`, which has a lazy `@ManyToOne` to `Note`. I added `@EntityGraph(attributePaths = {"quiz", "quiz.note"})` on the repository queries so Hibernate fetches attempts, quizzes, and notes in a single SQL `LEFT JOIN` query instead of executing 2N extra SELECTs.
5. **How does your backend handle LLM rate limits, slow responses, or markdown-wrapped JSON?**
   In `OpenRouterService`, I configured a 5-second connect timeout and 120-second read timeout on `RestTemplate`, implemented a 3-attempt retry loop with linear backoff (1s, 2s) when OpenRouter returns HTTP 429 or a transient network error, enforced JSON output via both prompt rules and `response_format: {"type": "json_object"}`, and stripped any ```` ```json ```` fences in `extractJsonFromResponse()` before parsing with Jackson inside a

`@Transactional` method.

6. **Explain the Leitner Spaced Repetition algorithm in `SpacedRepetitionService`.**
   Each flashcard has a `boxLevel` (1 to 5) and `nextReviewDate`. New cards start in Box 1 and are due immediately. When reviewed via `POST /api/flashcards/review`, a correct answer promotes the card to `Math.min(currentBox + 1, 5)`; a wrong answer resets it to Box 1. The next review is scheduled at `LocalDateTime.now().plusDays(INTERVAL_DAYS[newBox - 1])` where intervals are `{1, 2, 4, 8, 16}` days (i.e. 2^(box-1) days).
7. **Why is disabling CSRF in `SecurityConfig` safe in your architecture?**
   CSRF attacks rely on browsers automatically sending session cookies with cross-origin requests. Because our backend uses `SessionCreationPolicy.STATELESS` with zero cookies, and our frontend attaches the JWT explicitly in the `Authorization: Bearer` header via an Axios interceptor, a malicious third-party site cannot forge authenticated requests.
8. **Why did you store `onTimeUp` inside a `useRef` in `Timer.jsx`?**
   In `QuizPage.jsx`, `handleSubmit` is recreated whenever the user selects an answer because it depends on the `answers` state. Storing `onTimeUp` in `onTimeUpRef.current` allows the 1-second `setInterval` effect to depend only on `[totalSeconds]` (so the timer never stutters or resets when clicking options) while still calling the freshest `handleSubmit` closure when `secondsLeft === 0`.
9. **Why did you pass `headers: { 'Content-Type': undefined }` when uploading files in `NotesPage.jsx`?**
   Our shared Axios instance in `axios.js` sets a default header of `'Content-Type': 'application/json'`. When sending a binary `FormData` payload, passing `undefined` removes that default so the browser's native networking layer automatically generates `multipart/form-data; boundary=----WebKitFormBoundary...` with the exact MIME boundary delimiter required by Spring's `MultipartFile` resolver.
10. **How do your local, test, and production database environments differ?**
    In local development and production (Render), the app connects to serverless **TiDB Cloud (MySQL 8 compatible)** over TLS 1.2/1.3 using `ddl-auto=update`. In automated testing (`@ActiveProfiles("test")` and GitHub Actions CI), it switches to an **in-memory H2 database** in MySQL compatibility mode (`MODE=MySQL`) with `ddl-auto=create-drop`, running all 23 tests in ~2 seconds with complete isolation.